



anadolum
e K a m p ü s
ve

anadolu mobil

dilediğin yerden,
dilediğin zaman,
öğrenme fırsatı!



(ekampus.anadolu.edu.tr)



(mobil.anadolu.edu.tr)

ekampus.anadolu.edu.tr



Takvim



Duyurular



Ders
Kitabı (PDF)



Epub



Html5



Video



Canlı Ders



Sesli Kitap



Ünite
Özeti



Sesli Özet



Sorularla
Öğrenelim



Alıştırma



Deneme
Sınavı



Infografik



Etkileşimli
İçerik



Bilgilendirme
Panosu



Çıkmış Sınav
Soruları



Sınav Giriş
Bilgisi



Sınav
Sonuçları



Öğrenci
Toplulukları



AOSDESTEK
AÇIKÖĞRETİM DESTEK SİSTEMİ

Açıköğretim Sistemi ile ilgili
merak ettiğiniz her şey AOS Destek Sisteminde...

- Kolay Soru Sorma ve Soru-Yanıt Takibi
- Sıkça Sorulan Sorular ve Yanıtları
- Canlı Destek (Hafta İçi Her Gün)
- Telefonla Destek

aosdestek.anadolu.edu.tr

AOS DESTEK Sistemi İletişim ve Çözüm Masası

0850 200 46 10

www.anadolu.edu.tr



/AOFAnadolum



/Anadolu_Univ



instagram.com/anadoluuniv

T.C. ANADOLU ÜNİVERSİTESİ YAYINI NO: 3319
AÇIKÖĞRETİM FAKÜLTESİ YAYINI NO: 2179

BİLGİSAYAR VE PROGRAMLAMAYA GİRİŞ

Yazarlar

Doç.Dr. Cihan KALELİ (Ünite 1, 2)
Yrd.Doç.Dr. Alper BİLGE (Ünite 3, 4)
Yrd.Doç.Dr. Semih ERGİN (Ünite 5, 6)
Yrd.Doç.Dr. Efnan ŞORA GÜNAL (Ünite 7, 8)

Editör

Prof.Dr. Yaşar HOŞCAN

Bu kitabın basım, yayım ve satış hakları Anadolu Üniversitesine aittir.
“Uzaktan Öğretim” tekniğine uygun olarak hazırlanan bu kitabın bütün hakları saklıdır.
İlgili kuruluştan izin almadan kitabın tümü ya da bölümleri mekanik, elektronik, fotokopi, manyetik kayıt
veya başka şekillerde çoğaltılamaz, basılamaz ve dağıtılamaz.

Copyright © 2016 by Anadolu University
All rights reserved

No part of this book may be reproduced or stored in a retrieval system, or transmitted
in any form or by any means mechanical, electronic, photocopy, magnetic tape or otherwise, without
permission in writing from the University.

ÖĞRENME TEKNOLOJİLERİ AR-GE BİRİMİ

Öğretim Tasarımcısı

Prof.Dr. Tevfik Volkan Yüzer

Grafik Tasarım Yönetmenleri

Prof. Tevfik Fikret Uçar

Doç.Dr. Nilgün Salur

Öğr.Gör. Cemalettin Yıldız

Dil ve Yazım Danışmanı

Okt. Sedef Sezgin

Ölçme Değerlendirme Sorumluları

Öğr.Gör. Emrah Emre Özkeskin

Öğr.Gör. İbrahim Sarı

Kapak Düzeni

Prof.Dr. Halit Turgay Ünal

Grafiker

Özlem Çayırılı

Dizgi ve Yayına Hazırlama

Kitap Hazırlama Grubu

Bilgisayar ve Programlamaya Giriş

E-ISBN
978-975-06-2484-1

Bu kitabın tüm hakları Anadolu Üniversitesi'ne aittir.
ESKİŞEHİR, Ağustos 2018

3106-0-0-0-2009-V01

İçindekiler

Önsöz	ix
-------------	----

Bilgisayar ve Programlamanın Gelişimi 2

1. ÜNİTE

GİRİŞ	3
BİLGİSAYARLARIN GELİŞİMİ VE TARİHÇESİ	3
Birinci Nesil: Vakum Tüpleri	3
İkinci Nesil: Transistörler	5
Üçüncü Nesil: Tümlşik Devreler	6
Dördüncü Nesil: Mikroişlemciler	7
Moore Yasası	7
Beşinci Nesil: Yapay Zekâ	8
BİLGİSAYAR MİMARİSİ	8
Von Neumann Modeli	8
Von Neumann Engeli	9
BİLGİSAYARDA DONANIM VE YAZILIM	9
Donanım	9
Merkezî İşlem Birimi	10
Ana Bellek	11
İkincil Depolama Aygıtları	11
Girdi Aygıtları	11
Çıktı Aygıtları	11
Yazılım	11
Sistem Yazılımları	11
Uygulama Yazılımları	12
PROGRAMLAMA DİLLERİ	12
Programlama Dillerinin Hedefleri	12
Yazılımda Güvenilirlik	12
Yazılımda Sürdürülebilirlik	13
Yazılımda Verimlilik	13
Programlama Dillerinde Seviyeler	13
Özet	15
Kendimizi Sınayalım	16
Yaşamın İçinden	17
Kendimizi Sınayalım Yanıt Anahtarı	18
Sıra Sizde Yanıt Anahtarı	18
Yararlanılan ve Başvurulabilecek Kaynaklar.....	19

Bilgisayarda Veri Saklama 20

2. ÜNİTE

GİRİŞ	21
İKİLİ SAYI SİSTEMİ	21
İkili Sayı Sistemi Nedir?	21
Onlu Sayı Sisteminden İkili Sayı Sistemine Dönüşüm	21
İkili Sayı Sisteminden Onlu Sayı Sistemine Dönüşüm	22

İkili Sayı Sisteminde Dört İşlem	22
Toplama	22
Çıkarma	22
Çarpma	23
Bölme	23
BİLGİSAYARDA DİJİTAL VERİNİN SAKLANMASI	23
Sayıların Saklanması	24
Karakterlerin Saklanması	25
ASCII (American Standard Code for Information Language)	25
Genişletilmiş ASCII	27
Unicode	27
ASCII ve Unicode Sistemlerinin Karşılaştırılması	28
Diğer Veri Tiplerinin Saklanması	28
Görüntülerin Saklanması	28
Seslerin Saklanması	28
DEPOLAMA AYGITLARI	29
Birincil Depolama Aygıtları	30
Ana Bellek	31
İşlemci Ön Belleği	31
İşlemci Yazmaçları	32
İkincil Depolama Aygıtları	32
Sabit Disk	32
Hârici Sabit Disk	33
Flash Bellek	33
CD	34
DVD	34
Üçüncül Depolama Aygıtları	34
Çevrim Dışı Depolama Aygıtları	35
Özet	36
Kendimizi Sınayalım	37
Kendimizi Sınayalım Yanıt Anahtarı	38
Sıra Sizde Yanıt Anahtarı	38
Yararlanılan ve Başvurulabilecek Kaynaklar	39

3. ÜNİTE

Bilgisayarda Veri İşleme..... 40

GİRİŞ	41
VERİ İŞLEME NEDİR?	41
Veri, Enformasyon ve Bilgi Kavramları	41
Veri İşleme	42
Veri İşleme Döngüsü	43
Veri Toplama	43
Veri Hazırlama	43
Veri Girişi	43
Veri İşleme	43
Çıktı ve Yorumlama	43
Depolama	43

İŞLEMCİ MİMARİSİ	44
Merkezî İşlem Birimin Yapısı	44
Aritmetik ve Mantık Birimi	45
Kontrol Birimi	46
Yazmaçlar	46
Komut Kümesi Mimarisi	46
Komut Elemanları	47
Komut Çeşitleri	48
Komutların Gösterimi	48
PROGRAMLARIN ÇALIŞMA ESASLARI	49
Programların Ana Belleğe Kopyalanması	49
Komut Çevrimi	51
Özet	52
Kendimizi Sıneyalım	54
Okuma Parçası	55
Kendimizi Sıneyalım Yanıt Anahtarı	56
Sıra Sizde Yanıt Anahtarı	57
Yararlanılan ve Başvurulabilecek Kaynaklar.....	57

Programlamada Temel Kavramlar ve C'ye Giriş 58

4. ÜNİTE

GİRİŞ	59
ALGORİTMALAR	59
Algoritma Nedir?	59
Algoritmaların Özellikleri	60
Algoritmaların Programlamadaki Rolü ve Önemi	60
Algoritmaların Gösterimi	61
Konuşma Dili	61
Akış Şemaları	61
Sözde Kod	62
YÜKSEK SEVİYE PROGRAMLAMA DİLLERİ	62
Makine Dilinden Çevirici Dillere Geçiş	62
Programlamada Yüksek Seviye Dillere Geçiş	63
Derleyiciler ve Yorumlayıcılar	64
Derleyiciler ve Yorumlayıcılar Arasındaki Farklar	66
Kaynak Kodu	66
Yaygın Kullanılan Yüksek Seviye Programlama Dilleri	67
C PROGRAMLAMA DİLİ	67
C Dili Hakkında Temel Bilgiler	67
Program Yazım Kuralları	68
C Dilindeki Anahtar Kelimeler	68
C Dilindeki Operatörler	69
İlişkisel Operatörler	69
Mantıksal Operatörler	69
Matematiksel Operatörler	69
Atama Operatörleri	70

C Programlarının Temel Yapısı	70
Önişlemci Direktifleri	70
Ana Fonksiyon	70
Basit Bir C Programı	71
Özet	73
Kendimizi Sınayalım	75
Kendimizi Sınayalım Yanıt Anahtarı	76
Sıra Sizde Yanıt Anahtarı	76
Yararlanılan ve Başvurulabilecek Kaynaklar	77

5. ÜNİTE

C'de Veri Tipleri ve Tanımlamalar	78
GİRİŞ	79
Veri Nedir?	79
C Programlama Dilindeki Temel Veri Tipleri Nelerdir?	80
Sayılar	80
Tam Sayı Veri Tipleri	80
Kesirli Sayı Tipleri	82
Karakterler	82
Dizgiler (Strings)	83
C PROGRAMLAMA DİLİNDE DEĞİŞKEN ADLANDIRMA VE TANIMLAMALAR	84
Değişken İsimlendirilmesi Nedir?	84
Değişken İsimlendirilmesi Kuralları	85
Değişken İsimlendirilmesi Örnekleri	86
Değişkenlerin Ekranda Görüntülenmesi	87
Klavyeden Bir Değişkene Veri Girilmesi	89
enum Anahtar Sözcüğü	90
Mantıksal Değişken Tipleri	90
C PROGRAMLAMA DİLİNDE DEPOLAMA TÜRLERİ	90
Geçici Depolama	90
Kalıcı Depolama	91
Özet	92
Kendimizi Sınayalım	94
Kendimizi Sınayalım Yanıt Anahtarı	95
Sıra Sizde Yanıt Anahtarı	95
Yararlanılan ve Başvurulabilecek Kaynaklar	95

6. ÜNİTE

İşleçler, Koşul ve Yineleme Deyimleri.....	96
GİRİŞ	97
İşleç Nedir?	97
İşleç Çeşitleri	97
Aritmetik İşleçler	97
Toplama ve Çıkarma İşleçleri	98
Çarpma ve Bölme İşleçleri	98
Kalan İşleci	98
Artırma ve Eksiltme İşleci	99
Ön Ek ve Son Ek	99

İlişkisel İşleçler	100
Mantıksal İşleçler	101
Mantıksal VE (AND) İşleci	101
Mantıksal VEYA (OR) İşleci	102
Mantıksal DEĞİL (NOT) İşleci	103
Bitsel İşleçler	103
Bitsel VE (AND) İşleci	103
Bitsel VEYA (OR) İşleci	104
Bitsel ÖZEL VEYA (XOR) İşleci	104
Bitsel DEĞİL (NOT) İşleci	105
Sola ve Sağa Öteleme İşleçleri	105
Atama İşleci	106
Diğer İşleçler	106
İşleçler Arasındaki Öncelik İlişkisi	106
KOŞUL VE YİNELEME DEYİMLERİ	107
IF Deyimi	107
SWITCH Deyimi	109
FOR Deyimi	110
WHILE Deyimi	110
DO – WHILE Deyimi	111
Özet	112
Kendimizi Sınayalım	113
Kendimizi Sınayalım Yanıt Anahtarı	114
Sıra Sizde Yanıt Anahtarı	114
Yararlanılan ve Başvurulabilecek Kaynaklar	115

Diziler, Göstericiler ve Dizgiler..... 116

7. ÜNİTE

GİRİŞ	117
DİZİLER	117
Dizi Tanımlama	117
Dizilere Değer Atama	118
İki ve Üç Boyutlu Diziler	125
GÖSTERİCİLER	130
Gösterici Tanımlama	130
Göstericilerin Dizilerle Birlikte Kullanılması	132
Gösterici Aritmetiği ve sizeof() Fonksiyonu	133
DİZGİLER	135
Özet	137
Kendimizi Sınayalım	138
Kendimizi Sınayalım Yanıt Anahtarı	139
Sıra Sizde Yanıt Anahtarı	140
Yararlanılan ve Başvurulabilecek Kaynaklar	140

8. ÜNİTE

Fonksiyonlar	142
GİRİŞ	143
FONKSİYON TANIMLAMA VE ÇAĞIRMA	143
Fonksiyon tanımı	144
FONKSİYONLARA ARGÜMAN GEÇİRME	146
Fonksiyona Değer Yöntemi ile Argüman Geçirme	146
Fonksiyona Referans Yöntemi ile Argüman Geçirme	147
Dizileri Fonksiyona Argüman olarak Geçirme	148
C STANDART HAZIR KÜTÜPHANE FONKSİYONLARININ KULLANIMI	151
Matematiksel Kütüphane Fonksiyonları (math.h)	151
Dizgi ve Karakter Fonksiyonları (string.h, ctype.h)	152
Dinamik Bellek Ayırma Fonksiyonları	156
Özet	158
Kendimizi Sınayalım	159
Kendimizi Sınayalım Yanıt Anahtarı	161
Sıra Sizde Yanıt Anahtarı	161
Yararlanılan ve Başvurulabilecek Kaynaklar	161

Önsöz

Sevgili öğrenciler,

Günümüzde bilim ve teknolojik gelişimlerin hızlı ilerleyişi, en temel düzeyde, bilgisayar kullanımının yaygınlaşması ve bilginin hızlı paylaşımı ile mümkün olmuştur. Önceleri yalnızca hesaplama işlemlerinin hızlandırılması maksadıyla geliştirilen bilgisayarlar, bugün hayatın her alanında yaygınlaşmıştır. İnternet teknolojilerinin de geliştirilmesiyle bilgisayarlar, sadece hesaplama araçları olarak değil, bilginin yayılımını ve etkisini artıran bir platform olarak kullanılmaktadırlar.

Bilgisayarların en önemli özelliği, ihtiyaçlar kapsamında, isteğe uygun olarak programlanabilmeleridir. Bu kitap ile bilgisayarların veri depolama, veri üzerinde işlem yapma gibi becerilerinin fiziksel karşılıklarını kavrayacaksınız. Fiziksel donanımları kontrol etmek amacıyla yazılımın nasıl önem kazandığını anlayacak ve öncelikle bu donanımları yönetmek, daha sonra yararlı işlevler yerine getirmek üzere bilgisayar programları geliştirilmesinin yöntemleri hakkında bilgi sahibi olacaksınız. Ayrıca temel programlama işlemlerine başlanan bu kitap ile C dilinin sağladığı programlama yöntemleri hakkında bilgi sahibi olacaksınız.

Bu kitap ile siz öğrencilerimize günlük hayatımızın vazgeçilmez unsurları olan bilgisayarların nasıl çalıştığını ve nasıl programlandıklarını anlatan bir başvuru kaynağının ortaya konması hedeflenmiştir. Bu doğrultuda, kitabın sizlere ulaşmasında emeği geçen herkese, göstermiş oldukları çabadan dolayı teşekkür ederim.

Editör

Prof.Dr. Yaşar HOŞCAN

1

Amaçlarımız

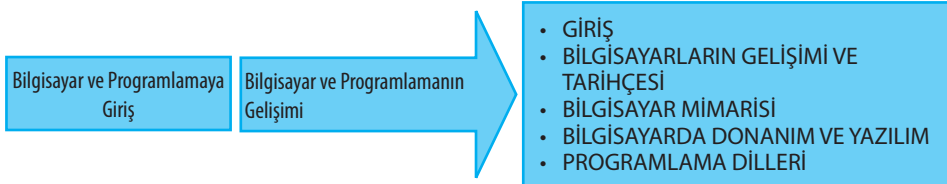
Bu üniteyi tamamladıktan sonra;

- 👁 Hayatımızın vazgeçilmez parçası olan bilgisayarların gelişimini ve tarihçesini tanımlayabilecek,
- 👁 Bilgisayarların temel çalışma prensiplerini açıklayabilecek,
- 👁 Bilgisayarları oluşturan bileşenleri ifade edebilecek,
- 👁 Programlama dillerinin temel özelliklerini listeleyebilecek ve çeşitli programlama dillerini karşılaştırabilecek bilgi ve becerilere sahip olabileceksiniz.

Anahtar Kavramlar

- Bilgisayar Nesilleri
- Von Neumann Modeli
- Merkezî İşlem Birimi
- Girdi/Çıktı Aygıtları
- Sistem Yazılımları
- Uygulama Yazılımları
- Programlama Dillerinin Özellikleri

İçindekiler



Bilgisayar ve Programlamanın Gelişimi

GİRİŞ

Bilgisayar ve Programlamanın Gelişimi ünitesinde işlenen konuların ana başlıkları bilgisayarların gelişimi ve tarihçesi, bilgisayar mimarisi, bilgisayarda donanım ve yazılım, ve programlama dilleridir.

Bilgisayar sistemlerini anlayabilmek ve bu sistemlerin temel özelliklerine hakim olabilmek için, bilgisayarların doğuşunu, gelişimini ve tarihçesini incelemek gerekir. Modern bilgisayarların tarihi “Bilgisayarların Gelişimi ve Tarihçesi” başlığında, bilgisayarın temel işlevsel yapısı ve tasarımı “Bilgisayar Mimarisi” başlığında anlatılmıştır. Bir bilgisayarı meydana getiren iki temel bölüm olan donanım ve yazılım hakkındaki temel bilgiler “Bilgisayarda Donanım ve Yazılım” başlığı altında incelenmiştir. Günümüzdeki modern teknolojilerin ve yazılımların oluşmasında önemli rolü olan programlama dilleri ise ünitenin son konusu olan “Programlama Dilleri” başlığında anlatılmıştır.

BİLGISAYARLARIN GELİŞİMİ VE TARİHÇESİ

Günümüz insanı için hayatın vazgeçilmez bir parçası olan bilgisayarlar, en basit tabirle, aritmetiksel ve mantıksal işlemler yapabilen, programlanabilir elektronik cihazlardır. Art arda veya eş zamanlı olarak birçok işlemi hızlı ve kolay bir şekilde yapabilen bilgisayarlar, insan hayatını kolaylaştıran ve değiştiren en önemli teknolojik buluşlardandır.

Bilgisayarların gelişim sürecine bakıldığında; basit bir hesap makinesi olan abaküsten, çağımızın en popüler cihazlarından olan tablet bilgisayarlara uzanan teknolojik bir devrim gözlemlenmektedir. Tarihte yapılmış ilk bilgisayar, 1946 yılında Amerika Birleşik Devletleri’nde açıklanmış, ENIAC (Elektronik Sayısal Entegreli Hesaplayıcı) adlı makinedir.

Modern bilgisayarların tarihi beş ana dönem altında incelenir:

- Birinci nesil bilgisayarlar (1940-1956): Vakum tüplerinin kullanıldığı ENIAC benzeri çok büyük cihazlardır.
- İkinci nesil bilgisayarlar (1956-1963): Transistörlere geçişin yapıldığı bilgisayarlardır.
- Üçüncü nesil bilgisayarlar (1964-1971): Tümüleşik devreler kullanılan ve kendi aralarında haberleşebilen bilgisayarlardır.
- Dördüncü nesil bilgisayarlar (1971 - günümüz): Günümüzde kullanılan bilgisayarların da dâhil olduğu modern bilgisayarlardır.
- Beşinci nesil bilgisayarlar (günümüz - yakın gelecek): Yapay zekâ ile donatılmış, gelişim süreci devam eden bilgisayarlardır.

Birinci Nesil: Vakum Tüpleri

Günümüzde kullanılan modern bilgisayarların temeli 1940’lı yıllarda atılmıştır. Lamba olarak da adlandırılan vakum tüplerinin kullanımına dayanan bu bilgisayarlar, devasa büyüklükte yer kaplayan elektronik cihazlardır.

Delikli kart: Üzerine açılan delikler sayesinde sayısal bilgi taşıyabilen, bu bilgiler aracılığıyla makinelere komut veren karton veya plastik parçalardır.

Birinci nesil bilgisayarlarda elektronik sistemi olarak vakum tüpleri, bellek olarak da manyetik silindirler kullanılmıştır. Bu bilgisayarların programlanmasında, bilgisayar tarafından algılanabilen en düşük seviye dil olan makine dili kullanılmıştır. Veri girişi kağıt şeritler ve **delikli kartlar** ile yapılırken, sonuçlar yazılı çıktı olarak alınmıştır.

Resim 1.1

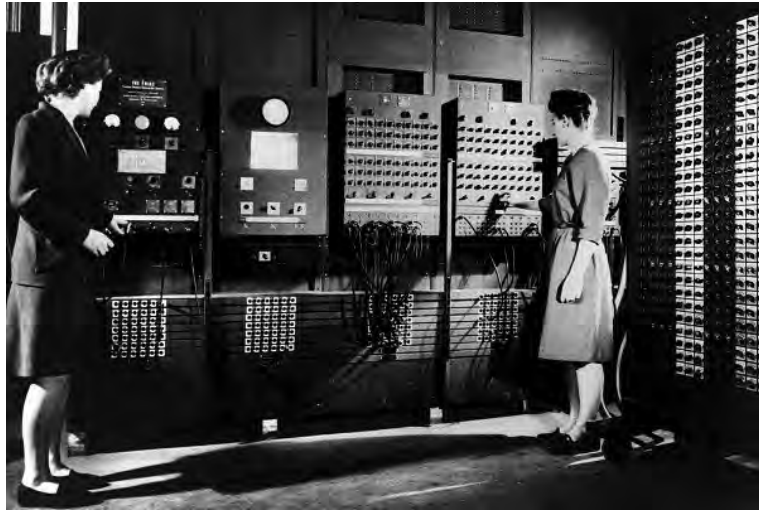
Birinci nesil bilgisayarlarda kullanılan vakum tüpleri



Birinci nesil bilgisayarların en önemli temsilcisi ENIAC adlı makinedir. Elektrikle çalışan ve elektronik veri işlemesine sahip ilk bilgisayar olan ENIAC, İkinci Dünya Savaşı sırasında duyulan gereksinimler nedeniyle, Amerika Birleşik Devletleri tarafından üretilmiştir. Yaklaşık 30 ton ağırlığında, 18000 vakum tüpü kullanan, 167 metrekare yer kaplayan devasa bir makinedir.

Resim 1.2

İlk dijital bilgisayar ENIAC ve programcılar



Kaynak: computerhistory.org

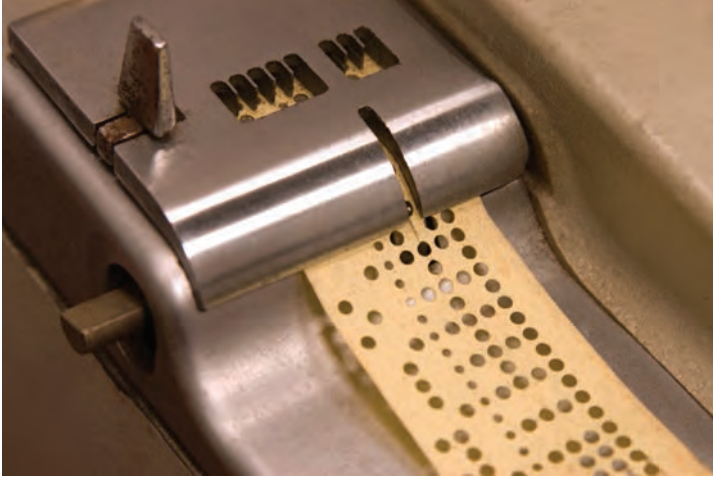
ENIAC, eşitlikleri karşılaştırabiliyor, dört işlemi çözebiliyor ve kare kök hesaplayabiliyordu. Belirtilen sayısal ve mantıksal yeteneklerinden dolayı, ENIAC ilk bilgisayar olarak kabul görmüştür.



DİKKAT

Resim 1.3

Veri girişinde kullanılan delikli kart örneği



Bu dönemde geliştirilen bilgisayarlardaki en önemli problem, fiziksel olarak çok fazla yer kaplamalarıydı. Ayrıca, işlemler sırasında çok fazla elektrik kullanılıyor ve yüksek miktarda ısı üretiliyordu. Bilgisayarlar tek seferde yalnız bir işlem yapabilme kapasitesine sahipti ve sonuç almak bazen günler sürebiliyordu. Belirtilen tüm bu sebeplerden dolayı, birinci nesil bilgisayarlarda işlem yapmak oldukça maliyetli olmuştur.

İkinci Dünya Savaşı sırasında geliştirilmeye başlanan ENIAC'ın ilk bilgisayar olarak kabul edilmesinin nedenleri nelerdir?



SIRA SİZDE

İkinci Nesil: Transistörler

Transistör, elektrik akımını kontrol edebilen, gerektiğinde anahtar olarak kullanılabilen yarı iletken bir elektronik devre elemanıdır. 1947 yılında, dünyanın önde gelen telefon şirketlerinden Bell laboratuvarlarında, Amerikalı fizikçiler John Bardeen, Walter Brattain ve William Shockley tarafından icat edilmiştir. Vakum tüplerine göre çok daha küçük, ekonomik, hızlı ve güvenilir olan transistörler, radyo, televizyon, telefon ve bilgisayar gibi araçların daha kolay üretilmesine olanak sağlamıştır.

Transistörün üretilmesiyle, bilgisayar teknolojisinde devrim yaşanmış ve ikinci nesil bilgisayarlar ortaya çıkmıştır. IBM, Honeywell ve Control Data gibi birçok firma ikinci nesil bilgisayarları üretmeye başlamıştır. Vakum tüpleri yerine transistör kullanımına geçilen bu bilgisayarlarda, fiziksel boyut küçülmüş ve elektrik tüketimi azalmıştır. Fakat bilgisayarlarda yapılan işlemler sırasında ortaya çıkan yüksek ısıdan kaynaklanan problemler, bu nesilde de devam etmiştir. Bu döneme ait bilgisayarlar IBM 7090, IBM 1400, CDC 164 olarak örneklendirilebilir.

Resim 1.4

Bir transistör çeşidi



Kaynak: sdigital-components.com

DİKKAT



Transistörlerin teknolojinin gelişimi üzerinde muazzam bir etkisi olmuştur. Transistörü geliştiren bilim adamları, 1956 yılında Nobel Fizik ödülünü kazanmıştır.

Çevirici dil: İnsanlar tarafından algılanabilen sembollerin kullanıldığı, alt seviye bir programlama dilidir. Programlaması karmaşık ve zor olan makine dilinin yerine geçmiştir. Çevirici dil ile yazılan bir program, çevirici (assembler) olarak adlandırılan başka bir program tarafından makine koduna dönüştürülür.

Tümleşik devre: Yarı iletken maddeler ile tasarlanmış, ince yüzeyler üzerine yerleştirilmiş bir elektronik devredir. Aynı veya farklı çeşit devre elemanlarının belirli bir düzen ve amaç doğrultusunda bir araya getirilmesiyle oluşturulur.

İkinci nesil bilgisayarlar döneminde yaşanan gelişmeler sadece transistörlere geçişle sınırlı kalmamıştır. Bu dönemde makine dilinden **çevirici dil** (assembly language) kullanımına geçilerek, programların daha kolay yazılması ve insanlar tarafından daha rahat anlaşılması sağlanmıştır. Üst düzey programlama dillerinin temelleri atılmış, COBOL ve FORTRAN programlama dillerinin ilk versiyonları oluşturulmuştur. Programlama konusunda yaşanan bu gelişmelerle birlikte bilgisayar programcısı, bilgisayar sistem uzmanı, program analisti gibi meslek tipleri oluşmuş ve yazılım endüstrisi doğmuştur.

Üçüncü Nesil: Tümleşik Devreler

Bilgisayarların elektronik donanımında transistörlerin kullanılması bilgisayar teknolojisi-ne seviye atlatırken, bazı problemleri de beraberinde getirmiştir. Kullanılan transistör sayısının fazlalığı sebebiyle, devreler karmaşık hâle gelmiş ve transistör kablolarını birbirine bağlamak zorlaşmıştır. Fiziksel kısıtlardan dolayı transistörlerin ve devrelerin küçülmesi olanaksız olmuştur. Bu problemler, 1958 yılında Jack Kilby tarafından geliştirilen tümleşik devrenin transistörlerin yerini almasıyla aşılmış, üçüncü nesil bilgisayarlarda **tümleşik devreler** kullanılmıştır.

Resim 1.5

Tümleşik devre çeşitleri



Kaynak: sdigital-components.com

Yarı iletken silikondan yapılan tümleşik devreler ile birlikte, transistörlerin fiziksel olarak küçültülmesi sağlanmış ve bir devre üzerine binlerce transistör yerleştirilebilmiştir. Bu gelişmenin sayesinde daha küçük ve hızlı bilgisayarlar üretilmiş ve üretim maliyetleri düşürülmüştür. Böylelikle, tarihte ilk defa bilgisayarların geniş kitlelere ulaşabilmesi sağlanmıştır. Üçüncü nesil bilgisayarların başlıca örnekleri IBM 360, IBM 370 ve PDP (Personal Data Processor) serileridir.

DİKKAT



Bilgisayarların küçülmesi, üretim maliyetlerinin düşmesi ve işletim sırasında harcanan elektriğin azalması kişisel bilgisayarların ortaya çıkması için gerekli ortamı hazırlamıştır.

Üçüncü nesil bilgisayarlar döneminde işletim sistemleri, veri girişi ve çıkışı araçları ile ilgili gelişmeler yaşanmıştır. Bilgisayarlarda işletim sistemi mimarisi oluşturulmuş, birçok programın aynı anda çalışması sağlanmıştır. Bilgisayar belleği işletim sistemi tarafından kontrol edilmiş, ve programlar tarafından ortak olarak kullanılmıştır. Daha önceki dönemlerde veri girişi için kullanılan delikli kart ve kağıt şeridin yerine klavye kullanımına geçilmiştir. İşlem sonuçlarının yazılı çıktı ile alınması son bulmuş, operasyonlar ve sonuçlar için monitörler devreye girmiştir.

İNTERNET



Vakum tüpleri ve transistörlerden tümleşik devrelere geçişin tarihi için http://www.nobelprize.org/educational/physics/integrated_circuit/history/index.html

Dördüncü Nesil: Mikroişlemciler

Günümüzde kullanılan modern bilgisayarları da kapsayan dördüncü nesil bilgisayar dönemi, **mikroişlemcilerin** üretilmesiyle başlamıştır. Genel amaçlı ilk mikroişlemci, Amerikalı bilim adamı Ted Hoff tarafından, 1971 yılında üretilen, yaklaşık 2300 transistöre sahip, Intel 4004 adlı işlemcidir. Mikroişlemcilerin geliştirilmesi, daha hızlı, daha küçük ve daha ucuz bilgisayarların yapılmasına imkan vermiştir.

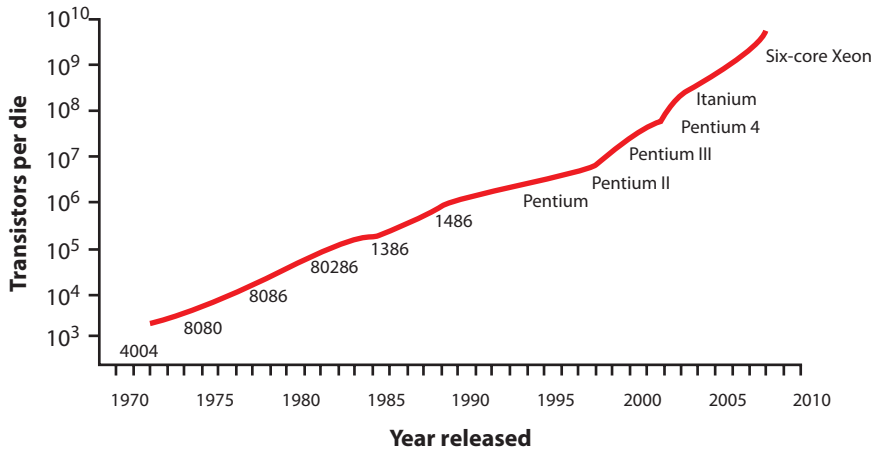
Mikroişlemci: Tek bir tümleşik devre üzerinde merkezi işlem biriminin (CPU) fonksiyonlarını gerçekleştiren, programlanabilir elektronik bileşendir.

Moore Yasası

Intel firmasının kurucularından Gordon Moore, 1965 yılında yayınlanan makalesinde tümleşik devreler içindeki transistör sayısının her yıl iki katına çıkacağını öne sürmüştür. On yıllık süreçte gözlemlerine devam eden Moore, transistör sayısının her iki yılda bir iki katına çıkacağına kanaat getirmiştir.

Mikroişlemcilerin tarihsel gelişimi incelendiğinde, Moore'un öngörüsünün şimdiye kadar doğru olduğu görülmektedir. Gelişmiş mikroişlemcilerdeki transistör sayısı milyonları geçmiştir. Yayınlandığı tarihten günümüze kadar geçerliliğini koruyan bu öngörü, bilişim dünyasında *Moore Yasası* olarak adlandırılır.

Resim 1.6



Moore Yasası doğrultusunda yıllara göre transistör sayıları

Kaynak: computerhistory.org

Dördüncü nesil bilgisayarlar döneminde, mikroişlemcilerde yaşanan gelişmeler, kişisel bilgisayarların üretilmesinin de önünü açmıştır. İlk kişisel bilgisayar, 1981 yılında IBM tarafından geliştirilmiştir. Takip eden dönemde, Apple firması da Macintosh adlı ürününü piyasaya sürmüş ve iki firma arasında büyük bir rekabet doğmuştur.

Modern işlemcileri üreten, piyasadaki pazar payına sahip başlıca firmalar hangileridir?



SIRA SİZDE

Kişisel bilgisayarların evlere, iş yerlerine ve okullara taşınmasıyla beraber, kullanıcı dostu gelişmeler yaşanmıştır. Kullanıcı grafik ara yüzleri oluşturularak, insanların monitör kullanım alanları genişletilmiştir. Avuç içi cihazlar ve fareler geliştirilmiş, kullanıcı işlemlerinin bu cihazlar ile komutlandırılması sağlanmıştır. Yakın yerlerde bulunan bilgisayarların daha etkin kullanılması için bilgisayarlar birbirine bağlanmış ve yerel ağlar oluşturulmuştur. Oluşan yerel ağların birbirine bağlanması ve tüm dünya geneline yayılması da Internet kavramı ortaya çıkmıştır.

Modern bilgisayar teknolojisini incelediğimizde, bilgisayarların ve bilgisayarlı elektronik cihazların günlük yaşamımızın vazgeçilmez parçaları olduğunu görmekteyiz. İletişim, bankacılık, eğitim, sağlık, mühendislik, sanayi ve ticaret gibi birçok sektörde üst düzey bilgisayar teknolojileri kullanılır. Günümüz insanı, her geçen gün gelişen ve yenilenen akıllı telefonlara, dizüstü bilgisayarlara ve tablet bilgisayarlara tanıklık etmektedir. Gelişen bağlantı altyapıları ve kablosuz teknolojiler sayesinde, her alanda Internet'e erişebilmek mümkündür.

İNTERNET



Modern işlemciler hakkında detaylı bilgiye ulaşmak ve gelişmeleri yakından takip etmek için <http://www.cpu-world.com>

Yapay Zekâ: Bilgisayarların ve makinelerin karmaşık problemlere insanlar gibi çözümler üretmesini sağlayan bilim dalıdır. İnsanlara özgü düşünme, karar verme, analiz etme gibi becerilerin makinelere kazandırılmasını hedefler.

Beşinci Nesil: Yapay Zekâ

Yakın gelecekte uygulama ve kullanım alanlarının yaygınlaşacağı beşinci nesil bilgisayarlar, **yapay zekâ** teknolojileri ile donatılmaktadır. Modern bilgisayarlar ses tanıma yöntemlerini kullanarak, verilen komutları algılayabilmekte ve gerekli cevapları verebilmektedir. Bilgisayarların hesaplama ve işleme gücünü arttırmak da bilim adamlarının yoğunlukla üzerinde çalıştığı bir konudur. Tüm bu çalışmalar ve ileri programlama teknikleriyle insan zekâ düzeyine yakın makineler geliştirilecektir.

BİLGİSAYAR MİMARİSİ

Bilgisayarın temel işlevsel yapısına ve tasarımına bilgisayar mimarisi adı verilir. Bilgisayar sisteminde yer alan bileşenlerin tasarım ve düzenlenme şekli, bilgisayar mimarisinde yer alır.

Modern bilgisayar dünyasında, bilgisayarların kullanım alanları ve yerleri oldukça değişiklik göstermektedir. Farklı ihtiyaçları karşılamak için farklı tasarımlar geliştirilmiştir. Tüm bu farklılıklara rağmen, modern bilgisayarlar, aynı mimari yapısını esas alır. Bu yapıya "**Von Neumann Modeli**" adı verilmiştir.

Von Neumann Modeli

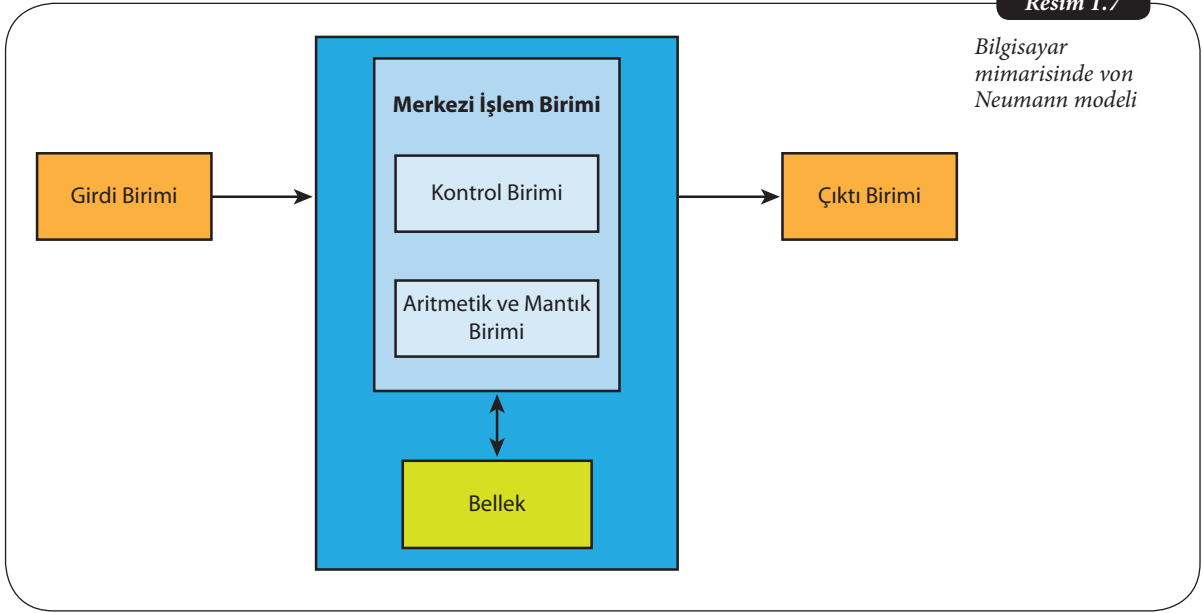
Modern bilgisayarların yapısını oluşturan mimari modeldir. Amerikalı bilim adamı John von Neumann tarafından, 1945 yılında ortaya konulmuştur.

İşlemci ve belleğin birbirinden ayrıldığı bu modelde, bilgisayarı oluşturan üç ana bileşen öngörülmüştür:

- **Girdi/Çıktı Birimleri:** Kullanıcı ile bilgisayar arasındaki iletişimi sağlayan birimlerdir. Kullanıcı komutlarının girilmesi ve işlem sonuçlarının alınması, bu birimlerin temel görevidir.
- **Merkezî İşlem Birimi:** Verilerin işlenmesinden sorumlu bileşendir. Programların çalışmasını ve işlemlerin yapılmasını sağlar.
- **Bellek:** Bilgisayarın kullanacağı veri deposudur. Bilgisayar tarafından işlenecek veriler, programlar ve komutlar, bu bileşende saklanır.

Resim 1.7

Bilgisayar
mimarisinde von
Neumann modeli



Von Neumann modelini kullanan ilk bilgisayar, Princeton Üniversitesi'nde yapılmıştır. IAS adı verilen bu bilgisayar, John von Neumann ve ekibi tarafından geliştirilmiştir.



DİKKAT

Von Neumann modelinde, bileşenler arasındaki iletişim elektronik sinyaller ile sağlanmaktadır. Sinyallerin iletimi, **veri yolu** (bus) adı verilen kabloların bilgisayar bileşenlerini bağlaması ile yapılır.

Veri yolu: Bilgisayarlar içindeki parçalar arasında veri transferine imkan veren devre sistemidir.

Von Neumann Engeli

İşlemci ve bellek arasındaki veri transfer hızı, bellek kapasitesine göre çok küçüktür. Bu transfer hızı, modern makinelerin işlemci hızıyla kıyaslandığında da oldukça yavaş kalmaktadır. Ortaya çıkan bu gecikmeden dolayı, işlemci bellekten alınacak veya belleğe gönderilecek veriyi beklemek durumunda kalır. İşlemci ve bellek arasındaki veri transferi dolayısıyla yaşanan bu kısıtlamaya *Von Neumann Engeli* adı verilmiştir.

Von Neumann engelini ortadan kaldırmak veya etkisini azaltmak için birçok çalışma yapılmıştır. Bilgisayar ve işlemci mimarisinde değişikliklere yol açan çeşitli yöntemler geliştirilmiştir. Bu yöntemlerin başlıcaları ön bellek kullanımı (caching), paralel işleme (parallel processing), çoklu işleme (multithreading) ve küme hâlinde işleme (pipelining) olarak listelenir.

Von Neumann modelinin günümüz bilgisayarlarına kazandırdığı en önemli özellik nedir?



SIRA SİZDE

BİLGİSAYARDA DONANIM VE YAZILIM

Bilgisayarlar, donanım (hardware) ve yazılım (software) adı verilen iki bölümden meydana gelir. Bir bilgisayar sistemini işletebilmek için, bu iki temel unsura ihtiyaç vardır.

Donanım

Bilgisayarın fiziksel ve elektronik yapısını oluşturan bileşenlere donanım adı verilir. Bir bilgisayarı fiziksel yapısına göre değerlendirdiğimizde, bilgisayarın birçok donanım bileşeninden oluşan bir sistem olduğunu görmekteyiz. Bu sistemi oluşturan parçaların her birinin farklı bir görevi bulunmaktadır. İşletilebilir bir bilgisayara sahip olmak için, bu parçalar belirli bir düzen içerisinde bir araya getirilmelidir. Modern bilgisayarları oluşturan beş temel donanım bileşeni vardır:

- Merkezî İşlem Birimi (CPU - Central Processing Unit)
- Ana Bellek (Main Memory)
- İkincil Depolama Aygıtları (Secondary Storage Devices)
- Girdi Aygıtları (Input Devices)
- Çıktı Aygıtları (Output Devices)

Resim 1.8

Bilgisayar sistemini oluşturan temel donanım bileşenleri



Merkezî İşlem Birimi

Bilgisayarda programların çalışmasını ve işlemlerin yapılmasını sağlayan temel bileşen merkezî işlem birimidir. Bilgisayarın beyni olarak da kabul edilen merkezî işlem birimi, bilgisayarın en önemli ve karmaşık parçasıdır.

Resim 1.9

Modern bir merkezî işlem biriminin üstten görünümü



Modern bilgisayarlarda, merkezî işlem birimi olarak mikroişlemciler görev yapmaktadır. Bu işlemcilerin görevi, işlenecek veriyi ve kullanılacak programı belleğe yüklemek, verinin işlenmesini sağlamaktır. Fiziksel olarak çok küçük, performans olarak çok hızlı olan mikroişlemcilerin üç ana parçası vardır:

- Kontrol Birimi (CU - Control Unit)
- Aritmetik ve Mantık Birimi (ALU - Arithmetic and Logic Unit)
- Yazmaçlar (Registers)

Ana Bellek

Ana bellek, bilgisayarın çalışma esnasında kullandığı alandır. Bilgisayarda bir program yürütülürken, programın kendisi ve programla ilgili veri ana bellekte tutulur.

Ana bellek, çoğunlukla rastgele erişimli bellek (RAM - Random Access Memory) olarak da adlandırılır. Bu tür belleklerde veriye erişim hızlı bir şekilde gerçekleşir. Günümüzde kullanılan kişisel bilgisayarların ana bellek kapasiteleri GB (Gigabyte) cinsinden belirtilir.

Ana bellekte yapılan işlemler kalıcı değildir. Bilgisayar kapatıldığında veya elektrik bağlantısı kesildiğinde, ana bellekteki içerik silinir.



DİKKAT

İkincil Depolama Aygıtları

Yardımcı bellek olarak da tanımlanan ikincil bellek, verinin uzun süre saklanmasına imkan sağlayan bellek türüdür. Bilgisayar kapatıldığında veya elektrik bağlantısı olmadığında, ikincil bellekteki bilgilerde herhangi bir kayıp yaşanmaz.

Bilgisayarda programlar ve önemli veriler, ikincil depolama aygıtlarında saklanır. Bu programlar veya veriler, ihtiyaç duyulması hâlinde, ikincil bellekten ana belleğe taşınarak kullanılır.

İkincil depolama aygıtlarının en sık kullanılan çeşidi sabit disklerdir. Genellikle tüm bilgisayarlarda gömülü bir sabit disk bulunmaktadır. Günümüzde sabit disklerin kapasitesi TB (Terabyte) seviyesine kadar çıkmıştır. Sıklıkla kullanılan diğer depolama aygıtları harici disk, CD (compact disc), DVD (digital versatile disc), manyetik teyp ve USB bellek olarak listelenebilir.

Girdi Aygıtları

Kullanıcılardan veya farklı cihazlardan bilgisayara gelen her türlü veriye girdi adı verilir. Bu verilerin toplanması ve bilgisayara iletilmesi, girdi araçları tarafından gerçekleştirilir.

Modern bilgisayarlarda en sık kullanılan girdi araçları klavye, fare, mikrofon, dijital kamera, tarayıcı ve barkod okuyucudur.

Çıktı Aygıtları

Bilgisayarın kullanıcılar veya farklı cihazlar için oluşturduğu her türlü veriye çıktı adı verilir. Üretilen verilerin belirli bir formata çevrilmesi ve sunulması çıktı araçları ile yapılır.

Modern bilgisayarlarda en sık kullanılan çıktı araçları monitör, yazıcı, hoparlör, kulaklık ve projektördür.

Yazılım

Bilgisayar sistemini oluşturan donanım bileşenlerini yönetmek ve kullanıcıların işlemlerini gerçekleştirmek için gerekli olan komutlar topluluğuna yazılım denir.

Bir bilgisayarın çalışabilmesi için yazılıma ihtiyacı vardır. Bilgisayarı açtığımız andan kapatana kadar geçen sürede yapılan her işlem, bir yazılım kontrolü altında gerçekleşir. Yazılım, sistem yazılımları ve uygulama yazılımları olmak üzere iki ana kategoride incelenir.

Sistem Yazılımları

Sistem yazılımları, bilgisayar kullanımı için gerekli ana fonksiyonları sağlayan, bilgisayar donanımına ve sistemin yürütülmesine yardımcı olan yazılımlardır. Sistem yazılımları üç gruba ayrılır:

- İşletim Sistemleri: Bilgisayarda işlem yapabilmek, bilgisayara bağlı birimleri kontrol edebilmek ve bilgisayar programlarını kullanabilmek için geliştirilmiş yazılımlardır. Windows Vista, Windows 7, Mac OS X, Fedora ve Ubuntu günümüzdeki popüler işletim sistemlerindendir.

- Hizmet Programları: Bilgisayar sistemini desteklemek amacıyla kullanılan yazılımlardır. Dosya sıkıştırma, virüs tarama, virüs temizleme, veri yedekleme gibi özel işlemleri yaparlar.
- Yazılım Geliştirme Araçları: Yazılım geliştirme, geliştirilen yazılımları test etme, var olan yazılımları güncelleme gibi amaçlar için, programcılar tarafından kullanılan yazılımlardır. Başlıca yazılım geliştirme araçları derleyiciler, çeviriciler ve yorumlayıcılardır.

Uygulama Yazılımları

Uygulama yazılımları, bilgisayarları günlük hayatımızın bir parçası yapan, bizlere birçok alanda çeşitli işlemler yapma olanağı veren yazılımlardır. Kullanıcılar, bilgisayar başında geçirdikleri zamanın büyük bir kısmında bu uygulamalardan yararlanırlar. Kelime işlemciler, resim düzenleme programları, web tarayıcıları, e-posta yöneticileri, video oyunları gibi birçok uygulama yazılımı bulunmaktadır.

SIRA SİZDE



Bir kullanıcının günlük yaşamda sıklıkla faydalandığı uygulama yazılımları neler olabilir?

PROGRAMLAMA DİLLERİ

Yazılımcıların belirli bir **algoritmayı** ifade etmek için kullandığı, yapılacak işlemi bilgisayara anlatmaya yarayan standart gösterimlere programlama dili denir. Bir programlama dilinin içerdiği kelimeler, kurallar ve talimatlar sayesinde bilgisayarlara özel işlemler yaptırılabilir.

Bilgisayar programlama tarihinde yüzlerce programlama dili geliştirilmiş ve kullanılmıştır. Teknolojinin sürekli değişmesi ve gelişmesi, farklı ihtiyaçların doğması, kullanım amaçlarının değişmesi gibi sebeplerden dolayı bir programlama dilinin sonsuza kadar kullanımda kalması mümkün değildir. Günümüzde popüler olarak kullanılan programlama dilleri C, C++, Python, Java, PHP, JavaScript ve Ruby olarak örneklendirilebilir.

Programlama Dillerinin Hedefleri

Programlama dilleri yazılım geliştirmek için kullanılan araçlardır. Yazılımın kalitesi ile programlama dili arasında doğrudan bir ilişki olduğu için, bir yazılım sisteminde olması gereken temel özellikler ile programlama dillerinin hedefleri paralellik gösterir. Bir programlama dilinin faydalı ve başarılı sayılabilmesi için yazılımda güvenilirlik, sürdürülebilirlik ve verimlilik sağlanabilmelidir.

Yazılımda Güvenilirlik

Bilgisayar programlamada, geliştirilen yazılımın güvenilir olması öncelikli hedeflerden biridir. Kullanıcılar yazılıma güven duymalı, çalışma esnasında programdan kaynaklanan hatalar ile karşılaşmamalıdır. Yazılım veya donanım kaynaklı bir problem yaşandığı takdirde, kullanıcıya destek verilebilmelidir.

Yazılımda güvenilirlik sağlayabilmek için, bir programlama dilinin aşağıdaki niteliklere sahip olması gerekmektedir:

- **Yazılabilirlik (Writability):** Programlama dilinin yazılabilirliği, programcının üretkenliğini etkileyen bir faktördür. Programlama dilinin insan dostu sembolleri ve komutları desteklemesi gerekir. Programcı problem çözmeye odaklanmalı, dilin kurallarında kaybolmamalıdır.
- **Okunabilirlik (Readability):** Programın kaynak kodu incelenirken, programın akışı ve mantığı takip edilebilmelidir. Programın okunabilirliği sayesinde, var olan hatalar tespit edilebilmelidir.

Algoritma: Bir problemin çözümünde uygulanan sistematik yöntemdir. Bir algoritmanın adımları açık, düzenli ve sıralı şekilde tanımlanmalıdır. Programlama dillerinin temeli algoritmaya dayanır.

- *Sadelik (Simplicity)*: Bir programlama diline hâkim olabilmek için, dilin sade ve basit olması gerekir. Dilin bu özelliği, dilin gücünü azaltabilir.
- *Güvenlik (Safety)*: Zararlı programlar oluşturabilecek komutlar, programlama dili tarafından kısıtlanabilmelidir. Dilin bu özelliği, dilin gücünü ve esnekliğini azaltabilir.
- *Sağlamlık (Robustness)*: Yazılım sistemlerinde hatalı veri girişi, geçersiz işlem, operatör dikkatsizliği gibi birçok durumla karşılaşılabilir. Bir programlama dili, istenmeyen durumlarda yapılabilecek işlemleri destekleyebilecek altyapıyı sunmalıdır.

Yazılımda Sürdürülebilirlik

Yazılımda sürdürülebilirlik, bir yazılımın değişen istek ve ihtiyaçlara cevap verebilme yeteneğidir. İş gücü, zaman ve maliyet gibi ekonomik kısıtlar, yazılımların sürdürülebilir olmasını gerekli kılar.

Bir yazılımın sürdürülebilir olması için, yazılımın kaynak kodunun kolay ve güvenli bir şekilde değiştirilebilmesi gerekir. Bu gereksinimi sağlamanın ilk şartı, programlama dilinin okunabilirlik ve sadelik özelliklerine sahip olmasıdır. Ayrıca kodun belirli bir bölümünde yapılan değişiklik, yalnız ilgili işlemleri etkilemeli, programın tüm yapısını bozmamalıdır.

Yazılımda Verimlilik

Verimlilik, tüm yazılım sistemleri için önemli bir gereksinimdir. Günümüz teknolojisinde donanım maliyetleri düşmüş olsa da, yazılım kapsamı oldukça genişlemiştir. Birçok sektörde iş yoğunluğunun büyük kısmı yazılım sistemlerine aktarılmıştır. Yazılımlarla yapılan işlemlerdeki artıştan dolayı, yüksek verimlilikte çalışan yazılımlara sahip olmak kaçınılmaz olmuştur.

Bir yazılımın verimliliğini belirleyen kriterlerden başlıcaları yazılımın çalışması için gereken bellek ve işlem hızıdır. Geliştirilen programların isteklere hızlı yanıt vermesi ve çalışma süresince düşük sistem kaynağı kullanması, tüm programlama dillerinde ortak amaçtır.

Yazılım geliştirme sürecinde ve sürdürülebilirlik aşamasında harcanan efor, verimliliği belirleyen bir diğer unsurdur. Programlama dilleri yeniden kullanılabilir ve portatif kodlar geliştirmeye imkan tanımalıdır. Bu olanak, programcılara zaman ve emek açısından büyük bir kolaylıktır.

Programlama dilleri hakkında detaylı inceleme D.A. Watt'ın "Programming Language Design Concepts" (New York: Wiley and Sons, 2004) kitabında mevcuttur.



K İ T A P

Programlama Dillerinde Seviyeler

Bilgisayar teknolojisinin gelişmesine paralel olarak, yazılım sistemlerinden ve otomasyonlardan beklenen hizmetler de artmıştır. Doğan ihtiyaçlara karşılık verebilmek ve sistemlerden en iyi verimi alabilmek için; programlama teknikleri geliştirilmiş ve yazılım geliştirme araçları iyileştirilmiştir. Programlama tarihinde, farklı amaçlara hizmet eden yüzlerce dil kullanılmıştır. Programlama dillerinin tarihçesini göz önüne alarak, programlama dillerini üç seviyede sınıflandırmak mümkündür:

- *Makine Seviyesi*: Programlama dillerinin birinci nesli olarak da adlandırılan bu seviyedeki diller, yalnızca 0 ve 1 değerlerinden oluşur. İnsanlar tarafından anlaşılması oldukça zor olan makine seviyesi programlama dilleri, bilgisayarların algılayabileceği şekilde tasarlanmıştır.

- **Çevirici Seviyesi:** Programlama dillerinin sınıflandırılmasında orta düzeyde bulunan dillerdir. Makine seviyesindeki diller ile aynı komutlara sahip olmalarına karşın, komutların ve değişkenlerin gösterimi için özel semboller bulunur. Çevirici dillerin bilgisayar tarafından anlaşılabilmesi için, o dile özel bir çevirici kullanılır. Çeviricinin görevi, programlama dilindeki komutları bilgisayarın algılayacağı makine kodlarına dönüştürmektir.
- **Yüksek Seviye:** İçerdikleri ifadeler, semboller ve kurallar sayesinde insanlar tarafından en rahat anlaşılan, en çok kullanılan programlama dilleridir. Bu seviyedeki diller ile geliştirilen programların bilgisayar tarafından anlaşılması için derleyiciler ve yorumlayıcılar kullanılır. Programların kaynak kodunun makine koduna çevrilmesi, derleyiciler ve yorumlayıcılar aracılığıyla yapılır.

Farklı düzeylerdeki programlama dilleri, birbirlerine karşı avantajlar veya dezavantajlar gösterebilir. Programlama dillerinin seviyelerine göre karşılaştırılması aşağıdaki tabloda gösterilmiştir:

Tablo 1.1
Farklı düzeydeki
programlama dillerinin
karşılaştırılması

Makine Seviyesi	Avantajlar Bilgisayar ile doğrudan iletişim yapılır. Derleyici veya yorumlayıcı yazılımlara ihtiyaç yoktur. Herhangi bir dönüşüm yapılmadığı için, programlar hızlı çalışır.
	Dezavantajlar Programlama dili ve makine mimarisi arasında bağımlılık vardır. Programların yazılması ve anlaşılması zordur. Geliştirilen programlar portatif değildir.
Çevirici Seviyesi	Avantajlar Makine seviyesindeki dillere göre daha anlaşılır bir yapı mevcuttur. Geliştirilen programlar portatiftir.
	Dezavantajlar Dildeki komutların kullanımını anlamak zahmetlidir. Programlama dili ve çevirici arasında bağımlılık vardır.
Yüksek Seviye	Avantajlar Bu seviyedeki diller, insanlar tarafından okunabilir ve anlaşılabilir. Yeni programlar geliştirme, hata ayıklama, güncelleme yapma gibi işlerin yapılması daha kolay ve güvenlidir. Geliştirilen programlar portatiftir.
	Dezavantajlar Diğer seviyedeki dillere göre daha fazla sistem kaynağı kullanır. Geliştirilen programlar daha yavaş çalışır.

Özet



Hayatımızın vazgeçilmez parçası olan bilgisayarların gelişimini ve tarihçesini tanımlamak.

Bilgisayar, aritmetiksel ve mantıksal işlemler yapabilen, programlanabilir elektronik bir cihazdır. Günümüzde hayatın vazgeçilmez bir parçası olan bilgisayarların tarihi beş ana nesil altında incelenir. Her bilgisayar nesli, o döneme damga vuran gelişme ile anılmaktadır. Bilgisayar nesillerini isimlendiren buluşlar sırasıyla, vakumlu tüpler, transistörler, tümleşik devreler, mikroişlemciler ve yapay zekâdır.



Bilgisayarların temel çalışma prensiplerini açıklamak.

Bilgisayarın temel işlevsel yapısına ve tasarımına bilgisayar mimarisi adı verilir. Modern bilgisayarlarda “Von Neumann Modeli” adı verilen mimari esas alınmıştır. Amerikalı bilim adamı John von Neumann tarafından ortaya konulan bu modelde, işlemci ve bellek birbirinden ayrılmıştır. Bilgisayarı oluşturan bileşenlerin merkezi işlem birimi, bellek ve girdi/çıkış birimleri olduğu kabul edilmiştir.



Bilgisayarları oluşturan bileşenleri tanımlamak.

Donanım, bilgisayarın fiziksel ve elektronik yapısını oluşturan bileşenlerdir. Modern bilgisayarları oluşturan başlıca donanım bileşenleri merkezi işlem birimi, ana bellek, ikincil depolama aygıtları, girdi aygıtları ve çıktı aygıtlarıdır.

Yazılım, bilgisayar sistemini oluşturan donanım bileşenlerini yönetmek ve kullanıcıların işlemlerini gerçekleştirmek için gerekli olan komutlar topluluğudur. Yazılım, sistem yazılımları (işletim sistemleri, hizmet programları ve yazılım geliştirme araçları) ve uygulama yazılımları (kelime işlemciler, web tarayıcıları, e-posta yöneticileri, video oyunları, vs.) olmak üzere iki ana gruba ayrılır.



Programlama dillerinin temel özelliklerini listelemek ve çeşitli programlama dillerini karşılaştırmak.

Yazılımcıların belirli bir algoritmayı ifade etmek için kullandığı, yapılacak işlemi bilgisayara anlatmaya yarayan standart gösterimlere programlama dili denir. Bir programlama dilinin öncelikli hedefi yazılımda güvenilirlik, sürdürülebilirlik ve verimlilik sağlamaktır. Günümüzde popüler olarak kullanılan programlama dilleri C, C++, Python, Java, PHP, JavaScript ve Ruby olarak örneklendirilebilir.

Programlama dillerinin tarihçesi göz önüne alınarak, programlama dillerini makine seviyesi, çevirici seviyesi ve yüksek seviye olmak üzere üç gruba ayırmak mümkündür. Makine seviyesindeki dillerin insanlar tarafından anlaşılması zordur. Bu seviyedeki diller, yalnızca 0 ve 1 değerlerinden oluşur. Çevirici seviyesindeki dillerde, komutların ve değişkenlerin gösterimi için özel semboller bulunur. Programlama dilindeki komutlar, makine kodlarına çevirici yazılımlar ile dönüştürülür. Yüksek seviyedeki diller, içerdikleri ifadeler, semboller ve kurallar sayesinde insanlar tarafından en rahat anlaşılabilir programlama dilleridir. Programların kaynak kodunun makine koduna çevrilmesi, derleyiciler ve yorumlayıcılar aracılığıyla yapılır.

Kendimizi Sınyalım

1. Aşağıdakilerden hangisi birinci nesil bilgisayarlar da yaşıyan problemlerden biri **değildir**?

- Makinelerin fiziksel boyutu
- İşlemler için gerekli elektrik miktarı
- İşlemler sırasında ortaya çıkan ısı miktarı
- Bellek olarak manyetik silindirlerin kullanılması
- İşlemleri sonuçlandırma süresi

2. Tümüleşik devrelerdeki transistör kapasitesinin artışı hakkında yaptığı öngörü ile bilgisayar tarihine geçmiş kişi aşağıdakilerden hangisidir?

- Alan Turing
- Bill Gates
- Gordon Moore
- Steve Jobs
- Nikola Tesla

3. Modern bilgisayar mimarisinin temelini oluşturan model, hangi bilim insanı tarafından ortaya konulmuştur?

- John von Neumann
- Albert Einstein
- Alan Turing
- Bill Gates
- Gordon Moore

4. Bilgisayarda verilerin işlenmesini sağlayan, bilgisayarın beyni olarak da tanımlanan donanım bileşeni aşağıdakilerden hangisidir?

- İşletim sistemi
- Ana bellek
- Merkezî işlem birimi
- Sabit disk
- Dokunmatik ekran

5. Aşağıdakilerden hangisi bilgisayarın girdi aygıtlarından biri **değildir**?

- Tarayıcı
- Yazıcı
- Barkod okuyucu
- Klavye
- Fare

6. Aşağıdaki yazılım türlerinden hangisi uygulama yazılımları grubuna ait **değildir**?

- Kelime işlemci
- Resim düzenleme programı
- Web tarayıcısı
- E-posta yöneticisi
- İşletim sistemi

7. Dosya sıkıştırma, veri yedekleme, virüs temizleme gibi özel işlemleri gerçekleştiren hizmet programlarının temel amacı aşağıdakilerden hangisidir?

- Yazılım geliştirmek
- Yazılım güncellemek
- Bilgisayar sistemini desteklemek
- Bilgisayar birimlerini kontrol etmek
- Kullanıcı işlemlerini kayıt altına almak

8. Bir yazılımın değışen istek ve ihtiyaçlara cevap verebilmesi, yazılımın hangi özelliğı ile ilgilidir?

- Sürdürülebilirlik
- Verimlilik
- Güvenilirlik
- Sağlamlık
- Sadelik

9. Aşağıdakilerden hangisi bir programlama dilinden beklenen niteliklerden biri **değildir**?

- Geliştirilen kodun rahatça okunabilir olması
- Geliştirilen kodun güvenli bir şekilde güncellenebilmesi
- Sistem kaynaklarının kullanımında verimli olması
- Programcının yaratıcılığını kısıtlaması
- Zararlı programlar oluşturabilecek işlemleri kısıtlaması

10. Bir programın kaynak kodu incelenirken, programın mantığını rahatça takip edebilmek için programlama dilinin aşağıdaki özelliklerden hangisine sahip olması gerekir?

- Yazılabilirlik
- Okunabilirlik
- Sadelik
- Güvenlik
- Sağlamlık

Yaşamın İçinden

“

Bilgisayarlar Daha Ne Kadar Küçülebilir?

Son 60 yıla baktığımızda, giderek küçülen transistörlerin veri işleme gücündeki artışta önemli rol oynadığını görüyoruz. Her biri çok küçük birer bilgisayar parçasına dönüştürülmüş moleküller, önümüzdeki 60 yıl içerisinde daha da büyük bir gelişmeyi tetikleyebilir mi?

Mikroelektronik endüstrisinin geleceği için büyük şeyler vaat eden atomik ölçülerde veri işlemede, atomik ölçekli devreler kullanılıyor ve bilgisayar süreçleri tek bir molekülün içinde yürütülüyor. Bu teknolojinin beraberinde getirdiği nano ve piko ölçekli bileşenlerdeki gelişmeler, bilgisayarların işlem gücünün daha da artmasına olanak sağlayacak. Günümüzde atomik ölçüde veri işleme çalışmalarını yürüten araştırmacıların durumu, transistörü icat edenlerin 1947'den önceki durumuna benzetilebilir.

Fransa Ulusal Bilim Araştırmaları Merkezi'ne bağlı Malzeme İşleme ve Yapısal Çalışmalar Merkezi'nde Nanobilim ve Piko-teknoloji Grubu (GNS) yöneticisi Cristian Joachim, “Hiç kimse bu işin sonunun nereye varacağını bilmiyor.” diyor. Joachim, Avrupa'da bu hedef doğrultusunda etkinlik gösteren 15 farklı akademik ve endüstriyel araştırma enstitüsünde çalışan araştırmacılardan oluşan bir ekibi koordine ediyor. Aslında 1990'lı yıllarda başlayan bir görevin devamı niteliğinde olan bu çalışmalar, günümüzde parasal desteğini Avrupa Birliği'nin Pico-Inside projesinden sağlıyor.

Modern bilgisayarların “motoru” olarak tanımlanabilecek geleneksel bir mikroişlemci'deki transistörler, doğru-yanlış sinyalleri üreten mantık kapılarının ve dijital devrelerin temel yapı taşlarıdır. Bir mantık kapısı yaratmak için birkaç transistör gerekir ve modern mikroişlemcilerde her biri 100 nanometre kadar olan transistörlerden milyarlarca bulunur. Transistörler küçülmeye devam ediyor ve bir işlemci'de bulunan transistör sayısı yaklaşık her iki yılda bir ikiye katlanıyor. Kuantum fiziği yasalarının klasik yöntemleri kullanarak daha çok küçülmeyi engellemeye başladığında soruna farklı bir bakış açısıyla yaklaşan atomik ölçüde veri işlemenin rolü daha belirginleşecek.

Joachim ve ekibi, atomların, moleküllerin ya da başka temel parçacıkların mantık kapısı, bellek ya da başka bir eleman olarak kullanılıp kullanılamayacağını araştırıyor. Bir molekülü ele alıp bilgisayar parçaları geliştirmeye yoğunlaşan bu ekibin asıl amacı tek bir molekülün içinde bir mantık kapısı oluşturabilmek. Joachim “Bir bilgisayar yapmak için kaç atom gerekir” sorusunu şu an yanıtlamayacaklarını fakat gün geçtikçe daha çok fikir sahibi olduklarını belirtiyor. Ekip, 14 transistörün işlevini görebilecek, 30 atomdan oluşan basit bir mantık kapısı tasarlamış durumda.

Farklı yaklaşımlarla atom ölçeğinde mantık kapıları yapma çalışmalarını sürdürüyorlar. Pico-Inside ekibinin çalışmalarının önemi bilim dünyasında yaygın olarak bilinmesine karşın Joachim hâlâ temel bir araştırma niteliğinde olduğuna dikkat çekerken mantık kapıları küçülmeye devam ettiği sürece mikroelektronik çalışmalarına gerek duyacağını ekliyor.

M. Ender Terzi

”

Kaynak: Bilim ve Teknik, Şubat 2009.

Kendimizi Sınavalım Yanıt Anahtarı

1. d	Yanıtınız yanlış ise “Bilgisayarın Gelişimi ve Tarihçesi” konusunu yeniden gözden geçiriniz.
2. c	Yanıtınız yanlış ise “Bilgisayarın Gelişimi ve Tarihçesi” konusunu yeniden gözden geçiriniz.
3. a	Yanıtınız yanlış ise “Bilgisayar Mimarisi” konusunu yeniden gözden geçiriniz.
4. c	Yanıtınız yanlış ise “Bilgisayarda Donanım ve Yazılım” konusunu yeniden gözden geçiriniz.
5. b	Yanıtınız yanlış ise “Bilgisayarda Donanım ve Yazılım” konusunu yeniden gözden geçiriniz.
6. e	Yanıtınız yanlış ise “Bilgisayarda Donanım ve Yazılım” konusunu yeniden gözden geçiriniz.
7. c	Yanıtınız yanlış ise “Bilgisayarda Donanım ve Yazılım” konusunu yeniden gözden geçiriniz.
8. a	Yanıtınız yanlış ise “Programlama Dilleri” konusunu yeniden gözden geçiriniz.
9. d	Yanıtınız yanlış ise “Programlama Dilleri” konusunu yeniden gözden geçiriniz.
10. b	Yanıtınız yanlış ise “Programlama Dilleri” konusunu yeniden gözden geçiriniz.

Sıra Sizde Yanıt Anahtarı

Sıra Sizde 1

ENIAC’ın ilk bilgisayar olarak kabul edilmesinin nedeni, sahip olduğu sayısal ve mantıksal becerilerdir. Bu bilgisayar, eşitlikleri karşılaştırabiliyor, dört işlemi çözebiliyor ve kare kök hesaplayabiliyordu.

Sıra Sizde 2

Günümüzde kullanılan işlemcileri üreten birçok firma bulunmaktadır. Bu firmaların başlıcaları Intel, AMD, Samsung, Qualcomm, NVIDIA ve Motorola’dır.

Sıra Sizde 3

Von Neumann modelini kullanan bilgisayarlar, aynı anda birden fazla program ve veriyi belleğe alabilirler. Bellekte depolanmış farklı programlar, işlemci tarafından geçiş yapılarak işlenebilmektedir.

Sıra Sizde 4

Bir kişinin sıklıkla kullandığı uygulamalar, kişinin ihtiyaçlarına, zevklerine, ilgi alanlarına göre değişiklik gösterebilir. Buna rağmen, hepimizin ortak kullandığı, günlük işlerimizde yardımcı uygulamalar vardır. Bu uygulamalar aşağıdaki listede örneklendirilmiştir:

- Web tarayıcıları: Google Chrome, Internet Explorer, Mozilla Firefox.
- Kelime işlemciler: Microsoft Office, OpenOffice, LibreOffice.
- Medya oynatıcılar: Winamp, VLC Player, Windows Media Player.
- Arama motorları: Google, Yandex, Yahoo.
- Sosyal medya uygulamaları: Facebook, Twitter, Instagram.

Sıra Sizde 5

Derleyici, yüksek seviyeli programlama dilinde yazılmış bir programın kaynak kodunu makine diline çeviren yazılımdır. Programcının geliştirdiği koddan, işlemcinin anladığı, 0 ve 1’lerden oluşan makine koduna dönüşüm yapılır.

Yorumlayıcı, programın kaynak kodundan makine koduna dönüşümünü anlık olarak yapan yazılımdır. Yorumlayıcılarda yazılım kısmı kısım ele alınır, kodun işlenmeyen satırları ile ilgilenilmez.

Yararlanılan ve Başvurulabilecek Kaynaklar

- Watt, D.A. (2004). **“Programming Language Design Concepts”**, New York: Wiley and Sons.
- Stallings, W. (2006). **“Computer Organization and Architecture”**, New Jersey: Prentice Hall.
- Campbell-Kelly, M., Aspray, W., Ensmenger, N., Yost, J.R.(2013). **“Computer: A History of the Information Machine”**, Colorado: Westview Press.
- Mitchell, J.C. (2002). **“Concepts in Programming Languages”**, Cambridge: Cambridge University Press.
- Bilim ve Teknik Dergisi. **“Bilgisayarlar Daha Ne Kadar Küçülebilir?”**, Şubat 2009.

2

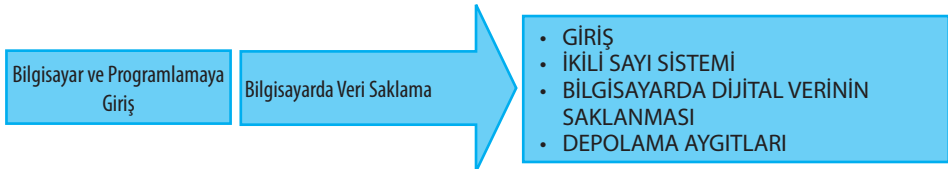
Amaçlarımız

- Bu üniteyi tamamladıktan sonra;
- İkili sayı sistemini açıklayabilecek,
 - İkili sayı sisteminde matematiksel işlemler yapabilecek,
 - Bit ve byte kavramlarını açıklayabilecek,
 - Sayı, karakter, görüntü ve ses verilerinin bilgisayarda nasıl saklandığını ifade edebilecek,
 - Bilgisayar sistemlerinde kullanılan depolama aygıtlarını tanımlayabilecek bilgi ve becerilere sahip olabileceksiniz.

Anahtar Kavramlar

- İkili Sayı Sisteminde Matematiksel İşlemler
- Bit ve Byte Kavramları
- ASCII
- Unicode
- Veri Depolama Yöntemleri
- Bilgisayar Belleği
- Sabit Disk
- Flash Bellek

İçindekiler



Bilgisayarda Veri Saklama

GİRİŞ

Bilgisayarda veri saklama ünitesinde işlenen konuların ana başlıkları ikili sayı sistemi, bilgisayarda dijital verilerin saklanması, ve depolama aygıtlarıdır.

Bilgisayar sistemlerinde verilerin işlenebilmesi ve devamlılığın sağlanabilmesi için verilerin saklanması ve depolanması gereklidir. Bilgisayarda veriler, ikili sayı sistemi ile gösterilir. Bu sayı sisteminin temel özellikleri ve bu sistemdeki matematiksel işlemler “İkili Sayı Sistemi” başlığında belirtilmiştir. Dijital veride temel kavramlar olan bit ve byte hakkında açıklamalar ve metin, ses, görüntü ve video formatlarının bilgisayarda saklanması ile ilgili genel bilgiler “Bilgisayarda Dijital Verilerin Saklanması” başlığı altında verilmiştir. Dijital verilerin geçici veya kalıcı olarak saklanmasına yarayan cihazlar ve bu cihazların temel özellikleri “Depolama Aygıtları” başlığında anlatılmıştır.

İKİLİ SAYI SİSTEMİ

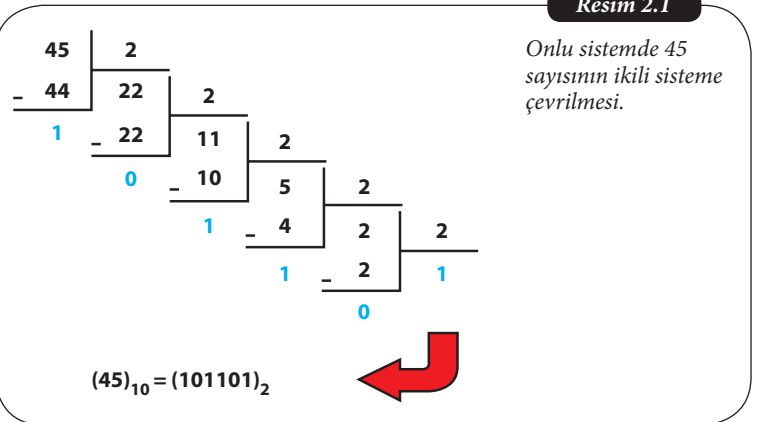
İkili Sayı Sistemi Nedir?

Günlük hayatta kullandığımız onlu sayı sistemindeki sayıların, 2 tabanında yazılmasıyla elde edilen sisteme *İkili Sayı Sistemi* adı verilir. Bu sistemdeki tüm sayılar, 0 ve 1 rakamlarının kullanımıyla ifade edilir. Örneğin onlu sistemdeki 45 sayısı, ikili sistemde 101101 sayısına karşılık gelir ve matematiksel olarak $(45)_{10} = (101101)_2$ şeklinde gösterilir.

Günümüzde kullanılan ikili sayı sistemi, 1679 yılında Alman filozof Gottfried Wilhelm von Leibniz tarafından ortaya konulmuştur. İkili sayı sistemine benzerlik gösteren sistemlerin varlığı, eski Mısır, Çin ve Hindistan uygarlıklarına kadar dayanmaktadır.

Onlu Sayı Sisteminden İkili Sayı Sistemine Dönüşüm

Onlu sayı sisteminde ifade edilmiş bir sayının ikili sayı sistemine çevrilmesi için 2'ye bölme işlemi uygulanır. Bölümün değeri 2'den az olan kadar, bölme işlemine devam edilir. Kalan değerlerinin tersten yazılmasıyla ikili sayı sistemindeki gösterim elde edilir.



İkili Sayı Sisteminden Onlu Sayı Sistemine Dönüşüm

İkili sayı sisteminde ifade edilmiş bir sayının onlu sayı sistemine çevrilmesi için her basamak 2'nin kuvveti ile çarpılır. Çarpma işlemi en sağ basamaktan ve 2^0 ile başlar. Her basamak geçişinde kuvvet 1 arttırılır. Çarpımlar sonucunda elde edilen değerler toplanarak onlu sayı sistemindeki sayı elde edilir.

Resim 2.2

İkili sistemde 101101 sayısının onlu sisteme çevrilmesi.

$$\begin{array}{r}
 101101 \\
 \begin{array}{l}
 \rightarrow 2^0 * 1 = 1 \\
 \rightarrow 2^1 * 0 = 0 \\
 \rightarrow 2^2 * 1 = 4 \\
 \rightarrow 2^3 * 1 = 8 \\
 \rightarrow 2^4 * 0 = 0 \\
 \rightarrow 2^5 * 1 = 32 \\
 + \\
 \hline
 45
 \end{array}
 \end{array}$$

$(101101)_2 = (45)_{10}$

İkili Sayı Sisteminde Dört İşlem

Bire tümleyen: İkili sistemdeki bir sayının her basamağının tersi alınarak elde edilen değerdir. Örneğin; 1011 sayısının bire tümleyeni 0100'dür.

Onlu sayı sistemine benzer bir şekilde, ikili sayı sisteminde de toplama, çıkarma, çarpma ve bölme işlemleri yapılabilmektedir. İkili sayı sisteminde dört işlemin yapılabilmesi için bazı kurallar ve teknikler geliştirilmiştir.

Toplama

İkili sayı sisteminde toplama işleminin kuralları aşağıda listelenmiştir:

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 10$ (İkinci basamağa geçen 1 sayısı "Elde" anlamına gelir.)

Çıkarma

İkili sayı sisteminde çıkarma işleminin kuralları aşağıda listelenmiştir:

- $0 - 0 = 0$
- $1 - 1 = 0$
- $1 - 0 = 1$
- $0 - 1 = 1 \rightarrow X$ (Üst basamaktan alınan "Borç" X ile ifade edilir.)

Resim 2.3

84 ve 37 sayılarının ikili sistemde toplanması ve çıkarılması.

$$(84)_{10} = (1010100)_2$$

$$(37)_{10} = (100101)_2$$

$$\begin{array}{r}
 1 \\
 1010100 \\
 + 100101 \\
 \hline
 1111001
 \end{array}$$

$$\begin{array}{r}
 x \quad xxxx \\
 1010100 \\
 - 100101 \\
 \hline
 101111
 \end{array}$$

$$(84+37)_{10} = (121)_{10}$$

$$(121)_{10} = (1111001)_2$$

$$(84-37)_{10} = (47)_{10}$$

$$(47)_{10} = (101111)_2$$

Çıkarma işleminde negatif değerler ile çalışabilmek için *bire tümleyen* (one's complement) ve *ikiye tümleyen* (two's complement) yöntemleri kullanılır.

Bire tümleyen yöntemiyle çıkarma işleminde, öncelikle çıkan sayının bire tümleyeni bulunur. Daha sonrasında ana sayı ile çıkan sayının bire tümleyeni toplanır. Elde edilen sonucun basamak sayısı, ana sayının veya çıkan sayının basamak sayısından büyükse sonucun en solundaki basamak silinir ve sonuca 1 eklenir. Sonuç 0 veya pozitif olacak şekilde bulunmuştur. Sonucun basamak sayısı, ana sayının veya çıkan sayının basamak sayısından büyük değilse sonucun bire tümleyeni alınır ve başına “-” işareti konulur. Sonuç negatif şekilde bulunmuştur.

İkiye tümleyen yöntemiyle çıkarma işleminde, öncelikle çıkan sayının ikiye tümleyeni bulunur. Daha sonrasında ana sayı ile çıkan sayının ikiye tümleyeni toplanır. Elde edilen sonucun basamak sayısı, ana sayının veya çıkan sayının basamak sayısından büyükse sonucun en solundaki basamak silinir. Sonuç 0 veya pozitif olacak şekilde bulunmuştur. Sonucun basamak sayısı, ana sayının veya çıkan sayının basamak sayısından büyük değilse sonucun ikiye tümleyeni alınır ve başına “-” işareti konulur. Sonuç negatif şekilde bulunmuştur.

Çarpma

İkili sayı sisteminde çarpma işleminin kuralları aşağıda listelenmiştir:

- $0 * 0 = 0$
- $0 * 1 = 0$
- $1 * 0 = 0$
- $1 * 1 = 1$

Bölme

İkili sayı sisteminde bölme işleminin kuralları aşağıda listelenmiştir:

- $0 / 0 = 0$
- $0 / 1 = 0$
- $1 / 0 = 0$
- $1 / 1 = 1$

Resim 2.4

35 ve 7 sayılarının ikili sistemde çarpılması ve bölünmesi.

$$(35)_{10} = (100011)_2$$

$$(7)_{10} = (111)_2$$

$$\begin{array}{r} 100011 \\ \times 111 \\ \hline 111 \\ 100011 \\ 100011 \\ + 100011 \\ \hline 11110101 \end{array}$$

$$\begin{array}{r} \text{xxx} \\ 100011 \overline{) 111} \\ - 111 \quad \overline{) 101} \\ \hline 00111 \\ - 111 \\ \hline 000 \end{array}$$

$$(35 * 7)_{10} = (245)_{10}$$

$$(245)_{10} = (11110101)_2$$

$$(35 / 7)_{10} = (5)_{10}$$

$$(5)_{10} = (101)_2$$

İkili sayı sisteminde yapılan dört işlemin kendine has özellikleri vardır. Bu işlemlere aşina olabilmek için konuyla ilgili alıştırma yapabilirsiniz.



DİKKAT

BİLGİSAYARDA DİJİTAL VERİNİN SAKLANMASI

Dijital elektronikte 1 değeri akımın varlığını, 0 değeri ise akımın yokluğunu temsil eder. Bir bilgisayarın farklı kısımları arasında iletişim kurulması da akım seviyesinin takibiyle sağlanır. Belirli zaman aralıklarında akım seviyesi dinlenerek, 1 ve 0 değerlerinden oluşan mesajlar ile haberleşilir.

Bilgisayar sistemlerinde tüm veriler 1 ve 0 kullanılarak gösterilir. Bir başka deyişle, dijital veriler, ikili sayı sistemine ait değerlerdir. Bu sistemdeki her basamak bit olarak adlandırılır. Verilerin bilgisayar kullanıcılarına metin, resim, ses, video gibi çeşitli formatlara dönüştürülmesi ise bilgisayar yazılımları ile yapılır.

Bit: Dijital verinin en küçük ve temel ünitesidir. Bir bit, yalnızca 0 veya 1 değerini alabilir.

Bilgisayar sistemlerinde hafıza, **byte** adı verilen küçük saklama birimlerine bölünmüştür. Bir byte 8 bitten oluşur ve bu alan yalnızca küçük bir sayıyı ya da bir harfi saklamak için yeterlidir. Modern bilgisayarda milyarlarca byte kapasiteli bellekler bulunmaktadır. Byte birimleri ile ilgili detaylı bilgi, Tablo 2.1'de verilmiştir.

Byte: 8 bitlik dizilim boyunca 0 ve 1 değerlerini kapsayan bellek ölçü birimidir. Bir byte ile 28 farklı dizilim oluşturulabilir.

Tablo 2.1
Byte birimleri.

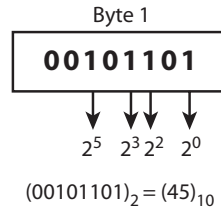
Ad	Sembol	Gerçek Byte Sayısı	Yaklaşık Byte Sayısı
Kilobyte	KB	2^{10}	10^3
Megabyte	MB	2^{20}	10^6
Gigabyte	GB	2^{30}	10^9
Terabyte	TB	2^{40}	10^{12}
Petabyte	PB	2^{50}	10^{15}
Exabyte	EB	2^{60}	10^{18}
Zettabyte	ZB	2^{70}	10^{21}
Yottabyte	YB	2^{80}	10^{24}

Sayıların Saklanması

Bilgisayar sistemlerinde sayıların saklanabilmesi için, saklanacak değerin ikili sayı sistemindeki karşılığının bulunması gerekir. İkili sayı sistemine dönüştürme sonucunda elde edilen bitler, bilgisayar hafızasındaki byte alanlarına kaydedilir.

Resim 2.5

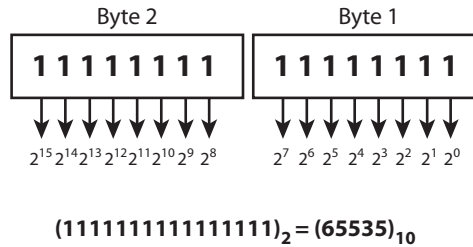
45 sayısının
bilgisayarda
saklanması.



Bilgisayarda saklanacak sayının değeri arttığında, hafıza alanından daha fazla yer kullanmak gerekir. Örneğin 2 byte ile 16 bitlik sayılar oluşturulabilir. 2 byte kullanarak saklanabilecek en büyük sayı 65535'tir. Bu sayıdan daha büyük sayıları saklamak için daha fazla byte kullanmak gereklidir.

Resim 2.6

2 byte kullanarak
saklanabilecek en
büyük sayı.



Bilgisayarda negatif tam sayıları ve reel sayıları saklamakta mümkündür. Negatif sayıların bilgisayar hafızasında saklanması için ikiye tükleyen yöntemi kullanılır. Reel sayıların bilgisayarında saklanmasında ise **kayan noktalı gösterim** (floating-point notation) kullanılır.

Karakterlerin Saklanması

Bilgisayar dünyasında harf, noktalama işareti, rakam ve bazı özel işaretler karakter olarak adlandırılır. Bilgisayardaki her karakter, belirli bir **karakter setine** aittir. Tüm veri tiplerinde olduğu gibi, karakterlerin bilgisayar hafızasında saklanabilmesi için ikili sayı sisteminde bir değere dönüştürülmesi gerekir.

Bir karakter bilgisayar hafızasında saklanırken, öncelikle **karakter kodlaması** (character encoding) yapılır. Bu işlem sonucunda elde edilen onlu düzendeki sayı, ikili sayı sistemine dönüştürülür.

Bir kullanıcı, klavyede “a” harfine bastığında, aşağıdaki işlemler sırasıyla gerçekleşir:

- Bilgisayar sistemi klavyeden yapılan karakter girişini algılar.
- Girilen harfin ASCII’ye göre karakter kodlaması yapılır.
- Kodlama sonucunda elde edilen “97” sayısı ikili sayı sistemine dönüştürülür.
- Dönüşüm sonucunda elde edilen “01100001” sayısı hafızada saklanır.

Kayan noktalı gösterim:

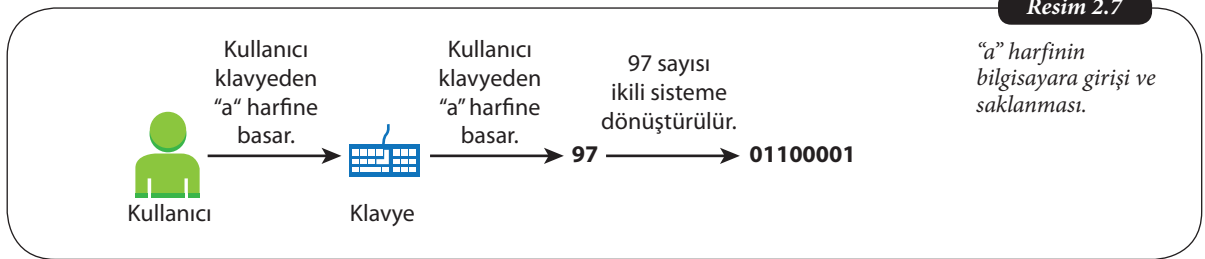
Reel sayıların bilgisayarda gösterilmesini sağlayan bir yöntemdir. Bu gösterimde, sayıların gerçek değerine en yakın şekilde bilgisayar ortamına taşınması amaçlanır.

Karakter seti:

Bilgisayar donanımı ve yazılımı tarafından algılanabilen, belirli karakterleri kapsayan kümedir. Kümedeki her karaktere karşılık gelen bir sayı vardır.

Karakter kodlama:

Bir karakterin, ait olduğu karakter setine göre şifrelenmesidir. Bilgisayar hesaplamalarında, veri saklamada, metinlerin iletiminde kullanılır.



ASCII (American Standard Code for Information Language)

Dilimize “Bilgi Değişimi İçin Amerikan Standart Kodlama Sistemi” olarak giren ASCII, Latin alfabesi üzerine kurulu, 7 bitlik bir karakter kodlama şemasıdır. İlk kez 1963 yılında ANSI (American National Standards Institute) tarafından standart olarak sunulmuştur. Bilgisayarlarda ve iletişim araçlarında kullanılan metinler, ASCII kodları ile temsil edilir.

ASCII, modern karakter kodlama şemalarının öncüsüdür. Günümüzde kullanılan şemalarda birçok farklı karakter de olmasına rağmen, genel olarak ASCII temel alınmıştır.

ASCII’de 33 adet kontrol karakteri ve 95 adet baskı karakteri olmak üzere toplam 128 karakter bulunur. 0-31 arasındaki sayılarla belirtilen kontrol karakterleri, metinlerin akışını kontrol eder. 32-126 arasındaki sayılarla belirtilen baskı karakterleri, ekranda görünen metinleri oluşturur. 127 sayısı ile belirtilen karakter ise silme işlemini yapan karakterdir.

Tablo 2.2
ASCII kodları, ikili
sistemdeki karşılıkları
ve ilgili karakterler.

ASCII KODU	BINARY KODU	SEMBOL	ASCII KODU	BINARY KODU	SEMBOL	ASCII KODU	BINARY KODU	SEMBOL
0	00000000	NUL	43	00101011	+	86	01010110	V
1	00000001	SOH	44	00101100	,	87	01010111	W
2	00000010	STX	45	00101101	-	88	01011000	X
3	00000011	ETX	46	00101110	.	89	01011001	Y
4	00000100	EOT	47	00101111	/	90	01011010	Z
5	00000101	ENQ	48	00110000	0	91	01011011	[
6	00000110	ACK	49	00110001	1	92	01011100	\
7	00000111	BEL	50	00110010	2	93	01011101	]
8	00001000	BS	51	00110011	3	94	01011110	^
9	00001001	HT	52	00110100	4	95	01011111	_
10	00001010	LF	53	00110101	5	96	01100000	`
11	00001011	VT	54	00110110	6	97	01100001	a
12	00001100	FF	55	00110111	7	98	01100010	b
13	00001101	CR	56	00111000	8	99	01100011	c
14	00001110	SO	57	00111001	9	100	01100100	d
15	00001111	SI	58	00111010	:	101	01100101	e
16	00010000	DLE	59	00111011	;	102	01100110	f
17	00010001	DC1	60	00111100	<	103	01100111	g
18	00010010	DC2	61	00111101	=	104	01101000	h
19	00010011	DC3	62	00111110	>	105	01101001	i
20	00010100	DC4	63	00111111	?	106	01101010	j
21	00010101	NAK	64	01000000	@	107	01101011	k
22	00010110	SYN	65	01000001	A	108	01101100	l
23	00010111	ETB	66	01000010	B	109	01101101	m
24	00011000	CAN	67	01000011	C	110	01101110	n
25	00011001	EM	68	01000100	D	111	01101111	o
26	00011010	SUB	69	01000101	E	112	01110000	p
27	00011011	ESC	70	01000110	F	113	01110001	q
28	00011100	FS	71	01000111	G	114	01110010	r
29	00011101	GS	72	01001000	H	115	01110011	s
30	00011110	RS	73	01001001	I	116	01110100	t
31	00011111	US	74	01001010	J	117	01110101	u
32	00100000		75	01001011	K	118	01110110	v
33	00100001	!	76	01001100	L	119	01110111	w
34	00100010	"	77	01001101	M	120	01111000	x
35	00100011	#	78	01001110	N	121	01111001	y
36	00100100	\$	79	01001111	O	122	01111010	z
37	00100101	%	80	01010000	P	123	01111011	{
38	00100110	&	81	01010001	Q	124	01111100	
39	00100111	'	82	01010010	R	125	01111101	}
40	00101000	(	83	01010011	S	126	01111110	~
41	00101001	)	84	01010100	T	127	01111111	
42	00101010	*	85	01010101	U			

Genişletilmiş ASCII

8 bitlik karakter kodlaması olan genişletilmiş ASCII şemalarında, standart ASCII karakterlerine ek olarak 128 karakter daha bulunur. Bu ek karakterler sayesinde özel semboller ve farklı dillere ait harfler standart bir şekilde kodlanabilmektedir.

Dünya üzerinde birçok farklı kültür ve dil bulunmaktadır. Her kültür ve dilin kendine has sembollere, işaretlere ve harflere sahip olabilmesinden dolayı, genişletilmiş ASCII sayesinde kullanılabilen ek karakterler de yetersiz kalabilmektedir. Bu problemi giderebilmek ve bölgesel karakterleri karşılayabilmek için ASCII çeşitlemeleri yapılmaktadır. Örneğin, Batı Avrupa bölgesinde bulunan İngilizce, Almanca, Fransızca, İtalyanca, İspanyolca gibi diller için ISO 8859-1 şeması kullanılırken, Türkçe için ISO 8859-9 şeması kullanılır.

Genişletilmiş ASCII kod şeması ile standart ASCII kod şemasının karakter sayılarındaki farklılığın sebebini açıklayınız.



SIRA SİZDE

Unicode

Türkçe’de “Evrensel Kod” olarak tanımlanan Unicode, her karakteri bir sayı ile ilişkilendiren bir bilişim endüstrisi standardıdır. Bu sistemde karakterler için altyapı, yazılım ve dil gibi etkenler gözetilmeksizin, her karaktere yalnız bir sayı ataması yapılır.

Unicode sisteminin temel amacı, bilgisayar metinlerini oluşturan sistemleri ortak bir çatı altında toplamak ve farklı karakter kodlama şemalarının birbirleriyle uyumlu çalışmasını sağlamaktır. Sistemin güncel versiyonu, 120.000’den fazla karakter ve sembolü bünyesinde barındırmaktadır.

Unicode sisteminde her bir karaktere atanan bir **kod noktası** vardır. Bu sistemde 016 ile 10FFFF₁₆ değerleri arasında tanımlanmış 1.114.112 adet kod noktası bulunmaktadır.

Evrensel kod sisteminde, kod noktaları ifade edilirken “U+” ve noktanın on altılı sayı sistemindeki karşılığı kullanılır. Mevcut sistemdeki karakterlerin on altılı sayı sistemindeki karşılıkları dört, beş veya altı haneli olabilir. Türkçe’deki özel karakterlerin Unicode sistemindeki gösterimleri Tablo 2.3’de verilmiştir.

Kod noktası: Karakter kodlamasında bir karakterin alabileceği sayısal değere verilen isimdir. On altılı sayı sisteminde ifade edilen kod noktaları, karakter kodlamasının kod uzayını oluşturur. ASCII için 016 ile 7F16 değerleri arasında tanımlanmış 128 adet kod noktası, genişletilmiş ASCII için 016 ile FF16 arasında tanımlanmış 256 kod noktası bulunur.

Karakter	Unicode Gösterim	Karakter	Unicode Gösterim
ğ	U+011F	Ğ	U+011E
ı	U+0131	İ	U+0130
ç	U+00E7	Ç	U+00C7
ö	U+00F6	Ö	U+00D6
ü	U+00FC	Ü	U+00DC
ş	U+015F	Ş	U+015E

Tablo 2.3
Türkçe karakterlerin
Unicode karşılıkları.

Unicode sisteminde karakterlere atanan değerler, ikili sayı sistemine doğrudan dönüşürülmez. Doğrudan dönüşüm için tüm karakterler eşit sayıda bit ile temsil edilmelidir. Bu gereklilik, Unicode sisteminde her karakterin 32 bit uzunluğunda olmasına yol açarak, bir karakterin 4 byte yer kaplamasına neden olur. Bu duruma engel olmak için UTF-8, UTF-16, UTF-32 gibi evrensel kod sistemine özel kodlama yöntemleri geliştirilmiştir. Bu yöntemler ile yapılan kodlamalarda, karakterlerin bit uzunlukları değişkenlik gösterir. Günümüzde en yaygın kullanılan karakter kodlama yöntemi UTF-8’dir.

ASCII ve Unicode Sistemlerinin Karşılaştırılması

ASCII karakter kodlaması ve Unicode sistemi, farklı karakterleri ikili sayı sistemine dönüştürmeye yarayan yöntemlerdir. Verilerin dijital ortamda okunması, yazılması, saklanması ve transfer edilmesi bu yöntemler aracılığıyla gerçekleşir.

ASCII ve Unicode arasındaki temel farklılıklar aşağıdaki listede verilmiştir:

- ASCII bir kodlama şeması iken, Unicode bir standarttır.
- ASCII'de kodlanan karakterlerin uzunluğu 8 bit olarak sabittir. Unicode'da ise bu uzunluk değişkendir.
- Unicode ile dünya üzerindeki dillerin büyük çoğunluğu temsil edilir. ASCII'de bu yeterlilik mevcut değildir.
- Unicode sistemi ASCII ile uyumludur. Unicode, kendi bünyesi altında ASCII kodlamasını barındırır.

SIRA SİZDE



ASCII'de kodlanan bir karakterin uzunluğu 8 bit olarak sabitken, Unicode'da bu uzunluk değişkendir. Bu temel farklılığın sebebini açıklayınız.

Diğer Veri Tiplerinin Saklanması

Bilgisayarda saklanan veri tipleri arasında, sayılar ve karakterler dışında birçok farklı dijital veri tipi bulunur. Dijital kamera, müzik çalar, DVD oynatıcı gibi dijital cihazların kullandığı veriler de bilgisayar tarafından işlenebilmeli ve saklanabilmelidir.

Görüntülerin Saklanması

Görüntü, bilgisayar hafızasında saklanan önemli veri tiplerinden birisidir. Bilgisayar sistemlerinde görüntüler, **piksel** (pixel) adı verilen noktacıkların birleşmesiyle meydana gelir.

Bir görüntünün bilgisayarda saklanabilmesi için, görüntüdeki her pikselin sayısal bir koda çevrilmesi gerekir. Pikselin rengini taşıyan bu sayısal kod, ikili sisteme dönüştürülerek bilgisayar hafızasında saklanır.

RGB renk grubundan bir piksel için kırmızı, yeşil ve mavi renk değerleri bulunur. Her değer 0-255 arası bir sayı olabildiği için, bir piksel 3 byte ile saklanabilir.

Piksel: Dijital görüntülerin en küçük parçası olan üçlü renk grubudur. Dijital görüntülerde RGB (Red Green Blue) renk grubundan bir piksel, kırmızı, yeşil ve mavi renklerine ait değerler ile gösterilir. Her renk için 0-255 arası bir değer tanımlanır.

Tablo 2.4
RGB renk grubundan piksellerin bilgisayar hafızasında saklanması.

	Byte 3	Byte 2	Byte 1
	KIRMIZI [7:0]	YEŞİL [7:0]	MAVİ [7:0]
Beyaz Piksel RGB=(255,255,255)	11111111	11111111	11111111
Siyah Piksel RGB=(0,0,0)	00000000	00000000	00000000
Kırmızı Piksel RGB=(255,0,0)	11111111	00000000	00000000
Yeşil Piksel RGB=(0,255,0)	00000000	11111111	00000000
Sarı Piksel RGB=(255,255,0)	11111111	11111111	00000000
Gri Piksel RGB=(128,128,128)	10000000	10000000	10000000

Seslerin Saklanması

Ses canlıların işitme organları tarafından algılanabilen, atmosferdeki periyodik basınç değişimleridir. Ses, bir ortamda ses dalgaları adı verilen mekanik dalgalar halinde yayılır. Ses oluşturan her madde veya cisim bir ses kaynağı olarak adlandırılır.

Resim 2.8

Ses dalgalarının görünümü.



Birçok farklı dijital cihaz tarafından girdi ve çıktı için kullanılan ses, bilgisayar ortamına aktarılırken, bilgisayarın anlayacağı ikili sayı sistemine dönüştürülmelidir. Bu dönüşümün yapılabilmesi için, ses dalgaları yatay olarak kesitlenir ve bu kesitlerden örnekler alınır. Bir başka deyişle ses, örnek adı verilen küçük parçalara ayrılır. Oluşturulan her bir örnek, ikili sayı sistemine çevrilir ve bilgisayar hafızasında saklanır.

Bir sesin dijital ortama alınırken parçalandığı örnek sayısı ile sesin kalitesi arasında doğrudan bir ilişki vardır. **Örnekleme oranı** (sample rate) olarak da adlandırılan bu değer arttıkça sesin kalitesi de artar. Sesin kalitesinin artmasıyla birlikte, bilgisayar hafızasında kapladığı yer de artmaktadır.

Modern cihazlarımızda dinlediğimiz kaliteli bir müzik şarkısı, saniyede 44100 örnek parçasına ayrılmıştır. Bu sayı, örnekleme oranı olarak 44.1kHz (kilohertz) olarak ifade edilir. Günümüzde yaygın olarak kullanılan diğer oranlar 88.2 kHz, 96 kHz ve 192 kHz'dir.

Örnekleme oranı: Analog ses dalgasının bir kesitte veya saniyede kaç kere örneklendiğini gösteren değerdir. Örnekleme oranı hertz cinsinden ifade edilir.

Bir ses dijital ortama aktarılırken, sesin orijinal kalitesine nasıl ulaşılır?



SIRA SİZDE

DEPOLAMA AYGITLARI

Dijital verilerin depolanması, bilgisayar sistemleri için temel bir ihtiyaçtır. Dijital verilerin geçici veya kalıcı olarak saklanmasını sağlayan cihazlara depolama aygıtları denir.

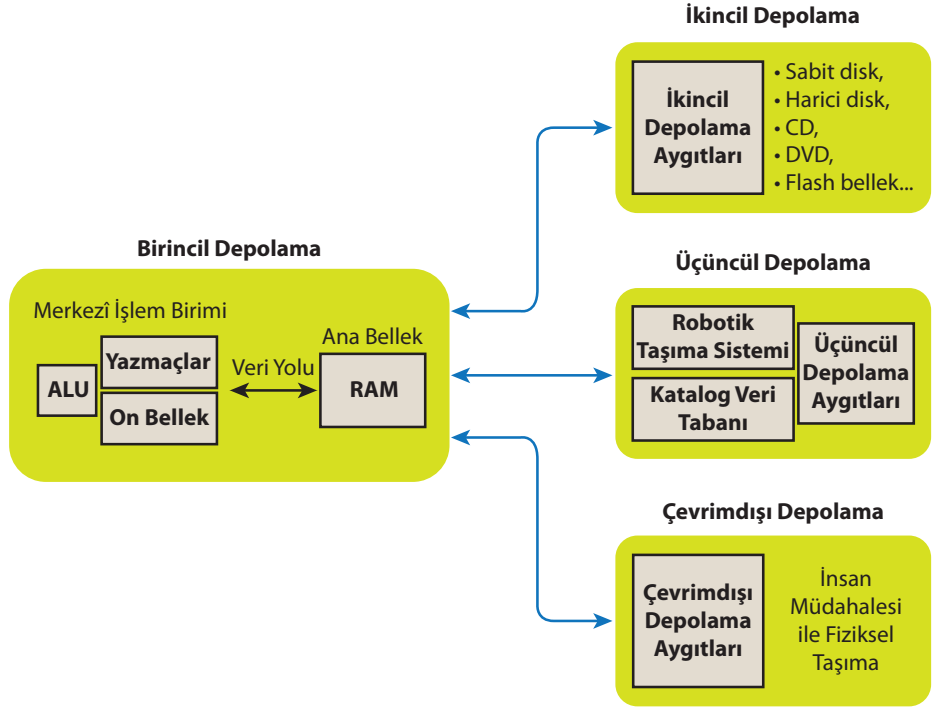
Bilgisayarda verilerin depolanmasında birincil depolama, ikincil depolama, üçüncül depolama ve çevrimdışı depolama olmak üzere dört ana grup vardır. Belirtilen her depolama grubunun özelliklerine ve amaçlarına göre farklılaştırılmış depolama aygıtları bulunur.

- **Birincil Depolama (Primary Storage):** Merkezî işlem birimi tarafından doğrudan erişilen ve veri işleme esnasında kullanılan bellek grubudur. Bilgisayarın ön belleği, ana belleği ve yazmaçlar birincil depolama aygıtlarıdır. Bu yöntemde veriler geçici olarak hafızaya alınır.
- **İkincil Depolama (Secondary Storage):** Merkezî işlem birimi tarafından doğrudan erişilemeyen, bilgileri kalıcı olarak saklayan depolama yöntemidir. Verilere bilgisayarın girdi/çıkı kanalları üzerinden erişilir.

- Üçüncül Depolama (Tertiary Storage): Çok büyük ölçekli verileri saklamaya ve arşivlemeye yarayan, insan müdahalesi gerektirmeyen depolama yöntemidir.
- Çevrimdışı Depolama (Offline Storage): Bilgisayar sistemi ile ilişkisi kesilmiş, verilerin kullanımı için insan müdahalesi gereken depolama yöntemidir.

Şekil 2.9

Bilgisayar sistemlerinde depolama yöntemleri.



Birincil Depolama Aygıtları

Bir bilgisayarda, merkezî işlem birimi tarafından doğrudan erişilebilen bellek grubuna birincil depolama aygıtları adı verilir. Bilgisayarın belleği olarak da tanımlanan birincil depolama aygıtlarındaki komutlar, merkezî işlem birimi tarafından sürekli dinlenir ve gerekli durumlarda çalıştırılır.

Bilgisayar sistemlerinde birincil depolama alanı olarak geniş kapasiteye sahip ana bellek, işlemcide yer alan ön bellek, ve yine işlemcide bulunan yazmaçlar (registers) kullanılır. Birincil depolama aygıtlarında saklanan veriler kalıcı değildir; bilgisayar sistemi kapatıldığında veya elektrik bağlantısı kesildiğinde, birincil depolamadaki bilgiler de kaybolur.

Birincil depolama aygıtları, bilgisayar performansını doğrudan etkileyen bileşenlerdir. Fiyat açısından pahalı kabul edilen birincil depolama aygıtlarının verimli ve düzgün kullanımı, bir bilgisayar sisteminden maksimum fayda sağlamak için önemli bir gereksinimdir.

DİKKAT



Birincil depolama aygıtları tarafından tutulan veriler, kalıcı olarak saklanamaz. Bilgisayar sisteminin güç bağlantısı kesildiğinde, hafızadaki verileri kaybolur. Verilerin kalıcı olarak saklanabilmesi için, diğer depolama türlerine ait bir aygıt (sabit disk, manyetik teyp, optik disk, vb.) kaydedilmesi gerekir.

Ana Bellek

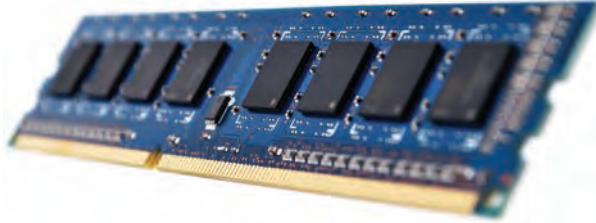
İngilizce'den dilimize **rastgele erişimli bellek** (random access memory) olarak da geçen, bilgisayarda bir programla ilgili komut ve verilerin depolandığı bellek türüdür. Ana bellek kısaca RAM olarak da belirtilir.

Rastgele erişimli bellek:

Hafızadaki herhangi bir bölüme rastgele ve hızlı bir şekilde erişilen bellek türüdür. Rastgele erişimli belleklerde, farklı iki veriye ulaşmak için harcanan süre, genellikle aynıdır.

Şekil 2.10

RAM belleğin yandan görünümü..



Bilgisayarın ana belleği, merkezî işlem birimine veri yolları üzerinden doğrudan bağlıdır. İşlemci, ana belleğin herhangi bir adresine ulaşabilir, bu adresteki bilgiyi okuyabilir ve değiştirebilir.

Bilgisayar sistemleri için ana belleğin temel fonksiyonları aşağıdaki listede özetlenmiştir:

- İşletim sisteminin çalışması için gerekli işletim sistemi bileşenleri ana bellekte korunur.
- Sistemde çalışmakta olan programlara ait komutlar ve veriler ana bellekte saklanır.
- Ana bellek, programların oluşturduğu sonuçları ve dijital çıktıları bünyesinde barındırır.
- İkincil depolama aygıtlarına ve çıktı birimlerine gönderilecek bilgiler ana bellekte saklanır.

Bilişim dünyasında gerçekleşen teknolojik gelişmelere paralel olarak, yazılım sektöründe de yenilikler ve değişimler yaşanmaktadır. Son yıllarda geliştirilen otomasyonlar ve uygulamalar, çok daha fazla sistem kaynağı kullanır hale gelmiştir.

Bilgisayarlardan yüksek performans almak ve yazılımlara ayak uydurabilmek için, büyük kapasiteli ana belleklerin geliştirilmesi zorunludur. Modern bilgisayar sistemlerinde RAM bellek kapasiteleri oldukça artmıştır. Günümüzde, ortalama bir bilgisayar kullanıcının ihtiyaçlarını karşılayabilecek bilgisayarlar için gerekli bellek miktarı 1 GB'tan başlamaktadır. Bir bilgisayar sistemi için gerekli ana bellek kapasitesi, bilgisayarda yapılan işlemler doğrultusunda görecelik göstermesine rağmen, son dönemlerde üretilen bilgisayarlardaki bellek miktarı 16 GB'a kadar yükselmiştir.

İşlemci Ön Belleği

Gordon Moore'un 1965 yılında işlemci kapasitesi hakkında öne sürdüğü teori, günümüze dek geçerliliğini korumuş ve bilişim dünyasında "Moore Yasası" adını almıştır. Bu yasada belirtildiği üzere, mikroişlemcilerdeki transistör sayısı her sene artacak, bu artış da işlemci performansını olumlu yönde etkileyecektir.

Moore Yasası'na paralel olarak, modern bilgisayarlardaki merkezî işlem biriminin veri işleme hızı oldukça artmıştır. Ana bellekten gelen veri hızının, merkezî işlem biriminin kapasitesinin altında kalmasından dolayı bir verimlilik kısıtlaması oluşur.

Merkezi işlem birimi ve ana bellek arasındaki hız farkından kaynaklanan problemin çözülebilmesi için işlemci içine yüksek hızlı hafızalar eklenmiştir. Çalışmakta olan programa ait komutların ve verilerin geçici olarak saklandığı yüksek hızlı hafızalara ön bellek adı verilir.

Bir bilgisayarın hızlı çalışması için ön bellekten kullandığı veri miktarının fazla olması gerekir. Bilgisayarda en sık kullanılan ve en son erişilen veriler, ana bellekten ön belleğe kopyalanır. Bu kopyalama işlemiyle ön bellek kullanımından en yüksek verimi almak hedeflenir.

Modern merkezi işlem birimlerinde farklı türde ve bağımsız ön bellekler bulunur. Ön bellekler, kullanım amacına göre komut ön belleği ve veri ön belleği olarak sınıflandırılır. Veri ön bellekleri de kendi aralarında L1, L2, L3 şeklinde farklı seviyelere sahip olabilir.

DİKKAT



Ön bellek, hız açısından ana belleğe göre oldukça üstündür. Fakat ön bellekte saklanabilecek veri miktarı kısıtlıdır. Bu özelliklerinden dolayı, ön belleğin fiyatı ana belleğin fiyatın göre daha pahalıdır.

İşlemci Yazmaçları

İşlemci içerisinde bulunan yazmaçlar, en hızlı fakat en küçük depolama üniteleridir. Genellikle 32 veya 64 bit uzunluğunda veri saklayabilen yazmaçlar, işlemci tarafından geçici saklama işlerinde kullanılır.

Birincil depolama aygıtları hiyerarşisinde en üst noktada bulunan işlemci yazmaçları, aritmetik işlemlerin yapılmasında önemli rol oynar. Yazmaçlar, ana belleğe göre hızlı oldukları için karmaşık hesaplamaların daha hızlı yapılabilmesini sağlar.

Bir işlemci bünyesinde farklı türlerde yazmaçlar bulunur. Yazmaçların en genel sınıflandırılması, kullanıcı erişimine açık yazmaçlar ve dâhili yazmaçlar olarak iki grupta yapılır.

İşlemci yazmaçları isimlendirilirken, bir yazmaçın hafızaya alabildiği bit uzunluğu kullanılır. Örneğin 16 bit kaydedebilen bir yazmaç “16-bitlik yazmaç” olarak adlandırılır.

SIRA SİZDE



4

Birincil depolamada kullanılan ana bellek, işlemci ön belleği ve işlemci yazmaçlarını işlem hızı, hafıza alanı ve fiyat açısından karşılaştırınız.

İkincil Depolama Aygıtları

İkincil depolama, dijital verilerin kalıcı olarak saklanmasını sağlayan depolama yöntemidir. Bilgisayar sistemi kapatıldığında veya bir elektrik kesintisi yaşandığında, ikincil depolama aygıtlarındaki bilgiler kaybolmaz. Bu özellik, ikincil depolamayı verilerin yedeklenmesi ve saklanması için ideal bir yöntem kılar.

Merkezi işlem birimi, ikincil depolama aygıtlarına doğrudan erişemez. Bilgisayar sistemlerinde ikincil depolama aygıtlarına erişim, girdi/çıkış kanalları üzerinden yapılır. İkincil depolama alanında bulunan veri, bu kanallar aracılığıyla birincil depolama alanına aktarılır ve işlemci tarafından kullanılır.

İkincil depolama aygıtlarında veri transfer hızı, birincil depolama aygıtlarına göre düşüktür. Fakat ikincil depolama aygıtları fiyat olarak daha ucuzdur ve hafıza alanı açısından daha yüksek kapasiteye sahiptir.

Sabit Disk

Üst üste konmuş metal plakalardan oluşan sabit diskler, günümüz bilgisayarlarının başlıca depolama üniteleridir. Sabit disklerin yapısında manyetik, elektronik ve mekanik bileşenler yer alır.

Şekil 2.11

Bir sabit diskin yakından görünümü.



Sabit disklerde verilerin saklanması, bir eksen etrafında dönebilen manyetik disk sayesinde gerçekleşir. Üst üste dizilen disk plakalarının her iki yüzü de bilgi depolamak için kullanılabilir. Her bir disk plakasına, plaka üzerinde gezinme kabiliyetine sabit manyetik kafalar yerleştirilmiştir. Bu kafalar sayesinde istenen adresten veriler okunur ve istenen adrese veriler yazılır.

Bilgisayar sistemlerinde bir yazılımın yürütülebilmesi için, yazılımla ilgili komut ve verilerin sabit diskten ana belleğe alınması gerekir. Yazılım için ihtiyaç duyulan alan ana bellek tarafından karşılanamadığında, bilgisayar sabit diskin bir bölümünü ana bellek gibi kullanabilir.

Sabit diskler son derece hassas cihazlardır. Çalışmakta olan bir sabit disk; fiziksel darbe, sarsıntı, güç kaybı, sıvı ile temas gibi dış etkenlerden dolayı arızalanma tehlikesiyle karşı karşıyadır. Bu gibi arızalarda veri bütünlüğünde bozulmalar veya kalıcı veri kayıpları yaşanabilir.

Günümüzde üretilen sabit diskler, fiziksel boyut olarak küçülmüş, hafıza kapasitesi olarak genişlemiştir. Kişisel bilgisayarlar için tercih edilen sabit disklerin kapasiteleri 400 GB ile 1 TB arasında değişmektedir.

Hâricî Sabit Disk

Hâricî sabit disk, bilgisayara bir kablo yardımıyla bağlanabilen ve bir yerden bir yere kolaylıkla taşınabilen sabit disklerdir. Rahat taşınabilme özelliğinden dolayı, hâricî sabit diskler “taşınabilir disk” olarak da adlandırılır.

Hâricî sabit diskler, bir bilgisayar sistemine genellikle **USB** (Universal Serial Bus) portları üzerinden bağlanır. Dâhilî sabit disklerle göre veri transferindeki yavaşlık, bu bağlantı yönteminden kaynaklanır.

Elektronik bileşenlerin küçülmesi sayesinde, hâricî sabit disklerin fiziksel olarak kapladığı alan oldukça azalmıştır. Modern ürünler rahatlıkla cepte taşınabilmektedir. Küçülen fiziksel boyutun yanı sıra, hâricî sabit disklerin hafıza kapasitelerinde de büyük bir artış vardır. Günümüzdeki popüler cihazların kapasiteleri 500 GB ile 4 TB arasında değişmektedir.

Flash Bellek

Flash bellek, yapı itibarıyla ana belleğe, kullanım itibarıyla sabit diske benzeyen bir bellek türüdür. Yapısal olarak tüm bileşenleri elektrondur, içerisinde hareket eden hiçbir parça bulunmaz. Bu özellik sayesinde, sabit disklerdeki fiziksel hassasiyet, flash belleklerde ortadan kalkar ve flash belleklerin kolaylıkla taşınmasına imkan verir.

USB: Dış donanımların bilgisayara bağlanmasını sağlayan, seri yapıda bir bağlantı biçimidir. Modern bilgisayarlarda en az 4 tane USB portu yer alır.

Taşıma kolaylığı ve hızlı veri aktarımı özellikleri sayesinde, flash bellekler sıklıkla tercih edilir. İlerleyen teknoloji ve yaygınlaşan kullanım alanı, flash belleklerin fiyatının düşmesini de sağlamıştır. Günümüzde flash belleklerin kapasiteleri 4 GB ile 128 GB arasında değişmektedir.

CD

CD (Compact Disc), optik teknoloji ile okuma ve yazma işlemleri gerçekleştirilen depolama aygıtıdır. CD teknolojisi, bilgisayar dünyasında yoğunlukla kullanılmıştır.

CD üzerindeki bilgilerin okunması, okuyucu üzerindeki lazer kafa tarafından gerçekleştirilir. Hareket kabiliyeti olan lazer kafa, tüm CD yüzeyinde hareket ederek, yarı geçirgen bir aynaya lazeri yansıtır.

CD teknolojisinde yalnız okunabilir, yazılabilir ve tekrar yazılabilir CD çeşitleri bulunur. Standart bir CD'nin depolama kapasitesi 650 MB ile 700 MB olarak değişebilir.

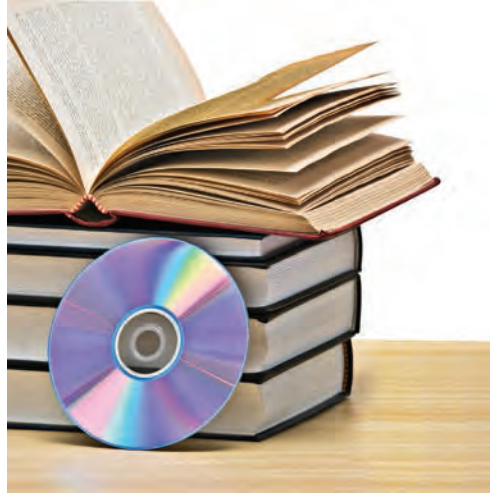
DVD

CD teknolojisinin gördüğü yoğun ilgiden sonra, daha geniş kapasiteli optik disklerin geliştirilmesi için çalışmalara başlanmıştır ve CD'lerin üst versiyonu olarak kabul edilen, daha fazla veri depolama kapasiteli DVD (Digital Versatile Disc)'ler ortaya çıkmıştır.

DVD'lerin depolama kapasitesi 4.7 GB ile 17.1 GB arasında değişmektedir. CD'lerden farklı olarak, bazı DVD'lerin her iki yüzeyine de kayıt yapılabilir ve bu olanak sayesinde depolama kapasitesi artar.

Şekil 2.12

Bir DVD'ye sığabilecek veri miktarı, onlarca kitapta bulunan bilgiden çok daha fazladır.



Üçüncül Depolama Aygıtları

Üçüncül depolama, genellikle çok büyük ölçekli verileri arşivlemek amacıyla başvuru bir veri saklama yöntemidir. Üçüncül depolama sistemlerinde verileri saklamak için disk veya teyp, verilerin fiziksel organizasyonunu içeren katalog veri tabanları, sistem sürücülerine disk veya teyp taşımakla görevli robot kollar bulunur.

Üçüncül depolamada verilerin kullanılması için insan müdahalesine ihtiyaç duyulmaz. Gerekli işlemler bilgisayar ve robot kollar aracılığıyla yapılır. Buna rağmen veriye erişim ve işleme hızı ikincil depolama aygıtlarına göre oldukça düşüktür. Bu nedenlerden dolayı, üçüncül depolama aygıtları çok büyük ve sıklıkla kullanılmayan verilerin arşivlenmesinde tercih edilir. Üçüncül depolama aygıtlarının başlıca örnekleri teyp kütüphaneleri ve optik disk kütüphaneleridir.

Resim 2.13



6000 teyp kartuş
içeren bir teyp
kütüphanesi ve taşıma
yapan robotik kol.

Kaynak:
Computational &
Information Systems
Lab.

Üçüncül depolama aygıtlarında bilgiye erişilmek istenildiği zaman, öncelikle verinin hangi disk veya teypte olduğunu tespit etmek gerekir. Bu gereksinimin yerine getirilebilmesi için katalog veri tabanı üzerinde arama yapılır. Yapılan arama sonucunda elde edilen yer bilgisi, bilgisayar tarafından robot kollara iletilir. Daha sonrasında robot kollar, gereken disk veya teybi sistem sürücüsüne taşır ve verinin işlenmesi için gerekli ortam hazırlanmış olur. Bilgisayarın veri üzerindeki işlemleri tamamlandığında, robot kollar tekrar devreye girer ve ilgili hafıza alanını ait olduğu bölüme geri götürür.

Çevrim Dışı Depolama Aygıtları

Çevrim dışı depolama, bir merkezî işlem biriminin kontrolü altında olmayan veri depolama türüdür. Çevrim dışı depolamada, verilerin bilgisayar sistemleri ile ilişkisi kesilmiştir. Bu yöntemde kullanılan depolama aygıtları, fiziksel olarak esas çalışma ortamından uzak mekanlara yerleştirilmiştir ve yalnız gerekli durumlarda kullanılır. Saklanan verilerin tekrar kullanılabilir hale gelmesi için insan müdahalesi gerekmektedir.

Çevrim dışı depolama aygıtları, verileri bir ortamdan başka bir ortama transfer etmek için kullanılır. Depolama ortamı bir bilgisayar sisteminden bağımsız olduğundan, fiziksel olarak nakil işlemi kolaylıkla yapılabilir. Çevrim dışı depolama aygıtlarının bu özelliği, onları veri yedekleme ve **felaket kurtarma** (disaster recovery) için önemli bir alternatif yapar. Çevrim dışı depolama ile yedeklenen veriler, farklı ve uzak bir ortamda saklandığında, verinin orijinal halinin bulunduğu yerdeki doğal afet, yangın, hırsızlık gibi risklerden kaçınılmış olunur.

Çevrim dışı depolamanın bilgi güvenliğine önemli bir faydası vardır. Bu depolama yöntemiyle saklanan veriler herhangi bir bilgisayara bağlı değildir; dolayısıyla verilere doğrudan erişmek de mümkün değildir. Bu kısıtlama, veri gizliliğini ve bütünlüğünü tehdit edebilecek yazılımsal saldırıları engellemiş olur.

Felaket kurtarma: Olası bir doğal afet, yangın, arıza, hırsızlık ve teknik problem gibi durumlarda bilişim sisteminin çalışabilir hale getirilmesi, kaybolan veya bozulan verilerin geri getirilmesi için izlenen süreçtir.

Günümüzde standart bilgisayar kullanıcıları, farkında olmasalar dahi çevrim dışı depolama yöntemine başvururlar. Birçok insan film, müzik, kitap, fotoğraf gibi kişisel verilerini hâricî disklerde, optik disklerde, flash belleklerde saklar veya yedekler.



DİKKAT

Özet



İkili sayı sistemini açıklamak.

İkili sayı sistemi, günlük hayatta kullanılan onlu sayı sistemindeki sayıların 2 tabanında yazılmasıyla elde edilir. Bu sistem, 1679 yılında Alman filozof Gottfried Wilhelm von Leibniz tarafından ortaya konulmuştur.



İkili sayı sisteminde matematiksel işlemler yapmak.

Onlu sayı sistemi ve ikili sayı sistemi arasında dönüşümler yapılabilir. Matematiksel dört işlemin, ikili sayı sisteminde yapılması da mümkündür. Negatif sayılarla işlem yapabilmek için bire tümleyen ve ikiye tümleyen yöntemleri kullanılır.

Bilgisayar sistemlerinde verilerin gösterimi, ikili sayı sistemi kullanılarak yapılır. Dijital veriler, bit olarak adlandırılan 1 ve 0 sayılarıyla gösterilir. Verilerin saklanması ise bilgisayar hafızasının byte adı verilen küçük bölümleri kullanılır. Bir byte, 8 bit bilgi depolar. Verilerin bilgisayar kullanıcılarına metin, resim, ses, video gibi çeşitli formatlara dönüştürülmesi ise programlar aracılığıyla gerçekleşir.



Bit ve byte kavramlarını açıklamak.

Bilgisayar sistemlerinde saklanması gereken dijital veriler, farklı formatlara sahiptir. Bilgisayar hafızasında en yaygın olarak saklanan formatlar sayı, karakter, görüntü ve sestir.

- Bir sayının bilgisayarda saklanabilmesi için ikili sayı sistemine dönüştürülmesi gereklidir. Sayının değeri arttıkça, hafıza alanında sayı için ayrılan byte sayısı da artar.
- Karakterler bilgisayar hafızasında saklanırken, öncelikle karakter kodlaması yapılır. Bu işlem sonucunda elde edilen onlu düzendeki sayı, ikili sayı sistemine dönüştürülür. Karakter kodlamasının yapılması için ASCII kodlama şeması, genişletilmiş ASCII kodlama şeması ve Unicode sistemi kullanılır.
- Görüntüler bilgisayarda saklanırken, görüntüyü oluşturan piksellerin sayısal değerlere dönüşümü yapılır. Piksel rengini ifade eden sayısal kodlar, ikili sayı sisteme dönüştürülerek bilgisayar hafızasında saklanır.
- Sesler dijital ortama alınırken, ses dalgaları yatay olarak kesitlenir ve bu kesitlerden örnekler alınır. Oluşturulan her bir örnek, ikili sayı sistemine çevrilir ve bilgisayar hafızasında saklanır. Seslerin aktarılması esnasında belirlenen örnekleme oranı, sesin kalitesini belirler. yer de artmaktadır.



Sayı, karakter, görüntü ve ses verilerinin bilgisayarda nasıl saklandığını ifade etmek.

Bilgisayar sistemlerinin organize olabilmesi için dijital verilerin geçici veya kalıcı olarak saklanması gereklidir. Bu gereksinimin karşılanmasını sağlayan birincil, ikincil, üçüncül ve çevrimdışı depolama yöntemleri vardır.

Merkezî işlem birimi tarafından doğrudan erişilebilen depolama yöntemi birincil depolamadır. Bu yöntemde kullanılan depolama aygıtları ana bellek, işlemci ön belleği ve işlemci yazmaçlarıdır. Bilgisayarın güç ünitesi devre dışı kaldığında, bu aygıtlardaki veriler kaybolur. Birincil depolama aygıtlarında veri transferi çok hızlı gerçekleşir. Fakat bu aygıtların depolama kapasitesi küçüktür ve fiyatları yüksektir.



Bilgisayar sistemlerinde kullanılan depolama aygıtlarını tanımlayabilecek

Merkezî işlem biriminin doğrudan erişemediği, dijital verilerin kalıcı olarak saklandığı depolama yöntemi ikincil depolamadır. Bilgisayar sistemi kapatıldığında veya bir elektrik kesintisi yaşandığında, ikincil depolama aygıtlarındaki bilgiler kaybolmaz. Merkezî işlem birimi, ikincil depolama aygıtlarına doğrudan erişemez; erişim için girdi/çıkış kanalları kullanılır. İkincil depolama aygıtlarında veri transfer hızı, birincil depolama aygıtlarına göre düşüktür. Fakat bu aygıtlar fiyat olarak daha ucuzdur ve hafıza alanı bakımından daha geniş kapasiteleri vardır. Başlıca ikincil depolama aygıtları sabit disk, CD, DVD, flash bellek, manyetik teyp olarak örneklendirilebilir.

Üçüncül depolama, çok büyük ölçekli verileri arşivlemek amacıyla kullanılan veri saklama yöntemidir. Üçüncül depolama sistemlerinde verileri saklamak için disk veya teyp, verilerin fiziksel organizasyonunu içeren katalog veri tabanları, sistem sürücülerine disk veya teyp taşımakla görevli robot kollar bulunur. Verilerin kullanılması için insan müdahalesine ihtiyaç duyulmaz. Gerekli işlemler bilgisayar ve robot kollar aracılığıyla yapılır.

Çevrim dışı depolama, genellikle veri yedekleme ve felaket kurtarma için kullanılan depolama yöntemidir. Depolama aygıtları, fiziksel olarak esas çalışma ortamından uzak mekanlara yerleştirilmiştir ve yalnız gerekli durumlarda kullanılır. Saklanan verilerin tekrar kullanılabilir hale gelmesi için insan müdahalesi gerekir. Bu depolama yöntemiyle saklanan veriler herhangi bir bilgisayara bağlı olmadığından veri gizliliğini ve bütünlüğünü tehdit edebilecek yazılımsal saldırılara karşı güvenlidir.

Kendimizi Sınayalım

1. Onlu sayı sisteminde 74 sayısının, ikili sayı sistemindeki karşılığı, aşağıdakilerden hangisidir?
 - a. 0110010
 - b. 0101110
 - c. 1001010
 - d. 1011100
 - e. 1100101
2. İkili sayı sisteminde 10011011 sayısının, onlu sayı sistemindeki karşılığı, aşağıdakilerden hangisidir?
 - a. 139
 - b. 155
 - c. 159
 - d. 178
 - e. 198
3. Aşağıdakilerden hangisi ASCII kodlama sistemine ait bir özellik **değildir**?
 - a. 7 bitlik bir kodlama şemasıdır.
 - b. Latin alfabesi üzerine kurulmuştur.
 - c. Kontrol karakterleri ile metinlerin akışı kontrol edilir.
 - d. Baskı karakterleri ile ekrandaki metinler oluşturulur.
 - e. Dünya üzerindeki tüm dillerin temsil edilmesi için yeterlidir.
4. RGB renk grubuna ait bir piksel, bilgisayar hafızasında kaç byte yer kaplar?
 - a. 1
 - b. 2
 - c. 3
 - d. 4
 - e. 5
5. Bir sesin dijital ortama alınırken saniyedeki örneklenme sayısını gösteren örnekleme oranı, aşağıdaki birimlerden hangisi ile belirtilir?
 - a. Kilometre
 - b. Kilowatt
 - c. Kilogram
 - d. Kilohertz
 - e. Kilobyte
6. Aşağıdakilerden ifadelerden hangisi RAM bellek için doğru **sayılamaz**?
 - a. Bellek kapasitesi, aynı anda çalışabilecek program sayısını etkiler.
 - b. Bellekteki herhangi bir alana rastgele erişim sağlanabilir.
 - c. İkincil depolama aygıtlarına göre daha pahalıdır.
 - d. Olası bir elektrik kesintisi durumunda, bellekteki bilgiler kaybolur.
 - e. Bellekteki bilgiler, işlemci tarafından değiştirilemez.
7. Bir bilgisayar sisteminde ikincil depolamaya ihtiyaç duyulmasının temel sebebi nedir?
 - a. Verileri kalıcı olarak saklamak.
 - b. Ana bellekte işlenen veri miktarını azaltmak.
 - c. Bilgisayar programlarının çalışmasını sağlamak.
 - d. Merkezi işlem birimini hızlandırmak.
 - e. Bilgisayar performansını arttırmak.
8. Aşağıdakilerden hangisi ikincil depolama aygıtlarından **değildir**?
 - a. CD
 - b. DVD
 - c. Sabit disk
 - d. Flash bellek
 - e. Ön bellek
9. Üçüncül depolama aygıtlarının yaygın kullanım amacı hangisidir?
 - a. Çok büyük ölçekli verileri arşivlemek.
 - b. Merkezi işlem birimine doğrudan veri ulaştırmak.
 - c. Ana bellekten gereksiz verileri kaldırmak.
 - d. Gizli bilgileri veri tabanında saklamak.
 - e. Sık kullanılan bilgilere hızlı bir şekilde erişmek.
10. Aşağıdakilerden hangisi çevrimdışı depolamanın özelliklerinden **değildir**?
 - a. Bir ortamdan başka bir ortama kolayca nakil edilebilir.
 - b. İnsan müdahalesi olmadan, doğrudan kullanıma hazırır.
 - c. Veri yedekleme ve felaket kurtarma amaçlı kullanılır.
 - d. Veri gizliliğine katkıda bulunur.
 - e. Veri bütünlüğüne katkıda bulunur.

Kendimizi Sınavalım Yanıt Anahtarı

- | | |
|-------|---|
| 1. c | Yanıtınız yanlış ise “İkili Sayı Sistemi” konusunu yeniden gözden geçirin. |
| 2. b | Yanıtınız yanlış ise “İkili Sayı Sistemi” konusunu gözden geçirin. |
| 3. e | Yanıtınız yanlış ise “Bilgisayarda Dijital Verinin Saklanması” konusunu yeniden gözden geçirin. |
| 4. c | Yanıtınız yanlış ise “Bilgisayarda Dijital Verinin Saklanması” konusunu yeniden gözden geçirin. |
| 5. d | Yanıtınız yanlış ise “Bilgisayarda Dijital Verinin Saklanması” konusunu yeniden gözden geçirin. |
| 6. e | Yanıtınız yanlış ise “Depolama Aygıtları” konusunu yeniden gözden geçirin. |
| 7. a | Yanıtınız yanlış ise “Depolama Aygıtları” konusunu yeniden gözden geçirin. |
| 8. e | Yanıtınız yanlış ise “Depolama Aygıtları” konusunu yeniden gözden geçirin. |
| 9. a | Yanıtınız yanlış ise “Depolama Aygıtları” konusunu yeniden gözden geçirin. |
| 10. b | Yanıtınız yanlış ise “Depolama Aygıtları” konusunu yeniden gözden geçirin. |

Sıra Sizde Yanıt Anahtarı

Sıra Sizde 1

Standart ASCII, 7 bitlik bir karakter kodlama şemasıdır. 7 bit kullanarak 2^7 tane farklı sayı oluşturulabilir. Dolayısıyla, standart ASCII’de 128 farklı karakter vardır.

Genişletilmiş ASCII şeması, 8 bit kullanılacak şekilde tasarlanmıştır. Bu tasarımda 2^8 tane farklı sayı oluşturulabilir. Dolayısıyla, genişletilmiş ASCII’de 256 farklı karakter vardır. Genişletilmiş ASCII ve standart ASCII’nin karakter sayısındaki farklılık, şemalarda kullanılan bit sayısı ile ilgilidir.

Sıra Sizde 2

Unicode sisteminde bir karaktere atanan sayı, doğrudan ikili sayı sistemine dönüştürülmez. Doğrudan dönüşüm yapabilmek için, her karakterin eşit sayıda bit içermesi gerekir. Unicode’da doğrudan dönüşüm yapılırsa, bir karakter 32 bit ile gösterilir; bu durumda bir karakter 4 byte yer kaplar. Bir karakterin kapladığı yer arttıkça, oluşturulan metinlerin boyutu da artacaktır. Bu problemin önüne geçmek için, karakterlerin değişken uzunlukta bit sayısına sahip olmasına izin veren kodlama teknikleri geliştirilmiştir.

Sıra Sizde 3

Bir ses dijital ortama aktarılırken, ses dalgaları belirli periyotlarla örneklerle bölünür. Periyot aralığı içerisindeki örnek sayısı ne kadar fazla olursa, sesin kalitesi de o kadar yüksek olur. Bir başka deyişle, örnekleme oranı büyük olan seslerde kalite de yüksek olur. Dijital ortama yüksek kaliteyle aktarılan sesler, sesin orijinal haline daha çok benzer.

Sıra Sizde 4

Birincil depolama aygıtlarında hız, kapasite ve fiyat bakımından bir hiyerarşi bulunur. Bu hiyerarşinin en tepesinde işlemci yazmaçları, ortasında işlemci ön belleği, en altında da ana bellek bulunur. Hiyerarşide yukarıdan aşağı gidildikçe hız azalır, kapasite artar, fiyat düşer. Bu bilgi doğrultusunda, işlemci yazmaçları en pahalı, en hızlı, en küçük depolama birimidir. Ön bellek de ana belleğe göre hızlı ve pahalıdır, fakat en fazla hafıza alanı ana bellekte bulunur.

Yararlanılan ve Başvurulabilecek Kaynaklar

- Matick, R.E. (1977). “**Computer Storage Systems and Technology**”, New York: Wiley.
- Balasa, F. (2010). “**Data Storage**”, Rijeka: Intech.
- Stallings, W. (2006). “**Computer Organization and Architecture**”, New Jersey: Prentice Hall.
- Wempen, F. (2015). “**Computing Fundamentals, Introduction to Computers**”, New York: Wiley.
- Petzold, C. (2000). “**Code: The Hidden Language of Computer Hardware and Software**”, Washington: Microsoft Press.

3

Amaçlarımız

Bu üniteyi tamamladıktan sonra;

- Veri, enformasyon ve bilgi kavramları arasındaki farkları açıklayabilecek,
- Veri işleme döngüsünü ve bu döngüyü oluşturan aşamaları betimleyebilecek,
- İşlemci mimarisini açıklayabilecek ve merkezi işlem biriminin ana parçalarını sıralayabilecek,
- Komut kümesi mimarisi ve program komutlarını tanımlayabilecek,
- Bir programın çalışmasında merkezi işlem birimi, ana bellek ve program arasındaki ilişkiyi çözümleyebileceksiniz.

Anahtar Kavramlar

- Veri, Enformasyon ve Bilgi
- Veri İşleme Döngüsü
- Merkezî İşlem Birimi
- Aritmetik ve Mantık Birimi
- Komut Kümesi Mimarisi
- Komut Çevrimi
- İşlemci, Bellek ve Program İlişkisi

İçindekiler

Bilgisayar ve Programlamaya
Giriş

Bilgisayarda Veri İşleme

- GİRİŞ
- VERİ İŞLEME NEDİR?
- İŞLEMCI MİMARİSİ
- PROGRAMLARIN ÇALIŞMA ESASLARI

Bilgisayarda Veri İşleme

GİRİŞ

Bilgisayarda veri işleme ünitesinde veri işlemenin tanımı yapılmış; işlemci mimarisi ve programların çalışma esasları anlatılmıştır.

Bilgi sisteminde yer alan temel kavramlar veri, enformasyon ve bilgidir. Bu kavramlar hakkında detaylı açıklama, veri işlemenin tanımı ve veri işleme döngüsü “Veri İşleme Nedir?” başlığında belirtilmiştir. Modern bilgisayarların beyni olarak tanımlanan merkezî işlem biriminin yapısı, özellikleri ve görevleri “İşlemci Mimarisi” başlığında açıklanmıştır. Programların ana belleğe alınması, program komutlarının çalıştırılması, komut çevrimi gibi temel programlama esasları ise ünitenin son ana başlığı olan “Programların Çalışma Esasları”nda incelenmiştir.

VERİ İŞLEME NEDİR?

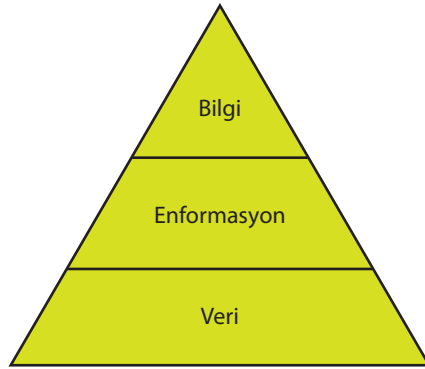
Veri, Enformasyon ve Bilgi Kavramları

Bir bilgisayar sisteminde verinin nasıl işlendiğini anlayabilmek için bilgi sistemindeki temel kavramların açıklanması gerekir. Birbirleriyle ilişkili olan ve yakın anlamlar içeren bu kavramlar veri, enformasyon ve bilgidir.

Veri, enformasyon ve bilgi kavramları üst üste eklenerek bilgi piramidini oluşturur. Bu piramidin basamaklarında yukarı doğru çıkılırken; içerik daralır, anlam genişler.

Resim 3.1

Bilgi piramidi.



Veri, ilişkide olduğumuz dünyanın durumlarını belirten ham olgulardır. Sayısal veya sözel bir simge hâlinde bulunabilen veri, tek başına bir anlam taşımaz. Enformasyon, verinin belirli bir konu çerçevesinde işlenmiş hâlidir. Seçilmiş verilerin bir anlam oluşturacak

şekilde işlenmesiyle enformasyon oluşur. Bilgi, enformasyonların birleşimiyle meydana gelir. Enformasyonu yorumlamak için gereken kuralların anlaşılmasıyla bilgi oluşur. Veri, enformasyon ve bilgi kavramlarını açıklamaya çalışan bir örnek Tablo 3.1'de verilmiştir.

Tablo 3.1
Veri, enformasyon ve bilgi kavramları için örnekler.

Kavram	Örnek
Veri	Yeşil ve kırmızı lambaları olan bir cihazdan, kırmızı ışık yayılmaktadır.
Enformasyon	Yayalar için kırmızı ışık yanmaktadır.
Bilgi	Yayaların geçişine izin yoktur, yayalar beklemelidir.

Veri İşleme

En basit tabirle veri işleme, veri üzerinde çeşitli işlemler uygulayarak ham veriden bilgi elde etmektir. Veri işlemede temel amaç, farklı kaynaklardan elde edilen verinin yönetilmesini sağlamak ve anlamlı bilgiye sahip olmaktır.

Veri işleme sürecinde ham olgular, simgeler ve semboller bir araya getirilerek, ortaya çıkan bileşenlerin birbirleriyle ilişkisi incelenir. Bu ilişkiler doğrultusunda da anlamlı bilgiler ortaya çıkar.

Girdi, işleme ve çıktı adımlarından oluşan veri işleme süreci, genel bir sistem olarak tarif edilebilir. Veri işleme sistemlerinde girdi olarak veri kullanılırken, çıktı olarak anlamlı bilgi beklenir.

Resim 3.2

Veri işleme sisteminin temel yapısı.



Dünya genelinde birçok farklı veri işleme sistemi bulunur. Veriyi bilgiye dönüştürme sürecinde kullanılan bu sistemler, veriyi kullanılabilir ve anlaşılabilir hâle getirir. Veri işleme sistemleri, verinin işlenme türüne göre üç gruba ayrılır:

- **Manuel veri işleme:** Bu sistemlerde verinin işlenmesi insanlar tarafından manuel olarak yapılır. Genellikle kâğıt ve kalem kullanılan, birçok ofiste görülen evrak kayıt yöntemi ve evrak klasörleri, veri işlemenin insanlar tarafından el ile yapıldığı sistemlere örnek olarak gösterilebilir.
- **Mekanik veri işleme:** İnsanların bir mekanik cihaz yardımıyla veri girişi ve veri işleme yaptığı sistemlerdir. Daktilo, hesap makinesi, muhasebe makinesi gibi cihazlar, bu sistemlerde sıklıkla kullanılır.
- **Elektronik veri işleme:** Veri işlemenin bilgisayarlar tarafından yapıldığı otomatik sistemlerdir. Hız, kolaylık, güvenilirlik ve erişilebilirlik bakımından sağladığı avantajlar sayesinde, elektronik sistemler günümüzün popüler veri işleme yöntemidir. Dijital verinin büyüklük ve kapsam olarak ileri seviyelere ulaşmasıyla, elektronik sistemlere olan ihtiyaç da oldukça artmıştır.

DİKKAT



Modern teknolojiye, dijital veri boyutları çok yüksek seviyelere çıkmıştır. Büyük verinin işlenebilmesi için elektronik veri işleme sistemlerine ihtiyaç duyulmaktadır.

SIRA SİZDE



Büyük veri nedir? Büyük veri ve bulut bilişim ilişkisini araştırınız.

Veri İşleme Döngüsü

Bir veri işleme sistemi temel olarak girdi, işleme ve çıktı adımlarıyla gösterilir. Fakat ham veriden bilginin elde edildiği veri işleme süreci, farklı bileşenlerin yer aldığı ve farklı işlemlerin yapıldığı daha detaylı aşamalardan oluşur.

Bilgisayar sistemlerinde veri işleme süreci, birbirini takip eden aşamalar hâlinde ilerler. Süreçteki aşamaların birleşerek oluşturduğu döngülere veri işleme döngüsü adı verilir.

Veri işleme döngüsü veri toplama, veri hazırlama, veri girişi, veri işleme, çıktı ve yorumlama ile depolama olmak üzere altı aşamadan meydana gelir.

Veri Toplama

Veri işleme döngüsünde ilk adım verinin toplanmasıdır. Bu aşamada yapılan işlemler, veri işleme sürecinin sonucuna doğrudan etki eder. Veri işleme sürecinde kullanılan verinin kalitesi, sürecin genel başarısı üzerinde de önemli paya sahiptir. Elde edilen verinin düzen ve amaca uygun olması, veri toplamanın temel ilkesidir.

Veri Hazırlama

Veri toplama aşamasından sonra gelen adım, verilerin hazırlanmasıdır. Çeşitli kaynaklardan toplanan verilerin işlenebilmesi ve analiz edilebilmesi için verilerin bir hazırlanma aşamasından geçmesi gerekir. Ham verinin doğruluk ve uygunluk açısından kontrolü, veri işlemenin başarısını doğrudan etkiler. Bu aşamada veriler, belirli formatlar doğrultusunda düzenlenir ve incelenir.

Veri Girişi

Bir verinin bilgisayar tarafından işlenebilmesi için, verinin dijital hâle getirilmesi gerekir. Bilgisayar sistemlerinin klavye, fare, tarayıcı, dijital kamera gibi girdi üniteleri sayesinde veri girişi yapılır. Veri girişi, kullanıcılar veya otomatik sistemler tarafından gerçekleştirilebilir. Bu aşamanın gerçekleştirilmesinde farklı yazılımlardan veya mevcut verilerden de yararlanılabilir.

Veri İşleme

Veri girişi sayesinde dijital hâle getirilen ham veri, bilgisayar sistemleri tarafından işlenebilir durumdadır. Veri işleme, bir programın veri üzerinde belirli yöntemler uyguladığı aşamadır. Programın bünyesinde bulunan komutlar, veri üzerinde çalışarak aktif hâle gelir. Veri büyüklüğü, veri uygunluğu, yazılım kalitesi, bilgisayarın işlem gücü gibi faktörler, veri işleme aşamasının sonuçlanmasını etkiler.

Çıktı ve Yorumlama

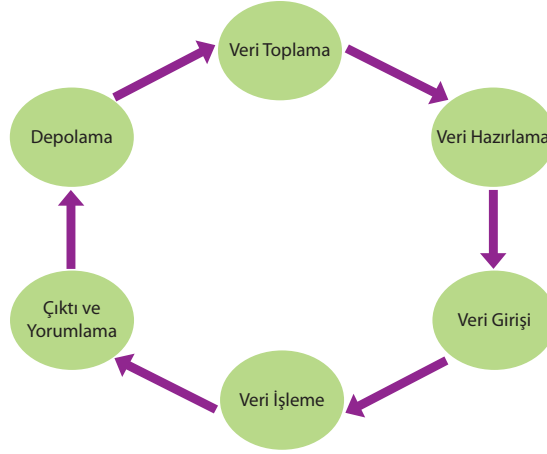
İşlenen veri, bilgisayar kullanıcısına iletilmelidir. Verilerin kullanıcıya aktarımında ses, görüntü, basılı materyal gibi birçok format kullanılır. Bilgisayar sistemlerinin yazıcı, monitör, projektör ve hoparlör gibi çıktı üniteleri sayesinde veriler kullanıcıya ulaşır ve kullanıcı tarafından yorumlanarak değerlendirilir. Kullanıcı sayesinde işlenmiş veriden enformasyon ve bilgi elde edilir.

Depolama

Veri işleme döngüsünün son adımında, işlenen veriler ve elde edilen sonuçlar depolanır. Tüm bilgisayar sistemlerinde depolama üniteleri bulunur. Bu sayede, veri işleme sürecinde elde edilen sonuçlar tekrar tekrar kullanılabilir. Bilgisayar sistemlerinde depolama yapılabilmesi, veri işleme sürecini kısaltan önemli bir faktördür.

Resim 3.3

Bilgisayar sistemlerinde veri işleme döngüsü.



SIRA SİZDE



Veri işleme döngüsünde veriden bilgi elde edilirken, insan faktörünün etkisini açıklayınız.

İŞLEMÇİ MİMARİSİ

Günümüz bilgisayarlarında merkezî işlem biriminin tüm fonksiyonları, yarı iletken bir tümleşik devrede toplanmıştır. Program komutlarını çalıştırmak ve verileri işlemek için gerekli olan tüm elektronik devreler, programlanabilir bir elektronik bileşen olan bu tümleşik devrenin içinde bulunur. Merkezî işlem birimi olarak görev yapan bu devre, kısaca “işlemci” olarak adlandırılır.

Modern teknoloji sayesinde işlemcilerin boyutları oldukça küçülmüştür. Günümüz bilgisayarlarındaki merkezî işlem birimleri, fiziksel boyut olarak bir posta pulu ebadına indirgenmiştir. Ulaşılan minyatür boyutlardan dolayı, işlemciler günümüzde mikroişlemci olarak anılmaktadır.

Bilgisayar sistemlerinde veri işleme, merkezî işlem birimi tarafından gerçekleştirilir. Mikroişlemciler, bellekten komut alır, komut çözer, aritmetik ve mantık işlemlerini gerçekleştirir, girdi/çıkıttı aygıtları ile iletişim kurar.

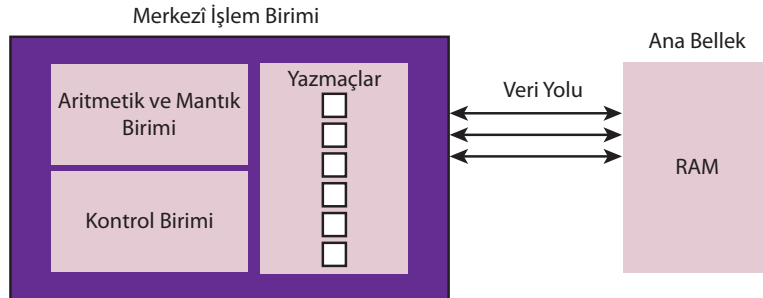
Merkezî İşlem Birimin Yapısı

Merkezî işlem birimi, işlenecek veriyi ve kullanılacak programı belleğe yükleyerek verinin işlenmesini sağlar. Fiziksel olarak çok küçük, performans olarak çok hızlı olan mikroişlemcilerin üç ana parçası vardır:

- Aritmetik ve Mantık Birimi (ALU - Arithmetic and Logic Unit)
- Kontrol Birimi (CU - Control Unit)
- Yazmaçlar (Registers)

Resim 3.4

Merkezî işlem biriminin yapısı ve ana bellek ile bağlantısı.



Aritmetik ve Mantık Birimi

Merkezi işlem biriminin aritmetik ve mantık işlemlerinden sorumlu bileşeni aritmetik ve mantık birimidir. Bu birim, tüm işlemcilerin temel yapı taşıdır. Günümüz işlemcilerinde çok karmaşık ve güçlü aritmetik ve mantık birimleri bulunmaktadır.

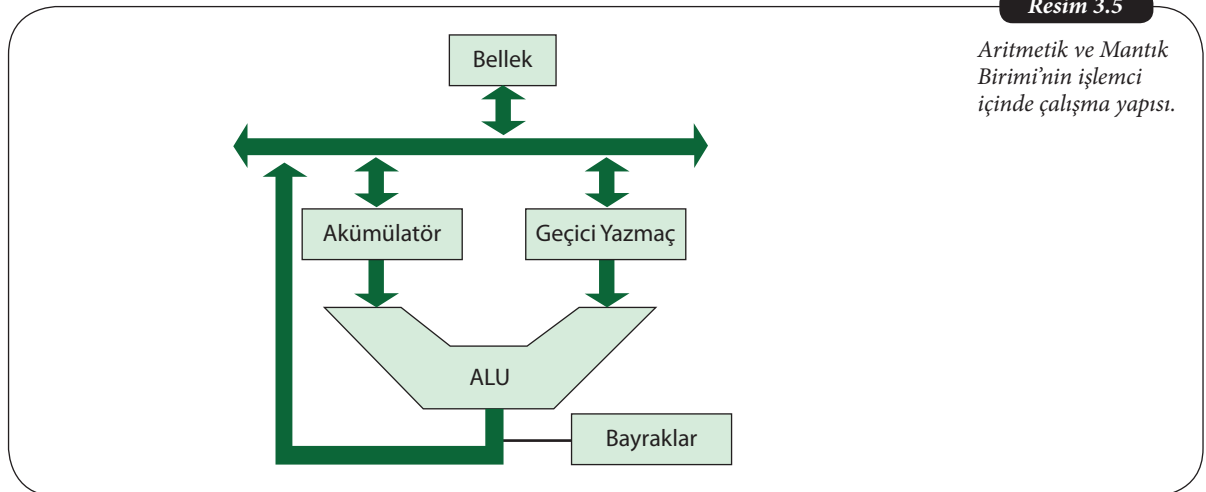
Bilgisayar biliminde kısaca ALU olarak ifade edilen aritmetik ve mantık biriminde toplama, çıkarma, çarpma, bölme gibi aritmetik işlemler ve AND, OR, XOR, NOT gibi mantık işlemleri gerçekleştirilir. Bu işlemlerin gerçekleştirilmesinde komutlarla birlikte mantık kapılarının oluşturduğu toplayıcılar, çıkarıcılar ve kaydırma yazmaçları kullanılır.

MANTIK İŞLEMİ	A	B	İFADE	SONUÇ
AND	0	0	A AND B	0
AND	0	1	A AND B	0
AND	1	0	A AND B	0
AND	1	1	A AND B	1
OR	0	0	A OR B	0
OR	0	1	A OR B	1
OR	1	0	A OR B	1
OR	1	1	A OR B	1
XOR	0	0	A XOR B	0
XOR	0	1	A XOR B	1
XOR	1	0	A XOR B	1
XOR	1	1	A XOR B	0

Tablo 3.2
Mantık işlemlerinden AND, OR ve XOR için mantıksal ifadeler ve sonuçlar.

Aritmetik ve mantık biriminde gerçekleşen bütün işlemler, kontrol sinyalleri aracılığıyla yapılır. İşlemlerin yapılması, kontrol biriminin gözetiminde ve eş zamanlı olarak gerçekleşir.

Resim 3.5'te aritmetik ve mantık biriminin işlemci içindeki çalışma yapısı gösterilmiştir. Aritmetik ve mantık biriminin bu yapı çerçevesinde ilişkide olduğu bileşenleri, akümülatör, geçici yazmaç, bayraklar ve bellektir.



Resim 3.5

Aritmetik ve Mantık Birimi'nin işlemci içinde çalışma yapısı.

Program sayacı: İşlemci tarafından işlenen komutun adresini taşıyan yazmaçtır. Komutlar bulunduğunda, program sayacının değeri 1 artırılır. Bilgisayar yeniden başlatıldığında, program sayacının değeri de 0'a çevrilir.

Kontrol Birimi

Bilgisayarda yapılan tüm işlemleri kontrol eden birimdir. Kontrol birimi; giriş ve çıkış birimlerinin denetimini, bellek ile ilgili işlemleri, komutların yorumlanması ve bilgisayarın bir bütün olarak çalışmasını sağlar.

Kontrol birimi, tüm komutları bellekte bulunmuş sıralarına göre işlemektedir. Bu sıranın belirlenmesi için **program sayacı** (program counter) adı verilen yazmaçtan yararlanır.

Bilgisayarda bir sonraki adımda işlenecek komutun adresi, program sayacında bulunur. Kontrol birimi, her komutu getirdikten sonra program sayacının değerini 1 artırır. İşlenmesi gereken komut, tek bir seferde işlenemeyecek kadar büyük olduğunda, program sayacı her iki bilgi alınışında bir kez artırılır.

DİKKAT



Bilgisayarların 8, 16, 32 veya 64 bitlik olması, işlenebilecek komut büyüklüğünü etkiler. Bir komutun 32 bitlik bir bilgisayarda işlenmesi için gereken süre, 16 bitlik bir bilgisayarda işlenmesi için gereken sürenin yaklaşık olarak yarısıdır.

Yazmaçlar

Kaydedici olarak da adlandırılan yazmaç, mikro işlemciler tarafından kullanılan dâhili geçici hafızalardır. İşlemci içerisinde bulunan özel amaçlı yazmaçlar, aritmetik işlemlerin yapılmasında önemli rol oynar. Yapısal özellikleri sayesinde, karmaşık hesaplamaların daha hızlı yapılabilmesini sağlar.

Birincil depolama aygıtlarının en hızlısı olan yazmaçlar, genellikle 32 veya 64 bit uzunluğunda veri depolayabilirler. İşlemci yazmaçları isimlendirilirken, bir yazmacın hafızaya alabildiği bit uzunluğu kullanılır. Örneğin; 32 bit kaydedebilen bir yazmaç "32-bitlik yazmaç" olarak adlandırılır.

SIRA SİZDE



Merkezî işlem birimini oluşturan ana parçaları listeleyiniz ve bu parçaların temel görevlerini yazınız.

Komut kümesi: Mikro işlemcinin tasarlanma ve üretim aşamalarında tanımlanan, mikro işlemci tarafından algılanabilen komutlar kümesidir. Günümüzde Intel, AMD, Samsung ve IBM gibi işlemci üreten birçok firma vardır. Her firmanın kendine ait bir komut kümesi bulunur. Dolayısıyla, farklı firmaların mikro işlemcileri aynı komutları algılayamayabilir.

Komut Kümesi Mimarisi

Bir bilgisayar sisteminin beyni olarak tanımlanan merkezî işlem birimi, bilgisayarın en önemli parçasıdır. Merkezî işlem birimi, yalnızca makine dilinde yazılmış komutları algılayabilir. Bu elektronik cihaz, ana bellekten veri okur, sayılar ile dört işlem gerçekleştirir, sayıları birbirleriyle karşılaştırır, veriyi belleğin bir adresinden farklı bir adresine taşır.

Merkezî işlem biriminin gerçekleştirdiği işlemlere bakıldığında, bu cihazın veri üzerinde basit işlemler yaptığı görülür. Bir beyin olarak tanımlanmasına rağmen, merkezî işlem birimi kendi başına ne yapması gerektiğini bilemez. Yapılması gereken işlemler, programlarda bir komut listesi hâlinde yer alır ve program tarafından merkezî işlem birimine bildirilir.

Bir programda yer alan her komut, merkezî işlem birimi tarafından yapılacak bir işleme denk gelir. Merkezî işlem biriminin yapabileceği işlemlerin makine dilinde karşılığı bulunur ve bu işlemlerin tümünü içeren kümeye **komut kümesi** (instruction set) adı verilir. Komut kümesi mimarisi, bilgisayar sistemlerinde donanım ve yazılım arasında ayırma katmanı olarak yer alır.

Resim 3.6

Komut kümesinin bilgisayar sistemindeki yeri.



Merkezi işlem birimi, kendi başına çalışamaz. Yalnızca makine dilindeki komutları algılayabilen bu cihaz, komut listesi içeren programlar tarafından yönlendirilir.



DİKKAT

Bir işlemcinin gerçekleştirebileceği işlemler ile işlemcinin komut kümesi arasında doğrudan bir ilişki vardır. İşlemcinin tasarım aşamasında, komut kümesinde yer alacak komutlar ve komut kümesi mimarisi belirlenmelidir. Komut kümesinin belirlenmesini sağlayan iki yaklaşım bulunur:

- **RISC (Reduced Instruction Set Computer):** Dilimize “Azaltılmış Komut Kümeli Bilgisayar” olarak geçen bu yaklaşımda, komut kümesinin olabilecek en yalın seviyeye indirgenmesi hedeflenir. Bu yaklaşıma göre, bir işlemcinin komut kümesinde temel komutların olması yeterlidir ve temel komutların çeşitli organizasyonu, her türlü karmaşık işlem yapılabilir. Bu yaklaşımın çıkış noktası, hızlı, verimli ve ucuz bilgisayarlar üretmektir. RISC modelini kullanan işlemci üretici firmalar, Power PC ve Sun Sparc olarak örneklendirilebilir.
- **CISC (Complex Instruction Set Computer):** Dilimize “Karmaşık Komut Kümeli Bilgisayar” olarak geçen bu yaklaşımda, işlemcilerin çok sayıda farklı komutu çalıştırabilecek kapasitede olması hedeflenir. Komut çeşitliliği sayesinde, gelişen ve karmaşıklaşan yazılımlar ile daha rahat başa çıkılabilir. Bu yaklaşım doğrultusunda tasarlanan işlemcilerin komut kümesinde, temel komutların yanı sıra, teknik olarak çok fazla ihtiyaç duyulmayan komutlar da bulunur. CISC modelini kullanan işlemci üretici firmalar, Intel ve AMD olarak örneklendirilebilir.

Komut kümesi mimarisinin şekillenmesini sağlayan RISC ve CISC yaklaşımları, uzun yıllar boyunca önemli bir tercih konusu olmuş ve işlemci firmaları arasında büyük bir çekişme ve rekabet yaratmıştır.

Zaman içerisinde gerçekleşen teknolojik gelişmeler doğrultusunda, CISC işlemcilerin üretim maliyetleri önemli ölçüde düşmüştür. Üretim maliyetlerindeki azalma, bu yaklaşım ile tasarlanmış işlemcileri kişisel ve dizüstü bilgisayarlarda birinci tercih yapmıştır. Modern bilgisayarların büyük kısmında CISC yaklaşımını kullanan, Intel ve AMD işlemcileri tercih edilir.

RISC yaklaşımı ile tasarlanmış işlemciler, genellikle daha az elektrik tüketirler. Elektrik ihtiyacı konusundaki bu önemli avantaj, bu yaklaşım ile tasarlanmış işlemcilerin cep telefonlarında, akıllı telefonlarda, dijital televizyonlarda ve navigasyon sistemlerinde kullanılmasını sağlar.

Komut Elemanları

Bir işlemcinin görevini yerine getirmesi için gerekli olan bilgi, program komutlarının içinde yer almalıdır. Program komutlarını oluşturan elemanlar, işlemci için gerekli olan bilgileri taşır. Makine dilindeki bir komutu oluşturan dört ana eleman aşağıda listelenmiştir:

- *İşlem kodu (Operation code - opcode):* Merkezî işlem birimi tarafından yapılacak işlemi belirler.
- *İşlenen kaynak referansı (Source operand reference):* Yapılacak işlemde girdi olarak kullanılan elemanlardır. Bir işlemde, bir veya birden çok işlenen kaynak olabilir.
- *İşlenen sonuç referansı (Result operand reference):* Bazı işlemlerde sonuç elde edilebilir. Bu işlemlerde, çıktı olarak işlenen sonuç kullanılır.
- *Sıradaki komut referansı (Next instruction reference):* İşlem tamamlandıktan sonra, işlemcinin hangi komutu bulması gerektiğini belirten elemandır.

Komut Çeşitleri

Bir bilgisayar sistemi, kullanıcıların gerçekleştirmek isteyeceği tüm veri işleme çeşitlerini karşılayabilecek bir komut kümesine sahip olmalıdır. Günümüzde kullanılan üst düzey programlama dilleri ile yazılmış programlar, bilgisayarın anlayacağı makine diline çevrilebilmelidir. Bu gereksinimi karşılamak için, farklı komut türleri oluşturulmuştur. Modern komut kümelerindeki komut çeşitleri aşağıdaki listede verilmiştir:

- *Veri işleme*: Aritmetik ve mantık komutları
- *Veri depolama*: Bellek komutları
- *Veri taşıma*: Girdi/Çıktı komutları
- *Kontrol*: Test ve dallanma komutları

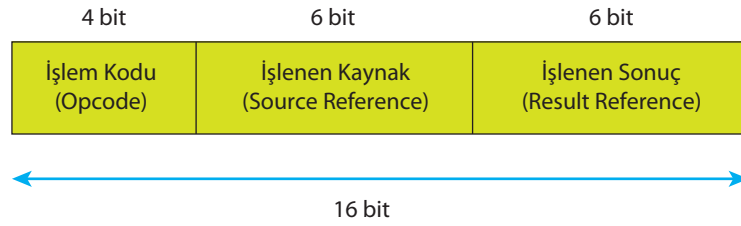
Aritmetik komutları sayısal verilerin işlenmesinde kullanılırken; mantık komutları kullanıcı tarafından işlenmesi istenen diğer veri tiplerini işlemekte kullanılır. İşlemci yazmaçları ve bellek arasında verinin taşınması, bellek komutları aracılığıyla gerçekleşir. Girdi/çıktı komutları ile program ve veriler belleğe taşınır, işlem sonuçları kullanıcıya gönderilir. Test komutları ile verilerin ve hesaplamaların değeri kontrol edilirken; dallanma komutları ile verilen kararlar doğrultusunda program akışı içerisinde dallanmalar yapılır.

Komutların Gösterimi

Bilgisayar sisteminde komutlar 0 ve 1'lerden oluşan dizilimler ile ifade edilir. Her komut, belirli alanlardan oluşur ve bu alanlar değişkenlik gösterebilir. Resim 3.7'de basit bir komut formatı gösterilmiştir.

Resim 3.7

Basit bir komut formatı.



İkili sayı sisteminde ifade edilen terimlerin insanlar tarafından anlaşılması kolay değildir. Makine dilindeki komutların programcılar tarafından rahatça anlaşılabilmesi ve program yazımının kolaylaşması için, komutların ifade edilmesinde sembolik gösterimler kullanılır. Bazı işlem kodlarının sembolik gösterimi Tablo 3.3'te verilmiştir.

Tablo 3.3
İşlem kodlarının
gösterimi için örnekler.

İşlem Kodu	İşlem Açıklaması (İngilizce)	İşlem Açıklaması (Türkçe)
ADD	Add	Toplama
SUB	Subtract	Çıkarma
MUL	Multiply	Çarpma
LOAD	Load data from memory	Bellekten veri yükleme
STOR	Store data to memory	Belleğe veri kaydetme

İşlem kodları dışındaki komut elemanlarının sembolik gösterimi de mümkündür. Örnek olarak, X adresinde bulunan bir değerden, Y adresinde bulunan bir değer çıkarılması SUB X, Y komutu ile gerçekleştirilir.

Komut elemanlarının sembollerle gösterilebilmesi, makine dili programlarının sembolik gösterimlerle yazılabilmesini sağlar. İşlem kodlarına ait sembollerin, ikili sayı sisteminde sabit bir karşılığı bulunur. Programcı, bu sembolleri ve işlenen verilerin bellek adreslerini kullanır.

PROGRAMLARIN ÇALIŞMA ESASLARI

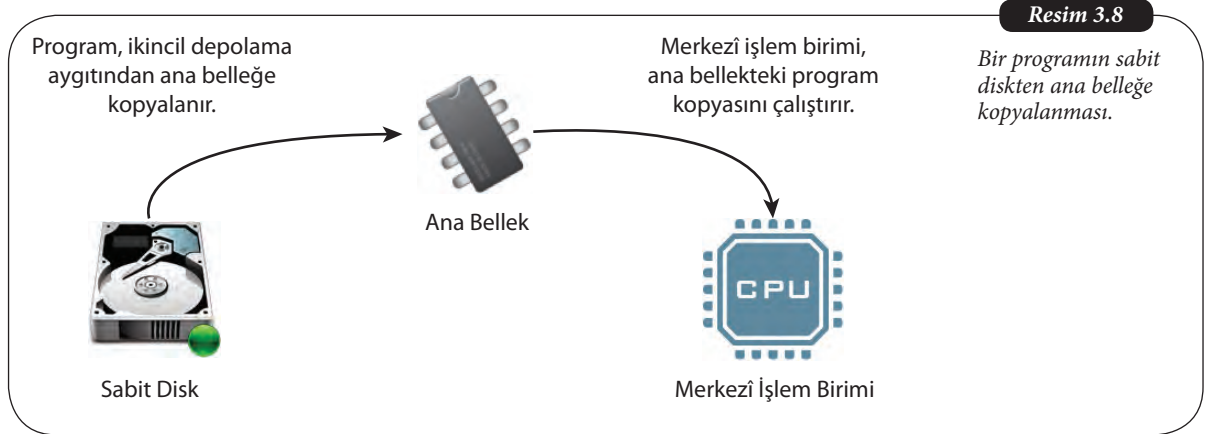
Programların Ana Belleğe Kopyalanması

Bilgisayarlar, kullanıcı isteklerini karşılamak için çeşitli programlara ihtiyaç duyar. Bilgisayar açıldığında, işletim sistemi ve birçok program arka planda çalışmaya başlar. Bilgisayar kapanıncaya kadar, bu programlar çalışmaya devam ederler.

Modern bilgisayar sistemlerinde yüzlerce programı bilgisayar sisteminde saklamak mümkündür. Bilgisayar sistemlerinde programların saklanması, genellikle ikincil depolama aygıtları ile yapılır. Bilgisayar kullanıcıları arasında “program yükleme” olarak adlandırılan bu işlem, bir programın web sayfası, CD, DVD, harici bellek gibi bir kaynaktan bilgisayarın sabit diskine kopyalanmasıdır. Bilgisayarın sabit diskinde veya farklı bir ikincil depolama alanında yeterli yer olduğu sürece, sisteme program yüklemeye devam edilebilir.

Bilgisayar sisteminde bir programın çalışması için, o programın başlatılması ve ana belleğe kopyalanması gerekir. Bir programın başlatılmasını sağlayan komut, işletim sistemi, kullanıcı veya farklı bir yazılım tarafından verilebilir.

Bir program başlatıldığında, programın çalıştırılabilir hâli ve programın çalışması için gereken öncelikli dosyalar, saklandığı ikincil depolama aygıtından bilgisayarın ana belleğine kopyalanır. Bu kopyalama işlemi sayesinde, merkezî işlem birimi programa doğrudan ve hızlıca ulaşabilir. Örnek olarak, bir web sayfasında gezinti yapmak için, kullanacağımız web tarayıcısının simgesine tıklarız. Bu işlem sırasında web tarayıcı uygulaması, bilgisayarın sabit diskinden bilgisayarın ana belleğine kopyalanır. Merkezî işlem birimi, ana bellekteki program kopyasına erişir ve programdaki komutları çalıştırır.



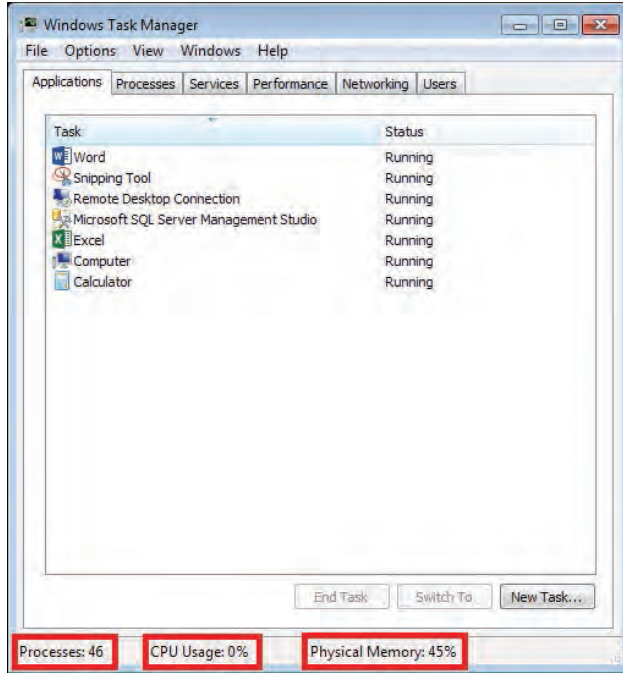
İşlemci teknolojisinde yaşanan gelişmeler, bilgisayar donanımındaki iyileşmeler, işletim sistemlerinin ve uygulama yazılımlarının gelişen özellikleri sayesinde, günümüz bilgisayarları çok güçlü hâle gelmiştir. Modern bilgisayar sistemleri, aynı anda birçok programı verimli şekilde çalışmasına izin vermektedir.

Sistem yazılımları için yeterli kaynak olduğu müddetçe, bilgisayarda aktif olarak çalışan program sayısı artırılabilir. Bilgisayarın çalışması esnasında, merkezî işlem biriminin ve ana belleğin faaliyetlerini gösteren yardımcı uygulamalar da bulunmaktadır.

Resim 3.9'da, Windows işletim sistemindeki görev yöneticisinden bir ekran alıntısı paylaşılmıştır. Bu görüntü, bilgisayarda ofis uygulamaları, uzak bağlantı uygulaması, veri tabanı yöneticisi, hesap makinesi gibi uygulama yazılımları çalışmaktayken alınmıştır. İşlem sayısını, merkezî işlem birimi kullanımını ve ana bellek kullanımını gösteren bilgiler kırmızı kutucuk içerisinde gösterilmiştir. Programlar çalışır durumdayken, ana belleğin %45'lik kısmı kullanılmaktadır. Ekran alıntısı anında programlarda herhangi bir iş yürütülmediği için, programlar rölanti (idle) durumdadır ve merkezî işlem biriminin uygulamalardan kaynaklanan bir iş yükü yoktur.

Resim 3.9

Windows işletim sistemindeki görev yöneticisinden bir ekran alıntısı



Program Komutlarının Çalıştırılması

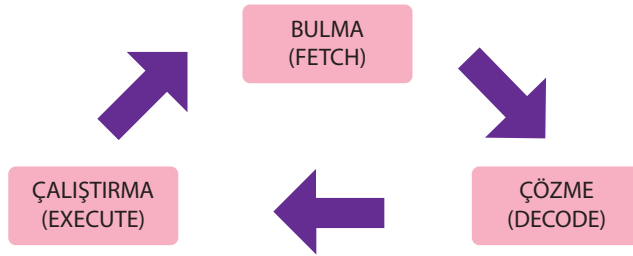
Bilgisayar sistemlerinde bir program ana belleğe kopyalandıktan sonra, programdaki komutlar merkezî işlem birimi tarafından çalıştırılabilir. Program komutlarının çalıştırılması için gerçekleşen adımlar aşağıda listelenmiştir:

- Bir programın başlatılması için merkezî işlem birimi, programa ait ilk komutu ana bellekten almalıdır.
- Merkezî işlem birimi, adres yoluna komutun ana bellekte bulunduğu adresi koyar.
- Adres yolundaki bellek adresi sayesinde komut ana bellekte bulunur.
- Komut, veri yolu üzerinden ana bellekten merkezî işlem birimine taşınır.
- Komut, merkezî işlem birimi tarafından çözülür.
- Çözme işlemi sayesinde, merkezî işlem birimi ne yapması gerektiğini belirler.
- Merkezî işlem birimi, gerekli işlemi gerçekleştirir.
- Merkezî işlem birimi, bilgisayarın diğer donanım bileşenlerine komutun işlendiğini belirten sinyaller gönderir.

Bir bilgisayarın merkezî işlem birimi, yukarıda belirtilen adımları sürekli olarak tekrarlamaktadır. Merkezî işlem birimi, bir komutu çalıştırdıktan sonra diğer bir komuta geçer ve bu döngü bilgisayar çalıştıkça devam eder. Bulma, çözme, çalıştırma adımlarından oluşan bu döngüye “Komut Çevrimi (Instruction Cycle)” adı verilir.

Resim 3.10

Üç adımda gerçekleşen komut çevrimi.



Merkezi işlem biriminin program komutlarını çalıştırması hakkında detaylı bilgi, W. Stallings'in "Computer Organization and Architecture" (New Jersey: Prentice Hall, 2006) kitabında mevcuttur.



K I T A P

Komut Çevrimi

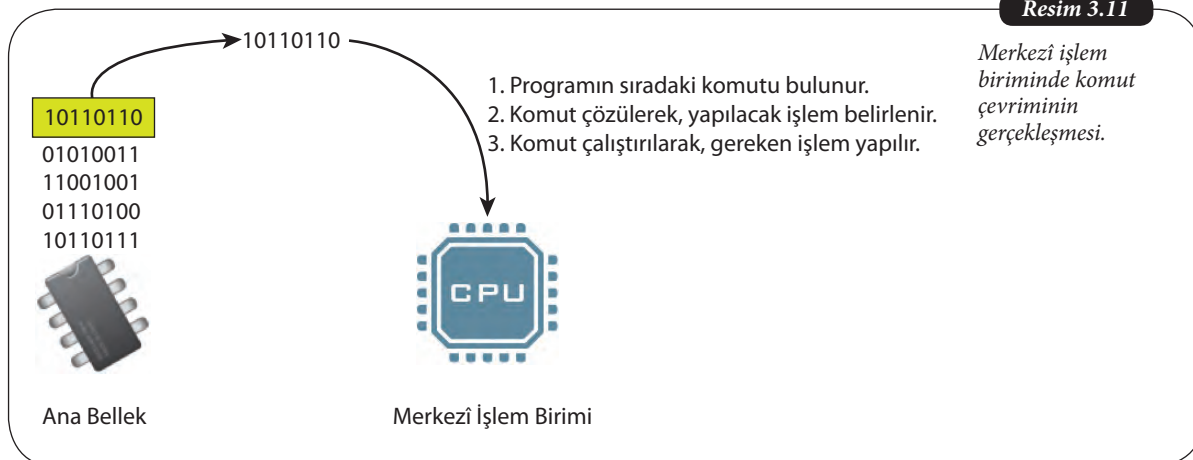
Komut çevrimi, bir programdaki komutların, merkezi işlem birimi tarafından çalıştırılması sürecine verilen isimdir.

Bilgisayar açık olduğu sürece tekrarlanan komut çevrimi, bilgisayar sistemlerindeki en temel işlemdir. Bir programdaki her komut için gerçekleşen bu çevrim, bulma, çözme ve çalıştırma adımlarından meydana gelir. Birçok kaynakta komut çevriminin bu üç adımla adlandırıldığı (Fetch-Decode-Execute Cycle) da görülmektedir.

- *Bulma (Fetch)*: Bir program, birçok makine dili komutunu içerir. Komut çevrimindeki ilk adımda, sıradaki komut ana bellekte bulunur ve merkezî işlem birimine taşınır. Komut, merkezî işlem biriminin yazmaçlarında geçici olarak saklanır.
- *Çözme (Decode)*: Makine dilindeki bir komut, merkezî işlem biriminin yapacağı işlemi belirten, ikili sayı sisteminde bir sayıdır. Bu aşamada merkezî işlem birimi, bellekten gelen komutu çözerek, hangi işlemi yapacağına karar verir.
- *Çalıştırma (Execute)*: Komut çevriminin son aşaması, makine diline dönüştürülmüş komutun çalıştırılmasıdır. Merkezî işlem birimi, işlenen komutla ilgili gereken işlemleri yapar. İşlemin sonucu farklı bir yazmaçta saklanır. İşlem tamamlandıktan sonra, merkezî işlem birimi yeni bir komut çevrimi için kendini hazırlar.

Resim 3.11

*Merkezî işlem
biriminde komut
çevriminin
gerçekleşmesi.*



Merkezî işlem birimi, komut çevriminde adres yolunu ve veri yolunu hangi aşamalarda ve ne amaçla kullanır?



SIRA SİZDE

Özet



Veri, enformasyon ve bilgi kavramları arasındaki farkları açıklamak.

Bilgi sistemini oluşturan temel kavramlar veri, enformasyon ve bilgidir. Birbirleriyle yakın anlamlar içeren bu kavramlar, üst üste eklenerek bilgi piramidini oluşturur. İlişkide olduğumuz dünyanın durumlarını belirten ham olgulara veri denir. Yalın hâlde bir anlam taşımayan verilerin belirli bir konu çerçevesinde işlenmesiyle enformasyon oluşur. Enformasyonların yorumlanarak birleştirilmesiyle de bilgi meydana gelir.



Veri işleme döngüsünü ve bu döngüyü oluşturan aşamaları betimlemek.

Ham veri üzerinde çeşitli işlemler uygulayarak bilgi elde edilmesine veri işleme adı verilir. Veri işleminin temel amacı, farklı kaynaklardan elde edilen verinin yönetilmesini sağlamak ve anlamlı bilgiye sahip olmaktır. Veri işleminin gerçekleştirilmesinde kâğıt ve kalem kullanımı gerektiren manuel yöntemler, daktilo ve hesap makinesi kullanımı gerektiren mekanik yöntemler ve bilgisayar ile gerçekleştirilen elektronik yöntemler kullanılabilir.

Bilgisayar sistemlerinde veri işleme süreci, birbirini takip eden aşamalar hâlinde ilerler ve bu aşamaların tümüne birden veri işleme döngüsü adı verilir. Veri toplama, veri hazırlama, veri girişi, veri işleme, çıktı ve yorumlama, depolama aşamaları veri işleme döngüsünü oluşturur.

Veri toplamada, veriler farklı kaynaklardan toplanır ve bir araya getirilir. Elde edilen verinin düzgün ve amaca uygun olması gereklidir. Veri hazırlamada, ham verinin uygunluk kontrolü ve belirli formatlar doğrultusunda düzenlemeler gerçekleştirilir. Veri girişinde, veriler dijital hâle getirilir. Bu işlemde klavye, fare, tarayıcı, dijital kamera gibi girdi ünitelerinden yararlanılır. Dijital hâle getirilen veri, veri işleme aşamasında programlar aracılığıyla işlenir. İşlenen veri, çıktı ve yorumlama aşamasında kullanıcıya iletir. Yazıcı, monitör ve hoparlör gibi çıktı üniteleri sayesinde, veriler kullanıcıya ulaşır ve kullanıcı tarafından yorumlanarak değerlendirilir. Bu sayede işlenmiş veriden enformasyon ve bilgi elde edilir. Veri depolamada, işlenen veriler ve elde edilen sonuçlar depolanır.



İşlemci mimarisini açıklamak ve merkezi işlem biriminin ana parçalarını sıralamak.

Bilgisayar sistemlerinde veri işleme, merkezi işlem birimi tarafından gerçekleştirilir. Merkezi işlem birimi, işlenecek veriyi ve kullanılacak programı belleğe yükleyerek, verinin işlenmesini sağlar. Merkezi işlem birimini oluşturan temel bileşenler aritmetik ve mantık birimi, kontrol birimi ve yazmaçlardır.

Merkezi işlem birimi, yalnızca makine dilinde yazılmış komutları algılayabilir. Bir beyin olarak tanımlanmasına rağmen, merkezi işlem birimi kendi başına ne yapması gerektiğini bilemez. Yapılması gereken işlemler, programlar tarafından merkezi işlem birimine bildirilir.



Komut kümesi mimarisi ve program komutlarını tanımlamak.

Bir programda yer alan her komut, merkezi işlem birimi tarafından yapılacak bir işleme denk gelir. Merkezi işlem biriminin yapabileceği işlemlerin makine dilinde karşılığı bulunur ve bu işlemlerin tümünü içeren kümeye komut kümesi denir. Komut kümesi mimarisi, bilgisayar sistemlerinde donanım ve yazılım arasında ayırma katmanı olarak yer alır.

İşlemcinin işlem kapasitesini belirleyen komut kümesi, RISC ve CISC olarak adlandırılan iki yaklaşım doğrultusunda tasarlanır. RISC yaklaşımında komut kümesinin olabilecek en yalın seviyeye indirgenmesi hedeflenir. Bu yaklaşıma göre, bir işlemcinin komut kümesinde temel komutların olması yeterlidir. Cep telefonlarında, akıllı telefonlarda ve dijital televizyonlarda bu yaklaşımı tercih eden işlemciler kullanılır. CISC yaklaşımında ise, işlemcilerin çok sayıda farklı komutu çalıştırabilecek kapasitede olması hedeflenir. Bu yaklaşım doğrultusunda tasarlanan işlemcilerin komut kümesinde, temel komutların yanı sıra, karmaşık komutlar da bulunur. Günümüz kişisel ve dizüstü bilgisayarlarındaki işlemciler, bu yaklaşım doğrultusunda geliştirilmiştir.

Bir işlemcinin görevini yerine getirmesi için gerekli olan bilgi, program komutlarının içinde yer alır. Makine dilindeki bir komutta işlem kodu, işlenen kaynak referansı, işlenen sonuç referansı ve sıradaki komut referansı bilgileri yer alır.

Kullanıcı isteklerini karşılayabilmek için, çeşitli komut tipleri oluşturulmuştur. Aritmetik komutları ile sayısal veriler işlenirken; mantık komutları ile diğer veri tiplerinin işlenmesi gerçekleştirilir. İşlemci yazmaçları ve bellek arasında verinin taşınması, bellek komutları aracılığıyla gerçekleşir. Girdi/çıkı komutları ile program ve veriler belleğe taşınır, işlem sonuçları kullanıcıya gönderilir. Test komutları ile verilerin ve hesaplamaların değeri kontrol edilirken; dallanma komutları ile verilen kararlar doğrultusunda program akışı içerisinde dallanmalar yapılır.



Bir programın çalışmasında merkezî işlem birimi, ana bellek ve program arasındaki ilişkiyi çözümlemek.

Modern bilgisayar sistemlerinde yüzlerce programı bilgisayar sisteminde saklamak mümkündür. Bilgisayar sistemlerinde programların saklanması, genellikle ikincil depolama aygıtları ile yapılır. Bilgisayar kullanıcıları arasında “program yükleme” olarak adlandırılan bu işlem, bir programın web sayfası, CD, DVD, harici bellek gibi bir kaynaktan bilgisayarın sabit diskine kopyalanmasıdır.

Bilgisayar sisteminde bir programın çalışması için, o programın başlatılması gerekir. Program başlatıldığında, programın çalıştırılabilir hâli ve programın çalışması için gereken öncelikli dosyalar, saklandığı ikincil depolama aygıtından bilgisayarın ana belleğine kopyalanır. Bu kopyalama işlemi sayesinde, merkezî işlem birimi programa doğrudan ve hızlıca ulaşabilir. Komut çevrimi, bir programdaki komutların, merkezî işlem birimi tarafından çalıştırılması sürecine verilen isimdir. Bilgisayar açık olduğu sürece tekrarlanan komut çevrimi, bilgisayar sistemlerindeki en temel işlemdir. Bir programdaki her komut için gerçekleşen bu çevrim, bulma, çözme ve çalıştırma adımlarından meydana gelir. Bulma adımında, sıradaki komut ana bellekte bulunur ve merkezî işlem birimine taşınır. Komut, merkezî işlem biriminin yazmaçlarında geçici olarak saklanır. Çözme adımında merkezî işlem birimi, bellekten gelen komutu çözerek, hangi işlemi yapacağına karar verir. Çalıştırma adımında, makine diline dönüştürülmüş komut çalıştırılır. İşlemin sonucu farklı bir yazmaçta saklanır. İşlem tamamlandıktan sonra, merkezî işlem birimi yeni bir komut çevrimi için kendini hazırlar.

Kendimizi Sıyalım

1. Aşağıdakilerden hangisi veri işleme döngüsünde yer alan aşamalardan biri **değildir**?

- Veri toplama
- Veri hazırlama
- Çıktı ve yorumlama
- Depolama
- Veri takibi

2. Aşağıdakilerden hangisi elektronik veri işleme sistemlerinin sağladığı faydalardan biri **değildir**?

- Otomatik yöntemler ile kullanıcılara kolaylık sağlamak.
- Büyük veriden bilgi elde edilmesini sağlamak.
- Verilerin daktilo ve kalem ile girilmesini sağlamak.
- Verilerin güvenli bir şekilde işlenmesine yardımcı olmak.
- Verilerin hızlı bir şekilde işlenmesini sağlamak.

3. Merkez işlem biriminin yapacağı işi belirleyen program komutunu oluşturan eleman aşağıdakilerden hangisidir?

- Adres yolu
- İşlem kodu
- İşlenen kaynak referansı
- İşlenen sonuç referansı
- Sıradaki komut referansı

4. Sayısal verilerin işlenmesinden sorumlu komut çeşidi aşağıdakilerden hangisidir?

- Dallanma
- Test
- Bellek
- Aritmetik
- Mantık

5. Aşağıdakilerden hangisi merkezî işlem biriminin görevlerinden biri **değildir**?

- Aritmetik işlemlerini gerçekleştirmek.
- Mantık işlemlerini gerçekleştirmek.
- Bellekten komut almak.
- Program komutlarını depolamak.
- Girdi/çıktı aygıtları ile iletişim kurmak.

6. Makine dilinde yazılmış bir programda, toplama işlemi için kullanılan sembol aşağıdakilerden hangisidir?

- ADD
- SUB
- MUL
- DIV
- LOAD

7. Komut kümesi mimarisinin oluşturulmasında kullanılan RISC ve CISC yaklaşımları ile ilgili aşağıdakilerden hangisi **yanlıştır**?

- CISC yaklaşımında, işlemcilerin çok sayıda farklı ve karmaşık komutu çalıştırabilecek kapasitede olması hedeflenir.
- RISC yaklaşımında, tasarlanan komut kümelerinin yalın olması hedeflenir.
- Intel ve AMD firmaları, ürettikleri işlemcilerde RISC yaklaşımını kullanır.
- Günümüzdeki kişisel bilgisayarlarda CISC yaklaşımını kullanan işlemciler tercih edilir.
- RISC yaklaşımı ile üretilen işlemcilerde elektrik tüketimi düşük seviyededir.

8. Bir programın bilgisayarda kalıcı olarak saklanmasını sağlayan bilgisayar bileşeni aşağıdakilerden hangisidir?

- Merkezî işlem birimi
- Ana bellek
- Modem
- Monitör
- Sabit disk

9. Bir program komutunun çalıştırılması esnasında gerçekleşen adımlar, aşağıdakilerden hangisinde doğru sırada verilmiştir?

- Çözme - Bulma - Çalıştırma
- Çözme - Çalıştırma - Bulma
- Bulma - Çalıştırma - Çözme
- Bulma - Çözme - Çalıştırma
- Çalıştırma - Çözme - Bulma

10. Bir program çalıştırıldığında, programın geçici olarak kopyalandığı bilgisayar bileşeni, aşağıdakilerden hangisidir?

- Ana bellek
- Sabit disk
- Flash bellek
- Harici disk
- Merkezî işlem birimi

Okuma Parçası

Büyük Veri Kahramanı: Veri Bilimci

1997 yılında NASA araştırmacıları Michael Cox ve David Ellsworth tarafından ilk defa kullanılan “büyük veri” ifadesi, gittikçe kapasitesi artan verilerin kontrolünün ve yorumlanmasının oluşturacağı problemlere dikkat çekmek için kullanılmıştı.

Büyük veriyi büyük yapan önemli üç faktör vardır: İşlenen verinin hacmi, verinin karmaşıklığı ve çeşitliliği ve bilgi giriş çıkış hızı. Büyük veri, bilinen ve çok kullanılan veri tabanı sistemlerinin işleyemeyeceği kadar geniş ve karmaşık veri kümelerinden oluştuğu için bu veriler istenilen düzeyde ne saklanabilir, ne de analiz edilebilir. Bu veri ancak doğru veri analiz araçları ve yetkin bir veri bilimciyle bulunduğu hızlı ve doğru analiz edilerek, bir şirketin ya da bir projenin geleceği için ihtiyaç duyduğu çıkarımların elde edilmesini sağlamaya yardımcı olur.

Büyük veri Google, Facebook ve LinkedIn gibi arama ve sosyal paylaşım platformlarının yüz milyonlarca hatta milyarlarca kişinin bilgilerini toplayıp şirketlerinin gelişimi ve müşteri memnuniyeti amacıyla kullanması ve bu yöntemle kullanıcı sayılarını artırması nedeniyle bir anda popüler oldu. Öte yandan uzun yıllardır ABD dâhil birçok ülkedeki büyük şirketler müşterilerinden topladıkları verileri şirketlerinin gelişimi için kullanmıyor, çöpe atıyorlardı. Ancak sosyal paylaşım platformlarının son zamanlardaki başarısının oluşturduğu örnek sayesinde bu verilerin kullanılmaya değer olduğu ortaya çıktı ve “büyük veri” devrimi oldu.

Aslında temelinde analiz olan her bilim dalı (örneğin uzay bilimleri, tıp, iklim bilim, fizik, istatistik) çok fazla parametre içeren değişkenlerle çalıştığı için, ortaya çıkan çok büyük veri setlerini analiz etmek zorunda kalır. Bu açıdan bilim dünyası büyük veri ile aslında çok daha önce tanışmıştı. Fakat hızla gelişen teknolojiye paralel olarak artık hayatın her alanında daha büyük ve karmaşık veri setleri ile başa çıkmak zorunda kalıyoruz. Sosyal medyada yer alan karmaşık veri setlerinin kullanım alanlarına şöyle bir örnek verilebilir. Facebook’un veri bilimcisi Carlos Diuk, iki insanın bir ilişkiye başlayıp başlamayacağını tahmin edebiliyor. Diuk’a göre “İlişkiye başlayacak iki kişi arasındaki Facebook iletilerinin sayısı ilişki başlamadan önceki 100 gün boyunca yavaş fakat düzenli bir artış gösteriyor. İlişki başladığında ise iletiler azalmaya başlıyor. Bir ilişki başlamadan 12 gün önce günde en yüksek 1,67 ileti gözlemlenirken, ilişki başladıktan sonraki ilk 85 günde ileti sayısının günde 1,53’e kadar düştüğü gözlemleniyor. Muhtemelen çiftler birlikte daha fazla zaman geçirmek istiyorlar ve dış dünyadaki etkileşimleri, sosyal medyadaki etkileşimlerine oranla daha da artıyor.”

Günümüzde yenilikçi fikri olan bir kişi rahatlıkla bir akıllı telefon uygulaması yazabilir ya da yazdırabilir. Yaklaşık 2 milyara yakın akıllı telefon kullanıcısı olduğunu düşünürsek, küçük bir reklam kampanyasıyla bir akıllı telefon uygulamasını belki 5000 kişinin telefonuna yüklemesi gayet olasıdır. Bundan sonra uygulamanın sahibi iki şey yapabilir: İlki, yeni müşteri gelmesini bekleyebilir. İkincisi, bir veri bilimci tutup programını kimler yüklemiş, bu kişiler ağırlıklı olarak hangi ülkelerde ve hangi şehirlerde, kaç yaşında, cinsiyetleri ne, programın kullanım sıklığı günün hangi saatinde artmış, programı kullanırken telefon yatay mı yoksa dikey mi tutulmuş gibi bilgileri kullanarak programını geliştirebilir ve kısa zamanda 50 bin ya da 500 bin kullanıcıya ulaşabilir.

Örneğin Google’ın bazı ülkelerdeki grip salgını, salgın başlar başlamaz haber verebilmesi yine veri biliminin kullanılması ile mümkün oluyor. İnsanlar grip olduğunda Google’da grip ile ilgili arama yapmaya başlıyor. Eğer bu arama sayıları belli bir bölgede hızla artıyorsa Google grip salgınının başladığını herkesten önce fark edebiliyor. Öte yandan aramalarda kullanılan sözcükler insanların ait oldukları kültürler, yaşayış tarzları gibi birçok konuda bilgiler verir ve hatta o kültürün geleceği hakkında dahi öngöründe bulunulmasını sağlayabilir. Aslında veri bilimi çok yeni bir alan olduğu için tanımı da kesin değil. Veri bilimci basitçe herhangi bir yazılım mühendisinden daha iyi istatistik ve herhangi bir istatistikçiden daha iyi yazılım mühendisliği bilen kişi olarak tanımlanabilir. Dolayısıyla yazılımcılar veri bilimi için ilk sırada tercih edilmiyor. Her ne kadar yazılımcılar çok iyi ve çok hızlı bilgisayar kodu yazıp yazılım üretebilseler de, veri bilimcinin esas görevi “çok iyi istatistik bilgisiyle verileri kullanarak sorunları çözmek hatta daha önceden sorun olduğu bile tahmin edilemeyen şeyleri öngörerek çözüm üretmek” olarak tanımlanabilir. Daha genel bir tanım yapmak gerekirse, veri bilimci kendini bilgisayar bilimi, istatistik, analitik modelleme ve matematik konularında geliştirmiş, çalıştığı alan ile ilgili kuvvetli sezgileri olan, güçlü iletişim yeteneğine sahip kişidir. Veri bilimci sadece sorunları fark etmekle kalmaz, aynı zaman da organizasyon için en önemli sorunları tespit etme konusunda da ayrı bir yeteneğe sahiptir.

McKinsey Global Enstitüsü’nün 2011 yılında yazdığı bir rapor, önümüzdeki dört yıl içinde sadece ABD’de 140.000 ile 190.000 arasında veri analizci ve veri bilimci açığı olacağını ve bunun yanı sıra bu analizleri değerlendirecek yaklaşık 1,5 milyon yönetici kadrosu açığı olacağını öngörüyor. Bugün sadece iki yıllık geçmişi olan bir büyük veri analizcisi ABD’de yılda ortalama 200.000-300.000 dolar arasında kazanabiliyor.

Bu demek oluyor ki, önümüzdeki yıllarda “veri bilimci” adıyla çok fazla iş pozisyonu açılacak ve yeterince yetişmiş insan olmayacak.

Peki, henüz üniversitelerde veri bilimi bölümleri açılmamışken, bu insanlar nasıl yetişecek ve bu iş gücü açığı nasıl kapanacak? Bu büyük, karmaşık ve farklı formatlardaki veri setlerini analiz edip bilimsel olarak yorumlayabilecek nitelikte, disiplinler arası çalışabilecek insanlara ihtiyaç var. İşte bu nedenle zaten mesleği gereği bu tarz büyük veri setleri ile çalışan, astronomi, fizik, matematik gibi bilim dallarında uzmanlaşmış kişiler, büyük veri analizi için biçilmiş kaftan olarak görülüyor. 2008 yılında ABD’de yaşanan ekonomik kriz sonrasında üniversitelerdeki kadrolu araştırmacı sayısının düşmesi nedeniyle özellikle doktorasını yapmış fizikçiler ve astronomlar kadro beklemek yerine yavaş yavaş endüstride bu tarz işleri tercih etmeye başladı. Zaten aşına oldukları bir araştırma alanı olduğundan, başarıları kısa sürede Silikon Vadisi’nde duyuldu ve şu anda veri bilimci kadroları için en revaçta olan adaylar arasındalar. Yetkin bir veri bilimci olmak için matematik, algoritma, veri tabanları, istatistik ve programlama gibi konularda uzmanlaşmış olmak gerekiyor. Bazı üniversitelerin veri bilimi alanında verdiği çevrimiçi kursları (örneğin; Coursera, Udacity) takip etmek ve şu an revaçta olan yazılımları ve programları (örneğin; Python, Hadoop, R) öğrenmek büyük verilerin analizi konusunda kişisel gelişim için iyi bir başlangıç olacaktır.

İlerleyen yıllarda çok daha önem kazanacak olan bu yeni bilim ve meslek dalı, üniversitelerin özellikle sayısal alan mezunları için iyi bir alternatif olacak. Arzu eden öğrencilerin veri bilimi konusunda kendilerini geliştirmesini ve meslek seçenekleri arasına veri bilimci olmayı da eklemelerini tavsiye ediyoruz. Çünkü büyük veriye sahip olan ve bu veriyi doğru ve hızlı bir şekilde işleyip analiz eden, aynı zamanda geleceğin de sahibi olacak.

Dr. Umut Yıldız, Selçuk Topal

Kaynak: Bilim ve Teknik, Nisan 2015.

Kendimizi Sınavalım Yanıt Anahtarı

- | | |
|-------|---|
| 1. e | Yanıtınız yanlış ise “Veri İşleme Nedir?” konusunu yeniden gözden geçiriniz. |
| 2. c | Yanıtınız yanlış ise “Veri İşleme Nedir?” konusunu yeniden gözden geçiriniz. |
| 3. b | Yanıtınız yanlış ise “İşlemci Mimarisi” konusunu yeniden gözden geçiriniz. |
| 4. d | Yanıtınız yanlış ise “İşlemci Mimarisi” konusunu yeniden gözden geçiriniz. |
| 5. d | Yanıtınız yanlış ise “İşlemci Mimarisi” konusunu yeniden gözden geçiriniz. |
| 6. a | Yanıtınız yanlış ise “İşlemci Mimarisi” konusunu yeniden gözden geçiriniz. |
| 7. c | Yanıtınız yanlış ise “İşlemci Mimarisi” konusunu yeniden gözden geçiriniz. |
| 8. e | Yanıtınız yanlış ise “Programların Çalışma Esasları” konusunu yeniden gözden geçiriniz. |
| 9. d | Yanıtınız yanlış ise “Programların Çalışma Esasları” konusunu yeniden gözden geçiriniz. |
| 10. a | Yanıtınız yanlış ise “Programların Çalışma Esasları” konusunu yeniden gözden geçiriniz. |

Sıra Sizde Yanıt Anahtarı

Sıra Sizde 1

Sosyal medya, ağ günlükleri, sistem kayıtları, fotoğraf ve video gibi kaynaklardan toplanan verinin, anlamlı ve işlenebilir hâline büyük veri adı verilir. Son yıllarda gittikçe önem kazanan büyük veri, dijital olarak çok büyük hacim kaplar ve yapısal olarak karmaşıktır.

Bulut bilişim, bilişim aygıtları arasında ortak bilgi paylaşımını sağlayan hizmetlerdir. Bulut bilişim hizmetlerinde uygulamalar, programlar, veriler sanal bir sunucuda depolanır ve internet aracılığıyla kolayca erişilir.

Bulut bilişim, büyük verinin işlenmesinde popüler bir yöntemdir. Bulut bilişim teknolojisiyle sunulan yüksek hızda veri işleme, yoğun hâlindeki verileri yönetebilme, ilişkisiz verilerden bilgi çıkarma gibi olanaklar sayesinde, büyük veri işlenebilmektedir.

Sıra Sizde 2

Veri işleme döngüsünde ham veriden işlenmiş veriye geçiş, veri işleme aşamasında gerçekleşir. Elde edilen işlenmiş veri, çıktı ve yorumlama aşamasıyla kullanıcıya ulaşır. Bu andan itibaren, bilgi edinimi için insan faktörü devreye girer. İşlenmiş veriden enformasyon ve bilgi elde etmek, kullanıcının yorumlamasına dayanır.

Sıra Sizde 3

Bir bilgisayar sisteminin beyni olarak görev yapan merkezi işlem birimi, üç ana parçadan meydana gelir. Bu parçalar, aritmetik ve mantık birimi, kontrol birimi ve yazmaçlardır.

Aritmetik ve mantık birimi, aritmetik ve mantık işlemlerinin yapılmasından sorumludur. Toplama, çıkarma, bölme ve çarpma gibi aritmetik işlemleri bu birim tarafından gerçekleştirilir. AND, OR, XOR, NOT gibi mantık işlemleri de bu birimin kontrolü altında gerçekleşir.

Kontrol birimi, bilgisayarda yapılan tüm işlemleri kontrol eder. Giriş ve çıkış birimlerinin denetimini, bellek ile ilgili işlemleri, komutların yorumlanması ve bilgisayarın bir bütün olarak çalışmasını sağlar.

Yazmaçlar, işlemci tarafından kullanılan dâhili geçici hafızalardır. Hız olarak çok yüksek, kapasite olarak çok düşük depolama aygıtlarıdır. Aritmetik işlemlerin yapılmasında önemli rol oynarlar.

Sıra Sizde 4

Program komutları, ana bellekten merkezi işlem birimine kopyalanmalıdır. Bu işlemi gerçekleştirmek için, komut çevriminin bulma aşamasında adres yolu ve veri yolu kullanılır. Merkezi işlem birimi, adres yoluna komutun ana bellekte bulunduğu adresi koyar. Komut, veri yolu ile ana bellekten merkezi işlem birimine taşınır.

Yararlanılan ve Başvurulabilecek Kaynaklar

- Brookshear, J.G. (2011). “**Computer Science: An Overview**”, Addison-Wesley.
- Patterson, D.A., Hennessy, J.L. (2008). “**Computer Organization and Design: Hardware/Software Interface**”, Morgan Kaufmann.
- Stallings, W. (2006). “**Computer Organization and Architecture**”, Prentice Hall.
- Young, R. (2009). “**How Computers Work: Processor and Main Memory**”, CreateSpace.
- Bilim ve Teknik Dergisi. “**Büyük Veri Kahramanı: Veri Bilimci**”, Nisan 2015.

4

Amaçlarımız

Bu üniteyi tamamladıktan sonra;

- Programlamanın temel unsuru olan algoritma kavramını açıklayabilecek,
- Algoritmaların gösterim yöntemlerini listeleyebilecek,
- Alt seviye programlama dillerinden yüksek seviye programlama dillerine geçişi ifade edebilecek,
- Derleyici ve yorumlayıcı kavramlarını tanımlayabilecek ve bu iki kavram arasındaki farkları açıklayabilecek,
- C programlama dilinin temel özelliklerini uygulayabilecek,
- C programlarının yapısını çözümleyebilecek bilgi ve becerilere sahip olabileceksiniz.

Anahtar Kavramlar

- Algoritma
- Sözde Kod
- Akış Şeması
- Derleyici
- Yorumlayıcı
- Kaynak Kodu
- Önişlemci Direktifi
- Ana Fonksiyon

İçindekiler

Bilgisayar ve Programlamaya
Giriş

Programlamada Temel Kavramlar
ve C'ye Giriş

- GİRİŞ
- ALGORİTMALAR
- YÜKSEK SEVİYE PROGRAMLAMA DİLLERİ
- C PROGRAMLAMA DİLİ

Programlamada Temel Kavramlar ve C'ye Giriş

GİRİŞ

Programlamada temel kavramlar ve C'ye giriş ünitesinde işlenen konuların ana başlıkları algoritmalar, yüksek seviye programlama dilleri ve C programlama dilidir.

Günlük yaşantımızda da sıklıkla karşılaştığımız algoritmaların programlamadaki yeri ve önemi, çeşitli algoritma örnekleri ve algoritmaların temel özellikleri “Algoritmalar” başlığı altında belirtilmiştir. Programlama dillerinin gelişme süreci, yüksek seviyeli programlama dilleri, derleyici ve yorumlayıcı kavramları “Yüksek Seviyeli Programlama Dilleri” başlığında anlatılmıştır. Günümüz işletim sistemlerinin neredeyse tamamında kullanılan, dünyanın en yaygın programlama dillerinden olan C programlama dilinin temel özellikleri, bu dil aracılığıyla program yazma kuralları ve yazılan programın çalıştırılma prensipleri “C Programlama Dili” başlığında gösterilmiştir.

ALGORİTMALAR

Algoritma Nedir?

Bir işin nasıl yapılacağını bildiren adımlar kümesine algoritma denir. Günlük hayatımızın büyük kısmında, farkında olmadan da olsa algoritmalar ile karşılaşırız. Bir yemeğin yapılmasındaki adımları içeren yemek tarifi, yerini bilmediğiniz bir restorana gitmenize yardımcı olan yol tarifi, yeni aldığınız elektronik cihazın kullanma kılavuzu günlük hayatta kullandığınız algoritmalara örnek olarak gösterilebilir.

Bir algoritma, yapılacak işe ait açıklamalardan ve işin yapılmasında takip edilecek adımlardan meydana gelir. Açıklama kısmında, işin tanımı yapılır ve işle ilgili detaylar belirtilir. Adımlar kısmında, işin başlangıcından sonuna kadar yapılacak işlemler sıralanır ve tarif edilir.

İki pozitif sayının ortak bölenlerinin en büyüğünü bulmak için kullanılan Euclid algoritmasının açıklaması ve adımları Resim 4.1’de gösterilmiştir.

Resim 4.1

İki pozitif sayının ortak bölenlerinin en büyüğü bulan Euclid algoritması.

Açıklama: Elimizde iki adet pozitif sayı vardır. Bu iki sayının ortak bölenlerinin en büyüğü bulunacaktır.

Adımlar: **Adım 1.** Sayılardan büyük olanı A, küçük olanı B olarak isimlendir.
Adım 2. A sayısını B sayısına böl, kalanı K olarak isimlendir.
Adım 3. K sayısı 0'dan farklı ise A sayısına B'nin değerini ata, B sayısına K'nın değerini ata, 2'nci adıma geri dön. K sayısı 0 ise, ortak bölenlerin en büyüğü B'nin değeridir.

Örnek: Elimizdeki sayılar 8 ve 12 olsun.
 Adım 1. $A = 12, B = 8$
 Adım 2. $A \% B = 4, K = 4$
 Adım 3. $K \neq 0, A = 8, B = 4$
 Adım 2. $A \% B = 0, K = 0$
 Adım 3. $K = 0$, ortak bölenlerin en büyüğü = 4

SIRA SİZDE

1

Algoritmalar, günlük hayatımızda sık sık karşımıza çıkar. Hayatımızın farklı anlarında, farklı şekillerde karşılaştığımız olaylardan algoritma kapsamına girebilecekler örnekler veriniz ve açıklayınız.

Algoritmaların Özellikleri

Programlamada kullanılan algoritmaların temel özellikleri aşağıdaki listede verilmiştir:

- **Girdi ve Çıktı Bilgisi:** Bir algoritmanın girdi ve çıktı bilgileri olmalıdır. Bu bilgiler, algoritma için tanımlı veri kümesine ait olmalıdır. Girdi bilgisi algoritmaya dışarıdan verilirken, çıktı bilgisi algoritma içinde üretilerek dışarıya verilir. Bazı algoritmalarda yalnızca çıktı bilgisi bulunabilir.
- **Açıklık:** Algoritmayı oluşturan adımlar doğru ve kesin bir şekilde tanımlanmalıdır.
- **Doğruluk:** Algoritmayı çalıştıran her girdi için doğru bir çıktı elde edilmelidir.
- **Sonluluk:** Bir algoritmanın mutlaka sonu olmalıdır. Girilen veri boyutundan bağımsız şekilde, algoritma adımları farklı bir aşamaya geçmeli veya sonlanmalıdır. Algoritmanın farklı adımları gerçekleştirilirken, algoritma sonsuz döngüye girmemelidir.
- **Verimlilik:** Algoritmayı oluşturan adımlar, belirli bir süre içinde tamamlanmalıdır.
- **Genellik:** Bir algoritma, aynı türdeki problemlerin hepsine uygulanabilir olmalıdır.

Algoritmaların Programlamadaki Rolü ve Önemi

Bilgisayarda bir işlemin gerçekleştirilmesi için, yapılacak işlemin merkezi işlem birimine bildirilmesi gereklidir. Bu gereksinim, programlar aracılığıyla gerçekleşir. Bilgisayarın yapacağı işleri tarif eden algoritmalar ile programlar oluşturulur. Programlar ve içerdikleri algoritmalar, bilgisayar sistemlerinde yazılım olarak adlandırılır.

Bilgisayar programlarının geliştirilmesinde algoritmaların büyük önemi vardır. Bir problemin çözümü için gerekli adımlar, algoritmalarda tanımlanır. Problemin çözümünü elde etmek için bir algoritma geliştirilemiyorsa, o problemin bilgisayar aracılığıyla çözülmesi de mümkün değildir.

Algoritmalar kendi aralarında da sınıflandırılabilir ve karşılaştırılabilir. Aynı işlevi gören algoritmalar, farklı adımlara sahip olabilir. Algoritmanın tasarımından ve yapısından kaynaklanan farklılıklar, algoritma performansını da etkiler. Programcılar, yapacakları işe

en uygun algoritmayı geliştirmek ve programın en verimli şekilde çalışmasını sağlamak durumundadır.

Algoritmalar, programların temel yapısını oluşturur. Bir programın hızlı ve verimli çalışması, programı oluşturan algoritma ile doğrudan ilişkilidir.



DİKKAT

Algoritmaların Gösterimi

Algoritmaların tanımlanmasında ve gösteriminde kullanılan değişik yöntemler mevcuttur. Yöntemlerden başlıcaları konuşma dili ile gösterim, akış diyagramları ile gösterim ve sözde kod ile gösterimdir.

Konuşma Dili

Bir algoritmanın açıklaması ve algoritmada yer alan adımlar, konuşma dili kuralları çerçevesinde gösterilebilir. Bu gösterim şeklinde, algoritma açık ve kesin bir dille tanımlanır. Algoritmada yer alan adımlar liste halinde yazılır.

Resim 4.1'de belirtilen örnekte, iki pozitif sayının ortak bölenlerinin en büyüğünü bulmayı sağlayan algoritma, konuşma dili ile gösterilmiştir. Algoritmanın tanımı ve adımları açıkça belirtilmiş ve konuşma dili kullanılarak ifade edilmiştir.

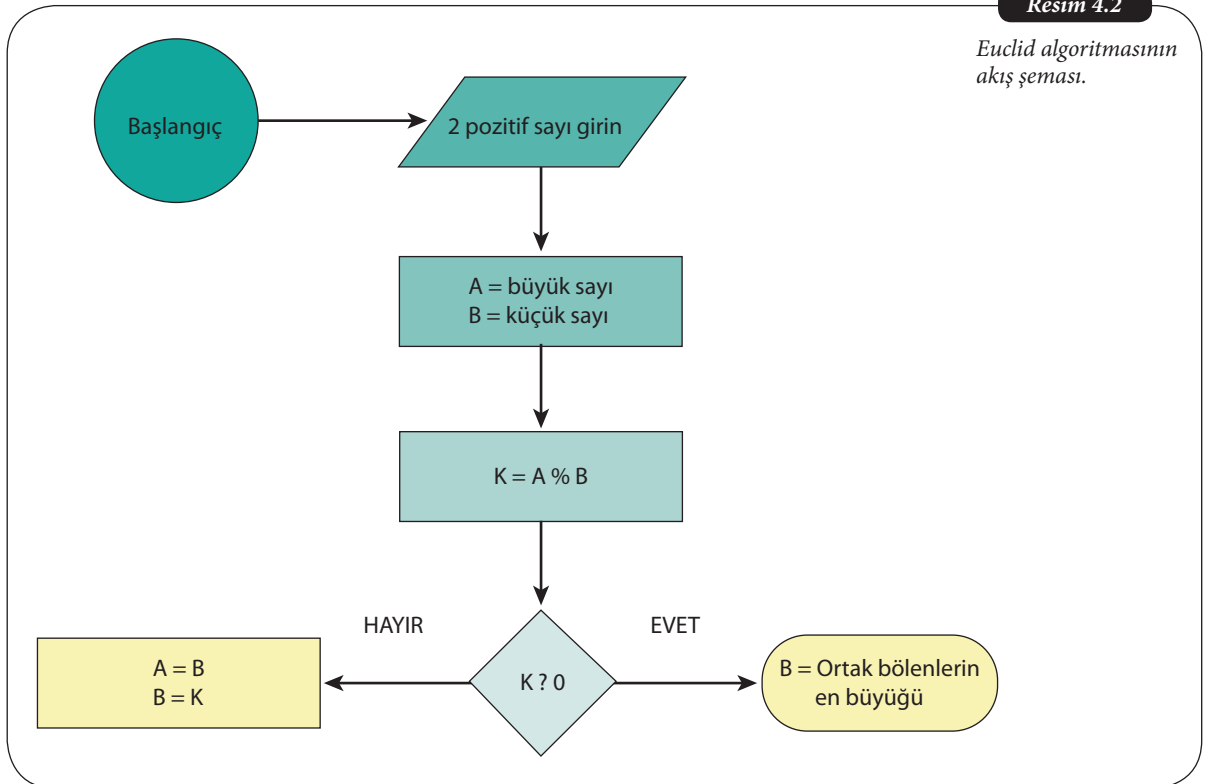
Akış Şemaları

Akış şeması, algoritmaların gösteriminde kullanılan faydalı bir yöntemdir. Bir akış şemasında algoritma adımlarını ifade eden kutucuklar, adımlar arası geçişleri gösteren oklar, karar verme mekanizmaları olarak kullanılan şekiller bulunabilir.

Akış şeması, bir algoritmanın görsel halini sunar. Bu sayede algoritmaların anlaşılması daha kolay hale gelir. Programcılar ve analistler tarafından yaygın olarak kullanılan akış şemalarını oluşturmak için birçok farklı çizim yazılımı bulunur.

Resim 4.2

Euclid algoritmasının akış şeması.





Ücretsiz versiyonuyla çeşitli çizimler yapılabilen, çevrim içi çizim aracı draw.io için <https://www.draw.io> adresini ziyaret edebilirsiniz.

Sözde Kod

Sözde kod (pseudocode), bir algoritma veya program oluşturulurken kullanılan, konuşma diline benzeyen ve programlama dillerinin detaylarından uzak anlatımlardır. Algoritmaların sözde kod ile gösterilmesinde, bir programlama diline benzeyen ifadeler kullanılır, ancak bu ifadeler bilgisayarın anlayabileceği ifadeler değildir.

Sözde kodlar, programlama mantığı ile konuşma dili cümlelerinin harmanlanmasıyla meydana gelir ve herkes tarafından kolaylıkla anlaşılabilir. Sözde kodu okuyan bir kişi, programlama dillerinin detaylarına takılmadan, algoritmanın çalışma mantığını anlayabilir.

Resim 4.3'te, iki pozitif sayının ortak bölenlerinin en büyüğünü bulmayı sağlayan algoritma, sözde kod ile gösterilmiştir.

- 1 numaralı adımda, algoritmanın tanımı yapılmış ve iki pozitif sayının girdi olarak kullanılacağı belirtilmiştir.
- 2 numaralı adımda, a sayısının b sayısına bölümünden kalan değer, k sayısına eşitlenmiştir.
- 3 ve 6 numaralı adımlar arasında, basit bir döngü oluşturulmuştur. Bu döngü, k sayısı 0'a eşit olana kadar devam edecektir.
- 7 numaralı adımda döngünün sonlandırıldığı belirtilmiştir.
- 8 numaralı adımda, b sayısı ekrana basılacaktır. Basılan bu sayı, algoritmanın girdileri için ortak bölenlerin en büyüğüdür.
- 9 numaralı adım, sözde kodun son adımıdır. Bu adımda, algoritmanın tamamlandığı belirtilmiştir.

Resim 4.3

Euclid algoritmasının sözde kodu.

```

1  procedure EUCLID(a, b : positive integers)
2      k = a mod b
3      while (k != 0)
4          a = b
5          b = k
6          k = a mod b
7      end while
8      print b
9  end procedure

```

YÜKSEK SEVİYE PROGRAMLAMA DİLLERİ

Makine Dilinden Çevirici Dillere Geçiş

Bilgisayarlar, yalnızca makine dilinde yazılmış programları çalıştırabilir. Makine dili komutları 0 ve 1 değerlerinden oluşan, insan tarafından okunması ve anlaşılması kolay olmayan komutlardır. Bu komutları kullanarak binlerce satırlık programlar yazmak ve gerektiği durumlarda bu programları incelemek, oldukça zahmetli, zaman gerektiren ve hataya müsait bir iştir.

Bilgisayar programlama tarihinde, makine dili komutlarını kullanarak program yazmanın zorlukları kısa sürede anlaşılmıştır. Bu zorluğun üstesinden gelebilmek ve prog-

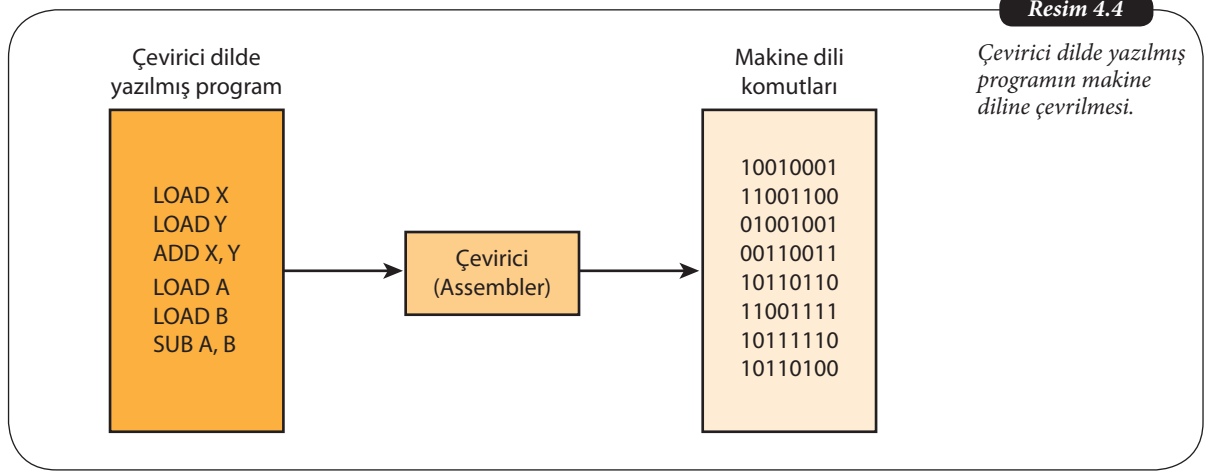
ramcılarının işini kolaylaştırmak için çevirici diller (assembly) geliştirildi. Bu dillerde makine dili komutlarının doğrudan kullanılması yerine, komutları ifade eden kısa kelimelere geçiş yapıldı. Örnek olarak, toplama işlemi için ADD, çıkarma işlemi için SUB, çarpma işlemi için MUL, veriyi bellekte farklı bir alana taşımak için MOV gibi kısa komutlar, makine dili komutlarının yerine geçti.

İşlemci üretici firmalar, kendi komut kümesi mimarilerini tasarlar ve geliştirir. Firmaların ürettikleri değişik işlemcilerin komut kümeleri de birbirinden farklı olur. Bilgisayar işlemcilerindeki bu farklılıklardan dolayı, çevirici dillerde de farklı türler bulunur.



DİKKAT

Çevirici dillerde yazılmış programlar, merkezî işlem birimi tarafından doğrudan çalıştırılmaz. Bu programları makine diline çeviren özel programlar vardır. Çevirici (assembler) olarak bilinen bu özel programlar sayesinde makine diline dönüşüm gerçekleştirilir ve program komutları çalıştırılır.



Programlamada Yüksek Seviye Dillere Geçiş

Çevirici diller, makine diline göre oldukça rahat bir programlama ortamı sağlamış olsa da; programcılarının karşılaştıkları zorluklar, yüksek seviyeli programlama dillerinin gelişimine kadar devam etmiştir.

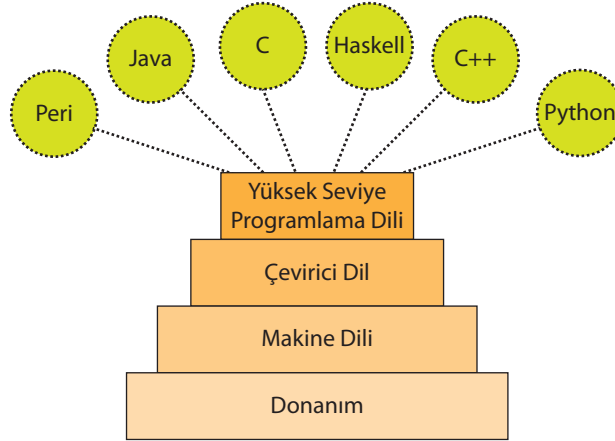
Yapı itibarıyla makine diline oldukça benzeyen çevirici diller, alt seviye programlama dilleri kategorisine girer. Çevirici dil kullanarak program geliştirmek için merkezî işlem biriminin yapısına iyi derecede hakim olmak gerekir. Makine dilinde olduğu gibi, çevirici dillerde yazılan programlar da çok fazla komut içerir.

Alt seviye programlama dillerinde yaşanan zorlukların önlenmesi için yüksek seviye programlama dilleri oluşturulmuştur. Temelleri 1950'li yıllarda atılan yüksek seviye programlama dilleri, güçlü ve karmaşık programların kolaylıkla geliştirilmesini sağlar. Bu dillerde program yazmak için merkezî işlem biriminin işleyişinin bilinmesine gerek yoktur. Yazılan programlardaki komut sayısı, alt seviye dillerdeki programlara göre oldukça azdır.

Bilgisayar sistemlerinde donanım ve programlama dilleri arasında bir hiyerarşi bulunmaktadır. Yüksek seviye programlama dilleri, bu hiyerarşinin en üst basamağında yer alır. Yüksek seviye programlama dillerinin, donanım ve programlama dilleri hiyerarşisindeki yeri Resim 4.3'te gösterilmiştir.

Resim 4.5

Donanım ve programlama dilleri arasındaki hiyerarşi.



Günümüzde kullanılan modern programlama dillerinde, insanlar tarafından rahatlıkla anlaşılan kelimeler ve simgeler kullanılır. Örnek olarak; C, C#, Python ve Java programlama dillerinde ekrana “Merhaba Dünya” gibi basit bir cümle yazdırmak için gerekli komutlar Tablo 4.1’de verilmiştir.

Tablo 4.1

C, C#, Python ve Java dillerinde ekrana cümle yazdırmak için gerekli komutlar.

Programlama Dili	Komut
C	printf (“Merhaba Dünya”);
C#	Console.WriteLine (“Merhaba Dünya”);
Python	print “Merhaba Dünya”
Java	System.out.print (“Merhaba Dünya”);

Yukarıda belirtilen işlemi makine dilinde veya çevirici dilde gerçekleştirmek oldukça zahmetli ve zordur. Yazılacak programın, bu işlemi gerçekleştirecek komutları ve merkezî işlem biriminin bilgisayarın çıktı üniteleri ile haberleşme yöntemini içermesi gereklidir. Makine dilinde veya çevirici dilde yazılacak program, yüzlerce komut içerecek ve hataya açık olacaktır.

Yüksek seviye programlama dilleri, programcılarının işini oldukça kolaylaştırır. Bu diller sayesinde, programcılar merkezî işlem biriminin yapısıyla ilgili işlemlerle uğraşmaktan kurtulur ve daha az komut ile çalışma imkanı bulur. Bu dillerde yazılan kodlar, programcılar tarafından rahatlıkla okunabilir ve anlaşılabilir. Bu sayede daha hızlı kod yazmak, kod yazımında daha az hata yapmak, yazılan kodları incelemek ve kontrol etmek mümkün olmuştur.

SIRA SİZDE



Derleyici: Bir programlama dilinde yazılmış kodu, genellikle makine diline çevirmek için kullanılan bilgisayar programıdır. Derleyicilerin temel görevi, çalışan bir program elde etmektir.

Yorumlayıcı: Bir programlama dilinde yazılmış kodu, doğrudan çalıştıran bilgisayar programıdır. Yorumlayıcılarda dönüştürme ve çalıştırma işlemleri gerçekleştirilir.

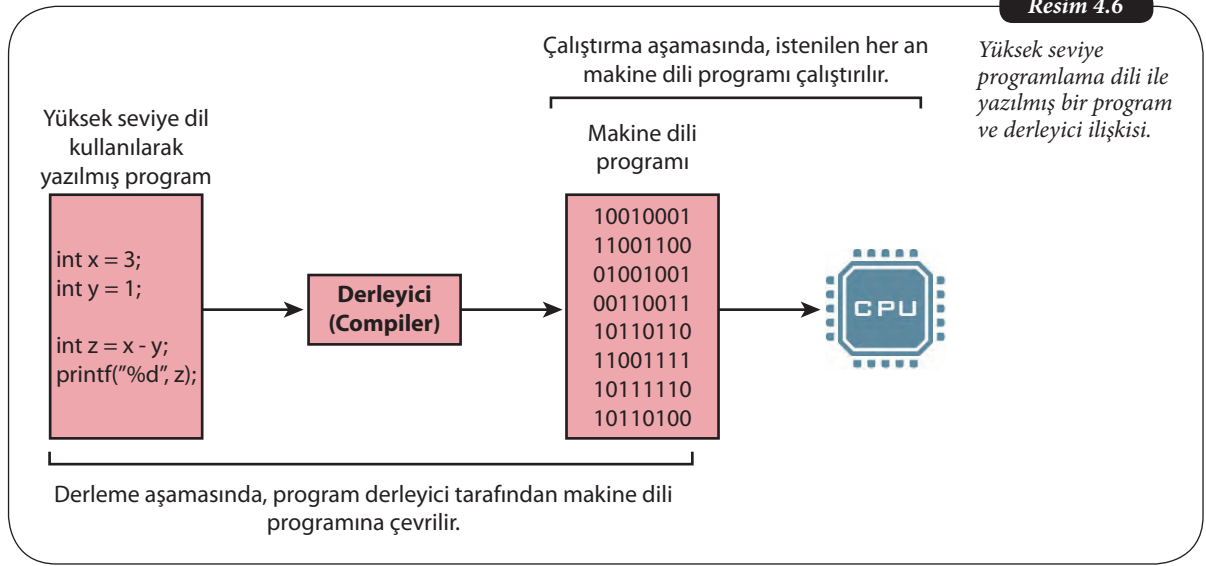
Günümüzde sıklıkla kullanılan yüksek seviye programlama dillerinden örnekler veriniz. Bu dillerden bir tanesi için, ekrana “Programlamaya Merhaba” cümlesini yazdıracak komutu gösteriniz.

Derleyiciler ve Yorumlayıcılar

Bilgisayar sistemlerinde merkezî işlem birimi, yalnızca makine dili komutlarını algılayabilir. Dolayısıyla, yüksek seviye programlama dili kullanılarak yazılan programların makine diline çevrilmesi gerekmektedir. Programın yazıldığı dilin özellikleri doğrultusunda, bu çevrimin yapılabilmesi için **derleyiciler** (compiler) veya **yorumlayıcılar** (interpreter) kullanılır.

Derleyiciler, yüksek seviye programlama dili kullanılarak yazılan bir programın makine diline çevrilmesinde kullanılan bilgisayar programlarıdır. Derleyici kullanılan programlama dillerinde, programın derlenmesi ve çalıştırılması iki ayrı süreç olarak işler. Bir program, bir defa derlendikten sonra, tekrar derlenme ihtiyacı duymaz. Derleme işlemi sonucunda elde edilen makine dili programı, istenildiği her an merkezî işlem birimi tarafından çalıştırılabilir.

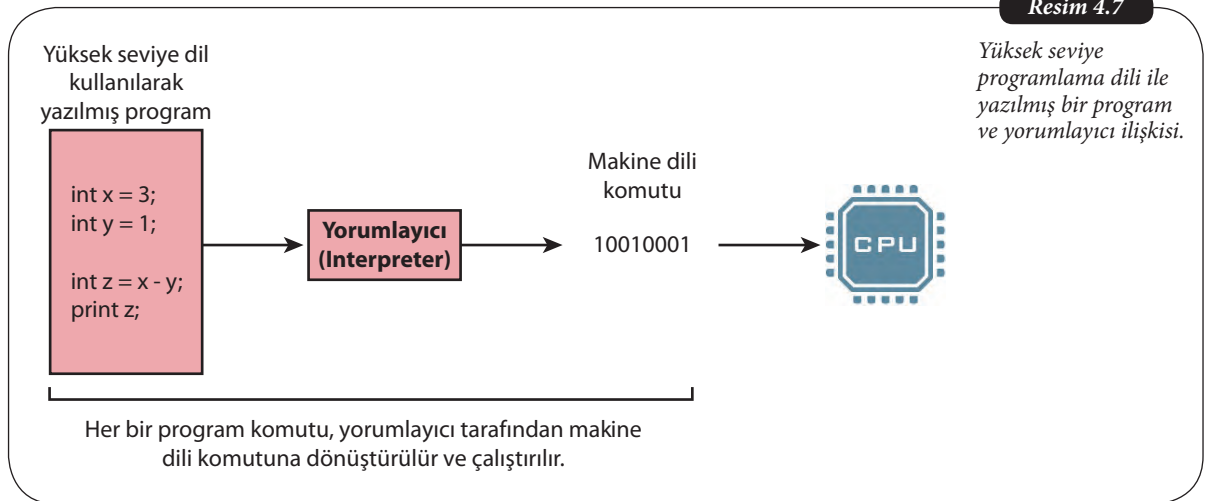
Resim 4.6



Derleyici kullanılan yüksek seviye programlama dillerine örnek olarak C, C++, Haskell, COBOL ve Fortran gösterilebilir.

Yorumlayıcılar, yüksek seviye programlama dili kullanılarak yazılan bir programdaki komutları dönüştürmekte ve çalıştırmakta kullanılır. Program komutları, yorumlayıcı tarafından okunur ve makine dili komutlarına dönüştürülür. Yorumlayıcı tarafından dönüştürülen her komut doğrudan çalıştırılır. Bu işlem, programdaki her komut için tekrarlanır.

Resim 4.7



Yorumlayıcı kullanılan yüksek seviye programlama dillerine örnek olarak Perl, PHP, Python, Ruby ve Smalltalk gösterilebilir.

Yorumlayıcı kullanan programlama dillerinde, makine dili komutları doğrudan çalıştırıldığı için ayrıca bir makine dili programı oluşturulmaz.



DİKKAT

Derleyiciler ve Yorumlayıcılar Arasındaki Farklar

Modern yazılım dünyasında program geliştirmek için genellikle yüksek seviye programlama dilleri kullanılır. Bu dillerde yazılan programlar, derleyiciler veya yorumlayıcılar aracılığıyla bilgisayarların anladığı makine komutlarına çevrilir. Temel olarak aynı amaç doğrultusunda işlev gören derleyiciler ve yorumlayıcılar arasında bazı farklar bulunur:

- Derleyiciler, bir programın tamamını makine diline çevirirken, yorumlayıcılar program komutlarını ayrı ayrı makine komutlarına dönüştürür.
- Derleyicilerde kaynak kodun analizi uzun sürerken, yorumlayıcılarda bu süre daha düşüktür. Buna karşılık; derleyicilerin çalışma hızı, yorumlayıcıların çalışma hızından yüksektir.
- Derleyicilerde programın tamamı tarandıktan sonra, var olan hata mesajları ortaya çıkar. Yorumlayıcılarda ise bir hata ile karşılaşılana kadar dönüştürme işlemi devam eder.
- Derleyicilerde orta seviyeli nesne kodları (intermediate object code) üretilirken, yorumlayıcılarda böyle bir üretime ihtiyaç yoktur. Bu farklılıktan dolayı, derleyicilerin bellek ihtiyacı daha fazladır.
- Derleyicilerde programların sürekli derlenmesine gerek yoktur. Yorumlayıcılarda ise programların her seferde dönüştürülmesi gerekir.

Kaynak Kodu

Yüksek seviye programlama dillerinde, bir program için programcılar tarafından yazılan komutlar topluluğuna **kaynak kodu** adı verilir. Kaynak kodu, kısaca kod olarak da adlandırılır.

ÖRNEK 1

C dilinde, ekrana “Merhaba Dünya” yazdıran bir programın kaynak kodu.

```
#include <stdio.h>

int main(void)
{
    printf("Merhaba Dünya");
    return 0;
}
```

IDE: Programcıların yazılım geliştirmesi için kapsamlı olanaklar sağlayan yazılım uygulamasıdır. Bir IDE'yi oluşturan başlıca bileşenler kod yazım editörü, derleyici, yorumlayıcı ve hata ayıklayıcıdır. Günümüzde sıklıkla kullanılan IDE örnekleri Eclipse, NetBeans, IntelliJ ve Visual Studio'dur. IDE, dilimize “Tümleşik Geliştirme Ortamı” olarak geçmiştir.

Bir programın yazım aşaması, program kodunun bir metin düzenleyici aracılığıyla oluşturulmasıyla başlar. Oluşturulan dosya, bilgisayarın sabit diskine kaydedilir. Programın yazıldığı dil, derlenen bir programlama dili ise program derleyici ile makine dili programına çevrilir. Programın yazıldığı dil, yorumlanan bir dil ise program yorumlayıcı ile makine komutlarına dönüştürülür. Bilgisayarın anladığı makine dili komutları elde edildikten sonra, merkezi işlem birimi bu komutları çalıştırır.

Günümüzde programların yazılması, kaynak kodlarının düzenlenmesi, programların test edilmesi ve çalıştırılması için **IDE** (Integrated Development Environment) adı verilen yazılımlar kullanılmaktadır.

Yaygın Kullanılan Yüksek Seviye Programlama Dilleri

Bilgisayar teknolojisinin gelişimiyle birlikte yüksek seviye programlama dillerinin çeşitleri de artmıştır. Programlama tarihinde, farklı amaçlara hizmet eden yüzlerce dil kullanılmıştır. Günümüze kadar yaygın kullanılan yüksek seviye programlama dillerinden bazıları Tablo 4.2'de listelenmiştir.

Programlama Dili	Açıklama
Ada	1970'li yıllarda geliştirilen, Amerika Birleşik Devletleri'nin savunma sanayisi için kullandığı bir dildir.
C, C ++	Bell Laboratuvarları tarafından geliştirilmiş, genel amaçlı ve popüler programlama dilleridir. C++, C dilinin devamı olarak tasarlanmıştır.
C#	Microsoft tarafından geliştirilmiş, günümüzün popüler programlama dillerindendir. Microsoft .NET platformu için uygulama geliştirmeye yönelik bir dildir.
Java	Açık kodlu, nesneye yönelik, platformdan bağımsız, çok popüler bir programlama dilidir.
JavaScript	Yoğunlukla web geliştirme işlerinde kullanılan, isim benzerliğine rağmen Java ile bir ilgisi olmayan dildir.
FORTRAN	1950'li yıllarda geliştirilen bu dil, ilk yüksek seviye programlama dilidir. Karmaşık matematiksel işlemleri gerçekleştirmek için kullanılmıştır.
Haskell	Salt işlevsel ve genel maksatlı bir programlama dilidir.
Perl	Yoğun şekilde metin işleme ve görüntü tanıma işlerinde kullanılabilen, güçlü ve pratik bir dildir.
PHP	Web teknolojisinde çok geniş kitleler tarafından kullanılan bir programlama dilidir.
Ruby	Günümüzde popülerliği artmakta olan, web sunucularında çalışmaya uygun programlar geliştirilebilen bir dildir.
Python	Ticari ve akademik uygulamalarda sıklıkla kullanılan, genel amaçlı bir dildir.

Tablo 4.2
Yaygın kullanılan yüksek seviye programlama dilleri.

C PROGRAMLAMA DİLİ

C Dili Hakkında Temel Bilgiler

C programlama dili, genel amaçlı bir programlama dilidir. 1970'lerin başında Dennis Ritchie tarafından, UNIX işletim sistemi için geliştirilmiştir. C programlama dili, günümüzdeki işletim sistemlerinin neredeyse tamamında kullanılan, dünyanın en yaygın sistem programlama dilidir.

UNIX işletim sisteminin popülerleşmesiyle birlikte, C dili de gelişmeye ve yaygınlaşmaya başlamıştır. İşletim sistemleri, derleyiciler ve hata ayıklayıcılar gibi aşağı seviyeli sistem programlarının geliştirilmesinde yoğun olarak C dili kullanılır. Genel olarak her alanda kullanılabilen bu dil, günümüzdeki nesneye yönelik programlama dillerinin (C++, Java, C#, vs.) ve betik dillerinin (JavaScript, PHP, vs.) esin kaynağıdır.

C dili gelişimini tamamlamış ve ANSI (American National Standards Institute) ve ISO (International Organization for Standardization) kurumları tarafından standart hale getirilmiş bir dildir. C dilinde yazılmış bir program herhangi bir değişikliğe uğramadan veya çok az bir değişimle, geliştirildiği ortamdan farklı bir ortamda derlenebilir ve çalıştırılabilir. Bu özellik, C dilini taşınabilir kılar. Örnek olarak, Windows işletim sisteminde yazılan bir C kodu, Linux işletim sisteminde de derlenebilir ve çalıştırılabilir.

C dili, yapısı itibarıyla güçlü ve esnek bir dildir. İyi bir yazılım geliştirme ortamına sahip olan bu dil, özel komut ve veri tipi tanımlamasına izin verir. Geliştirilen C kodları, fonksiyonlardan meydana gelir. Yazılan bu fonksiyonların tekrar tekrar kullanımı ve taşınabilmesi mümkündür.

Program Yazım Kuralları

- C dili, büyük ve küçük harflere karşı duyarlıdır. Bir komutun büyük veya küçük harflerle yazılması arasında fark vardır. Örnek olarak, “TOPLAM”, “toplam”, “Toplam” kelimelerinin hepsi, derleyici tarafından farklı olarak algılanır.
- C dilindeki tüm komutlar ve anahtar kelimeler, küçük harfler ile yazılır. Bu kurala istisna olarak, yalnızca #define komutu vardır.
- Programlar hakkında açıklamalar, kodun içerisinde yer alabilir. Açıklamalar, derleyici tarafından göz ardı edilir ve işlenmez.
- Açıklama satırları /* ile başlayıp, */ ile biter. Açıklamalar, birden fazla satırdan oluşabilir.
- C dilindeki her komut satırı, noktalı virgül ile biter.

C Dilindeki Anahtar Kelimeler

Anahtar kelimeler (keywords), C dili derleyicileri için özel anlam taşıyan programlama elemanlarıdır. Programlama diline özel anahtar kelimeler, o dilde yazılan programlarda tekrar tanımlanamaz ve değişken ismi olarak kullanılamaz. C dilinde 32 adet anahtar kelime vardır ve bu kelimelerin hepsi küçük harfle yazılır. Resim 4.9’da, C dilinde kullanılan anahtar kelimeler gösterilmiştir.

Resim 4.8

C dilindeki anahtar kelimeler.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
contt - const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

C dilindeki anahtar kelimeler, dört grup altında sınıflandırılabilir:

- *Veri tipi:* char, const, double, enum, float, int, long, short, signed, struct, union, unsigned, void, volatile.
- *Bellek:* auto, extern, register, static, typedef.
- *Deyim:* break, case, continue, default, do, else, for, goto, if, return, switch, while.
- *İşleç:* sizeof.

Kullanılan makine ve derleyiciye bağlı olarak, C dilinde başka anahtar kelimeler de yer alabilir. Bu durum, C diline has bir özelliktir. Standart dışındaki anahtar kelimeler, genellikle _ karakteri ile başlar.

Anahtar kelimeler, programlama diline özel tanımlardır. Bir dilde etkin şekilde program geliştirmek için, anahtar kelimelere ve bu kelimelerin uygun kullanımlarına hakim olmak gerekir. Bu bölümde belirtilen anahtar kelimeleri detaylı olarak araştırmanızda fayda vardır.



DİKKAT

C Dilindeki Operatörler

İlişkisel Operatörler

İlişkisel operatörler, iki değişkeni karşılaştırmak için kullanılır. İlişkisel operatörlerde yer alan değişkenlerin birbirleriyle mukayese edilebilir olması gerekir. Örneğin, iki sayı arasında ilişkisel operatörler kullanılabilirken, bir sayı ve bir metin arasında ilişkisel operatörler kullanılamaz. İlişkisel operatörlerin kullanıldığı karşılaştırmalar, doğru (true) veya yanlış (false) değerleri ile sonuçlanır. C dilinde tanımlı ilişkisel operatörler Tablo 4.3'te gösterilmiştir.

C Dilindeki İfade	İşlem	Örnek
>	Büyük	3 > 5 (false)
<	Küçük	3 < 5 (true)
>=	Büyük veya eşit	3 >= 5 (false)
<=	Küçük veya eşit	3 <= 5 (true)
==	Eşitlik karşılaştırma	3 == 5 (false)
!=	Eşit değil	3 != 5 (true)

Tablo 4.3

C dilinde tanımlı ilişkisel operatörler.

Mantıksal Operatörler

Mantıksal operatörler, mantıksal deyimler ile işlem yapmak için kullanılır. Bu operatörlerin kullanılacağı işlemlerde, deyimlerin doğru veya yanlış değerlerini alması gereklidir. İşlemlerin sonuçları da doğru ve yanlış değerlerini alacaktır. C dilinde tanımlı mantıksal operatörler Tablo 4.4'te gösterilmiştir.

C Dilindeki İfade	İşlem
&&	Ve
	Veya
!	Değil

Tablo 4.4

C dilinde tanımlı mantıksal operatörler.

Matematiksel Operatörler

C dilinde matematiksel işlemleri gerçekleştiren matematiksel operatörler bulunur. Bu dilde tanımlı matematiksel operatörler Tablo 4.5'te gösterilmiştir.

C Dilindeki İfade	İşlem	Örnek
+	Toplama	10 + 5 = 15
-	Çıkarma	10 - 5 = 5
*	Çarpma	10 * 5 = 50
/	Bölme	10 / 5 = 2
%	Modüler bölme	10 % 5 = 0
++	arttırma 1	10 ++ = 11
--	azaltma 1	10 -- = 9

Tablo 4.5

C dilinde tanımlı matematiksel operatörler.

DİKKAT



Bir arttırma ve bir azaltma işlemlerinde, operatörün ifadenin sağında veya solunda olmasına göre anlam değişir. `x++` ifadesinde, `x` değeri kullanılır ve `x`'in değeri 1 arttırılır. `++x` ifadesinde, `x`'in değeri 1 arttırılır ve artırılmış `x` değeri kullanılır. Aynı durum, bir azaltma işlemi için de geçerlidir.

Atama Operatörleri

Atama operatörleri, değişkenlere değer atamak için kullanılan simgelerdir. C dilinde tanımlı atama operatörleri Tablo 4.6'da gösterilmiştir.

Tablo 4.6
C dilinde tanımlı atama operatörleri.

C Dilindeki İfade	Açıklama
=	Sağdaki değer, soldaki değişkene atanır.
+=	Soldakine sağdaki eklenir, sonuç soldakine atanır.
-=	Soldakinden sağdaki çıkarılır, sonuç soldakine atanır.
*=	Soldaki ile sağdaki çarpılır, sonuç soldakine atanır.
/=	Soldaki sağdakine bölünür, sonuç soldakine atanır.
%=	Soldaki ile sağdakinin modüler bölünmesinin sonucu, soldakine atanır.

C Programlarının Temel Yapısı

Bir C programını oluşturan iki ana bölüm bulunur. Bu bölümler önışlemci direktifleri (preprocessor directives) ve ana fonksiyondur (main function).

Önişlemci Direktifleri

Derleyicinin bazı özel işlemleri gerçekleştirmesini sağlayan direktiflere önışlemci direktifleri denir. Derleme işleminin ilk aşamasında, önışlemci derleyiciye bu direktifleri ulaştırır. Her önışlemci direktifinin farklı bir görevi vardır.

Önişlemci direktifleri # işareti ile başlar. `#include` ve `#define`, en sık kullanılan önışlemci direktifleridir.

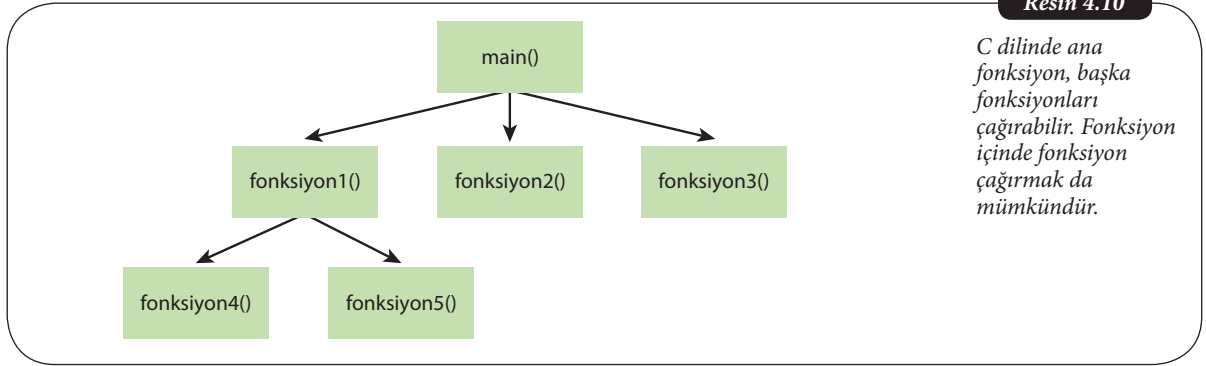
- **#include:** C dilinde en çok kullanılan fonksiyonlar, .LIB uzantılı kütüphane dosyalarında (library file) saklanır. Bu sayede fonksiyonların tekrardan yazılmasına gerek kalmaz, ihtiyaç halinde doğrudan kullanılır. Kütüphane dosyalarındaki fonksiyonların prototipleri, .H uzantılı başlık dosyalarında (header file) tutulur. Kütüphane dosyalarındaki fonksiyonların, programa dahil edilmesi için `#include` direktifi kullanılır. Örnek olarak, C dilinde ekrana yazı yazdırmak için `printf` fonksiyonunun kullanılması gerekir. Bu fonksiyonun prototipi `stdio.h` adlı başlık dosyasında bulunur. Fonksiyonun kullanılabilmesi için, kaynak kodun baş kısmına `#include <stdio.h>` komutu eklenmelidir.
- **#define:** C dilinde makro yaratmak için kullanılan önışlemci direktifidir. Makronun adını tanımlayarak, makroya bir karakter dizisini değer olarak atar. Derleyici, makronun adının geçtiği kod parçalarını makronun değeriyle değiştirir. Örnek olarak, `#define YEAR 2015` komutu ile YEAR geçen komutlarda 2015 değerinin kullanılması sağlanır.

Ana Fonksiyon

C programları fonksiyonlardan oluşur. Bu dilde yazılan her programın en az bir fonksiyonu olmak zorundadır.

C programlarının hepsinde **main()** fonksiyonu bulunur. Programın ilk çağrılan fonksiyonu olan ana fonksiyon, programda gövde görevini görür. Programda kullanılacak diğer fonksiyonlar, ana fonksiyon tarafından çağrılır. Bir fonksiyonun içerisinde başka bir veya birkaç fonksiyonun çağırılması mümkündür.

Resim 4.10

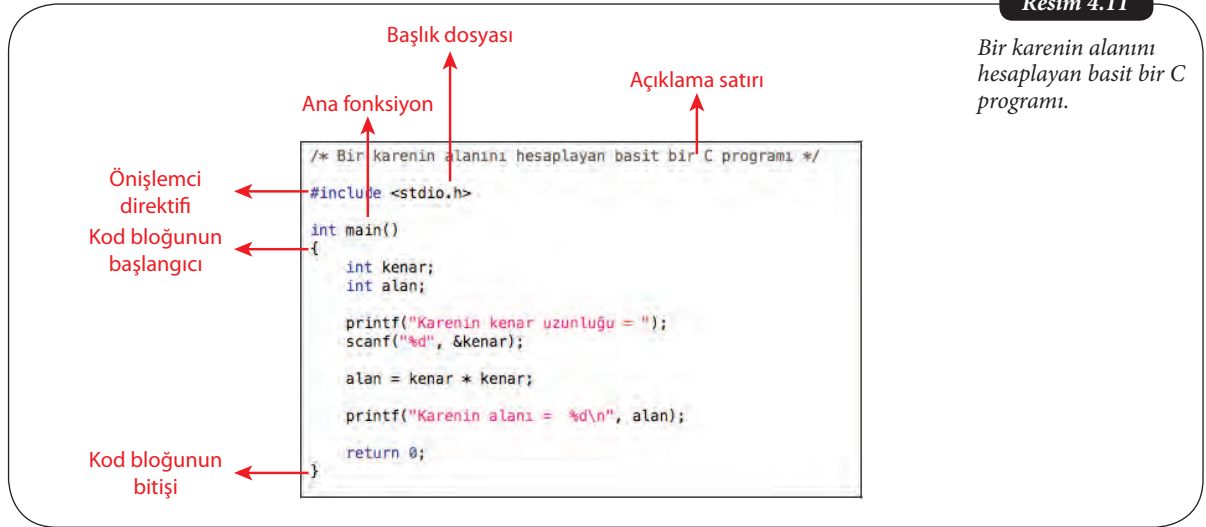


C dilinde ana fonksiyon, başka fonksiyonları çağırabilir. Fonksiyon içinde fonksiyon çağırarak da mümkündür.

Basit Bir C Programı

Resim 4.11'de bir karenin alanını hesaplayan, basit bir C programı gösterilmiştir. Bu program, kullanıcının ekrana karenin kenar uzunluğunu girmesini bekler. Girilen kenar uzunluğuna göre, karenin alanını hesaplar ve ekrana yazdırır. Kaynak kodun üzerinde kırmızı ile çizilen oklarla programın yapısını oluşturan temel bileşenler gösterilmiştir.

Resim 4.11



Bir karenin alanını hesaplayan basit bir C programı.

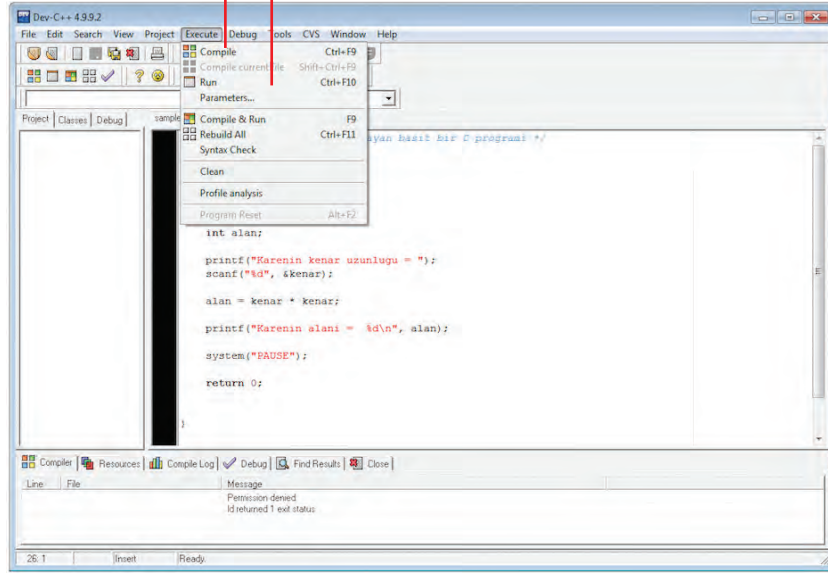
Bu bölümde gösterilen basit C programı, bir derleyici ile derlenmiştir. Derleme işlemi sonucunda, çalıştırılabilir bir dosya oluşturulmuş ve program bu dosya aracılığıyla çalıştırılmıştır. Programın derlenmesi için Dev-C++ kullanılmıştır.

Resim 4.12

Dev-C++ kullanarak programın derlenmesi ve çalıştırılması.

Kodun derlenmesi için gerekli komut

Programın çalıştırılması için gerekli komut

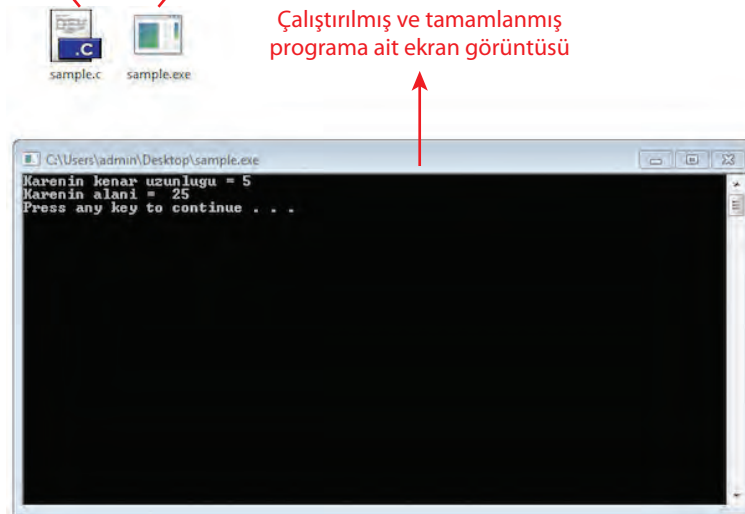
**Resim 4.13**

Programın çalıştırılması ve tamamlanması.

Programın kaynak kodunun saklandığı dosya

Çalıştırılabilir dosya (EXE)

Çalıştırılmış ve tamamlanmış programa ait ekran görüntüsü



Özet



Programlamanın temel unsuru olan algoritma kavramını açıklamak.

Algoritma, bir işin nasıl yapılacağını bildiren adımlar kümesidir. Bir algoritma, yapılacak işe ait açıklamalardan ve işin yapılmasında takip edilecek adımlardan meydana gelir. Algoritmalar, yalnız programlamada değil, günlük hayatımızda da sıklıkla karşımıza çıkar. Basit bir yemek tarifi veya bir elektronik cihazın kullanma kılavuzu, günlük hayatta karşımıza çıkan algoritmalara örnektir.

Algoritmaların girdi ve çıktı bilgileri olmalıdır. Algoritmaların adımları net bir şekilde tanımlanmalı, her girdi için doğru bir çıktı elde edilmelidir. Algoritmanın adımları gerçekleştirilirken algoritma sonsuz döngüye girmemeli, algoritma mutlaka sonlanmalıdır. Algoritma, belirli kriterler doğrultusunda verimli çalışmalı ve aynı tür problemlere uygulanabilir olmalıdır.



Algoritmaların gösterim yöntemlerini listelemek.

Algoritmaların gösteriminde çeşitli yöntemler bulunur. En sık kullanılan gösterim yöntemleri, konuşma dili ile gösterim, akış diyagramları ile gösterim ve sözde kod ile gösterimdir.

Bir algoritmanın açıklaması ve algoritmada yer alan adımlar, konuşma dili kuralları çerçevesinde gösterilebilir. Bu gösterim şeklinde, algoritma açık ve kesin bir dille tanımlanır. Algoritmada yer alan adımlar liste halinde yazılır.

Akış şeması, algoritmaların gösterimi için ideal bir yöntemdir. Bir akış şemasında algoritma adımlarını ifade eden kutucuklar, adımlar arası geçişleri gösteren oklar, karar verme mekanizmaları olarak kullanılan şekiller bulunur. Bu görsel öğeler sayesinde algoritmaların anlaşılması daha kolay hale gelir.

Sözde kod, bir algoritma veya program oluşturulurken kullanılan, konuşma diline benzeyen ve programlama dillerinin detaylarından uzak anlatımlardır. Programlama mantığı ile konuşma dili cümlelerinin bir araya getirilmesiyle sözde kodlar oluşur. Bu kodlar, herkes tarafından kolaylıkla anlaşılabilir. Sözde kodu okuyan bir kişi, programlama dillerinin detaylarına takılmadan, algoritmanın çalışma mantığını anlayabilir.



Alt seviye programlama dillerinden yüksek seviye programlama dillerine geçişi ifade etmek.

Bilgisayarlar, yalnızca makine dilinde yazılmış programları çalıştırabilir. Makine dili komutlarının insanlar tarafından okunması ve anlaşılması zordur. Bu dili kullanarak binlerce satırlık programlar yazmak oldukça zahmetli ve hataya müsait bir iştir.

Makine dilinden kaynaklanan zorlukların üstesinden gelebilmek için çevirici diller geliştirilmiştir. Çevirici dillerde program komutları kısa kelimelerle gösterilir. Bu dillerde yazılmış programlar, çevirici olarak bilinen özel programlar sayesinde makine diline dönüştürülür ve program komutları çalıştırılır.

Çevirici dil kullanarak program geliştirmek için merkezi işlem biriminin yapısına iyi derecede hakim olmak gerekir. Makine dilinde olduğu gibi, çevirici dillerde yazılan programlar da çok fazla komut içerir. Alt seviye programlama dillerinde yaşanan zorlukların önlenmesi için yüksek seviye programlama dilleri oluşturulmuştur. Bu dillerde program yazmak için merkezi işlem biriminin işleyişinin bilinmesine gerek duyulmaz. Yazılan programlardaki komut sayısı, alt seviye dillerdeki programlara göre oldukça azdır. Yüksek seviye programlama dilleri, güçlü ve karmaşık programların kolaylıkla geliştirilmesini sağlar.



Derleyici ve yorumlayıcı kavramlarını tanımlamak ve bu iki kavram arasındaki farkları açıklamak.

Merkezi işlem birimi, yalnızca makine dili komutlarını algılayabilir. Dolayısıyla, yüksek seviye programlama dili kullanılarak yazılan programların makine diline çevrilmesi gerekmektedir. Bu çevrimin yapılabilmesi için derleyiciler veya yorumlayıcılar kullanılır. Derleyiciler, yüksek seviye programlama dili kullanılarak yazılan bir programı makine diline çeviren bilgisayar programlarıdır. Bir program, bir defa derlendikten sonra, tekrar derlenme ihtiyacı duymaz. Derleme işlemi sonucunda elde edilen makine dili programı, istenildiğinde merkezi işlem birimi tarafından çalıştırılabilir.

Yorumlayıcılar, yüksek seviye programlama dili kullanılarak yazılan bir programdaki komutları dönüştürmekte ve çalıştırmakta kullanılır. Program komutları, yorumlayıcı tarafından okunur ve makine dili komutlarına dönüştürülür. Yorumlayıcı tarafından dönüştürülen her komut doğrudan çalıştırılır. Bu işlem, programdaki her komut için tekrarlanır.

Derleyiciler ve yorumlayıcılar arasındaki temel farklar:

- Derleyiciler, bir programın tamamını makine diline çevirirken, yorumlayıcılar program komutlarını ayrı ayrı makine komutlarına dönüştürür.
- Derleyicilerde kaynak kodun analizi uzun sürer ama program hızlı çalışır. Yorumlayıcılarda analiz süresi düşüktür ama program yavaş çalışır.
- Derleyicilerde programın tamamı tarandıktan sonra, var olan hata mesajları ortaya çıkar. Yorumlayıcılarda ise bir hata ile karşılaşılana kadar dönüştürme işlemi devam eder.
- Derleyicilerde orta seviyeli nesne kodları üretilirken, yorumlayıcılarda böyle bir üretime ihtiyaç yoktur. Bu farklılıktan dolayı, derleyicilerin bellek ihtiyacı daha fazladır.



C programlama dilinin temel özelliklerini uygulamak.

C programlama dili, genel amaçlı bir programlama dilidir. İşletim sistemleri, derleyiciler ve hata ayıklayıcılar gibi aşağı seviyeli sistem programlarının geliştirilmesinde yoğun olarak C dili kullanılır. Günümüz işletim sistemlerinin neredeyse tamamında bu dil kullanılmaktadır. C dili, ANSI ve ISO kurumları tarafından standart hale getirilmiştir.

C dili, taşınabilir bir programlama dilidir. Bu dilde yazılmış bir program, herhangi bir değişikliğe uğramadan veya çok az bir değişimle, geliştirildiği ortamdan farklı bir ortamda derlenebilir ve çalıştırılabilir.

C dili, yapısı itibarıyla güçlü ve esnek bir dildir. İyi bir yazılım geliştirme ortamına sahip olan bu dil, özel komut ve veri tipi tanımlamasına izin verir. Geliştirilen C kodları, fonksiyonlardan meydana gelir. Yazılan bu fonksiyonların, tekrar tekrar kullanımı ve taşınabilmesi mümkündür.

Bir komutun büyük veya küçük harfle yazılması, C dilinin derleyicileri için farklılık belirten bir faktördür. Dildeki tüm komutlar ve anahtar kelimeler küçük harflerle yazılır. Programlar hakkında açıklamalar, kodun içerisinde yer alabilir. Birden fazla satırdan oluşabilen açıklamalar, derleyici tarafından göz ardı edilir ve işlenmez.

C dilinde 32 adet anahtar kelime vardır. Bu kelimeler, derleyiciler için özel anlam taşıyan programlama elemanlarıdır. Anahtar kelimeler, yazılan programlarda tekrar tanımlanamaz ve değişken olarak kullanılamaz. Program kodlarının oluşturulması için kullanılan operatörler, C dilinde dört ayrı gruba ayrılır. C dilinde kullanılan operatörler mantıksal, matematiksel, ilişkisel ve atama operatörleridir.



C programlarının yapısını çözümlemek.

C dilinde yazılan programlarda iki temel bileşen bulunur. Bu bileşenler, önışlemci direktifleri ve ana fonksiyondur.

Önışlemci direktifleri, derleyicinin bazı özel işlemleri gerçekleştirmesini sağlar. Derleme işleminin ilk aşamasında, önışlemci derleyiciye bu direktifleri ulaştırır. Her önışlemci direktifinin farklı bir görevi vardır. Önışlemci direktifleri # işareti ile başlar. #include ve #define, en sık kullanılan önışlemci direktifleridir.

Yapısal bir dil olan C dilinde, programlar fonksiyonlardan oluşur. Her programın en az bir fonksiyonu olmak zorundadır. Programlarının hepsinde main() fonksiyonu bulunur. Programın ilk çağrılan fonksiyonu olan ana fonksiyon, programda gövde görevini görür. Programda kullanılacak diğer fonksiyonlar, ana fonksiyon tarafından çağrılır. Bir fonksiyonun içerisinden başka bir veya birkaç fonksiyonun çağrılması mümkündür.

Kendimizi Sıyalım

1. Bir işin yapılması için gerekli adımları içeren kümeye ne ad verilir?
 - a. Algoritma
 - b. Program
 - c. Yazılım
 - d. Girdi
 - e. Çıktı
2. Algoritma özellikleri ile ilgili verilen bilgilerden hangisi doğru **değildir**?
 - a. Bir algoritma, aynı türdeki problemlerin hepsine uygulanabilmelidir.
 - b. Algoritmalar sonsuz döngüye girebilmelidir.
 - c. Algoritma adımları açık bir şekilde belirlenmelidir.
 - d. Bir algoritma, dışarıdan girdi alabilmelidir.
 - e. Bir algoritma, dışarıya çıktı verebilmelidir.
3. Aşağıdakilerden hangisi, yüksek seviye programlama dillerine duyulan gereksinimin sebeplerinden biri **değildir**?
 - a. Makine kodlarının anlaşılmasının zor olması.
 - b. Alt seviye diller ile yazılan programların çok hızlı çalışması.
 - c. Çevirici diller ile program yazmanın hataya açık olması.
 - d. Alt seviye diller ile yazılan programların çok fazla komut içermesi.
 - e. Programcılarının işini kolaylaştırma gerekliliği.
4. Yüksek seviye programlama dili ile yazılmış bir programın tamamını makine dili programına çeviren program, hangi seçenekte belirtilmiştir?
 - a. Kaynak kodu
 - b. Hata ayıklayıcı
 - c. Metin düzenleyici
 - d. Derleyici
 - e. Yorumlayıcı
5. Derleyici ve yorumlayıcı arasındaki farkları belirten ifadelerden hangisi doğru **değildir**?
 - a. Derleyicilerin bellek ihtiyacı, yorumlayıcıların bellek ihtiyacından fazladır.
 - b. Yorumlayıcılar program komutlarını ayrı ayrı makine kodlarına dönüştürürken, derleyiciler programın tamamını makine diline çevirir.
 - c. Yorumlayıcılarda orta seviyeli nesne kodları üretilirken, derleyicilerde böyle bir üretim yapılmaz.
 - d. Kodun analiz edilmesi, derleyicilerde daha uzun sürer.
 - e. Derleyicilerin çalışma hızı, yorumlayıcıların çalışma hızından yüksektir.
6. Aşağıdakilerden hangisi yüksek seviye programlama dillerinden **değildir**?
 - a. C
 - b. C++
 - c. Java
 - d. Python
 - e. Assembly
7. C programlama dilinin özellikleri göze alındığında, aşağıda belirtilen ifadelerden hangisi geçerli **değildir**?
 - a. Yapısal bir dil olan C, fonksiyonlardan oluşur.
 - b. C dilinde özel komut ve veri tipi tanımlamaları yapılabilir.
 - c. C programlama dili, büyük ve küçük harflere karşı duyarlıdır.
 - d. Bir işletim sisteminde derlenen C kodu, farklı bir işletim sisteminde derlenemez.
 - e. Program kodları içerisinde yer alan açıklamalar, birden fazla satırdan oluşabilir.
8. Aşağıdakilerden hangisi C programlama dilinin ilişkisel operatörlerinden **değildir**?
 - a. Büyük eşit ($\geq$)
 - b. Küçük eşit ($\leq$)
 - c. Toplayarak atama ($+=$)
 - d. Eşitlik karşılaştırma ($==$)
 - e. Eşit değil ($!=$)
9. Bir C programını oluşturan iki ana bileşen, hangi seçenekte belirtilmiştir?
 - a. Derleyici ve ana fonksiyon
 - b. Derleyici ve anahtar kelimeler
 - c. Ana fonksiyon ve anahtar kelimeler
 - d. Önışlemci direktifleri ve operatörler
 - e. Önışlemci direktifleri ve ana fonksiyon
10. Aşağıda belirtilen anahtar kelimelerden hangisi, kategori olarak diğerlerinden farklı bir gruba aittir?
 - a. for
 - b. double
 - c. break
 - d. while
 - e. switch

Kendimizi Sınavalım Yanıt Anahtarı

1. a Yanıtınız yanlış ise “Algoritmalar” konusunu yeniden gözden geçiriniz.
2. b Yanıtınız yanlış ise “Algoritmalar” konusunu yeniden gözden geçiriniz.
3. b Yanıtınız yanlış ise “Yüksek Seviye Programlama Dilleri” konusunu yeniden gözden geçiriniz.
4. d Yanıtınız yanlış ise “Yüksek Seviye Programlama Dilleri” konusunu yeniden gözden geçiriniz.
5. c Yanıtınız yanlış ise “Yüksek Seviye Programlama Dilleri” konusunu yeniden gözden geçiriniz.
6. e Yanıtınız yanlış ise “Yüksek Seviye Programlama Dilleri” konusunu yeniden gözden geçiriniz.
7. d Yanıtınız yanlış ise “C Programlama Dili” konusunu yeniden gözden geçiriniz.
8. c Yanıtınız yanlış ise “C Programlama Dili” konusunu yeniden gözden geçiriniz.
9. e Yanıtınız yanlış ise “C Programlama Dili” konusunu yeniden gözden geçiriniz.
10. b Yanıtınız yanlış ise “C Programlama Dili” konusunu yeniden gözden geçiriniz.

Sıra Sizde Yanıt Anahtarı

Sıra Sizde 1

İnsanlar, günlük yaşamlarının her alanında algoritmalarla karşılaşır. ATM’den para çekme, çamaşır makinesiyle çamaşır yıkama, arabaya benzin doldurma, yemek yapma gibi günlük faaliyetler bile bir algoritma ile ifade edilebilir. Nakit paraya ihtiyacımız olduğunda, bir banka ATM’sinden para çekeriz. Bu olayı bir algoritma ile aşağıdaki şekilde ifade edebiliriz:

- Hesabınız olan bankanın en yakın ATM’sine gidin.
- ATM kuyruğuna girin, sıra size gelene kadar bekleyin.
- Sıra size gelince, kartınızı makineye yerleştirin.
- Şifre ekranına şifrenizi doğru olarak girin.
- Para çekme menüsüne girin.
- Hesabınızda bakiyeden yeterli miktarı seçin.
- Paranızı ve kartınızı alın.
- Kuyruktan çıkarak işleminizi sonlandırın.

Sıra Sizde 2

Modern programlama döneminde kullanılan birçok yüksek seviye programlama dili vardır. Bu dillerden yaygın olarak kullanılanlardan bazıları C, C++, C#, Java, JavaScript, Perl, PHP, Ruby, Python, Objective-C ve Prolog’dur.

Ekrana yazı yazdırmak, yüksek seviye programlama dillerinde temel komutlardan bir tanesidir. Bu işlem için, genellikle tek bir satırlık komut yeterli olur. Ekrana “Programlamaya Merhaba” yazdırmak için aşağıdaki komutlar kullanılabilir:

- C → printf(“Programlamaya Merhaba”);
- C++ → cout << “Programlamaya Merhaba”;
- Java → System.out.print(“Programlamaya Merhaba”);
- Perl → print “Programlamaya Merhaba”;

Sıra Sizde 3

Derleyici kullanan bir programlama dili için programın yazım aşaması, program kodunun bir metin düzenleyici aracılığıyla oluşturulmasıyla başlar. Oluşturulan dosya, bilgisayarın sabit diskine kaydedilir. Derleyici programın tamamını okur ve analiz eder. Programın kaynak kodunda bir hata yoksa, derleme işlemi gerçekleşir ve program, makine dili programına çevrilir. Merkezî işlem birimi, makine diline çevrilmiş programı çalıştırır. Programın kaynak kodunda bir hata olduğu takdirde, hata ile ilgili bildirim programcıya iletilir.

Yararlanılan ve Başvurulabilecek Kaynaklar

Deitel, P., Deitel, H. (2009). **“C: How to Program”**, Prentice Hall.

Kochan, S.G. (2004). **“Programming in C”**, Sams Publishing.

Cormen, T.H., Leiserson, C.E., Rivest, R.L, Stein, C. (2001). **“Introduction to Algorithms, Second Edition”**, McGraw-Hill Science/Engineering/Math.

5

Amaçlarımız

Bu üniteyi tamamladıktan sonra;

- C programlama dilindeki veri türlerini ve bu türler arasındaki farkı anlayabilecek,
 - C programlama dilindeki değişkenlere isim verilirken hangi kuralların takip edilmesi gerektiğini açıklayabilecek,
 - C programlama dilindeki değişkenlerin programcı tarafından ilk defa nasıl tanımlanabileceğini anlayabilecek,
 - C programlama dilindeki değişkenlere program içinde ön değerin nasıl verildiğini ifade edebilecek,
 - C programlama dilindeki depolama türlerini anlayabilecek,
 - C programlama dilinde herhangi bir değişkene tercih edilen depolama türünde nasıl tanımlama yapılacağını açıklayabilecek
- bilgi ve becerilere sahip olabileceksiniz.

Anahtar Kavramlar

- Tam Sayı Veri
- Kesirli Sayı Veri
- Dizgi
- Değişken
- Mantıksal Değişken
- Geçici Depolama
- Kalıcı Depolama

İçindekiler

Bilgisayar ve Programlamaya
Giriş

C'de Veri Tipleri ve Tanımlamalar

- Giriş
- C PROGRAMLAMA DİLİNDE DEĞİŞKEN ADLANDIRMA VE TANIMLAMALAR
- C PROGRAMLAMA DİLİNDE DEPOLAMA TÜRLERİ

C'de Veri Tipleri ve Tanımlamalar

GİRİŞ

Veri Nedir?

Programlama ya da diğer adıyla yazılım, bilgisayarın donanıma nasıl davranacağını anlatan, bilgisayara yön veren komutlar, kelimeler, aritmetik işlemlerdir. Diğer bir tanım verecek olursak programlama, bilgisayar programlarının yazılması, test edilmesi ve bakımının yapılması sürecine verilen isimdir.

Programlama, bir programlama dilinde yapılır. Bu programlama dili, Java gibi yüksek seviyede bir dil olabileceği gibi C, assembly ve bazı durumlarda makine dili de olabilir. Yazılan kaynak kodu, genellikle bir derleyici (compiler) yardımıyla belirli bir donanımda çalıştırılabilir hâle getirilir. Ayrıca kaynak kodu, bir yorumlayıcı (interpreter) yardımıyla derlemeye gerek duyulmadan satır satır da çalıştırılabilir. Derleyici, yazılan programları okuyup içerisinde mantıksal veya yazımsal (syntax) hatalar olup olmadığını bulan, bulunduğu hataları kullanıcıya göstererek programın düzeltilmesine yardım eden, hata yoksa programı çalıştırıp sonucunu gösteren birer platformdur. Bu platformda, daha önceden tanımlanmış veri tipleri kullanılmak zorundadır. Bilgisayara girilen işlenmemiş (ham) bilgilere veri denir. Bilgisayarın alabildiği, işleyebildiği, sonuç üretebildiği ve saklayabildiği her şey veridir.

Bazı programlama dilleri veri tiplerini önceden ayırır, bazı programlama dilleri ise ayırmaz. Teknik olarak veri tipini önceden belirleyen programlama dilleri, belleğe yerleşecek verileri, orada kaplayacakları alanlara göre gruplara ayırır. Programlama dilleri, bu veri gruplarını pratikteki kullanımlarına göre adlandırır. Örneğin; karakterler, tam sayılar, kesirli sayılar vb. gibi yalın tipler yanında dizim (array) gibi birleşik veri yapıları bu ayrımı yapan her dilde vardır. Fortran, Pascal, C/C++, Java vb. programlama dilleri veri tiplerini önceden ayıran dillerdir. Python, Ruby gibi programlama dilleri ise bu ayrımı önceden yapmaz; önce veriyi belleğe yerleştirir, sonra onun tipini belirler.

Bu iki yöntemin avantajlı ve dezavantajlı kabul edilebilecek yanları vardır. Verileri önceden tiplerine ayırmanın avantajları şöyle sıralanabilir:

- Her veri tipine yetecek ve ancak o kadar bellek alanı ayrılır.
- Programcı, bellekte ayrılacak adresin büyüklüğünü bilerek değişkenlerini tanımlar.
- Her değişkene, bellekte bir yer ayrılır. Değişken etkin olduğu sürece, o adrese bir başka veri yazılamaz. Programcı her istediğinde değişkenin değerini değiştirebilir, yeni değer atayabilir. Bu işlem değişkenin adresini değiştirmez.

- Her veri tipi üzerinde yapılacak işlemler farklıdır. Örneğin, sayılarda yapılan işlemlerle karakter verilerinde (metinlerde) yapılan işlemler farklıdır. Bu fark gözetenilerek veri tipleri üzerinde işlem tanımlanır.

Bu iyi yönlerine karşın, verileri önceden tiplerine göre ayırmayan dillerin avantajları ise şöyle sıralanabilir:

- Programcı veri tipleriyle uğraşmaz, gerekli verileri girer ve sadece kullanır.
- Aynı adlı değişkene her istediğinde aynı ya da başka tipten veri atanabilir.
- Bir veri belleğe bir kez konulur. O değeri kullanan değişkenler, o değeri ortak kullanır. Böylece belleğe bir veri ancak bir kez girer. Ortaya çıkan bu durum, belleğin çok ekonomik kullanılmasını sağlar.

C programlama dili, verileri önceden tiplere ayıran dillerdendir.

C Programlama Dilindeki Temel Veri Tipleri Nelerdir?

C programlama dilinde toplam üç tane temel veri tipi vardır:

- Sayı
- Karakter
- Dizgi (String)

Bu veri tipleri çeşitli alt gruplara ayrılırlar. Alt grupların oluşturulmasında ortak bir kuraldan söz edilemez. Farklı derleyiciler, farklı sınıflandırmalar ve farklı adlar kullanmıştır. Onları bilinen adlarına ve bellekte kapladıkları büyüklüklere göre sınıflandırmak yeterli olacaktır.

Özet olarak bu sınıflandırma, Tablo 5.1'de verilmiştir. Tablodaki tüm veri tipleri, C programlama dilinde çok kullanıldığı için dile gömülü gelen, dolayısıyla kütüphaneden çağrılması gerekmeyen veri tipleridir.

Tablo 5.1
C programlama dili için
veri tipleri

Veri Türü	Programlama Editöründe Kullanımı
Tam Sayılar	int
Karakterler	char
Mantıksal	false == 0, true != 0
Kesirli Sayılar	float, double
Yapısal Tipler	array, string, functions, struct, files, pointers

Tablo 5.1'de verilen veri tipleri için bir standart yoktur. Ama her programlama dilinde karakterler (character), tam sayılar (integer), kesirli sayılar (float) temel veri tipleridir. Dizim (array), dizgi (string), fonksiyon (function), dosya (file), işaretçi (pointer) vb. veri tipleri birleşik (yapısal) tiplerdir. Bazı programlama dilleri dizgi (string) veriyi de temel veri tipi olarak ele alır. C programlama dilinde ise dizgi (string) veri tipi karakter dizimi olarak tanımlanır.

Sayılar

Sayılar, tam sayılar ve kesirli sayılar diye ikiye ayrılır. Onlar da kendi içlerinde alt gruplara ayrılır. Bu sınıflandırma, sayıya bellekte ayrılacak yerin büyüklüğüne göre değişir. Alt grupların adları ve sayıları için bir standart yoktur. Farklı diller farklı gruplamalar yapar. C programlama dilinde tam sayıların ve kesirli sayıların alt grupları Tablo 5.2 ve Tablo 5.3'te verilmiştir.

Tam Sayı Veri Tipleri

Tablo 5.2'de tam sayılar gruplanırken aynı bellek büyüklüklerine karşılık gelen farklı adlar verilmiştir. Bu durum, çok sayıda C derleyicisi olmasına ve donanım niteliklerine bağlı

olarak oluşmuştur. Örnek olarak, bazı kişisel bilgisayarlarda (PC) derleyiciler `int` veri tipine 2 byte ayırırken bazı işletim sistemleri ise 4 byte ayırır.

Tam sayılar için “`int`” anahtar sözcüğü kullanılır. Bu anahtar sözcüğün iki alt grubu “`long`” ve “`short`” diye adlandırılır. Bazı derleyiciler ise “`long int`” ve “`short int`” terimlerini kullanmayı tercih eder. Yukarıdaki paragrafta da bahsedildiği gibi, derleyiciye ve donanımına bağlı olarak değişmekle birlikte, `short` tam sayılar için en az 16 bit (2 byte), `long` tam sayılar için en az 32 bit (4 byte) bellek alanı ayrılır. “`int`” veri tipi, bazı donanımlarda 16 bit, bazı donanımlarda ise 32 bit olabilir. Ama hiç bir derleyici veya donanım “`short`” veri tipine “`int`” tipe ayırdığından fazlasını ayırmaz.

Aksi ifade edilmedikçe, bütün tam sayı tipleri işaretli (signed) sayılır; yani hem negatif hem de pozitif değerler alırlar. “`char`” tipler, derleyiciye bağlı olarak işaretli ve işaretsiz olabilirler. O nedenle “`signed char`” ve “`unsigned char`” diye belirtmek en uygunu olur.

Tam sayılara ait C programlama dilinde tanımlama örnekleri şu şekildedir:

```
char ders sube;
unsigned char harf;
signed char ad;
short int prog sinav notu;
long int acikogretim ogr no;
short x, y;
long mesafe;
unsigned char klavye tusu;
unsigned int boy;
unsigned short ev kapi no;
unsigned long hesap no;
```

Veri Türünün Adı	Veri Türü	Bellekte Kapladığı Alan	Alabileceği Değer Aralığı
char	Karakterler	1 Byte	-128'den 127'ye kadar
signed char	Karakterler	1 Byte	-128'den 128'e kadar
short int	Küçük Tam Sayılar	2 Byte	-32768'den 32767'ye kadar
long int	Küçük Tam Sayılar	4 Byte	-2147483648'den 2147483647'ye kadar
short	Küçük Tam Sayılar	2 Byte	-32768'den 32767'ye kadar
long	Büyük Tam Sayılar	4 Byte	-2147483648'den 2147483647'ye kadar
unsigned char	İşaretsiz Karakterler	1 Byte	0'dan 255'e kadar
unsigned int	İşaretsiz Tam Sayılar	2 Byte	0'dan 65535'e kadar
unsigned short	İşaretsiz Küçük Tam Sayılar	2 Byte	0'dan 65535'e kadar
unsigned long	İşaretsiz Büyük Tam Sayılar	4 Byte	0'dan 4294967295'e kadar
enum	Numaralanmış Sayılar	2 Byte	0'dan 65535'e kadar

Tablo 5.2
C programlama dilinde tam sayı veri tipleri

Bu arada, sayıların alt gruplara ayrılışının standart olmadığı unutulmamalıdır. C program dilinde durum böyle iken Java programlama dilinde gruplandırma ve grup adları kesindir, donanıma ya da derleyiciye göre değişmez. Tam sayılar 8, 16, 32 ve 64 bitlik oluşlarına göre dört alt gruba ayrılır. Python'da programcı, sayıları alt gruplara ayırmakla uğraşmaz.

Kesirli Sayı Tipleri

C programlama dilinde kesirli sayılar (fractional numbers) üç gruba ayrılır:

- float
- double
- long double

Peki bu kesirli sayılar ne demektir? Kesirli sayılar, kayan nokta (floating point) gösterimi diye adlandırılan yöntemle farklı biçimlerde (format) yazılabilir. Örnek verilerek istenirse

$$123,4567 = 1,234567 \times 10^2 = 12345,67 \times 10^{-2}$$

sayıları arasında hiçbir fark yoktur. Virgül ayıracını dilediğimiz yere kaydırabiliriz. Matematik ilminde, kesirli olsun veya olmasın her sayı bir gerçel sayıdır (reel sayı). Fakat bilgisayarlardaki sayıların büyüklükleri sonludur.

Özetle, ondalık içeren sayılarda (kesirli sayı) “float” kullanılır. Sayımız küsuratlıysa veya başta tam sayı ama sonrasında değiştirdiğimizde küsuratlı olma ihtimali varsa float kullanırız. Örneğin, 2,54 sayısı; 1,921352 sayısı; 123951,231 sayısı.

Diğer bir kesirli veri cinsi de “double” olarak isimlendirilir. Eğer küsurlu bir sayı kullanıyorsak, mümkün olduğunca double kullanmayı tercih etmeniz önerilmektedir. Eğer üzerinde çalıştığımız veride ondalık basamak kısmında çok hassas davranılması gerekiyorsa en fazla ondalık basamağa imkân veren “long double” veri türü tercih edilmelidir. Tablo 5.3'te ondalık (kesirli) sayıların alt grupları ve kaç haneye kadar ondalık basamağı içerisinde barındırabildikleri verilmiştir.

Tablo 5.3
C programlama dilinde
ondalık (kesirli) sayı
veri tipleri

Veri Türünün Adı	Veri Türü	Bellekte Kapladığı Alan	Alabileceği Değer Aralığı
float	Az sayıda ondalık basamaklı kesirli sayılar	4 Byte	3.4E +/-38 (7 haneli)
double	Orta miktarda ondalık basamaklı kesirli sayılar	8 Byte	3.4E +/- 308 (15 haneli)
long double	Çok sayıda ondalık basamaklı kesirli sayılar	10 Byte	1.7E +/- 4932 (19 haneli)

Tablo 5.3'te “3.4E +/-38 (7 haneli)” demek, $3,4 \times 10^{\pm 38}$ demektir. Yani E harfi, genellikle e^x veya $\exp(x)$ sembolleriyle gösterilen ve yaklaşık değeri 2,718 olan Euler sayısını temsil eder. Haneden kastedilen ise kesirli sayıdaki ondalık basamak sayısıdır. Mesela, “1,2345678” sayısı 7 adet ondalık haneye sahip olan kesirli bir sayıdır.

Karakterler

Klavyede her tuşa dokunulduğunda bir karakter yazılmış olur. Karakterler, C programlama dilinde “char” veri türü ile tanımlanır. Örneğin,

```
char ch1, ch2;
```

iki tane char değişkeni tanımlar. Bir karakter sabiti tek tırnak içine yazılan bir karakterdir.

Örneğin,

```
ch1 = 'A';
ch2 = 'b';
```

atama ifadeleri ch1 değişkeninin içine A karakterini ve ch2 değişkeninin içine ise b karakterini koyar. Bu değişken isimlendirme ve değişkene bir sabit atama konularını ilerleyen sayfalarda başka örneklerle de anlatacağız. Boşluk da bir karakterdir. Boşluk karakteri ' ' karakteri sabiti ile ifade edilebilir. Her bir rakam da bir karakter olabilir. '0', '1', '2', ..., '9' rakamların karakter olarak gösterimidir. Rakam karakterleri, normal tam sayı rakamlarından tamamen farklıdır. Onlar bilgisayar tarafından tamamen iki farklı değer olarak yorumlanır. Örneğin, '1' ile gösterilen bir rakamının karakter hâli tam sayı 1'den farklıdır. Birincisi bir karakter, ikincisi ise bir tam sayı olarak C programlama dili tarafından yorumlanır. Aşağıdaki gibi bir karakter değişkene bir tam sayı koymaya kalkarsak C derleyicisi hata vermese de ch1 değişkeni istediğimiz '5' karakteri yerine ASCII tablosundaki 5. karaktere denk gelen karakteri içerecektir.

```
ch1 = 5;
```

Bütün bastırılabilir karakterlere ek olarak bastırılamaz karakterler de vardır. Bu özel karakterlerin sabitleri bir escape (kaçış) karakter kullanılarak verilir. Kullanılan escape karakteri '\ ' karakteridir. Bazı özel karakterler ve onların sabitleri aşağıda verilmiştir:

- '\n' : Return ya da satır sonunu gösteren karakter. Klavyeden return ya da enter tuşuna basıldığında gönderilen karakterdir. Bu karakteri ekrana yazdırmaya kalktığımızda bir sonraki satıra geçilecektir.
- '\t' : Klavyeden tab tuşuna basıldığında gönderilen karakterdir. Eğer bu karakteri ekrana yazmaya kalkarsak belirli miktarda boşluk bırakacaktır.
- '\"' : Tek tırnak karakteri ' '.
- '\\ ' : Ters yatay çizgi (backslash) karakteri, yani \.

C programlama dilinde bütün karakterlerin 0 ile 255 değeri arasında değişen bir kodu vardır. Diğer bir deyişle C programlama dilinde 256 değişik karakter olabilir. C programlama dilinde kullanılan bu kodlama sistemi, ASCII kodlama sistemidir. ASCII kodlama sisteminde kullanılan ilk 128 karakter, İngilizce harfler, rakamlar ve çok kullanılan diğer bazı karakterlerden oluşur. Bu kodlama sistemi ile aynı zamanda bütün karakterler bir sıraya konmuş olurlar. Bir karakterin ASCII kodunu bulabilmek için o karakteri "int" veri türüne çeviririz. Örneğin,

```
ch = (int) 'A';           // ch'in değeri 65 olacaktır.
ch = (int) 'Z';           // ch'in değeri 90 olacaktır.
ch = (int) 'a';           // ch'in değeri 97 olacaktır.
ch = (int) 'z';           // ch'in değeri 122 olacaktır.
ch = (int) '0';           // ch'in değeri 48 olacaktır.
ch = (int) '9';           // ch'in değeri 57 olacaktır.
ch = (int) ' ';           // ch'in değeri 32 olacaktır.
```

Burada da görüldüğü gibi, bu kodlama sisteminde 'A' karakteri 'Z' den önce gelmektedir (65 < 90) ve bütün büyük harfler sıralamada küçük harflerden önce gelmektedir. Rakamlar da büyük harflerden önce gelmektedirler. İki karakter değeri karşılaştırılırken onların ASCII kodlarına bakılır. Eğer ASCII kodları birbirine eşit ise o iki karakter birbirine eşittir.

Dizgiler (Strings)

Dizgiler karakterlerden oluşan dizilerdir. Karakter dizilerinin her bir elemanı da bir karakterdir. Diğer dizilerde olduğu gibi, bir dizginin de her bir elemanına ayrı ayrı erişebiliriz. Bir dizginin parçalarına erişebiliriz, bir karakterin bir dizgi içindeki yerini bulabiliriz ve buna benzer birçok işlemi dizgiler üzerinde yapabiliriz.

C programlama dilinde dizgileri bir dizi tanımlar gibi tanımlarız. Örneğin,

```
char ad soyad[20];
```

bildirimi ile ad_soyad dizgisi tanımlanıyor. Bu dizgi, indisleri 0 ile 19 arasında değişen herhangi bir karakter dizisini saklayabilir. Bu tanımlama ile derleyici bellekte toplam 20 karakterlik bir bölge ayıracaktır. ad_soyad olarak adlandırdığımız değişken de bu ayrılan bellek bölgesinin başlangıç adresini tutacaktır. Bu bölgedeki ilk karaktere 0, ikinci karaktere 1, üçüncü karaktere 2 vb. indisler ile erişebiliriz. Bu bölgede en çok 19 karakterlik bir dizgi tutabiliriz. Çünkü C programlama dilinde her dizginin sonunu belirtmek için dizginin en son karakterinden sonra '\0' (boş karakter) konur. Bu karakter de bellekte bir elemanlık yer kaplar. Dizgilerin kullanım alanları çok geniş olduğu için C programlama dilinde diğer veri türündeki dizilerden farklı olarak dizgilerin ilk değerleri tanımlama sırasında kısaca iki çift tırnak içinde belirtilebilir ("abc" gibi). Örneğin,

```
char ornek[15] = "bir dizgi";
```

ifadesi ile en çok 14 karakter uzunluğunda ornek adlı bir dizgi tanımlayıp ilk değer olarak "bir dizgi" sözcüklerini verebiliriz. Bu durumda ornek dizgisinin bellekteki görünümü aşağıdaki gibi olacaktır:

ornek

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
b	i	r	'	d	i	z	g	i	'\0'					

Bu dizgide, 10 ve 14. indisler arası beş indis mecburen boş kalacaktır.

Bu şekilde de görüldüğü gibi ornek dizgisinin 9 indisli 10. elemanı '\0' yani boş karakterdir (null character). C programlama dilinde bütün dizgilerin sonu bu özel karakter ile belirtilir. C programlama dilindeki dizgilerle ilgili bütün işlevler boş karakterden sonraki elemanlarda olan değerleri hiç dikkate almazlar.

ornek dizgisinin her elemanına indisini kullanarak erişebiliriz. Örneğin ch "char" veri türünde bir değişken ise

```
ch = ornek[2];
```

atama ifadesiyle değerini 'r' yapabiliriz.

İki boyutlu karakter dizileri tanımlayarak dizgi dizileri de tanımlanabilir. Örneğin,

```
char hafta_gunleri [7][10] = {'Pazartesi', 'Salı', 'Carsamba', 'Perseme', 'Cuma', 'Cumartesi', 'Pazar'};
```

her elemanı en çok 10 karakter uzunluğunda bir dizgi olan 7 elemanlı bir dizi tanımlar ve bu diziye ilk değerlerini verir.

SIRA SİZDE



Dizgilerin kullanım alanları çok geniştir. C programlama dilinde diğer veri türündeki dizgilerden (array) farklı olarak dizgilerin ilk değerleri tanımlama sırasında kısaca iki çift tırnak içinde belirtilebilir ("deneme" gibi). Fakat tanımlayacağımız dizginin boyutu, tanımlanan ilk değerinden küçük olursa ne olur? Örneğin,

```
char mobilya[5] = "sandalye";
```

ifadesi ile tanımlanan dizginin bellekteki görünümü nasıl olacaktır?

C PROGRAMLAMA DİLİNDE DEĞİŞKEN ADLANDIRMA VE TANIMLAMALAR

Değişken İsimlendirilmesi Nedir?

Programcı nasıl bir program yazıyor olursa olsun, programın tasarlanması aşamasında en çok dikkat etmesi gereken faktörlerden biri değişken tanımlamalarıdır. Değişkenin tipi-

nin ve isminin yanlış seçilmesi o an için bir problem teşkil etmeyebilir ancak aradan belli bir zaman geçtikten sonra çok büyük sorunlara neden olabilir. Öyle ki kesirli işlemler (floating) yapılacağı hâlde birtakım değişkenler tam sayı (integer) belirlenmişse ve test aşamasında kesirli sonuçlar vermeyen değerler girildiğinde o an için program düzgün çalışıyormuş gibi gözükabilir. Ayrıca değişkenlere anlamsız isimler verildiğinde yine o an için bir sorun çıkmasa da programcının bir süre sonra yeniden aynı programla uğraşmaya başladığında hangi değişkenin ne amaçla kullanıldığını bulabilmek için programı başından sonuna adım adım takip etmesi gerekebilir. Büyük bir programlama projesinde karmaşa yüzünden adım adım takip de mümkün olmayabilir. Bu tür durumlarla karşılaşmamak için değişkenlere her zaman anlamlı isimler vermek daha doğrudur. O anda 'x yolu' diye aklınızdan geçen bir şey varsa o değişkene 'x' adını vermek yerine 'x yolu' demek sonrası için daha iyi olacaktır.

Değişken İsmiendirilmesi Kuralları

Değişkenler, bilgisayarın belleğinde bilginin saklandığı gözlere verilen sembolik adlardır. Bir C programında, bir değişken tanımlandığında bu değişken için bellekte bir yer ayrılır. Her değişkenin tuttuğu değer nasıl bir veri olduğunu gösteren (önceki bölümde anlatılan) bir veri tipi vardır.

C programlama dilinde, değişkenler ve sabitler programın başında bulunmalıdır. Bazı uygulamalarda değişkenin bir başlangıç değerinin olması istenir. Böyle durumlarda değişken bildirilirken başlangıç değeri verilebilir. Örneğin:

```
char isim = 'X', z; /* değer atamak zorunlu değil */
int sayi = 0, n;
float toplam = 0.0, sonuc = 22.14;
```

C programlama dilinde kullanılan değişken isimlendirme kuralları aşağıdaki gibi sıralanmıştır:

- C programlama dili komutları değişken adı olarak verilemez (Örneğin; if, for, case gibi).
- Değişken ismi birden çok kelimeden oluşacaksa aralarına boşluk verilmez. Örneğin, "maksimum sayı" adında bir değişken tanımlanamaz ama "maksimum_sayı" ya da "MaksimumSayı" gibi değişken tanımlanabilir.
- Değişken adları en fazla 32 karakterden oluşabilir. 32 karakterden uzun değişken adları ilk 32 karakter değerlendirilir. Geriye kalan karakterler işleme tabi tutulmaz.
- Değişken adları İngiliz alfabesinde bulunan karakterler (A-Z) veya (a-z) ya da rakamlar (0-9) ile yazılmalıdır. Türkçe karakterler, özel karakter veya boşluk karakteri kullanılamaz.
- Değişken adları herhangi bir rakam ile başlayamaz. İlk karakter bir harf olmalıdır. Sonrakiler ise rakamlardan oluşabilir.
- Aşağıda verilen kelimeler ANSI C'nin anahtar kelimeleridir (key words) ve değişken ismi olarak kullanılamaz:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Değişken İsimlendirilmesi Örnekleri

Örnek 1:

```
int x;  
int a,b,c;  
float OrtalamaNot;
```

Eğer aynı değişkenlere bir başlangıç değeri atayarak programda kullanmak istiyorsak, değişkenlerin başlangıç değeri atanmış tanımlamaları aşağıdaki gibidir:

```
int x = 5;  
int a = 2, b = 5, c = 1;  
float OrtalamaNot = 50.5;
```

Örnek 2:

```
#include <stdio.h>  
main() {  
    char karakter = 65;  
    printf(' ' %c ' ', karakter);  
}
```

Bu program ekrana 65 sayısının ASCII karakter kodu olan A harfini yazar. Burada yazılan “printf” fonksiyonu ekrana bir değişkenin içerisindeki değeri yazdırmak için kullanılır ve ilerleyen kısımlarda detaylı bir biçimde anlatılacaktır.

Örnek 3:

```
#include <stdio.h>  
main() {  
    char karakter = 'A';  
    printf(' ' %c ' ', karakter);  
}
```

Bu program da ekrana A harfini yazar.

Örnek 4:

```
#include <stdio.h>  
main() {  
    int karakter = (int) 'A';  
    printf(' ' %d", karakter);  
}
```

Bu program ekrana 65 sayısını yazar çünkü karakter değişkeni tam sayı olarak tanımlanmıştır. Burada yazılan “printf” fonksiyonunun içerisinde “%c” yerine “%d” yazılmıştır ve bu farklılığın sebebi de ilerleyen kısımlarda detaylı bir biçimde anlatılacaktır.

Örnek 5:

```
#include <stdio.h>  
main(){  
    float pi = 3,1415;  
    printf(' ' %f ' ', pi);  
}
```

Bu program ekrana 3,1415 sayısını yazar çünkü değişken “float” şeklinde (ondalık sayı) tanımlanmıştır.

Örnek 6:

```
#include <stdio.h>
main(){
    int pi = int (3,1415);
    printf(" %d", pi);
}
```

Bu program ekrana 3 sayısını yazar çünkü değişken “int” şeklinde (tam sayı) tanımlanmıştır. Değişkenin sadece tam sayı kısmını tekrar yazdırır.

Değişkenlerin Ekranda Görüntülenmesi

Temel giriş/çıkış fonksiyonları, bütün programlama dillerinde mevcuttur. Bu tür fonksiyonlar, kullanıcıya ekrana veya yazıcıya bilgi yazdırmasına ve bilgisayara klavyeden veri girişi yapmasına izin verir. Temel giriş/çıkış fonksiyonları kullanılırken stdio.h başlık dosyası (header file) programın başına eklenmelidir. Bu kısımda, öncelikle en çok kullanılan giriş fonksiyonu, “printf” komutu anlatılacaktır.

Standart C programlama dili kütüphanesinde bulunan printf() fonksiyonu, değişkenlerin tuttuğu değerleri, onların adreslerini veya bir mesajı ekrana belli bir düzenle (format) yazdırmak için kullanılan fonksiyondur. Daha önce yazılan örnek programlarda printf() fonksiyonundan yararlanmıştık. Şimdi de bu fonksiyonun nasıl kullanıldığına bakalım.

printf() fonksiyonunun genel yazım biçimi aşağıdaki şekildedir:

```
int printf (const char *format,...);
```

Bu biçim biraz kafa karıştırıcı şekildedir ve genelde programcılıkla ilk defa karşılaşanlar için anlaması oldukça zordur. Bunun yerine basit olarak ekrana Hata oluştu!.. şeklinde bir mesaj yazdırma işlemi yapalım:

```
printf('Hata Oluştı!..');
```

Çoğu zaman ekrana, programda kullanılan bir değişkenin değeri yazdırılmak istenir. Örneğin ekrana bir tam sayı değişkeninin içeriğini bastırmak için,

```
....
int x = 12;
printf('x in değeri %d dir', x);
....
```

kodu yazılması gerekir. Bu program parçasının ekran çıktısı şöyle olacaktır:

x'in değeri 12'dir.

Bu örnekte printf fonksiyonuna iki parametre aktarılmıştır. Birincisi ekranda gösterilecek ve çift tırnaklar arasına yazılan ifadeler, ikincisi ise ekranda sayısal değeri gösterilmek istenen değişkendir (x). printf() fonksiyonu içerisine üç farklı parametre alabilir:

- Düz metin. Örneğin,
printf("Ben okula gitmedim");
- Kontrol karakterleri (escape sequence). Değişkenlerin ve sabitlerin nasıl yazılacağını belirtmek veya imlecin alt satıra geçirilmesi gibi bazı işlemlerin gerçekleştirilmesi için kullanılır. Örneğin,
printf('\tBen okula gitmedim\n');

Bu kontrol karakterleri aşağıda listelenmiştir:

Karakter	Anlamı
\a	Ses üretir (alert).
\b	İmleci bir sola kaydırır.
\f	Sayfayı atla ve bir sonraki sayfanın başına geçirir (formfeed).
\n	Bir alt satıra geçirir (newline).
\r	Satır başı yaptırır (carriage return).

\t Yatay TAB (horizontal TAB).
 \v Dikey TAB (vertical TAB).
 \" Çift tırnak karakterini ekrana yazdırır.
 \' Tek tırnak karakterini ekrana yazdırır.
 \\ \ karakterini ekrana yazdırır.
 %% % karakterini ekrana yazdırır.

- Tip belirleyici (conversion specifier). % işareti ile başlar ve bir veya iki karakterden oluşur (%d gibi). Ekrana yazdırılmak istenen değişkenin veri tipi, % işaretinden sonra belirtilir. Örneğin,

```
printf('x in değeri %d dir', x);
```

Bu tip belirleyici karakterler aşağıda listelenmiştir:

Tip Karakteri	Anlamı	Yazdırılacak Veri Tipi
%c	tek bir karakter	char
%s	karakter dizisi (string)	char
%d	işaretli tam sayı	int, short
%ld	uzun işaretli sayı	long
%u	işaretsiz tam sayı	unsigned int, unsigned short
%lu	işaretsiz büyük tam sayı	unsigned long
%f	az haneli ondalık sayı	float
%lf	çok haneli ondalık sayı	double

Tip karakterlerini kullanarak birden çok veri tipi yazdırılabilir. Örneğin,

```
...
int grade = 60;
float pi = 3.1415;
char ch = 'A';
printf('grade = %d , pi = %f ve ch = %c dir', grade, pi, ch);
...
printf fonksiyonu kullanımı Örnek 7'de verilmiştir.
```

Örnek 7:

```
#include <stdio.h>
main()
{
int a = 2, b = 10, c = 50;
float f = 1.05, g = 25.5, h = -0.1, yuzde;
printf('3 tamsayi : %d %d %d\n', a, b, c);
printf('3 tamsayi [TAB] : %d \t%d \t%d\n', a, b, c);
printf('\n');
printf('3 ondalik sayi (yanyana) : %f %f %f\n', f, g, h);
printf('3 ondalik sayi (altalta) : \n%f\n%f\n%f\n\n', f, g, h);
yuzde = 220 * 25/100.0;
printf('220 nin %%25 i %f dir\n', yuzde);
printf('%%f/%f isleminin sonucu = %f\n', g, f, g / f);
printf('\nprogram sonunda beep sesi cikar...\a');
}
```

Bu örnekteki programın ekran çıktısı:

```
3 tamsayi : 2 10 50
3 tamsayi [TAB] : 2 10 50
```

3 ondalik sayi (yanyana) : 1.050000 25.500000 -0.100000

3 ondalik sayi (altalta) :

1.050000

25.500000

-0.100000

220 nin %25 i 55.000000'dır.

25.500000/1.050000 isleminin sonucu = 24.285715

program sonunda beep sesi çıkar.

printf fonksiyonunun geri dönüş değeri "int" tipindedir. Bu geri dönüş değeri çıktının kaç karakter olduğunu gösterir. Yani, printf fonksiyonu, *format ile tanımlanmış karakter topluluğunun kaç bayt olduğu hesaplar. Bu durum, Örnek 8'deki program ile anlatılmıştır:

Örnek 8:

```
#include <stdio.h>
main()
{
    int karSay;
    int sayi = 1234;
    karSay = printf('Ugurlu sayim = %d\n',sayi);
    printf('Ust satirda karakter sayisi: %d dir\n', karSay);
}
```

Bu örnekteki programın ekran çıktısı ise:

Ugurlu sayim = 1234

Ust satirda karakter sayisi: 20'dir.

Klavyeden Bir Değişkene Veri Girilmesi

Birçok programda ekrana verilerin bastırılmasının yanı sıra klavyeden veri okunması gerekebilir. scanf() fonksiyonu klavyeden veri okumak için kullanılan fonksiyondur. printf() gibi scanf() fonksiyonu da üstte verilen kontrol karakterlerini ve tip belirleyicileri kullanır.

Örneğin klavyeden bir x tam sayısı okumak için,

```
scanf('%d',&x);
```

satırını yazmak yeterli olacaktır. Burada & işareti adres operatörü olarak adlandırılır.

Klavyeden iki farklı sayı okunmak istendiğinde scanf() fonksiyonu şöyle kullanılabilir:

```
scanf('%d %f',&x,&y);
```

veriler klavyeden

16 1.56

ya da

16 1.56

veya

16

1.56

şeklinde girilebilir. Örnek 9'da scanf() fonksiyonunun kullanımı gösterilmiştir.

Örnek 9:

```
#include <stdio.h>
main()
{
    int t;
    float g;
    printf('Bir ondalik sayi girin:'); scanf('%f',&g);
    printf('Bir tamsayi girin: '); scanf('%d',&t);
```

```
printf('\n');
printf('\t %f * %f = %f\n',g,g,g*g);
printf('\t %d * %d = %d\n',t,t,t*t);
}
```

Bu örnekteki programın ekran çıktısı ise:

Bir ondalık sayı girin: 1.34

Bir tamsayı girin : 12

1.340000 * 1.340000 = 1.795600

12 * 12 = 144

enum Anahtar Sözcüğü

enum anahtar sözcüğü İngilizcedeki enumerate sözcüğünün kısaltmasıdır. Bir grup sabiti sıralayıp sıra numarası verme eylemini yapar. Örneğin,

```
enum arkadaşlar {ALI, VELI, AYSE};
```

enum deyimi { } içindeki üç arkadaşın sıra numarası verir. Özel sıra numarası verilmemişse ilk sabite 0 numarası verilir. Sonrakiler, yazılış sırasına göre 1'er artarak numara alır. Buna göre ALI = 0, VELI = 1, AYSE = 2 olur. Sıralanan nesneleri büyük harflerle yazmak zorunlu değildir. Eğer istersek sıra numaralarını kendimiz belirleyebiliriz:

```
enum arkadaşlar {ALI = 5, VELI = 4, AYSE = 8};
```

dersek ALI = 5, VELI = 4, AYSE = 8 olur. { } içindeki sabitleri belirten arkadaşlar adı yerine, adlandırma kuralına uyarak, istediğiniz başka bir ad verebilirsiniz (örneğin, dostlar).

SIRA SİZDE

2

Klavyeden girilen vize ve final sınavlarının notlarından bir öğrencinin dönem not ortalamasını hesaplayan bir C programı yazınız. Bu programda, öğrencinin dönem not ortalamasına vize sınavının katkısı %40 ve final sınavının katkısı %60 olsun.

Mantıksal Değişken Tipleri

Mantıksal değerler, ikili mantık sisteminde yanlış (false) ve doğru (true) değerleridir. Bu değerlere, mantık konusunu matematiksel yapı olarak tanımlayan İngiliz matematikçi George Boole'un anısına boolean değerler denilir. Bilgisayarın mantıksal deyimleri işleyip, karar verebilmesi için boolean değerler her dilde önem taşır. C#, Java, C++ gibi üst düzey diller, hatta C99 ile belirlenen standartlarda mantıksal işlemler için bool (Boolean) tipinde bir veri tipi tanımlanmıştır. Bu veri tipi, sadece true ve false değerlerini alabilir. İlişkisel operatörler, bu mantıksal sonuçlardan birini döndürür. Ancak C dilinde özel bir mantıksal veri tipi yoktur. Bunun yerine, herhangi tam sayı olabilen değer;

- Sıfır ise FALSE (Yanlış),
- Sıfırdan farklı ise TRUE (Doğru)

kabul edilir. Bu değişkenler ile ilgili kullanım bilgileri, ilerleyen ünitelerde daha detaylı bir şekilde anlatılmıştır.

C PROGRAMLAMA DİLİNDE DEPOLAMA TÜRLERİ

C programlama dilinde değişkenler, yaşam sürelerine göre iki sınıfa ayrılır:

- Geçici Depolama (Volatile Storage)
- Kalıcı Depolama (Non-Volatile Storage)

Geçici Depolama

“auto” (automatic) ve register depo sınıflarına konulan değişkenler, ait oldukları program bloğunun (program kod parçası) işi bitince bellekten silinirler. “auto” depo sınıfına konulacak değişkenlerin tanımlanması örneğin aşağıdaki şekilde yapılır:

```
auto float x, y;
```

Bir program bloğunun içinde tanımlaması yapılan değişkenler “auto” depo sınıfındandır. Onları ayrıca “auto” diye tanımlamaya gerek yoktur. O nedenle, bir program bloğunun içinde yukarıdaki tanımlamayı,

```
float x, y;
```

gibi de yazabiliriz. Register’ler (yazmaçlar), CPU (Central Processing Unit – Merkezî İşlem Birimi) içinde veri tutmaya yarayan küçük depolardır. Makine diline dönüşen kodlar çalışırken çok kullanılan veriler bir yazmaca alınır; böylece işlemlerin daha hızlı yapılması sağlanır. Yazmaca konulması istenen değişkenlerin tanımlanması, örneğin aşağıdaki şekilde yapılır:

```
register int m, n;
```

Ne var ki boş register yoksa derleyici (compiler) bu isteğe uyamaz. Aslında, gelişkin derleyiciler çok kullanılan verileri kendiliğinden yazmaca koyabilme yeteneğine sahiptir. Bu tür gelişkin derleyicilerde, programcının yazmaca konulacak değişkeni tanımlamasına gerek kalmaz.

Geçici depolama, belleğin ekonomik kullanımı açısından önemlidir. Büyük programlarda çok sayıda değişken ve program bloku bulunur. Bazı değişkenler çok kısa süre için kullanılır, bazı değişkenler ise program boyunca kullanılır. İş biten değişkenlerin bellekten silinmeleri, belleğin o alanlarının başka işler için kullanılmasına olanak sağlar. O sebeple, iyi bir program, iş biten değişkenleri bellekten yok edecek biçimde yazılmak zorundadır.

Kalıcı Depolama

“extern” ve “static” depo sınıflarına konulan değişkenler, program süresince silinmeden bellekte kalırlar. Global değişkenler ön tanımlı (default) olarak “extern” değişken sayılırlar. Onlar program boyunca bellekte saklanırlar. O nedenle, global değişkenleri “extern” diye tanımlamaya gerek olmaz. Ancak bazı durumlarda bir blok içindeki değişkenin program boyunca bellekte kalması istenebilir. O zaman bu tür bir değişken “static” diye tanımlanmak zorundadır. “extern” ve “static” depo sınıfına konulacak değişkenlerin tanımlanması örneğin aşağıdaki şekilde yapılır:

```
extern int tc_kimlik_no;
static char ad_soyad[20];
```

Sabitleri istediğiniz sırada numaralayabileceğinizi görmek için aşağıdaki programa benzer programlar yazıp deneyiniz.



SIRA SİZDE

```
#include <stdio.h>
main ( )
{
enum takimler {BJK = 1, FB, GS = 2, TS};
printf ( 'BJK = %d\n' , BJK ) ;
printf ( 'FB = %d\n' , FB ) ;
printf ( 'GS = %d\n' , GS ) ;
printf ( 'TS = %d\n' , TS ) ;
}
```

Özet



C programlama dilindeki veri türlerini ve bu türler arasındaki farkı anlamak.

Bazı programlama dilleri veri tiplerini önceden ayırır bazı programlama dilleri ise ayırmaz. Teknik olarak veri tipini önceden belirleyen programlama dilleri, belleğe yerleşecek verileri, orada kaplayacakları alanlara göre gruplara ayırır. Programlama dilleri, bu veri gruplarını pratikteki kullanımlarına göre adlandırır. Örneğin; karakterler, tam sayılar, kesirli sayılar vb. gibi yalın tipler yanında dizim (array) gibi birleşik veri yapıları bu ayrımı yapan her dilde vardır. Fortran, Pascal, C/C++, Java, vb. programlama dilleri veri tiplerini önceden ayıran dillerdir. Python, Ruby gibi programlama dilleri ise bu ayrımı önceden yapmaz; önce veriyi belleğe yerleştirir, sonra onun tipini belirler.

C programlama dili, verileri önceden tiplere ayıran dillerdendir. C programlama dilinde toplam üç tane temel veri tipi vardır:

- Sayı
- Karakter
- Dizgi (String)

Bu veri tipleri çeşitli alt gruplara ayrılırlar. Alt grupların oluşturulmasında ortak bir kuraldan söz edilemez. Farklı derleyiciler, farklı sınıflandırmalar ve farklı adlar kullanmıştır. Onları bilinen adlarına ve bellekte kapladıkları büyüklüklere göre sınıflandırmak yeterli olacaktır.



C programlama dilindeki değişkenlere isim verilirken hangi kuralların takip edilmesi gerektiğini açıklamak.

Programcı nasıl bir program yazıyor olursa olsun, programın tasarlanması aşamasında en çok dikkat etmesi gereken faktörlerden biri değişken tanımlamalarıdır. Değişkenin tipinin ve isminin yanlış seçilmesi o an için bir problem teşkil etmeyebilir ancak aradan belli bir zaman geçtikten sonra çok büyük sorunlara neden olabilir. Öyle ki kesirli işlemler (floating) yapılacak hâlde birtakım değişkenler tam sayı (integer) belirlenmişse ve test aşamasında kesirli sonuçlar vermeyen değerler girildiğinde o an için program düzgün çalışıyormuş gibi gözükülebilir. Ayrıca değişkenlere anlamsız isimler verildiğinde yine o an için bir sorun çıkmasa da programcının bir süre sonra yeniden aynı programla uğraşmaya başladığında hangi değişkenin ne amaçla kullanıldığını bulabilmek için programı başından sonuna adım adım takip etmesi gerekebilir. Büyük bir programlama projesinde karmaşa yüzün-

den adım adım takip de mümkün olmayabilir. Bu tür durumlarla karşılaşmamak için değişkenlere her zaman anlamlı isimler vermek daha doğrudur. O anda 'x yolu' diye aklınızdan geçen bir şey varsa o değişkene 'x' adını vermek yerine 'x yolu' demek sonrası için daha iyi olacaktır.

C programlama dilinde kullanılan değişken isimlendirme kuralları aşağıdaki gibi sıralanmıştır:

- C programlama dili komutları değişken adı olarak verilemez (Örneğin; if, for, case gibi).
- Değişken ismi birden çok kelimeden oluşacaksa aralarına boşluk verilmez. Örneğin, "maksimum sayı" adında bir değişken tanımlanamaz ama "maksimum_sayı" ya da "MaksimumSayı" gibi değişken tanımlanabilir.
- Değişken adları en fazla 32 karakterden oluşabilir. 32 karakterde uzun değişken adları ilk 32 karakteri değerlendirilir. Geriye kalan karakterler işleme tabi tutulmaz.
- Değişken adları İngiliz alfabesinde bulunan karakterler (A-Z) veya (a-z) ya da rakamlar (0-9) ile yazılmalıdır. Türkçe karakterler, özel karakter veya boşluk karakteri kullanılamaz.
- Değişken adları herhangi bir rakam ile başlayamaz. İlk karakter bir harf olmalıdır. Sonrakiler ise rakamlardan oluşabilir.
- Aşağıda verilen kelimeler ANSI C'nin anahtar kelimeleridir (key words) ve değişken ismi olarak kullanılamaz:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while



C programlama dilindeki değişkenlerin programcı tarafından ilk defa nasıl tanımlanabileceğini anlamak.

Değişkenler bilgisayarın belleğinde bilginin saklandığı gözlemlenen sembolik adlardır. Bir C programında, bir değişken tanımlandığında bu değişken için bellekte bir yer ayrılır. Her değişkenin tuttuğu değer nasıl bir veri olduğunu gösteren bir veri tipi vardır.

C programlama dilinde, değişkenler ve sabitler programın başında bulunmalıdır. Bazı uygulamalarda de-

ğişkenin bir başlangıç değerinin olması istenir. Böyle durumlarda değişken bildirilirken başlangıç değeri verilebilir. Örneğin:

```
char isim = 'X', z; /* değer atamak zorunlu değil */
int sayi = 0, n;
float toplam = 0.005, sonuc = 22.14;
```



C programlama dilindeki değişkenlere program içinde ön değer nasıl verildiğini ifade etmek.

Eğer değişkenlere bir başlangıç değeri atayarak programda kullanmak istiyorsak, değişkenlerin başlangıç değeri atanmış tanımlamaları aşağıdaki gibidir:

```
int x = 5;
int a = 2, b = 5, c = 1;
float donem_sonu_not_ortalama = 50.5;
```



C programlama dilindeki depolama türlerini anlamak.

C programlama dilinde değişkenler, yaşam sürelerine göre iki sınıfa ayrılır:

- Geçici Depolama (Volatile Storage)
- Kalıcı Depolama (Non-Volatile Storage)



C programlama dilinde herhangi bir değişkene tercih edilen depolama türünde nasıl tanımlama yapılacağını açıklamak.

“auto” (automatic) ve register depo sınıflarına konulan değişkenler, ait oldukları program bloğunun (program kod parçası) işi bitince bellekten silinirler. Register’ler (yazmaçlar), CPU içinde veri tutmaya yarayan küçük depolardır. “auto” depo sınıfına konulacak değişkenlerin tanımlanması, örneğin aşağıdaki şekilde yapılır:

```
auto float x, y;
```

Yazmaca (register’a) konulması istenen değişkenlerin tanımlanması örneğin aşağıdaki şekilde yapılır:

```
register int m, n;
```

Geçici depolama, belleğin ekonomik kullanımı açısından önemlidir. Büyük programlarda çok sayıda değişken ve program bloku bulunur. Bazı değişkenler çok kısa süre için kullanılır, bazı değişkenler ise program boyunca kullanılır. İş biten değişkenlerin bellekten silinmeleri, belleğin o alanlarının başka işler için kullanılmasına olanak sağlar. O sebeple, iyi bir program, iş biten değişkenleri bellekten yok edecek biçimde tasarlanmak zorundadır.

“extern” ve “static” depo sınıflarına konulan değişkenler, program süresince silinmeden bellekte kalırlar. Global değişkenler, ön tanımlı (default) olarak “extern” değişken sayılırlar. “extern” ve “static” depo sınıfına konulacak değişkenlerin tanımlanması örneğin aşağıdaki şekilde yapılır:

```
extern int tc_kimlik_no;
static char ad_soyad[20];
```

Kendimizi Sıneyalım

1. Aşağıdakilerden hangisi C programlama dilinde kullanılan tam sayı bir veri türü **değildir**?

- float int
- signed char
- short int
- unsigned char
- enum

2. Eğer “double” türünde bir kesirli sayı veri tipi kullanıyorsanız **en fazla** kaç haneye kadar ondalık sayı saklayabilirsiniz?

- 5
- 7
- 9
- 15
- 19

```
#include <stdio.h>
main() {
    int karakter = (int) 'A';
    printf('%d', karakter);
}
```

3. Yukarıdaki program kod parçası ekrana aşağıdakilerden hangisini yazdırır?

- A
- 65
- a
- A65
- Aa

4. printf() fonksiyonunda “\n” kontrol karakterinin amacıyla ilgili aşağıdaki ifadelerden hangisi doğrudur?

- Ses üretir.
- İmleci bir sola kaydırır.
- Sayfayı atlatır ve bir sonraki sayfanın başına geçirir.
- Bir alt satıra geçirir.
- Satır başı yaptırır.

```
#include <stdio.h>
main(){
    int pi = int (3,1415);
    printf('%d', pi);
}
```

5. Yukarıdaki program kod parçası ekrana aşağıdakilerden hangisini yazdırır?

- 3,1415
- 3,1415000
- 3
- 3,0000000
- 3,14

6. scanf() fonksiyonundaki “&” işaretinin amacıyla ilgili aşağıdaki ifadelerden hangisi doğrudur?

- Klavyeden girilecek bir tam sayıyı gösterir.
- Bir kesirli sayıyı ekrana bastırır.
- Bir dizgiyi (string) ekrana bastırır.
- Klavyeden girilecek bir karakteri gösterir.
- Klavyeden girilecek bir değişkenin adres operatördür.

```
enum mobilyalar {SEHPA = 5, DOLAP, VITRIN, MASA,
SANDALYE = 6};
```

7. Yukarıdaki program kod parçasındaki değişken tanımlaması sonucu MASA değişkeninin içine atanan sayı ne olur?

- 5
- 5,5
- 6
- 7
- 8

8. Aşağıdaki atamalardan hangisi C programlama dilindeki geçici depolama sınıflarından biri **değildir**?

- auto float = 3,1415;
- register int eskisehir_plaka_kodu = 26;
- extern int ogrenci_no;
- float f = 1.05, g = 25.5, yuzde;
- int pi = int (3,1415);

9. Geçici depolama için aşağıdaki ifadelerden hangisi **yanlıştır**?

- Belleğin ekonomik kullanımı açısından önemlidir.
- İşi biten değişkenlerin bellekten silinmelerini sağlar.
- “auto” ve “register” depo sınıflarına konulan değişkenler, geçici olarak depolanan değişkenlerdir.
- Geçici depolamada, değişkenler ait oldukları program bloğunun (program kod parçası) işi bitince bellekten silinirler.
- Program süresince silinmeden bellekte kalırlar.

10. Global değişkenler ön tanımlı (default) olarak hangi değişken türünde sayılırlar?

- auto
- extern
- register
- static
- unsigned int

Kendimizi Sınavalım Yanıt Anahtarı

- | | |
|-------|---|
| 1. a | Yanıtınız yanlış ise “Veri Tipleri” konusunu yeniden gözden geçiriniz. |
| 2. d | Yanıtınız yanlış ise “Veri Tipleri” konusunu yeniden gözden geçiriniz. |
| 3. b | Yanıtınız yanlış ise “C Programlama Dilinde Değişken Adlandırma ve Tanımlamalar” konusunu yeniden gözden geçiriniz. |
| 4. d | Yanıtınız yanlış ise “C Programlama Dilinde Değişken Adlandırma ve Tanımlamalar” konusunu yeniden gözden geçiriniz. |
| 5. c | Yanıtınız yanlış ise “C Programlama Dilinde Değişken Adlandırma ve Tanımlamalar” konusunu yeniden gözden geçiriniz. |
| 6. e | Yanıtınız yanlış ise “C Programlama Dilinde Değişken Adlandırma ve Tanımlamalar” konusunu yeniden gözden geçiriniz. |
| 7. e | Yanıtınız yanlış ise “C Programlama Dilinde Değişken Adlandırma ve Tanımlamalar” konusunu yeniden gözden geçiriniz. |
| 8. c | Yanıtınız yanlış ise “C Programlama Dilinde Depolama Türleri” konusunu yeniden gözden geçiriniz. |
| 9. e | Yanıtınız yanlış ise “C Programlama Dilinde Depolama Türleri” konusunu yeniden gözden geçiriniz. |
| 10. b | Yanıtınız yanlış ise “C Programlama Dilinde Depolama Türleri” konusunu yeniden gözden geçiriniz. |

Sıra Sizde Yanıt Anahtarı

Sıra Sizde 1

C programlama dilinde diğer veri türündeki dizgilerden (array) farklı olarak dizgilerin ilk değerleri tanımlama sırasında kısaca iki çift tırnak içinde belirtilebilir (“deneme” gibi). Fakat tanımlayacağımız dizginin boyutu, tanımlanan ilk değerinden küçük olursa ne olur?

```
char mobilya[5] = "sandalye";
```

ifadesi ile tanımlanan dizginin bellekteki görünümü ancak “sand” olacaktır. Çünkü bu ifade ile en çok 4 karakter uzunluğunda deneme adlı bir dizgi tanımlayabiliriz.

Sıra Sizde 2

Sizden istenilen C programı şu şekilde olacaktır:

```
main()
{
    int sinav1, sinav2,
    float donem_notu;
    printf('Vize sinav notunu giriniz:');
    scanf('%d', &sinav1);
    printf('Final sinav notunu giriniz:');
    scanf('%d', &sinav2);
    donem_notu = sinav1*0.4 + sinav2*0.6;
    printf('\n Notlarin Ortalamasi: %f', donem_notu);
}
```

Sıra Sizde 3

Sabitleri istediğiniz sırada numaralayabilirsiniz.

```
main ( )
{
    enum takimler {BJK = 1, FB, GS = 2, TS};
    printf ('BJK = %d\n', BJK) ;
    printf ('FB = %d\n', FB) ;
    printf ('GS = %d\n', GS) ;
    printf ('TS = %d\n', TS) ;
}
```

Bu programda ekran çıktısı şu şekilde görünecektir:

```
BJK = 1
FB = 2
GS = 2
TS = 3
```

Yararlanılan ve Başvurulabilecek Kaynaklar

- Deitel, P., Deitel, H. (2009). “C: How to Program”, Prentice Hall.
- Kochan, S.G. (2004). “Programming in C”, Sams Publishing.
- Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C. (2001). “Introduction to Algorithms, Second Edition”, McGraw-Hill Science/Engineering/Math.
- Karaçay, T., Karaçay A. (2008). “C# ile Nesne Programlama”, Seçkin Yayıncılık.
- Aslan, K. (1998). “A'dan Z'ye C Klavuzu”, Pusula Yayıncılık.

6

Amaçlarımız

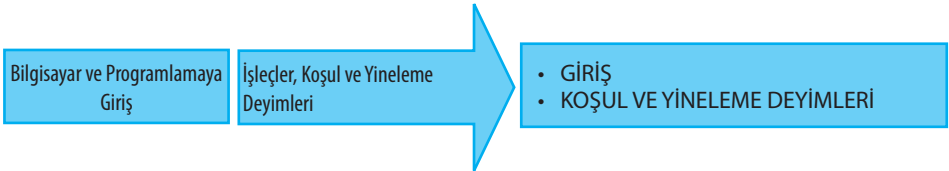
Bu üniteyi tamamladıktan sonra;

- C programlama dilinde kullanılan işleçleri tanımlayabilecek,
 - İşlevlerine göre işleçleri sınıflandırabilecek ve işlem önceliklerini açıklayabilecek, Koşul deyimlerini ve nasıl kullanılacaklarını anlatabilecek,
 - Yineleme deyimlerinin yapılarını ve hangi durumlarda hangi yineleme deyiminin kullanılması gerektiğini açıklayabilecek
- bilgi ve becerilere sahip olabileceksiniz.

Anahtar Kavramlar

- İşleç
- İşlenen
- Ön Ek ve Son Ek
- İşlem Öncelikleri
- Koşul Deyimleri
- Yineleme Deyimleri

İçindekiler



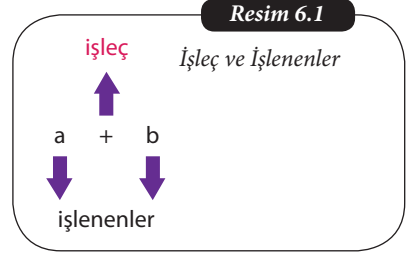
İşleçler, Koşul ve Yineleme Deyimleri

GİRİŞ

İşleç Nedir?

İşleçler, derleyiciye bazı özel matematiksel veya mantıksal fonksiyonları icra etmesini söyleyen simgelerdir. İşleçler program içinde kendi başlarına anlam ifade etmezler. Her işlecin anlamlı hâle gelebilmesi için bir veya birden fazla değişken veya sabit ile birlikte kullanılması gerekmektedir. Bu değişken veya sabitlere işlenen denir.

C dili yerleşik işleçler açısından zengin bir yapıya sahiptir. Sıklıkla kullanılan bazı işleçler Tablo 6.1'de gösterilmiştir.



$a = b + c$	İşleçler: $= +$
$z < t$	İşleç: $<$
$a++$	İşleç: $++$
$a b$	İşleç: $ $

Tablo 6.1
Örnek işleçler

İşleç Çeşitleri

İşleçleri iyi anlayabilmek için işlevlerine göre sınıflandırabiliriz:

- Aritmetik İşleçler
- İlişkisel İşleçler
- Mantıksal İşleçler
- Bit İşleçleri
- Atama İşleci
- Diğer İşleçler

Aritmetik İşleçler

Aritmetik işleçler denildiğinde akla ilk olarak dört işlem işleçleri yani toplama, çıkarma, çarpma ve bölme gelir. Bu işleçler hem tam sayı hem de kayan noktalı veri tiplerinde kullanılır. Kullanışları matematikteki kullanımlarına çok yakındır. Bu işleçlerin dışında daha az karşılaşılan aritmetik işleçler de bulunmaktadır.

Toplama ve Çıkarma İşleçleri

İki işleneni toplamak veya birbirinden çıkarmak için kullanılırlar. C bir değişken, A ve B de iki farklı işleneni göstermek üzere, + ve - işleçlerinin kullanımı Tablo 6.2'de gösterilmiştir.

Tablo 6.2
+ ve - işleçlerinin
kullanımı

+	$C = A + B$
-	$C = A - B$

Tablo 6.2'nin ilk satırında + işleci ile A ve B'nin değerleri toplanmakta ve bulunan sonuç = işleci ile C değişkenine atanmaktadır. Örneğin, A'nın değeri 5 ve B'nin değeri de 3 olmak üzere,

$$C = A + B$$

işleminin sonucu $C = 8$ 'dir. Benzer şekilde - işleci ile A ve B'nin değerlerinin farkı alınmakta ve bulunan sonuç = işleci ile C değişkenine atanmaktadır. Bir önceki örnekteki A ve B değerleri için,

$$C = A - B$$

işleminin sonucu $C = 2$ 'dir.

Çarpma ve Bölme İşleçleri

İki işleneni çarpmak veya birbirine bölmek için kullanılırlar. C bir değişken, A ve B de iki farklı işleneni göstermek üzere, * ve / işleçlerinin kullanımı Tablo 6.3'te gösterilmiştir.

Tablo 6.3
* ve / işleçlerinin
kullanımı

*	$C = A * B$
/	$C = A / B$

Tablo 6.3'ün ilk satırında * işleci ile A ve B'nin değerleri çarpılmakta ve bulunan sonuç = işleci ile C değişkenine atanmaktadır. Örneğin, A'nın değeri 10 ve B'nin değeri de 2 olmak üzere,

$$C = A * B$$

işleminin sonucu $C = 20$ 'dir. Benzer şekilde / işleci ile A ve B'nin değerlerinin birbirine bölümü ile elde edilen sonuç = işleci ile C değişkenine atanmaktadır.

$$C = A / B$$

işleminin sonucu $C = 5$ 'tir.

SIRA SİZDE



1

$x = 5 - 20 / 4 + 5$; ile verilen ifadede x değerini bulunuz.

Kalan İşleci

İki işlenenin birbirine bölümü sonucu kalan değerini hesaplayan bu işlece *kalan işleci* denir. % simgesi ile gösterilir. Dört işlem işleçlerine benzer şekilde, iki işlenenin arasına yerleştirilerek kullanılır.

$$C = A \% B$$

Şeklindeki bir ifadede A'nın B'ye bölümünden kalan değer C değişkenine atanır. Örneğin, A'nın değeri 15 ve B'nin değeri 7 olduğu durumda $C = 1$ olur.

% işleci * ve / işleçleri ile aynı önceliğe sahiptir. Bu sebeple + veya - işleçlerinden önce değerlendirilir. Örneğin,

$$K = 8 + 7 \% 4$$

ifadesinde ilk önce % işleci değerlendirilir, sonrasında sırasıyla + ve = işleçleri işleme alınır. İşlemin sonucu $K = 11$ 'dir.

= işleci dört işlem işleçlerinden ve kalan işlecinden daha az önceliğe sahiptir. Eğer böyle olmasaydı = işleci diğer işleçlerden önce etki eder ve C değişkenine ilk işlenenin değeri atanırdı.



DİKKAT

$y = 2 + 5 * 14 \% 3$; ile verilen ifadede y değerini bulunuz.



SIRA SİZDE

Artırma ve Eksiltme İşleci

Genellikle birçok uygulamada, ama özellikle sayım işleminin gerektiği uygulamalarda sıklıkla rastlanılan işleçlerdir. Bir değişkenin değerini 1 artırmak veya 1 azaltmak için kullanılırlar. İşleç yardımı olmadan bir değişkenin değerini 1 artırmak için yapılması beklenen işlem aşağıdaki gibidir:

$$\text{sayac} = \text{sayac} + 1$$

fakat aynı işlem artırma işleci kullanılarak daha pratik bir şekilde yapılabilir:

$$\text{sayac}++$$

İki durumda da aynı sonuç elde edilir. Eksiltme işleci de aynı şekilde kullanılır.

Kalan işleci sadece tam sayı değişkenlerle işlem yapar!

Ön Ek ve Son Ek

Artırma ve eksiltme işleçleri iki farklı şekilde kullanılabilir: *ön ek* ve *sonek* olarak. Önek işlecin değişkenin önünde, sonек ise arkasında yer alması demektir. Bu iki durum arasında başka bir işleç olmadan (yalın) olarak kullanıldıklarında hiçbir fark yoktur. Kısacası, ++K ile K++ ifadeleri K değerini 1 artırırlar. Aynı şekilde --K ile K-- ifadeleri de K değerini 1 azaltırlar. Fakat başka işleçler ile beraber kullanıldıklarında farklı anlamlar kazanırlar. Örneğin,

$$\text{ToplamArac} = \text{AracSayisi} * ++\text{sayac}$$

ifadesinde çarpma işlemi sayac değişkeninin değeri 1 artırıldıktan sonra yapılacaktır. Burada önek kullanılarak işlem önceliği kazandırılmıştır. Eğer ki sonек kullanılmış olsaydı, yani,

$$\text{ToplamArac} = \text{AracSayisi} * \text{sayac}++$$

Bu kez öncelikli olarak çarpma işlemi yapılacaktı ve ardından sayac değişkeninin değeri 1 artırılacaktı.

ToplamArac = AracSayisi * ++sayac; ifadesinde Aracsayisi değişkeninin değeri 15 ve sayac değişkeninin değeri ise 5'tir. İfade işleme alındıktan sonra tüm değişkenlerin değerleri ne olur? Aynı işlemi sonек artırma işleci kullanıldığı durum için de yapınız.



SIRA SİZDE

İlişkisel işleçler aritmetik işleçlerden daha düşük önceliklidirler.

İlişkisel İşleçler

İki değeri karşılaştıran işleçler *ilişkisel işleçler* olarak adlandırılır. Karşılaştırmalarda küçüktür, büyüktür, eşittir, küçük eşittir gibi ilişkiler kullanılır. İşlem sonucunda doğru veya yanlış değer döndürülür. Örnek olarak eşitlik açısından karşılaştırılan iki değeri birbirine eşittir (doğru) ya da değildir (yanlış).

Tablo 6.4
İlişkisel İşleçler

>	Büyüktür
<	Küçüktür
==	Eşittir
!=	Eşit değildir
>=	Büyüktür veya eşittir
<=	Küçüktür veya eşittir

DİKKAT



Eşittir ilişkisel işleci iki tane eşittir işareti (==) oluşmaktadır. Atama işleci olan tek eşittir işareti (=) ile karıştırılmamalıdır.

C dilinde ilişkisel işleçler koşulun sağlanması durumunda 1, sağlanmaması durumunda ise 0 değerini verirler. Şimdi bir örnekle ilişkisel işleçlerin nasıl çalıştığını inceleyelim.

Örnek 1:

Resim 6.2

İlişkisel işleçler örnek 1

```
A = 15;
B = 10;

K = A < B;
L = A == B;
M = A > B;
```

Verilen örnekte, 1. ve 2. satırlarda A ve B değişkenlerine değer ataması yapılmakta 4. 5. ve 6. satırlarda ise A ve B değerlerinin karşılaştırılması yapılmaktadır. İşlemler yapıldığında **K = 0**, **L = 0**, ve **M = 1** olacaktır. İşleçlerin işlem önceliği ile ilgili de bir örnek yapalım.

Örnek 2:

Resim 6.3

İlişkisel işleçler örnek 2

```
X = 5;
Y = 20;

Z = X >= Y / X + 1;
```

Bu örnekte dikkat edilmesi gereken nokta, aritmetik işleçlerin ilişkisel işleçlere göre önceliğinin olmasıdır. Bu sebeple ilk önce bölme işleci, sonrasında toplama ve ardından ilişkisel işleç dikkate alınacak ve en son atama işleci ile Z'ye değer ataması yapılacaktır. Buradaki işlem sonucunda **Z = 1** olacaktır. İşlem sırası Resim 6.3'te gösterilmiştir.

Resim 6.4

İlişkisel işleçler örnek 2'ye ait işlem sırası

```
1 Y / X = 4
2 4 + 1 = 5
3 X >= 5 = 1
4 Z = 1
```

Z = 6 == 6 < 10 + 1; ifadesi işlendiğinde Z değeri ne olur?



SIRA SİZDE

Mantıksal İşlemler

Şu ana kadar iki işlem grubunu inceledik: Aritmetik işlemler ve ilişkisel işlemler. Şimdi diğer bir işlem grubu olan **mantıksal işlemleri** ele alacağız. Mantıksal işlemler, *true* ve *false* (doğru ve yanlış) olmak üzere sadece iki adet değer tutabilen boolean tipi değişkenlerin, mantıksal olarak birbiri ile değerlendirilmesine yararlar. Bu açıklamayı birkaç cümle ile daha anlaşılır kılabiliriz. Örneğin, “*Kemal okula araba ile gitti.*” ifadesi boolean bir değere sahiptir. Diğer bir deyişle bu ifade ya doğrudur ya da yanlıştır. Bir diğer ifade de “*Bakım sebebiyle bugün tramvaylar çalışmayacak.*” şeklinde olabilir. Bu iki ifadeyi mantıksal işlemler yardımıyla işleme tabi tutabiliriz: *Kemal okula araba ile gittiyse VE bakım sebebiyle bugün tramvaylar çalışmayacaksa* o zaman Ahmet de otobüsle gitmek zorundadır. Burada iki ifadeyi birbiri ile bağlayan “ve” yani “and” bağlacıdır. Burada verilen iki ifadenin alacağı farklı değerler sonucu bir değer belirlenir. Öyle ki sadece iki ifadenin de *doğru* olması durumunda Ahmet otobüsle gitmek zorundadır.

Mantıksal işlemler boolean türündeki işlenenlerle işlem yaparlar.

&&	Mantıksal AND (ve)
	Mantıksal OR (veya)
!	Mantıksal NOT (değil)

Tablo 6.5
Mantıksal İşlemler

Üç adet mantıksal işlem vardır (bk. Tablo 6.5). Bu işlemler ve, veya, değil kelimelerinin İngilizce karşılıkları olarak AND, OR ve NOT olarak isimlendirilirler.

Mantıksal VE (AND) İşlemi

Mantıksal VE işlemi &&, iki işlenenin ikisi de doğru ise *doğru* (1), herhangi biri veya ikisi de yanlış ise *yanlış* (0) değerini üretir. Bu işleme ait doğruluk tablosu Tablo 6.6’da verilmiştir.

A	B	A && B
Yanlış	Yanlış	Yanlış
Yanlış	Doğru	Yanlış
Doğru	Yanlış	Yanlış
Doğru	Doğru	Doğru

Tablo 6.6
VE (AND) işlemine ait doğruluk tablosu

Şimdi bu işlemin çalışma mantığını örneklerle inceleyelim.

Örnek 1:

```
A = 0 && 0;
B = 5 && 0;
C = 0 && -2;
D = -1 && 3;
```

Resim 6.5

VE işlemi Örnek 1

Resim 6.5’te VE işlemi kullanılan dört farklı ifade verilmiştir. Bunlardan ilk ifadeye iki işlenenin de değeri sıfırdır (yani yanlıştır). Dolayısıyla işlem sonrası A = 0 olur. Benzer şekilde ikinci ve üçüncü ifadelerde de işlenenlerden biri sıfır değerini almaktadır ve bu sebeple sonuçlar B = 0, C = 0 olur. Son ifadeye iki işlenen de sıfırdan farklıdır yani doğrudur. Tablo 6.6’da da gösterildiği üzere iki doğrunun VE işlemi ile işlemi sonucunda doğru değeri üretilir: D = 1’dir.

Örnek 2:

Resim 6.6

VE işleci Örnek 2

```
sayac = 10;
A = sayac <= 15 && sayac > 5;
```

Bu örnekte mantıksal işleç ilişkisel işleçlerle birlikte kullanılmaktadır. İlişkisel işleçlerin mantıksal işleçlere göre işlem önceliği yüksektir. Bu sebeple işlem sırası Resim 6.7'deki şekilde gibi olacaktır.

Resim 6.7

VE işleci Örnek 2'ye
ait işlem sırası

```
1 sayac <= 15 // doğrudur (1)
2 sayac > 5   // doğrudur (1)
3 A = 1 && 1
4 A = 1
```

Mantıksal VEYA (OR) İşleci

Mantıksal VEYA işleci `||`, işlenenlerinden birinin doğru olması durumunda *doğrudur*. Sadece iki işlenenin de yanlış olması durumunda *yanlış* değerini verir. Farklı kombinasyonlar sonucu oluşacak değerler, Tablo 6.7'de verilen doğruluk tablosunda daha iyi bir şekilde gözlemlenebilir.

Tablo 6.7
VEYA (OR) işlecine ait
doğruluk tablosu

A	B	A B
Yanlış	Yanlış	Yanlış
Yanlış	Doğru	Doğru
Doğru	Yanlış	Doğru
Doğru	Doğru	Doğru

Şimdi bu işlecin çalışma mantığını örneklerle inceleyelim.

Örnek 1:

Resim 6.8

VEYA işleci Örnek 1

```
A = 0 || 0;
B = 0 || 9;
C = -2 || 0;
D = 3 || 5;
```

Resim 6.8'de VEYA işleci kullanılan dört farklı ifade verilmiştir. Bunlardan ilk ifadede iki işlenenin de değeri sıfırdır (yani yanlıştır). Dolayısıyla işlem sonrası $A = 0$ olur. Diğer ifadelerin tümünde bir işlenen sıfırdan farklı olduğu için tüm işlemlerin sonucu doğrudur: $B = 1$, $C = 1$, $D = 1$ 'dir.

Örnek 2:

Resim 6.9

VEYA işleci Örnek 2

```
AracSayisi = 2;
AracBul = AracSayisi < 5 || AracSayisi == 1;
```

Bu örnekte mantıksal VEYA işleci ilişkisel işleçlerle birlikte kullanılmaktadır. İlişkisel işleçlerin mantıksal işleçlere göre işlem önceliği yüksektir. Bu sebeple işlem sırası Resim 6.10'daki şekilde olacaktır.

Resim 6.10

```
1 AracSayisi < 5 // doğrudur
2 AracBul = 1
```

VE işleci Örnek 2'ye
ait işlem sırası

Dikkat edileceği üzere VEYA işlecinin sağ tarafında yer alan işlenen işleme alınmadan doğrudan AracBul isimli değişkene 1 ataması yapıldı. Bunun sebebi şudur: VEYA işlecinin ilk önce sol tarafındaki işlenen ele alınır, eğer doğruysa sağ tarafındaki işlene bakılmaksızın sonuç değeri 1 olarak belirlenir.

Y = 6 + 5 >= 8 && 3 - 1 <= 4; ifadesi işlendiğinde Y değeri ne olur?



SIRA SİZDE

Mantıksal DEĞİL (NOT) İşleci

Mantıksal DEĞİL işleci !, sadece tek bir işlenen alır ve işlenenin değerini mantıksal olarak tersine çevirir. Kısacası, doğru (true) olan bir ifadeyi yanlış (false), yanlış olan bir ifadeyi doğru yapar. Mantıksal DEĞİL işlecinin öncelik durumu diğer mantıksal işleçlerden farklılık gösterir. Öyle ki diğer mantıksal işleçlerin önceliği ilişkisel işleçleri düşük olsa da bu durum mantıksal DEĞİL için geçerli değildir. Bunun sebebi tekli (tek işlene sahip) işleç olmasıdır. Sonuç olarak, DEĞİL işleci ilişkisel işleçlerle kullanılırken parantezler yardımıyla işlem önceliği konusunda düzenleme yapmak gerekebilir.

Değil (NOT) işleci sadece tek işlenen alır ve örnek şeklinde kullanılır. ÖRN: !A, !sayac ...

A	!A
Yanlış	Doğru
Doğru	Yanlış

Tablo 6.8
DEĞİL (NOT) işlecinin
ait doğruluk tablosu

Örnek 1:

İlk ifadede 3 değeri mantıksal olarak doğrudur. Mantıksal DEĞİL işleci kullanılarak yanlış değere çevrilmiştir. Bu sebeple A = 0'dır. İkinci ifadede işlem önceliği değil işlecindedir. Sıfırın değeri yanlıştır, değil işleci bunu doğruya çevirir yani 1 değerini üretir. 4 değeri 1 değerinden büyük olduğu için B = 1 olur.

Resim 6.11

```
A = !3;
B = 4 > !0;
```

DEĞİL işleci Örnek 1

Bitsel İşleçler

Bit seviyesinde işlem yapan işleçlere *bitsel işleçler* denir. Bu işleçler sadece tam sayı türünden olan işlenenlerle çalışır, double ve float türündeki işlenenler ile işlem yapılamaz. Bit işleçlerinin listesi Tablo 6.9'da verilmiştir.

&	Bitsel AND (VE)
	Bitsel OR (VEYA)
^	Bitsel XOR (Özel VEYA)
~	Bitsel NOT (DEĞİL)
<<	Sola Öteleme
>>	Sağa Öteleme

Tablo 6.9
Bitsel İşleçler

Bitsel VE (AND) İşleci

Bitsel VE işleci & simgesi ile gösterilir. Kullanımı mantıksal VE işleci ile aynıdır. Tek farklılık işlemlerin bit seviyesinde yapılıyor olmasıdır. VE işlemini karşılıklı bitler için uygulanır.

Örnek 1:**Resim 6.12**

Bitsel VE işleci
Örnek 1

```
A = 25;      // 00011001
B = 12;      // 00001100

C = A & B;
```

Onluk sistemde değerleri verilen A ve B değişkenlerinin, ikilik sistemdeki karşılıkları: A = 00011001 ve B = 00001100'dır. Bitsel VE işlecini bu iki işlenene uyguladığımızda, karşılıklı her bit için VE işlemi yapılmış olur. Böylelikle C = 00001000 olarak bulunur. Bu da onluk sistemde 8'e karşılık gelir.

Resim 6.13

Bitsel VE işlecinin
uygulanması
Örnek 1

```
00011001 // A
00001100 // B
-----
00001000 // C = A & B
```

Bitsel VEYA (OR) İşleci

Bitsel VEYA işleci | simgesi ile gösterilir. Kullanımı mantıksal VEYA işleci ile aynıdır. Tek farklılık işlemlerin bit seviyesinde yapılıyor olmasıdır. VEYA işlemi karşılıklı bitler için uygulanır.

Örnek 1:**Resim 6.14**

Bitsel VEYA işleci
Örnek 1

```
A = 25;      // 00011001
B = 12;      // 00001100

C = A | B;
```

Bitsel VE örneğinde olduğu gibi onluk sistemde değerleri verilen A ve B değişkenlerinin, ikilik sistemdeki karşılıkları: A = 00011001 ve B = 00001100'dır. Bitsel VEYA işlecini bu iki işlenene uyguladığımızda, karşılıklı her bit için VEYA işlemi yapılmış olur. Böylelikle C = 00011101 olarak bulunur. Bu da onluk sistemde 29'a karşılık gelir.

Resim 6.15

Bitsel VEYA işlecinin
uygulanması
Örnek 1

```
00011001 // A
00001100 // B
-----
00011101 // C = A | B
```

Bitsel ÖZEL VEYA (XOR) İşleci

Bitsel ÖZEL VEYA işleci ^ simgesi ile gösterilir. Bu işleç işlenenlerinin karşılıklı bitleri aynı ise 0, farklı ise 1 değerini üretir. Doğruluk tablosu Tablo 6.10'da verilmiştir.

A	B	A ^ B
Yanlış	Yanlış	Yanlış
Yanlış	Doğru	Doğru
Doğru	Yanlış	Doğru
Doğru	Doğru	Yanlış

Tablo 6.10
Bitsel ÖZEL VEYA
işlecine ait doğruluk
tablosu

Örnek 1:

```
A = 25;      // 00011001
B = 12;      // 00001100

C = A ^ B;
```

Resim 6.16

Bitsel ÖZEL VEYA
işleci
Örnek 1

A ve B değişkenlerine atanan değerler Bitsel VE örneğinde olduğu gibidir. Resim 6.17’de gösterildiği gibi bitsel ÖZEL VEYA işlecini bu iki işlenene uyguladığımızda, C = 00010101 olarak bulunur. Bu da onluk sistemde 21’e karşılık gelir.

```
00011001 // A
00001100 // B
-----
00010101 // C = A ^ B
```

Resim 6.17

Bitsel ÖZEL VEYA
işlecinin uygulanması
Örnek 1

A = 00010010 ve B = 01001011 olmak üzere $X = A \& B \wedge B$; ifadesi işlendiğinde X değeri ne olur?



SIRA SİZDE

Bitsel DEĞİL (NOT) İşleci

Bitsel DEĞİL işleci ~ simgesi ile gösterilir. Tek işlenene uygulanır ve örnek şeklinde kullanılır. Uygulandığı işlenenin bitlerini tersine çevirir. Yani 0’ları 1, 1’leri 0 yapar.

Örnek 1:

```
A = 25;      // 00011001
C = ~A;
```

Resim 6.18

Bitsel DEĞİL işleci
Örnek 1

A değişkeninin değerine bitsel DEĞİL işleci uygulandığında, 0 lar 1 ve 1 ler 0 olacak şekilde değişim yapılır ve C = 11100110 sonucu elde edilir.

Sola ve Sağa Öteleme İşleçleri

Bitlerin sayı içerisindeki yerlerini değiştirmek için sola (<<) ve sağa (>>) öteleme işleçleri kullanılır. Her iki işleç de iki işlenene uygulanırlar. Sola öteleme işleminde bütün bitler öteleme sayısınca sola kaydırılır. Öteleme yaparken bit boyutunu aşan soldaki bitler kaybolurken, sağ taraftan 0 değerli bitlerle ekleme yapılır. Sağa öteleme işleminde de aynı işlemler soldan sağa doğru yapılır. Her iki işlecin çalışmasını örneklerle inceleyelim.

Örnek 1:

```
A = 25;      // 00011001
C = A << 1;
```

Resim 6.19

Sola Öteleme

Burada A değişkeninin değeri 1 kez sola ötelenecektir. Yani C değişkeninin değeri 00110010 şeklinde olacaktır. Görüldüğü gibi tüm bitler sola 1 adım kaydırıldıktan sonra sağ taraftan bir adet 0 ile besleme yapıldı.

Örnek 2:

Resim 6.20

Sağa Öteleme
Örnek 2

```
A = 25; // 00011001
C = A >> 3;
```

Burada A değişkeninin değeri 3 kez sağa ötelenecektir. Bitler adım adım 3 kez sağa kaydırıldığında C değeri elde edilir. Resim 6.20'de yapılan işlemler gösterilmiştir.

Resim 6.21

Sağa öteleme adımları
Örnek 2

```
A      ... 00011001
A >> 1  ... 00001100
A >> 2  ... 00000110
A >> 3  ... 00000011

C = 00000011
```

Atama İşleci

Atama işleci = , adından da anlaşılacağı üzere değer ataması için kullanılır. Kısaca sağ tarafında yer alan değeri üretir denilebilir. Önceki bölümlerde sıklıkla kullandığımız atama işleci, aşağıdaki örnekte bir değişkene değer ataması yapmak için kullanılmıştır.

Örnek 1:

Resim 6.22

Atama işleci

B = 12; Bu örnekte B değişkenine 12 değeri ataması yapılmaktadır.

Diğer İşleçler

Yukarıdaki bölümlerde tanıtılan işleçler dışında, gösterici işleçleri, işlemli atama işleçleri ve virgül, öncelik işleçleri gibi işleçler de bulunmaktadır.

İşleçler Arasındaki Öncelik İlişkisi

Matematikte sıklıkla karşımıza çıkan işlem önceliği, aynı şekilde işleçlerin kullanımı sırasında da ortaya çıkmaktadır. Birden fazla işlecin aynı satırda kullanıldığı durumlarda eğer parantez kullanılmamışsa işleçlerin kendi aralarındaki işlem önceliğine bakılarak karar verilir. Örneğin *çarpma* işlecinin *çıkarma* işlemine göre, çıkarma işlecinin ise *atama* işlemine göre bir önceliği vardır. Bir örnekle açıklayalım:

```
A = B - C * D
```

ile verilen ifade, derleyiciler tarafından aşağıdaki sırayla ele alınır.

1. işlem: C * D
2. İşlem: B - (1. işlem)
3. işlem: A = (2. işlem)

Eşit önceliğe sahip olan çarpma ve bölme gibi işleçlerde ise işlem önceliği C dili için soldan sağa doğrudur. Örneğin:

$$K = X * Y / Z$$

ile verilen ifadede işlem önceliği çarpma işlemine aittir. Yani işlem şu sırada yapılır:

```
1. işlem: X * Y
2. İşlem: (1. işlem) / Z
3. işlem: K = (2. işlem)
```

Tüm işleçler arasındaki öncelik ilişkisi Tablo 6.11'de verilmiştir. Öncelik sırası soldan sağa ve yukarıdan aşağıya doğru azalmaktadır.

İşleçler	Öncelik
!, ++, --,	En yüksek
*, / , %	
+, -	
<<, >>	
<, >, <=, >=	
==, !=	
&, ^,	
&&	
=	En düşük

Tablo 6.11
İşleçlerin Öncelik Sıraları

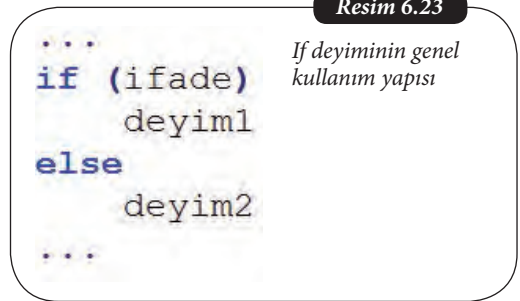
KOŞUL VE YİNELEME DEYİMLERİ

Program akışını kontrol etmek için deyimlere ihtiyaç vardır. Bu kısımda birçok programlama dilinde yer bulan karşılaştırma ve yineleme deyimlerine yer vereceğiz.

IF Deyimi

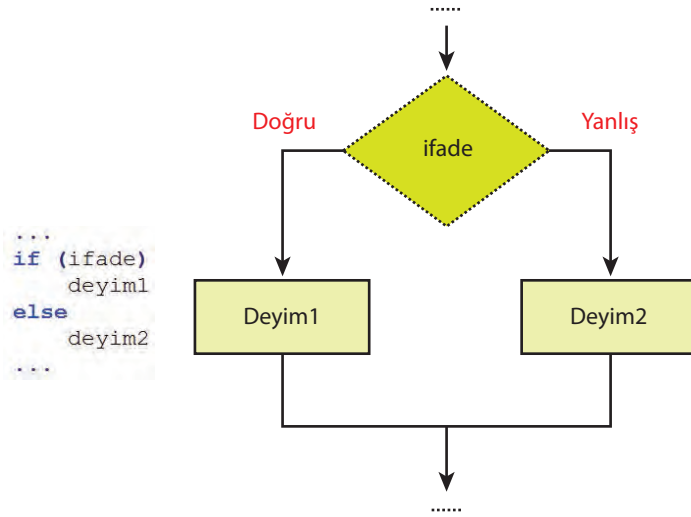
Program içerisinde karşılaştırma yapılması gerektiği durumlarda koşul deyimleri kullanılır. Yaygın olarak kullanılan koşul deyimlerinden birisi de *if* deyimidir. En sade şekliyle if deyimi, if sözcüğünün yanına yazılmış parantez içindeki bir ifade ve sonrasında gelen *deyimden* oluşur. İfadenin alabileceği iki değer mevcuttur: *doğru* veya *yanlış*. Eğer ki bu iki durum için de program da bir akış planlanıyorsa, bu durumda if deyimine else sözcüğü de eklenir. If deyiminin genel kullanım yapısı Resim 6.23'te verilmiştir.

Programda if deyimi icra edildiğinde işlemler şu sırada ilerler: if sözcüğünün yanındaki parantez içindeki ifade hesaplanır. Hesaplama sonrasında *doğru* veya *yanlış* olmak üzere bir mantıksal sonuç çıkar. Eğer sonuç doğruysa deyim1 icra edilir ve deyim2 icra edilmeden program akışına devam eder. Eğer sonuç yanlışsa deyim1 icra edilmeden doğrudan deyim2 icra edilir ve program akışına devam eder. If deyimine ait akış şeması Resim 6.24'te verilmektedir.



Resim 6.24

If deyiminin akış şeması



```

...
if (ifade)
    deyim1
else
    deyim2
...

```

If deyimini daha iyi anlayabilmek için örneklere bakalım ve adım adım çözümlerini inceleyelim.

Örnek 1:

Verilen bir sayının tek mi çift mi olduğunu ekrana yazdıran kod parçasını yazalım.

Resim 6.25

If deyimi Örnek 1

```

if (sayi % 2 == 0)
    printf("Verilen sayı çifttir.\n");
else
    printf("Verilen sayı tektir.\n");

```

Burada ilk önce sayının 2 ile bölünmesinden kalan değer kalan işleci yardımı ile hesaplanıyor ve 0'a eşit olup olmadığı kontrol ediliyor. Eğer eşitse ilk deyim işleme alınıyor. Yani ekrana "Verilen sayı çifttir." yazılıyor, eğer eşit değilse ekrana "Verilen sayı tektir." yazılıyor.

DİKKAT



If ile kullanılan birden fazla deyim varsa { ve } karakterleri ile bloklama yapılmalıdır.

If deyiminde yer alan ifadeler *else if* yardımıyla artırılarak daha detaylı koşullandırma yapmak mümkündür.

Örnek 2:

Metrekare cinsinden alan bilgisi verilen bir odanın *büyük*, *orta* ve *küçük* olarak sınıflandırılmasını yapalım.

Resim 6.26

```
if (Odanin_alani < 10)
    printf("Küçük boyutlu oda.\n");
else if (Odanin_alani > 10) && (Odanin_alani < 30)
    printf("Orta boyutlu oda.\n");
else if (Odanin_alani > 30)
    printf("Büyük boyutlu oda.\n");
```

If – else if
Örnek 2

Bu örnekte ilk olarak if değerinin yanındaki ifade değerlendirilir. Hesaplama sonucunda doğru değer elde edilirse ekrana “Küçük boyutlu oda” yazılır. Yanlış değer elde edilirse ilk else if değerinin yanındaki ifade hesaplanır. Doğru değer elde edilirse ekrana “Orta boyutlu oda” yazılır. Yanlış değer elde edilirse sonraki else if değerinin yanındaki ifade hesaplanır. Doğru değer elde edilirse “Büyük boyutlu oda” yazılır. Eğer yanlış değer elde edilirse birşey yapmadan program sonlandırılır.

sayi isimli değişkenin değeri 10'dan küçükse ekrana “Sayı 10'dan küçüktür.” yazan, değilse “Sayı 10'dan büyüktür” yazan bir if deyimi yazın.



SIRA SİZDE

SWITCH Deyimi

Birçok if – else yapısının kullanılmasını gerektirecek kadar fazla seçenek mevcutsa ve tüm kararlar aynı seçim değerine göre belirleniyorsa, bu durumlarda switch deyimini kullanmak iyi bir seçenek olabilir. If deyimi ile üretilebilecek karar sayısı doğru ve yanlış olmak üzere iki tanedir. Fakat switch deyimi ile istenilen sayıda karar üretmek mümkündür.

Resim 6.27

```
switch(ifade) {
    case secenek1: Deyim1; break;
    case secenek2: Deyim2; break;
    ...
    default: Deyim; break;
}
```

Switch deyiminin
genel kullanım yapısı

Derleyici işlem sırası şu şekildedir: İlk önce *switch* sözcüğünün yanındaki parantez içindeki ifade hesaplanır. Bu hesaplama sonucunda sayısal bir değer üretilmesi beklenir. Bulunan sayısal değer hangi *case* sözcüğünün yanındaki seçenek ile aynı değerde ise o satıra gidilir ve buradaki deyim işlenir. Eğer ki seçeneklerden hiçbiri hesaplanan ifade değeri ile aynı değilse o zaman *default* satırına gidilir ve buradaki deyim işlenir. Satır sonlarında bulunan *break* sözcüğü, program akışının diğer *case* ifadelerine geçmemesini sağlar.

Örnek 1:

Telefon bankacılığında sıklıkla karşılaştığımız seçim rutinini küçük bir kod parçası ile gerçeklemeye çalışalım.

Resim 6.28

```
switch(tus) {
    case 1: printf("Müşteri temsilcisi"); break;
    case 2: printf("Şifre değişimi"); break;
    case 3: printf("Kayıp bildirimi"); break;
    ...
    default: printf("Tekrar dinle"); break;
}
```

Switch deyimi
Örnek1

Switch deyiminde switch ifadesi ordinal türde (integer, char) olmalıdır.

Verilen örnekte *tus* değişkeni seçim yapılmasını sağlayan telefon düğmesini temsil etmektedir. Müşteri belirtilen tuşlardan hangisine basarsa program o işleme ait bilgiyi ekrana yazdıracak. Örneğin, müşteri 2 nolu tuşa bastığında program ekrana “Şifre değişimi” yazacak ve duracak. Eğer listede olmayan bir tuşa basarsa, bu kez ekrana “Tekrar dinle” yazılacak. Burada basit bir şekilde sistemin çalışma mantığı örneklenmeye çalışıldı. Ekrana yazdırmak yerine gerçekten müşteri temsilcisine bağlayacak programı C diline hakim olmaya başladığınızda yazabilirsiniz.

DİKKAT



Switch ifadesinde yer alan değişken ile seçenek değişkenlerinin aynı türde olmaları gerekmektedir.

FOR Deyimi

Programımızın bir bölümünün sabit bir sayıda yinelenmesini istediğimizde *for* deyimini kullanırız. **For** deyimi *for* anahtar sözcüğü ile başlar. Anahtar sözcükten sonra içinde üç farklı deyimin yer aldığı parantezler gelir.

Resim 6.29

For deyiminin genel kullanım yapısı

```
for(ifade1; ifade2; ifade3)
    Deyim;
```

Burada *ifade1*, döngü değişkenine ilk değer ataması yapılması amacıyla kullanılır. Sadece 1 kez çalıştırılır. *İfade 2* mantıksal bir değere sahiptir. Doğru değere sahip olduğu sürece döngü devam eder. Her yine-

lemede işlem görür. *İfade 3* döngü değişkeninin değerinin artırılması veya azaltılması amacıyla kullanılır. Her yinelemede işlem görür. Örneğin Resim 6.29'da verilen kod parçasında:

Resim 6.30

For deyimi
Örnek 1

```
for(i = 0; i < 10; i++)
    printf("Merhaba");
```

Döngü değişkeni *i* dir. Parantez içindeki ilk ifadede *i* değişkenine 0 değeri atanmıştır. Noktalı virgülden sonra ikinci ifade başlar ve bu ifadede *i* değişkeninin değeri 10'dan küçükse deyim işlenir ve ekrana “Merhaba” yazdırılır. Ardından son ifade olan *i* değişkenini 1 artırma işlenir ve ikinci ifadeye gidilir. Bu döngü *i* değerinin 10'dan küçük olmadığı duruma kadar devam eder.

DİKKAT



for sözcüğünün bulunduğu satırın sonuna noktalı virgül koyulmaz.

SIRA SİZDE



8

45 ile 55 arasındaki sayıları ekrana yazdıracak bir *for* döngüsü yazın.

WHILE Deyimi

Resim 6.31

While deyiminin genel kullanım yapısı

```
while(ifade)
    deyim;
```

Bir kod parçasının ne kadar tekrarlanacağını döngüye girene kadar bilmiyorsak *while* deyimini kullanırız. While sözcüğünün yanındaki parantez içinde yer alan ifade doğru olduğu sürece deyim işlenmeye devam eder.

Örnek1:

Resim 6.32'de verilen kod parçasında kareleri 30'dan küçük sayılar ekrana yazdırılıyor.

Resim 6.32

(a) While deyimi
Örnek1
(b) Çıktısı

```
sayi    = 1;
karesi  = 1;
while(karesi < 30){
    printf("%d\n", sayi);
    ++sayi;
    karesi = sayi * sayi;
}
```

1
2
3
4
5

Döngüye girmeden önce sayı ve karesi değişkenlerine değer ataması yapılıyor. While deyiminde yer alan ifade karesi değişkeninin 30'dan küçük olduğu durumlarda doğru değer döndürür ve program akışı döngüde kalır. Döngü içindeki ilk deyim o anki sayı değişkeninin değerini ekrana yazdırır. Sonraki deyim sayı değişkeninin değerini 1 artırır. Son olarak karesi değişkeninin değer ataması yapılır ve while ifadesine tekrar gidilir. Bu işlem while ifadesinin değeri yanlış olana kadar devam eder. Bu örnekte sayı değişkeninin değeri 6 olduğunda döngü sonlanacaktır. Program sonlandığında elde edilecek çıktı Resim 6.28 (b)deki şekilde olacaktır.

DO – WHILE Deyimi

For ve while döngülerinde ifadenin doğruluğu döngünün başında hesaplanıyordu. Bu durumda ifade yanlışsa döngüye girilmeden program akışına devam ediyordu. Fakat bazı durumlarda ifadenin değerine bakılmaksızın döngünün en az bir kez çalıştırılması istenir. İşte böyle bir durumda *do – while* deyimini kullanmak doğru olacaktır. Öyle ki döngü *do* anahtar sözcüğü ile başlar ve *while* anahtar sözcüğüne kadar devam eder. While sözcüğünün yanında bulunan ifade değeri doğru olduğu sürece döngü devam eder. Resim 6.29'da *do – while* deyiminin genel kullanım yapısı verilmiştir.

Örnek 1:

Başlangıçta değeri 5 olarak verilen A değişkeninin değerini 10'a kadar birer birer artıran ve A'nın aldığı değerleri ekrana yazdıran program parçasını inceleyelim.

Resim 6.33

```
do{
    deyim1;
    deyim2;
    ...
}while(ifade);
```

do – while deyiminin genel kullanım yapısı

Resim 6.34

```
A = 5;
do{
    printf("A'nın değeri: %d\n", A);
    A = A + 1;
}while(A < 10);
```

do – while deyimi Örnek1

Burada ilk olarak A değişkenine 5 değerinin ataması yapılır. Sonrasında A'nın mevcut değeri ekrana bastırılır ve sonrasında 1 artırılır. Son olarak while sözcüğüne gelindiğinde, parantez içinde verilen ifade hesaplanır ve doğru değer verirse başa dönülüp işlemler tekrarlanır. Eğer yanlış değer elde edilirse döngüden çıkılır. Örnek 1 için verilen kod parçasının çalıştırılması sonucu Resim 6.31'de verilen ekran görüntüsü elde edilir.

Resim 6.35

```
A'nın değeri: 5
A'nın değeri: 6
A'nın değeri: 7
A'nın değeri: 8
A'nın değeri: 9
```

do – while deyimi Örnek1 ekran çıktısı

Özet



C programlama dilinde kullanılan işleçleri tanımlamak. Derleyiciye bazı özel matematiksel veya mantıksal fonksiyonları icra etmesini söyleyen simgelere işleç denir. Tek olarak kullanıldıklarında anlamlı olmayan işleçler, bir veya iki adet işlenen ile kullanılmalıdır.



İşlevlerine göre işleçleri sınıflandırabilecek ve işlem önceliklerini açıklamak.

İşleçleri işlevlerine göre sınıflandırmak mümkündür. **Aritmetik İşleçler:** Aritmetik işleçler denilince ilk akla gelen, cebirdeki dört işlemin (toplama, çıkarma, çarpma, bölme) programlamadaki karşılığını oluşturan işleçlerdir. Ayrıca bu gruba kalan işleci ve artırma, eksiltme işleçlerini de dahil edebiliriz. Artırma ve eksiltme işleçleri tek işlenen ile örnek veya sonek şeklinde kullanılırlar. Diğer işleçler ise anlamlı hâle gelmek için iki işlenene ihtiyaç duyarlar. Kalan işleci sadece tam sayı türündeki işlenenlerle kullanılabilir. Aritmetik işleçlerin kendi aralarındaki öncelik sırası, yüksekten düşüğe: ++, --, *, /, +, - şeklindedir.

İlişkisel İşleçler: İki değeri karşılaştırmak için kullanılan işleçlerdir. Mutlaka iki işlenen ile birlikte kullanılırlar. Büyüktür (>), küçüktür (<), eşittir (==), eşit değildir (!=), büyüktür veya eşittir (>=) ve küçüktür veya eşittir bu gruba giren işleçlerdir. Karşılaştırma sonucunda doğru veya yanlış bir değer üretilir. Üretilen doğru sonucun sayısal karşılığı 1, yanlış sonucun ise 0'dır. İlişkisel işleçler aritmetik işleçlere kıyasla daha düşük önceliklidirler.

Mantıksal İşleçler: Boolean türündeki değişkenlerin mantıksal olarak birbiri ile değerlendirilmesini sağlayan işleçlerdir. Mantıksal VE (AND), VEYA (OR), DEĞİL (NOT) bu grubun işleçleridir. DEĞİL işleci tek işlenen ile örnek şeklinde kullanılırken, VE ve VEYA işleçleri iki işlenene ihtiyaç duyarlar. Mantıksal işleçler için verilen doğruluk tabloları bu işleçlerin anlaşılmasını kolaylaştırmaktadır. VE ve VEYA işleçleri aritmetik ve ilişkisel işleçlere göre daha düşük önceliktedirler. Fakat DEĞİL işleci en yüksek önceliğe sahiptir.

Bitsel İşleçler: Bit seviyesinde işlem yapan işleçlerdir. Sadece tam sayı türündeki işlenenlerle kullanılabilirler. Bitsel VE (&), Bitsel VEYA (|), Bitsel ÖZEL VEYA (^), Bitsel DEĞİL (~), Sola öteleme (<<) ve Sağa öteleme (>>) bu işleç grubundadır. Kullanımları mantıksal işleçlerle benzerlik gösterir. Farklılık bit seviyesinde işlem yapılmasıdır. İki işlenen sayı bütün olarak değil karşılıklı bitler olarak ele alınır.

Atama İşleci: İki işlenenin arasına koyularak kullanılan atama işleci (=), sağdaki değer soldaki değişkene atanması işini yapar. Bu işleç diğer işleçlere göre en düşük önceliğe sahiptir.



Koşul deyimlerini ve nasıl kullanılacaklarını anlatabilmek. Program akışını kontrol etmek ve karşılaştırma yapmak için koşul deyimleri kullanılır. En bilinen koşul deyimini IF'tir. Program akışı if deyimine geldiğinde iki dala ayrılmak durumundadır. IF içinde yer alan ifadenin hesaplanması sonucunda **doğru** veya **yanlış** bir değer üretilir. Doğru olması durumunda bir deyim veya deyim grubu, yanlış olması durumunda başka bir deyim veya deyim grubu işleme alınır. Koşul deyimleri program içinde her defasında sadece bir kere işlenirler. Seçilen dal program akışını belirler ve diğer dal programa dâhil edilmez. Bir seçim değişkenine karşılık ikiden fazla karar verilmesi gerektiği durumda yani ikiden fazla dallanma yapılması gerektiğinde, switch deyimini kullanılır. Switch deyimini ile istenilen sayıda karar üretmek mümkündür. Dikkat edilmesi gereken switch deyimindeki seçim ifadesinin ordinal türde (integer, char) olmasıdır.



Yineleme deyimlerinin yapılarını ve hangi durumlarda hangi yineleme deyiminin kullanılması gerektiğini açıklamak.

Programda bir bölümün sabit sayıda yinelenmesini istediğimizde for deyimini kullanırız. For sözcüğünün yanında bulunan 3 deyim for yinelemesinin çalışma şartlarını belirler. Bunlardan ilki yineleme değişkenine atanan ilk değeri, ikincisi yineleme devam etme şartını ve üçüncüsü yineleme değişkenini artırma veya azaltma işlecini içerir. For yinelemesi içinde birden fazla deyim kullanılması durumunda bloklaya yapılması gerekmektedir. Yinelenmesini istediğimiz kod parçasının ne kadar tekrarlanacağını bilmiyorsa while deyimini kullanabiliriz. While sözcüğünün yanında yer alan ifade doğru olduğu sürece yineleme işlemi devam eder. For ve while deyimlerinde yineleme şartını belirten ifadenin test edilmesi döngünün başında yapılır. Bu sebeple koşul oluşmadığında döngüye hiç girilmeden program akışına devam edebilir. Fakat bazı durumlarda koşula bakılmaksızın döngünün en az bir kere çalıştırılması istenir. Böyle durumlarda ifadenin test edilmesinin döngü sonunda yapıldığı do-while deyiminin kullanılması doğru olacaktır.

Kendimizi Sınavalım

- Derleyiciye bazı özel matematiksel veya mantıksal fonksiyonları icra etmesini söyleyen simgelere ne ad verilir?
 - Yazılım
 - İşlenen
 - İşleç
 - Önek
 - Sonek
- Aşağıdakilerden hangisi Aritmetik İşleçlerden biri **değildir**?
 - Toplama (+)
 - Çıkarma (-)
 - Kalan (%)
 - Büyüktür (>)
 - Çarpma (*)
- İki değerin karşılaştırılmasında kullanılan işleç türü aşağıdakilerden hangisidir?
 - Aritmetik İşleçler
 - Mantıksal İşleçler
 - İlişkisel İşleçler
 - Bit İşleçleri
 - Atama İşleçleri
- Aşağıdaki işleçlerden hangisi diğerlerine göre daha yüksek önceliğe sahiptir?
 - Toplama (+)
 - Çarpma (*)
 - Küçüktür (<)
 - Mantıksal Ve (&&)
 - Mantıksal Değil (!)
- && işleci hakkında aşağıda ifadelerden hangisi **doğrudur**?
 - Tek işlenen ile birlikte kullanılır.
 - İşlem önceliği toplama(+) işlecinden yüksektir.
 - İki nümerik değeri çarpar.
 - İki boolean değeri karşılaştırır.
 - İki boolean değeri birleştirir.
- $A = 8 + 2 < 3 + !1$; ifadesi icra edildiğinde A'nın değeri aşağıdakilerden hangisidir?
 - 0
 - 1
 - 5
 - 9
 - 14
- Aşağıdaki ifadelerden hangisi **doğrudur**?
 - If deyiminde ifadeyi yazarken sonuna noktalı virgül koyulur.
 - If deyimi yinelemeli bir deyimdir.
 - Sadece if – else sözcükleri kullanılarak ikiden fazla koşul değerlendirilebilir.
 - If deyiminde ifadenin değeri doğru veya yanlış olmak zorundadır.
 - Switch deyimi döngü şeklinde çalışır.
- Switch deyiminde ifadenin veri tipi aşağıdakilerden hangisidir?
 - double
 - float
 - integer
 - long double
 - const double
- For deyiminde ifade içinde kaç tane deyim vardır?
 - 1
 - 2
 - 3
 - 4
 - 5

```
A = 8;
do{
    B = A * 2;
    A = A + 1;
}while(A < 10);
```

- Yukarıda verilen kod parçası icra edilip program durduğunda B'nin değeri ne olur?
 - 8
 - 16
 - 18
 - 20
 - 24

Kendimizi Sınyalım Yanıt Anahtarı

1. c Yanıtınız yanlış ise “İşleçler” konusunu yeniden gözden geçiriniz.
2. d Yanıtınız yanlış ise “İşleçler” konusunu yeniden gözden geçiriniz.
3. c Yanıtınız yanlış ise “İşleçler” konusunu yeniden gözden geçiriniz.
4. e Yanıtınız yanlış ise “İşleçler” konusunu yeniden gözden geçiriniz.
5. e Yanıtınız yanlış ise “İşleçler” konusunu yeniden gözden geçiriniz.
6. a Yanıtınız yanlış ise “İşleçler” konusunu yeniden gözden geçiriniz.
7. d Yanıtınız yanlış ise “Koşul Deyimleri” konusunu yeniden gözden geçiriniz.
8. c Yanıtınız yanlış ise “Koşul Deyimleri” konusunu yeniden gözden geçiriniz.
9. c Yanıtınız yanlış ise “Yineleme Deyimleri” konusunu yeniden gözden geçiriniz.
10. c Yanıtınız yanlış ise “Yineleme Deyimleri” konusunu yeniden gözden geçiriniz.

Sıra Sizde Yanıt Anahtarı

Sıra Sizde 1

$x = 5 - 20 / 4 + 5$ ifadesinde işlem önceliği bölme işlecine aittir. İşlem sırası ve işlem sonucu aşağıdaki şekilde olmalıdır.

```
1 20 / 4 = 5
2 5 - 5 = 0
3 0 + 5 = 5
4 x = 5
```

Sıra Sizde 2

$y = 2 + 5 * 14 \% 3$; ifadesinde * ve % işleçlerinin önceliği aynı olduğu için soldan sağa doğru öncelik verilir. Yani işlem sırası aşağıdaki şekilde olur.

```
1 5 * 14 = 70
2 70 % 3 = 1
3 2 + 1 = 3
4 y = 3
```

Sıra Sizde 3

`ToplamArac = AracSayisi * ++sayac`; ifadesinde işlem önceliği artırma işlecindedir. Bu sebeple ilk önce sayac değışkeninin değeri 1 artırılır sonra AracSayisi ile çarpma işlemi yapılır. İşlem sırası ve sonucu aşağıdaki şekildedir.

```
1 ToplamArac = 15 * ++5; // işleme girmeden önce
2 ToplamArac = 15 * 6;
3 ToplamArac = 90;
```

Artırma işlecini sonek olarak kullandığımızda ise önce çarpma yapılır sonra sayac değışkeninin değeri 1 artırılır.

```
1 ToplamArac = 15 * 5++;
2 ToplamArac = 75;
```

Görüldüğü üzere ToplamArac değışkeni hesaplandıktan sonra sayac değeri 1 artırılır ve 6 olur.

Sıra Sizde 4

$Z = 6 == 6 < 10 + 1$; ifadesinde öncelik aritmetik işleç olan toplamaya aittir. Sonrasında sırasıyla eşittir ve küçüktür işleçleri işlenir. İşlem sırası aşağıdaki şekildedir.

```
1 10 + 1 = 11
2 6 == 6 = 1
3 1 < 11 = 1
4 Z = 1
```

Sıra Sizde 5

$Y = 6 + 5 >= 8 \ \&\& \ 3 - 1 <= 4$; biliyoruz ki ilişkisel işleçler mantıksal işleçlere göre önceliklidir. Aritmetik işleçler de ilişkisel işleçlere göre önceliklidir. Bu kurallara göre ifade işlendiğinde aşağıdaki sonucu elde ederiz.

```
1  6 + 5 = 11
2  3 - 1 = 2
3  11 >= 8 = 1
4  2 <= 4 = 1
5  1 && 1 = 1
6  Y = 1
```

Sıra Sizde 6

Bitsel işleçlerde önce VE sonra ÖZEL VEYA işleci işlenir.

A	=	00010010		A & B	=	00000010
B	=	01001011	⇒	B	=	01001011
A & B	=	00000010		A & B ^ B	=	01001001

Sıra Sizde 7

```
if (sayi < 10)
    printf("Sayı 10'dan küçüktür.");
else
    printf("Sayı 10'dan büyüktür.");
```

Sıra Sizde 8

```
for (sayi = 45; sayi < 55; sayi++)
    printf("%d\n", sayi);
```

Yararlanılan ve Başvurulabilecek Kaynaklar

Aslan K., (2001). "A'dan Z'ye C Kılavuzu", Pusula.

Deitel, P., Deitel H. (2010). "C++ How to Program", Pearson.

Lafore R. (2002). "Object Oriented Programming", Sams.

7

Amaçlarımız

Bu üniteyi tamamladıktan sonra;

- Program kodlarında tek boyutlu dizi tanımlamalarını, dizilere ilk değer atamayı, kod içerisinde ihtiyaç duyulduğunda, dizi elemanlarının içeriklerinin değiştirilmesi işlemlerini açıklayabilecek,
- İki ve üç boyutlu dizi tanımlamalarını ve çok boyutlu dizilerin elemanlarına değer atanması işlemlerini anlatabilecek,
- Göstericilerin tanımlamalarını ve diziler ile birlikte kullanımlarını ifade edebilecek,
- Karakterleri içeren dizi olarak kabul edilen dizgileri tanımlayabilecek, karakter dizgileri ile işlem yapabilecek bilgi ve becerilere sahip olabileceksiniz.

Anahtar Kavramlar

- Dizi
- İndis
- İlk Değer Atama
- Bellek
- Çok Boyutlu Dizi
- Gösterici
- Karakter
- Dizgi

İçindekiler

Bilgisayar ve Programlamaya
Giriş

Diziler, Göstericiler ve Dizgiler

- GİRİŞ
- DİZİLER
- GÖSTERİCİLER
- DİZGİLER

Diziler, Göstericiler ve Dizgiler

GİRİŞ

Kitabımızın bu ünitesinde, programlamada çok önemli bir yere sahip olan Diziler, Göstericiler ve Dizgiler konularına değinilmektedir. Her bir konu detaylı şekilde ele alınırken, aynı zamanda açıklayıcı örnekler, Sıra Sizde çalışmaları ve alıştırma soruları ile de desteklenmiştir

DİZİLER

C Programlamada diziler, aynı tipte olan birbirleri ile ilişkili verilerin veya değışkenlerin belli bir indis sırasına göre tek bir değışken adı altında bir arada tutulmasını sağlayan yapılardır. Kodlama ihtiyacına göre, program kodunun çalıştırılmasından itibaren işlem yapılacak veri listesinin belli bir sıraya göre düzenli bir şekilde hafızada tutulmasını ve erişilmesini sağlarlar. Bu bölümde dizilerle ilgili genel tanımlamalar ve işlemler öncelikle tek boyutlu diziler üzerinde açıklanacak olup, ardından çok boyutlu dizi yapılarından da bahsedilecektir.

Dizi Tanımlama

Önceki bölümlerde bahsedilen, veri değışken türlerini tanımlarken, tam sayı tipinde dört farklı değışkene ihtiyaç olduğunda, “*int k1, k2, k3, k4, k5 ;*” şeklinde tanımlama yapılabiliyordu. Ancak bu değışkenlerin hepsi aynı türde ve özellikle de bu değışkenler sırasıyla belli bir algoritmaya göre hesaplamaya tabi tutulacak ise,

int k[5] ;

şeklinde bir dizi tanımlanması daha kullanışlı bir çözüm sunmaktadır. Çünkü program kodlarımızda, ne kadar çok değışken olursa, kod blokları içerisinde kontrol edilmesi de bir o kadar zor olacaktır. Bu örnekte yalnızca 5 farklı değışken için bir dizi oluşturulmuş olmasına rağmen, değışken sayısının çok fazla olup. (örneğin 1000), elle tek tek tanımlanması imkânsız olan durumlarda dizilerin kullanımının önemi ortaya çıkmaktadır. Böylece yinelemeler ve daha sonraki bölümde anlatılacak olan fonksiyonlarla birlikte daha kolay ve düzenli bir veri kullanımına olanak sağladığı görülecektir.

Yukarıda tanımlanan *k* dizisinin elemanlarının hafızadaki dizilim sırası,

78	11	20	65	-30
----	----	----	----	-----

şeklinde kabul edilirse, dizinin elemanları $k[0]=78$, $k[1]=11$, $k[2]=20$, $k[3]=65$, $k[4]=-30$ şeklinde olacaktır.

Dizi tanımlanırken genel ifade,

tip dizi_adı[BOYUT] ;

şeklindeir. *tip* ifadesi *int*, *char*, *float*, *double* vb. değışken türlerinden herhangi birisi olabilir. Köşeli parantez [] ifadesi içerisinde belirtilen, *BOYUT* ise dizi içerisinde saklanacak, işlem yapılacak olan toplam veri sayısı olup, tanımlanan dizinin genel kapasitesini-boyutunu ifade eder. Buna göre, *int k[5]* ifadesi, tam sayı türünde 5 elemanlı *k* adında bir dizi tanımlar.

Dizilerde dikkat edilmesi gereken önemli bir nokta, *BOYUT* kadar elemana sahip bir dizinin ilk elemanının indisi 0 (*sıfır*) ile başlamasından dolayı, dizinin son elemanının indisi değeri sayısal olarak *BOYUT-1* olmaktadır. *int k[5]*; örneğimizde, ilk eleman *k[0]* olmakla birlikte, elemanlar *k[1]*, *k[2]*, *k[3]* şeklinde devam ederek sonuncu, yani dördüncü eleman ise *k[4]* nolu dizi elemanı olacaktır.

Dizinin içinde yar alan elemanların her biri ayrı bir değişken olarak kabul edilmesinden dolayı, değişkenler için geçerli, daha önceki bölümlerde belirtilen değişken tanımlama kuralları dizinin elemanları için de geçerlidir.

Dizilere Değer Atama

Bir dizi *int k[5]*; şeklinde yalnızca isim ve kapasite olarak tanımlandığında, hafızada yalnızca ona ait bir yer tahsis edilmiş olur. Ancak dizi elemanlarına bir değer ataması yapılmaz. Bu nedenle dizinin kapasitesine göre içindeki verilerin sıfırlanması veya atanacak değeri belirlemek amacıyla ilave olarak kodlamaya gereksinim duyulmaktadır. Daha sonraki satırlarda hataya sebep olmaması açısından, genellikle, dizi tanımlanmasının hemen ardından değer sıfırlama ya da yeni değer atama işlemini mutlaka yapmak gerekir.

Dizi elemanlarının ilk değer atanması-sıfırlanması ile ilgili kod örneği aşağıda yer alan

Örnek 7.1 *de bulunmaktadır.*

Örnek 7.1: *Dizi tanımlama, for döngüsü ile diziye ilk değer atama*

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/ /*tanımlanmıştır.*/
int main(void)
{
    int k[ 5];          /*k adında dizi tanımlaması*/
    int i;              /* sayaç-indis değeri*/
    printf("İlk tanımlama sonrası k dizisi elemanları:\n");
    /*Acıklama*/
    for ( i = 0; i < 5; i++ ) /* k dizisinin elemanlarını ekrana yazan yineleme*/
    { printf("k[ %d] = %d\n", i , k[ i]); }
    for ( i = 0; i < 5; i++ ) /*0 dan 4' e kadar toplam 5 adet yineleme indisi*/
    { k[ i] = 0; }          /*Sifirlama*/

    printf("Sifirlama sonrası k dizisi elemanları:\n");    /*Acıklama*/
    for ( i = 0; i < 5; i++ ) /*k dizisinin elemanlarını ekrana yazar*/
    { printf("k[ %d] = %d\n", i , k[ i]); }
    getch();          /*Program çıktısını ekranda bekletmek için*/
    return 0; }
```

Program Çıktısı (Örnek 7.1)

Tanımlama Sonrası k dizisi elemanları:

```
k[0] = 0
k[1] = 0
k[2] = 2686832
k[3] = 328
k[4] = 320
```

Sıfırlama Sonrası k dizisi elemanları:

```
k[0] = 0
k[1] = 0
k[2] = 0
k[3] = 0
k[4] = 0
```

Değişken tanımlama kuralları önceki ünitelerde anlatılmaktadır. Detaylı bilgi için ilgili bölüme bakınız.

Örnek program kodlarındaki satırların bitiminden sonra `/*` ve `*/` işaretleri arasında yer alan metinler derleyici tarafından program satırı olarak dikkate alınmaz ve derlenmez. Bu işaretler arasına yazılan metinler ilgili satırı açıklamak amacıyla yazılmıştır.



DİKKAT

Örnek 7.1'de, 5 elemana sahip k dizisinin ilk olarak `int k[5]`; şeklinde tanımlanmasından hemen sonra, değer atama öncesinde, sayısal olarak hangi değerlere sahip olduğu aşağıdaki,

```
for ( i = 0; i < 5; i++ ) /* k dizisinin elemanlarını ekrana yazan yineleme */
{ printf( "k[%d] = %d\n", i, k[i] ); }
```

satırları yardımıyla ekrana yazdırılmaktadır. Bu amaçla daha önce Ünite 6'da anlatılan *for* döngü-yinelemesi kullanılmıştır. Böylece *for* yinelemesinde kullanılan i indis değeri yardımıyla, k dizisinin 0. indisinden 4. indisine kadar tüm elemanlarına erişilebilmektedir. *printf* komutu yardımıyla da her elemanın ekrana yazdırma işlemi gerçekleştirilmektedir. *for* yinelemesinde i değeri her seferde $i++$ ile 1 arttırılmaktadır. Yineleme süresince toplamda 5 defa *printf* satırı aktif olmaktadır.

Main fonksiyonun son bölümünde kullanılan *getch()* fonksiyonu kodun derlenip çalıştırılmasından sonra ekrana yazdırma işlemi gerçekleştiriliyorsa bu metinlerin ekranda gösterilmesini sağlar. *getch()* fonksiyonunu kullanabilmek için `#include <conio.h>` şeklinde *conio.h* kütüphane dosyasının önışlemciler bölümüne eklenmesi gerekmektedir.

$k[\%d]$ içinde kullanılan i sayaç değerine karşılık gelen $\%d$ değerleri mutlaka tam sayı olmalıdır.



DİKKAT

Kodun devamında ise;

```
for ( i = 0; i < 5; i++ ) /*0 dan 4' e kadar toplam 5 adet yineleme indisi*/
{ k[ i ] = 0; } /* Sıfırlama */
```

yinelemesi ile de k dizisinin elemanlarının her biri sıra ile sıfırlanmaktadır. Kodun bu bölümünde, yalnızca değer ataması yapılmakta olup ekrana yazdırma işlemi bulunmamaktadır.

Son bölümde yer alan *for* yinelemesi;

```
for ( i = 0; i < 5; i++ ) /*k dizisinin elemanlarını ekrana yazar*/
{ printf( "k[%d] = %d\n", i, k[i] ); }
```

bölümü ile, ilk bölümdeki *for* yinelemesinde yapılan işlemin aynısı gerçekleştirilerek, sıfırlama işlemi sonrasında k dizisinin içeriği tekrar ekrana yazdırılmaktadır.

Örnek 7.1' deki kod satırları derleyici ile derlenerek çalıştırıldığında, program kodunun devamında verilen *program çıktısında* yer alan satırlar ekrana yazdırılacaktır. Kod içerisinde her *for* yinelemesi öncesinde kullanılan *printf* fonksiyonu ile hangi işlemin gerçekleştirilmekte olduğu ekrana yazdırılmaktadır. Bu program çıktısı incelendiğinde, dizilerin elemanlarına ilk değer atanmasının önemiyle ilgili dikkat çekici bir detay yer almaktadır. Çünkü içeriğine hâkim olunmayan dizi elemanlarını kullanarak işlem yapmak kodun yanlış ve beklenmeyen sonuçlar vermesine sebep olacaktır. Örnek 7.1' in program çıktısından da anlaşılacağı üzere, tanımlanan ve ilk değer atanmadan önce, k dizisinin bazı elemanları ($k[0]$ ve $k[1]$) tesadüfen 0 değerine sahip olarak gözüke de, diğer değerler, beklenmeyen, oldukça farklı sayısal değerlere sahiptir. Bu nedenle tanımlanan bir dizi, kodun devamında daha sonra başka bir hesaplamada kullanılacak ise, bu kullanım öncesi, dizinin elemanlarının ilk anda sıfırlanması büyük önem kazanmaktadır.

Örnek 7.1' de yer alan 3 farklı *for* yinelemelerinin her biri birbirinden bağımsız olmakla birlikte, her biri tek satırlık işlemlere sahip olmasından dolayı aslında `{ }` süslü parantezlere gerek bulunmamasına rağmen karışıklık oluşturmaması açısından `{ }` ler kullanılmıştır. *for* yinelemelerinin her birinde indis değeri i olarak kullanılmasında bir sakınca bulunmamaktadır. Çünkü her *for* yinelemesi kendi içerisinde 0'dan 4'e kadar saymakta, sonraki *for* yinelemesine geçildiğinde, i sayaç değeri tekrar 0 dan başladığından bir karışıklığa sebep olmamaktadır.



DİKKAT

Dizilerin indislerinin kullanımı ile ilgili olarak, $k[i]$ dizi ifadesindeki dizinin kaçınıcı elemanı olduğunu gösteren i indis değeri, kod içerisindeki kullanımında, yalnızca sayısal değer olmak zorunda değildir. Kod içerisinde değişik sembolik ifade yapıları şeklinde de kullanılabilir. Bu kullanımına örnek olarak aşağıda Örnek 7.2 içerisindeki kod yer almaktadır.

Örnek 7.2: Dizi tanımlama, sıfırlama, for döngüsü ile değer atama

```
#include <stdio.h>
#include <conio.h>          /* getch fonksiyonunu kullanabilmek için */
int main(void)
{
    int k[5];               /* k adında dizi tanımlaması */
    int i;                  /* sayac-indis değeri */
    printf("Tanımlama sonrası k dizisi elemanları:\n");
    for ( i = 1; i <= 5; i++ )    /* k dizisinin elemanlarını ekrana yazan yineleme */
    { printf(" k[ %d] = %d\n", (i-1) , k[ i-1] ) ; }

    for ( i = 1; i <= 5; i++ )    /* 0 dan 4'e kadar toplam 5 adet indis değeri */
    { k [ i-1] = 0; }            /* Sıfırlama */

    printf("Sıfırlama sonrası k dizisi elemanları:\n");

    for ( i = 1; i <= 5; i++ )    /* k dizisinin elemanlarını ekrana yazan yineleme */
    { printf(" k [ %d] = %d\n", (i-1) , k[ i-1] ) ; }

    printf("k dizisi elemanlarına değer atama:\n");

    for ( i = 1; i <= 5; i++ )    /* 0 dan 4'e kadar toplam 5 adet indis değeri */
    { k [ i-1] = 3* (i-1) ;
      printf(" k[ %d] = %d\n", (i-1) , k[ i-1] );          /* Indise bağlı değer atama, 3*indis */
    }

    getch();                 /* Program çıktısını ekranda bekletebilmek için */
    return 0; }

```

Program Çıktısı (Örnek 7.2):

Tanımlama Sonrası k dizisi elemanları:

k[0] = 0
k[1] = 0
k[2] = 2686832
k[3] = 328
k[4] = 320

Sıfırlama Sonrası k dizisi elemanları:

k[0] = 0
k[1] = 0
k[2] = 0
k[3] = 0
k[4] = 0

k dizisi elemanlarına değer atama:

k[0] = 0
k[1] = 3

```
k[2] = 6
k[3] = 9
k[4] = 12
```

Örnek 7.1'deki program kodunda, dizi elemanlarını ekrana yazdırırken ve sıfırlarken yineleme ifadesinde indis değeri 0 (sıfır), dan başlatılarak kullanılmıştır. Örnek 7.2 de ise, *i* değeri 1 (bir) den başlatılmış olmasına rağmen dizi elemanının içeriğine ulaşılırken *i-1* şeklinde indis ayarlaması yapılarak doğru indis değerleri ile dizinin elemanlarına ulaşılmıştır.

Örnek 7.2'de kod detaylı olarak incelendiğinde, öncelikle *k* dizisi tanımlaması sonrası Örnek 7.1'deki koda benzer olarak dizi elemanları,

```
for ( i = 1; i <= 5; i++ ); /*k dizisinin elemanlarini ekrana yazan yineleme*/
{ printf( "k[%d] = %d\n", (i-1), k[i-1] ); }
```

satırları yardımıyla ekrana yazdırılmıştır. Yukarıda da belirtildiği üzere for döngüsünde *i* indis değeri 1'den saymaya başladığı için indise ulaşılırken *i-1* ifadesi kullanılmıştır. Çünkü daha önce de belirtildiği üzere dizilerin elemanları 0. indisten başlamaktadır.

Kodun son bölümde yer alan,

```
for ( i = 1; i <= 5; i++ ) /* 0 dan 4' e kadar toplam 5 adet indis degeri */
{ k[i-1] = 3*(i-1);
  printf( "k[%d] = %d\n", (i-1), k[i-1] ); /*Indise bağlı değer atama, 3*indis*/ }
```

yinelemesinde ise, *k* dizisinin elemanlarına kendi indisine bağlı bir formül (*indis değerinin 3 katı*) ile değer ataması gerçekleştirilmiştir. Kod derlenip çalıştırıldığında elde edilecek program çıktısı kodun alt bölümünde yer almaktadır. Bu çıktıdan da görüldüğü üzere, *k* dizisinin elemanları *k[0]=0, k[1]=3, k[2]=6, k[3]=9, k[4]=12* şeklinde ilgili indis değerinin sayısal olarak 3 katı şeklinde atanmıştır.

Dizinin elemanları sıfırlama yapılmadan, doğrudan indisin 3 katı ile çarpma gerçekleştirilmiş olması durumunda değerler yanlış hesaplanırdı.



DİKKAT

Yukarıdaki Örnek 7.2 nin aksine,

```
for ( i = 1; i <= 5; i++ ) /* 0 dan 4' e kadar toplam 5 adet indis degeri */ { k[i-1] = 3*(i-1);
  printf("k[%d] = %d\n", (i-1), k[i-1]); /*Indise bağlı değer atama*/ }
```

Kod bölümünde

```
{ k[i] = 3*i; printf("k[%d] = %d\n", i, k[i]); /*Indise bağlı değer atama*/ }
```

kullanılması durumunda k dizisinin elemanları hangi değerlere sahip olacaktır?



SIRA SİZDE

Dizilerin boyut bilgilerinde kullanılabilecek ve programlamada büyük esneklik sağlayan bir diğer yapı *# define* ifadesidir. Main fonksiyonundan önce önışlemci tanımlılık bölümünde kullanılan bu yapı önışlemci komutudur.

#define BOYUT 5 ifadesi, sembolik sabit olan *BOYUT*'un değerinin 5 olduğunu belirtir. Önışlemci tarafından, kodun derleme işleminden önce dikkate alınarak *BOYUT* yazan her yere 5 ifadesini yerleştiren bir tanımdır.

Kodlar yazılırken pratik olarak değişim gerekli olduğunda, dizilerin maksimum boyutları(kapasiteleri) gibi değişebilir değerlerde genel ifadeler kullanmak büyük kolaylık sağlayacaktır. Örneğin kod ilk yazıldığında ve çalıştırıldığında, içinde tanımlanan dizinin 100 elemana sahip olması yeterli iken, daha sonraki kod güncellemelerinde dizi elemanının kapasitesinin 1000'e çıkarılması gerekirse, dizi boyutunun kullanıldığı her yerde 100 sayısının 1000 sayısı ile ayrı ayrı değiştirilmesi gerekecektir. Bu değişikliğin yerine *# define BOYUT 1000* ifadesi ile bu işlem çok daha pratik bir şekilde, tek bir satırdaki değişiklikle

gerçekleştirilebilmektedir. `#define BOYUT 100` ifadesindeki, `100` değerini `1000` sayısı ile değiştirmek yeterli olacaktır.

Aşağıda yer alan Örnek 7.3'te `#define` komutu kullanılarak dizinin kapasitesi-boyutu belirlenmiş olup, önceki iki örnekten (Örnek 7.1 ve Örnek 7.2) farklı olarak dizi elemanlarına değer atama işlemi klavyeden kullanıcı girişleri ile yapılmaktadır.

Örnek 7.3: Dizilerde `#define` tanımının kullanılması, dizi elemanlarının kullanıcı tarafından klavye girişi ile atanması

```
#include <stdio.h>
#include <conio.h>          /* getch fonksiyonunu kullanabilmek için */
#define BOYUT 5
int main(void)
{
    int k [ BOYUT ]; /* k adında dizi tanımlaması */
    int i;           /* sayac-indis değeri */

    printf("k dizisi elemanlarına %d adet değer atayınız\n", BOYUT);

    for ( i = 1; i <= BOYUT; i++ ) /* 0 dan 4' e kadar toplam 5 adet indis */
    {
        printf(" k[ %d] değerini giriniz:", (i-1) );
        scanf ("%d ", &k[i-1] );
    }
    printf("\n k dizisine klavyeden girdiğiniz %d adet sayı değerleri :\n", BOYUT);

    for ( i = 1; i <= BOYUT; i++ ) /* 0 dan 4' e kadar toplam 5 adet indis */
    {
        printf(" k[ %d] = %d \n", (i-1) , k[i-1] ); }

    getch(); /*Program çıktısını ekranda bekletebilmek için */
    return 0; }
```

Program Çıktısı (Örnek 7.3)

k dizisi elemanlarına 5 adet değer atayınız

k[0] değerini giriniz: 10

k[1] değerini giriniz: 20

k[2] değerini giriniz: 30

k[3] değerini giriniz: 40

k[4] değerini giriniz: 50

k dizisine klavyeden girdiğiniz 5 adet sayı değerleri :

k[0] : 10

k[1] : 20

k[2] : 30

k[3] : 40

k[4] : 50

DİKKAT



Örnek 7.3 de, `scanf` fonksiyonu ile klavyeden kullanıcının her girişinden sonra, bir sonraki değeri girebilmek için ENTER tuşuna basılması gerekmektedir.

Örnek 7.3'teki kodun detayları incelendiğinde,

```
for ( i = 1; i <= BOYUT; i++ ) /*0 dan 4' e kadar toplam 5 adet indis */
{ printf(" k[%d] değerini giriniz:",(i-1) ); scanf ("%d",&k[i-1] ); }
```

kod bölümü ile, sırasıyla k dizisinin her bir elemanına klavyeden giriş sağlandığı görülmektedir. `scanf ("%d", &k[i-1] );` ifadesi klavyeden giriş yapılmasını sağlamaktadır. `define`

işlemcisi ile BOYUT 5 olarak belirlendiği için, toplam 5 adet değer girişi için for yinelemesi ile değer atanmaktadır.

```
for ( i = 1; i <= BOYUT; i++ ) /* 0 dan 4' e kadar toplam 5 adet indis*/
{ printf( "k[%d] = %d\n", (i-1), k[i-1] ); }
```

kod bölümü ise, önceki örneklerde olduğu gibi dizi elemanlarını ekrana yazdırmak için kullanılır.

Dizi tanımlamalarında üzerinde durulması gerekli bir diğer önemli durum ise, aynı anda, aynı türden birden fazla dizi tanımlamasıdır. Bu durumla ilgili olarak, çoklu değişken tanımlamada da geçerli olan kurallar diziler için de geçerli olup her yeni dizi birbirinden virgül işareti “,” ile ayrılarak tanımlanmaktadır.

Örnek olarak, “`int k[5], not[50];`” şeklindeki tanımlamada tam sayı türünden, birbirinden bağımsız içeriklere sahip, 5 elemanlı *k* ve 50 elemanlı not dizileri aynı satırda tanımlanabilmektedir.

Dizi elemanlarına değer atanmasında kullanılan bir diğer yöntem ise, ilk olarak dizi tanımlaması yapılırken, süslü parantezlerin {, } kullanılmasıdır. Örneğin *arasinav1* adında 5 tane değere sahip bir dizinin eleman değerleri önceden biliniyorsa, doğrudan atanmak istendiğinde,

```
int arasinav1[5]={50, 80, 60, 90, 70};
```

şeklinde belirtilebilir.

Ancak, bu doğrudan atama işlemleri dizi tanımlaması yapılırken, {, } arasına dizinin eleman sayısından daha az değişken değeri yazılması durumunda eksik bırakılan değerler en sondaki değerler kabul edilerek bu değerlere doğrudan 0 değeri atanır. Örnek olarak, `int arasinav2[5]={60,70,80};` şeklindeki bir tanımlamada *a[3].* ve *a[4].* elemanlar 0 kabul edilir.

`int ort[5]={ 0 };` şeklindeki tanımlamada ise tüm elemanlar 0 değerini alır. Bu tanımlamada, ilk değer *ort[0].* eleman otomatik olarak 0 değerini alır, sonraki elemanlar yukarıdaki *arasinav2* dizisinde olduğu gibi eksik tanımlandığı için doğrudan sıfırlanır. Diğer bir ifade ile, diziye en az bir değer atanması ile birlikte geri kalan diğer değerler de otomatik olarak sıfırlanmaktadır.

`int final [ ]={40, 70, 50, 30, 80};` tanımlamasında ise {} parantezler içerisinde 5 adet sayı tanımlanmış olup, [] köşeli parantez içerisinde dizinin kapasitesi ile ilgili bir değer yazılmamış olmasına rağmen {} parantezleri içerisinde 5 değer olduğu için final dizisinin boyutu derleyici tarafından 5 olarak algılanır.

int k[4]={40, 70, 50, 30, 80}; şeklinde { }ler arasına, dizinin kapasitesinden daha fazla sayı yazılması durumunu yazacağınız program koduyla analiz ediniz.



SIRA SİZDE

Dizilere doğrudan atama işlemi ve farklı dizilerin elemanları arasında aritmetik işlem yapılması, elde edilen sonuçların da yine başka bir üçüncü dizinin elemanlarına atanması ile ilgili örnek satırlar aşağıda Örnek 7.4'te yer almaktadır.

Örnek 7.4: Dizilere ilk değer atama, dizi elemanları ile hesaplama,

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
#define BOYUT 5
int main(void)
{
    int arasinav1 [ BOYUT]={ 50, 80, 60, 90, 70}; /*arasinav1 adinda
dizi tanimlamasi*/
    int arasinav2 [ BOYUT]={ 60,70, 80}; /*arasinav2 adinda dizi ta-
nimlamasi*/
    int final[ ]={40,70,50,30,80}; /*final adinda dizi tanimlamasi*/
    float ort[ BOYUT]={ 0 };
    int i; /*sayac-indis degeri*/

    printf("%d ogrenciye ait Arasinav-Final-Ortalama Not Listesi\
n\n", BOYUT);
    printf("SIRA_NO ARASINAV_1 ARASINAV_2 FINAL ORTALAMA\n");
    printf("*****\n");
    for ( i = 0; i < BOYUT; i++ ) /* 0 dan 4' e kadar toplam 5 adet
indis degeri */

    { ort [ i ]= ( (arasinav1[ i]*0.3) + (arasinav2 [ i ] * 0.3 ) + (
final[ i ] * 0.4 ) );

/*not
ortalamaesi hesaplaniyor*/

    printf("%2d %10d %10d %8d %11.2f \n", (i+1), arasinav1[ i
], arasinav2 [ i ], final [ i ], ort [ i ]);
    }
    getch(); /*Program ciktisini ekranda bekletebilmek için*/
    return 0; }
```

Program Çıktısı (Örnek7.4):

5 ogrenciye ait Arasinav-Final-Ortalama Not Listesi

SIRA NO	ARA SINAV 1	ARA SINAV 2	FİNAL	ORTALAMA
1	50	60	40	49.00
2	80	70	70	73.00
3	60	80	50	62.00
4	90	0	30	39.00
5	70	0	80	53.00

Örnek 7.4'te yer alan kod satırlarında, 5 öğrenciye ait arasinav1, arasinav2, final değerleri ilk olarak dizi tanımlaması sırasında doğrudan atanmıştır. arasinav2 dizisinde yalnızca 3 adet not değeri tanımlanmıştır. Son iki öğrencinin arasinav2 ye girmediği kabul edilmekte ve yukarıda anlatılan tanımlama yapısından dolayı son iki eleman sıfır (0) değerlerine sahip olacaklardır. ort dizisi ise tanımlama esnasında ilk değeri 0 yapılarak tüm elemanları sıfırlanması sağlanmıştır. Kod içerisinde daha sonra, arasinavların sırasıyla ikisi de %30, final sınavı ise %40 oranında katkısı sağlanarak öğrencinin genel ortalama notu ort dizisinin elemanlarına atanacak şekilde hesaplanmıştır. Bu örnek kod içinde, her üç dizide de aynı indisli elemanlar aynı öğrenciye ait olduğu kabul edilmektedir. Tüm değerler ve hesaplama sonucu çıkan değerler, formatlı bir tablo şeklinde program çıktısında ekranda listelenmektedir.

`printf` fonksiyonu içinde yer alan `%2d %10d %10d %8d %11.2f` gibi ifadeler ekrana yazdırılacak değişkenlerin formatlı-hizalı olarak yazılmasını sağlamaktadır. Art arda boşluk bırakmadan `%d%d` şeklindeki kullanımda ekranda bitişik yazılacaklardır. Kullanım ile ilgili detaylı bilgiyi C Programlama Dili:Doç.Dr.İbrahim Güney-2. Baskı kitabında Bölüm 2'den alabilirsiniz.



K İ T A P

İki ve Üç Boyutlu Diziler

İki boyutlu diziler, tek boyutlu dizilerden farklı olarak birden fazla satır ve sütunlardan oluşan tablolardır. Diğer bir ifade ile, Şekil 7.1 de gösterildiği şekilde hücrelerden oluşan yapılardır. Şekil 7.1 de hem tek boyutlu hem de iki boyutlu dizi gösterimi bulunmaktadır. Aynı kapasite-eleman sayısına sahip, tek boyutlu ve iki boyutlu diziler kapasite açısından karşılaştırılırsa, tek boyutlu bir dizinin elemanlarının dizilimi $1 \times m \times n$ şeklinde iken, iki boyutlu dizinin elemanlarının dizilimi ise $m \times n$ şeklindedir.

Şekil 7.1

Tek Boyutlu ve
İki Boyutlu Dizi
Gösterimi

b[0]	b[1]	b[2]	b[3]
------	------	------	------

Tek Boyutlu Dizi

b[0][0]	b[0][1]	b[0][2]	b[0][3]
b[1][0]	b[1][1]	b[1][2]	b[1][3]
b[2][0]	b[2][1]	b[2][2]	b[2][3]
b[3][0]	b[3][1]	b[3][2]	b[3][3]
b[4][0]	b[4][1]	b[4][2]	b[4][3]

İki Boyutlu Dizi (5x4)

Tek boyutlu dizilerde indis değerleri tek bir yönde birer birer artarken, iki boyutlu dizilerde iki yönde de indis değerleri değişmektedir. Böylece, iki boyutlu bir dizideki bir elemanın içeriğine erişmek için hangi satır ve sütun değerine sahip ise, iki boyutlu bir düzlemdeki bir koordinat şeklinde; dikeyde y , yatayda x bir değişken gibi kabul edilir ise, $b[y][x]$ şeklinde, x ve y indis değerleri ile ulaşılır. Dizinin satır ve sütun için ilk indis değeri tek boyutlu dizilerde olduğu gibi yine 0'dan başlamaktadır. m satır, n sütun sayısı olmak üzere, $m \times n$ 'lik bir dizinin $m \times n$ adet farklı değere sahip eleman sayısı mevcuttur. Her satırda, sonuncu eleman $b[y][n-1]$, her sütunda sonuncu eleman $b[m-1][x]$ elemandır.

Çok boyutlu dizilerin tanımlanması, ilk değer ataması, tek boyutlu dizilerle benzerlikler taşımaktadır. Ancak iki boyutlu dizilerin elemanlarına erişiminde yineleme kullanımı, indis elemanlarına ulaşmak için, satır ve sütun için birbirinden farklı sayaç-indis değerleri kullanmak gerekir.

İki boyutlu dizinin elemanlarına erişimde $b[y][x]$ yerine yanlışlıkla $b[y, x]$ şeklindeki kullanım hatalı bir kullanımdır. C Programlamada derleyici tarafından $b[y, x]$ ifadesi $b[x]$ olarak algılanır.



D İ K K A T

Süslü parantez $\{, \}$ yapıları kullanılarak, tek boyutlu dizilerde olduğu gibi iki boyutlu dizilerde de dizinin ilk tanımlanması aşamasında ilk değer atanabilmektedir. En dışarda $\{ \}$ lerin içinde alt $\{ \}$ ler yardımıyla iki boyutlu dizinin satır elemanları belirlenebilir.

$float\ matematik\ [5][4] = \{ \{40, 60, 60\}, \{50, 80, 85\}, \{70, 50, 80, 0\}, \{90, 70, 70, 0\}, \{80, 50, 90\} \};$

şeklinde tanımlanan matematik dizisinin elemanları,
 ilk alt parantezdeki değerler $\{40, 60, 60\}$, 0. satıra,
 ikinci alt parantezdeki değerler $\{50, 80, 85\}$, 1. satıra,
 üçüncü alt parantezdeki değerler $\{70, 50, 80, 0\}$, 2. satıra,
 dördüncü alt parantezdeki değerler $\{90, 70, 70, 0\}$, 3. satıra,
 beşinci alt parantezdeki değerler $\{80, 50, 90\}$, 4. satıra
 ait değerlerdir.

Tanımlanan dizi matematik dersindeki kayıtlı 5 öğrenciye ait 1. arasınav, 2. arasınav, final notları ile dönem sonu notlarını içerdiği kabul edilirse 5 öğrenci için de, 0. sütun, 1. *arasınav*, 1. sütun 2. *arasınav*, 2. sütun *final*, tüm değerleri 0 olarak tanımlanan 3. sütun ise *dönem sonu* notlarını içeren sütunlar olarak varsayılabilir. 0., 1. ve 4. satırlarda son değer sıfır (0) olarak otomatik olarak oluşturulur. Çünkü dizinin sütun sayısı 4 olarak belirtilmiştir.

$float\ turkce[5][4] = \{ \{10, 50, 100, 0\}, \{40, 80, 50\}, \{50, 30, 30, 0\}, \{80, 70, 0, 0\}, \{90, 95\} \};$

ifadesinde $turkce[3][2]$. değeri 0 ve $turkce[4][2]$. değerleri 0 olarak atanacaktır. Bu durumla ilgili olarak, 4. ve 5. öğrenciler final sınavına girmediği için sınav notlarının bulunmadığı varsayılabilir.

Bu iki dizinin içerdiği değerleri Matematik ve Türkçe derslerini alan aynı 5 öğrenciye ait sınav notları olarak değerlendirilir ise, aşağıda Şekil 7.2’de yer alan tablodaki değerler gibi ele alınabilir. Son kolonda yer alan 5 öğrenciye ait, ilk anda sıfır olarak gösterilen dönem sonu notları ise, 1. *arasınav*, 2. *arasınav*, *final* notlarının aritmetik ortalaması olarak kod içerisinde daha sonra hesaplanabilir. Yukarıda anlatıldığı üzere Türkçe dersinde 4. ve 5. öğrenciler final sınavına katılmadıkları için bu sınavlar dizilerde sıfır(0) olarak içerilmektedirler. Yukarıda bahsedilen matematik ve turkce dizilerinin kullanımı ile ilgili kod örneği ve program çıktısı aşağıda Örnek 7.5’te bulunmaktadır. *matematik* ve *turkce* dizilerinde 1. *Arasınav*, 2. *arasınav* ve *final* notları ilk değer atama yöntemi ile atanmış olup, dönem sonu notu kod içerisinde hesaplanarak her iki dizinin son kolonuna atanmıştır.

Şekil 7.2

Sıra No	1. Ara Sınav	2. Ara Sınav	Final	Dönem Sonu
1	40	60	60	0
2	50	80	85	0
3	70	50	80	0
4	90	70	70	0
5	80	50	90	0

Matematik Dersi Öğrenci Not Listesi

Sıra No	1. Ara Sınav	2. Ara Sınav	Final	Dönem Sonu
1	10	50	100	0
2	40	80	50	0
3	50	30	30	0
4	80	70	0	0
5	90	95	0	0

Türkçe Dersi Öğrenci Not Listesi

Örnek 7.5: İki boyutlu matematik ve türkce dizilerinde donem sonu notu hesaplama ve değerleri ekrana yazdırma

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek icin*/
#define OGR 5
int main()
{ float matematik[ OGR][ 4]={ { 40, 60, 60},{ 50, 80, 85}, { 70, 50,
80,0},{ 90, 70, 70, 0}, { 80,50,90} };

float turkce[ OGR][ 4]={ { 10,50, 100, 0},{ 40, 80, 50}, { 50, 30, 30,
0},{ 80, 70, 0, 0},{ 90, 95} };

int i,j; /*sayac-indis degeri*/

/* matematik dersi*/
/*donem sonu notu hesaplama 3 notun ortalamasi*/

j=0;
for ( i = 0; i < OGR; i++ )
{ matematik[ i][ j+3]=( matematik[ i][ j] + matematik[ i][ j+1] +
matematik[ i][ j+2] ) /3;}

printf(" Matematik Dersi Notlar \n"); /*aciklama*/
printf(" 1.ASinav 2.ASinav Final Donem Sonu\n"); /*acikla-
ma*/
printf("***** \n");
for ( i = 0; i < OGR; i++ ) /*matematik dizisinin elemanla-
rini ekrana yazar*/
{ for( j=0; j<4; j++)
{ printf(" %8.2f", matematik[ i][ j] ); }
printf("\n");
}

/* turkce dersi*/
/*donem sonu notu hesaplama 3 notun ortalamasi*/
j=0;
for ( i = 0; i < OGR; i++ )
{ turkce[ i][ j+3]=( turkce[ i][ j] + turkce[ i][ j+1] + turkce[ i][ j+2]
)/ 3;}

printf(" \n Turkce Dersi Notlar \n");
/*aciklama*/
printf(" 1.ASinav 2.ASinav Final Donem Sonu\n"); /*aciklama*/
printf("*****\n");
for ( i = 0; i < OGR; i++ ) /*turkce dizisinin elemanlarini
ekrana /*yazar*/
{ for( j=0; j<4; j++)
{ printf ( " %8.2f", turkce[ i][ j] ) ; }
printf("\n");
}
getch();
return 0;
}
```

Program Çıktısı (Örnek 7.5):**Matematik Dersi Notlar**

1. Ara Sınav	2. Ara Sınav	Final	Dönem Sonu
40.00	60.00	60.00	53.33
50.00	80.00	85.00	71.67
70.00	50.00	80.00	66.67
90.00	70.00	70.00	76.67
80.00	50.00	90.00	73.33

Türkçe Dersi Notlar

1. Ara Sınav	2. Ara Sınav	Final	Dönem Sonu
10.00	50.00	100.00	53.33
40.00	80.00	50.00	56.67
50.00	30.00	30.00	36.67
80.00	70.00	0.00	50.00
90.00	95.00	0.00	61.67

Örnek 7.5'te yer alan

```
j=0;
for ( i = 0; i < OGR; i++ )
{ matematik[i][j+3] = ( matematik[i][j] + matematik[i][j+1] + matematik[i][j+2])
/3;}
```

kod bölümü ile matematik dizisinin 3 indis nolu, öncesinde değer içermeyen son sütundaki elemanlar, 3 sınavın ortalaması alınarak hesaplanmakta ve ilgili elemana atanmaktadır. Atama işleminin ardından,

```
for ( i = 0; i < OGR; i++ ) /*matematik dizisinin elemanlarini ekrana yazar*/
{ for( j=0; j<4; j++)
{ printf(“ %8.2f”, matematik[i][j]); }
printf(“\n”); }
```

kod bölümü ile, iç içe kullanılan *for* yinelemeleri ile dizinin içerdiği elemanlar satır ve sütunlar üzerinde ilerleyerek ekrana yazdırılmaktadır. Bu yineleme sayesinde sırasıyla, matematik[0][0], matematik[0][1], matematik[0][2], matematik[0][3], matematik[1][0], matematik[1][1], matematik[1][2], matematik[1][3], matematik[2][0], matematik[2][1], matematik[2][2], matematik[2][3], matematik[2][0], matematik[2][1], matematik[2][2], matematik[2][3], matematik[4][0], matematik[4][1], matematik[4][2], matematik[4][3], elemanlarına ulaşılmaktadır. İçerde yer alan *j* sayaç değerini kullanan *for* yinelemesi her sütundaki elemanları üzerinde hareket ederken, en dışarda yer alan *i* indisli *for* yinelemesi ise dizinin satırlarını üzerinde hareket etmektedir. Program kodunda matematik dizisi için yapılan işlemlerin aynısı türkçe dizisi için de gerçekleşmiştir. Elde edilen not ortalamaları her iki ders için de tablolar hâlinde Örnek 7.5'in program çıktısında yer almaktadır.

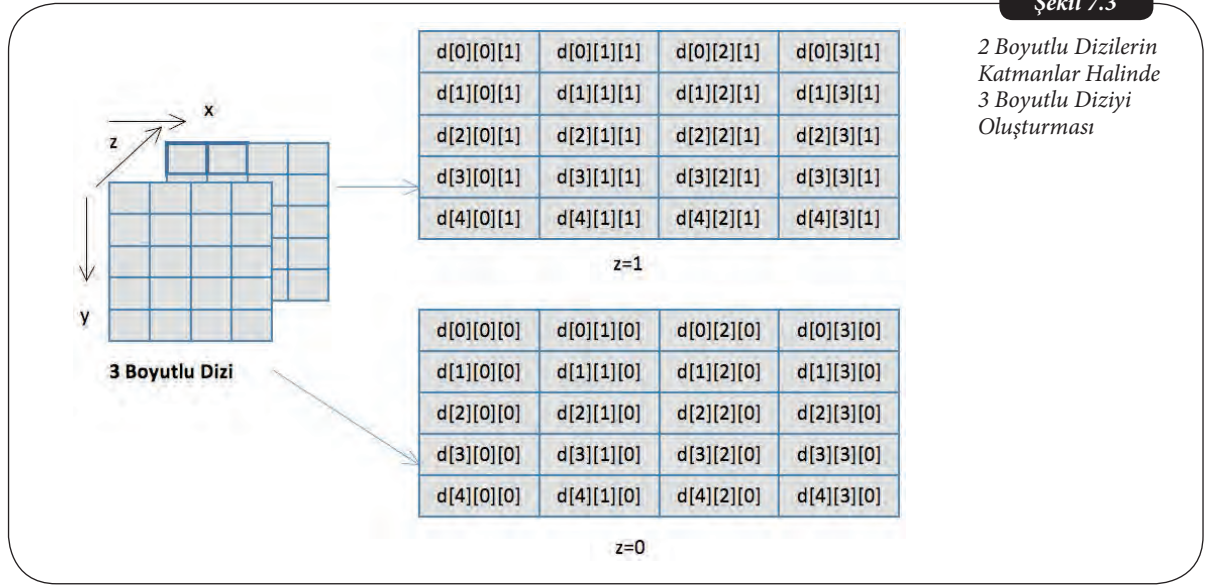
SIRA SİZDE



5x4 lük D dizisi “ float D [5][4] = { 40, 60, 60 50, 80, 85, 70, 50, 80, 0, 90, 70, 70, 0, 80, 50}; “ şeklinde tanımlanmış kabul edilirse, sayıların satır ve sütunlara dizilimi nasıl olacaktır?

Şimdiye kadar tek ve iki boyutlu diziler hakkında bilgi verilerek bunlarla ilgili örnekler açıklanmıştır. İki boyutlu dizilerin katmanlar hâlinde bir araya gelmesiyle oluşan 3 boyutlu diziler de öncekiler gibi aynı özelliklere sahiptir. Verilerin daha düzenli bir şekilde bir arada

tutulmasını sağlarlar. Örnek 7.5'te tanımlanan *matematik* ve *turkce* dizileri 3 boyutlu bir dizide bir arada toplanmak istenirse, dersler adında bir dizi tanımlanıp, *matematik* dizisi dersler dizisinin 3. boyuttaki 0. indisinde, *turkce* dizisi ise 3. boyuttaki 1. indisinde yer alabilir. Şekil 7.3'te, 2 boyutlu dizilerin bir araya getirilerek oluşturulduğu 3 boyutlu dizi yapısına ait bir çizim yer almaktadır. Dersler dizisinin elemanlarına *dersler[y][x][z]* şeklinde ulaşılabılır. *x* ve *y* ile ulaşılan ilk 2 boyut iki boyutlu dizilerle aynı kullanım şekline sahiptir. Tek fark *z* ile gösterilen kaçınıcı katman olduğunu gösteren değişkendir. 3 boyut yer aldığı için, elemanlar arasında gezinmek için iç içe 3 adet *for* yinelemesi kullanılabilir.



Aşağıda yer alan Örnek 7.6'da 3x2'lik dizilerin 2 katman halinde bir araya gelerek 3 boyutlu bir dizi oluşturulması ile ilgili örnek bir program kodu yer almaktadır. Örnek kodun program çıktısında dizideki elemanların dizilimi gösterilmektedir.

Örnek 7.6: 3 Boyutlu dizinin elemanlarının ekrana yazdırılması

```
#include <stdio.h>
#include <conio.h>

int main()
{
    int i, j, k;
    int s[2][3][2] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 };

    for(k=0; k<2; k++){
        printf("katman: %d\n", k);

        for(i=0; i<3; i++){

            for(j=0; j<2; j++){
                printf("%d ", s[k][i][j]);
                printf("\n");
            }
        }

        getch();
        return 0;
    }
}
```

/*ekranda görüntüleme için*/

Program Çıktısı: (Örnek 7.6)

```

katman:0
1 2
3 4
5 6
katman:1
7 8
9 10
11 12

```

GÖSTERİCİLER

C Programlamada tanımlanan değişkenler, tiplerine göre bellekte değişik byte uzunluklarında alanlar kaplamaktadırlar. Normal değişkenler içeriklerinde programlarda kullanılacak değişkenlerin sayısal değerini içerirken, göstericiler bağlantıda olduğu değişkenlerin adreslerini içerirler. Böylece dolaylı olarak değişkenlere ulaşmayı sağlarlar. Göstericiler, programda fonksiyonlar arasında aynı anda birden fazla değişken aktarımı gibi işlemlerde oldukça büyük kolaylık sağlayan yapılardır.

Gösterici Tanımlama

Göstericiler (ingilizce ifadesi: pointerlar) programlarda kullanımlarından önce tanımlanmaları gerekmektedir. Gösterici bildiriminde, normal değişken bildiriminden farklı olarak, gösterici adı önünde ‘*’ karakteri kullanılmaktadır.

*tip *degisken_adı;*

şeklinde tanımlanmaktadır. *tip* değişkenler için de kullanılan *int*, *float*, *char* vb. türlerinden herhangi birisi olabilir. Ancak dikkat edilmesi gereken nokta, gösterici ile göstericiye bellek adresi atanacak değişkenin veri tipi aynı olmalıdır.

Aşağıda yer alan

```

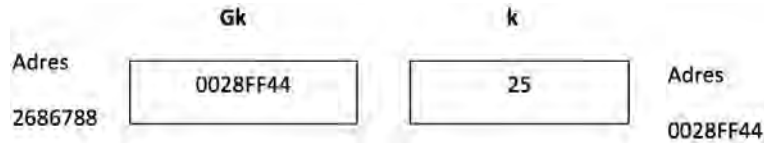
int k;
int *Gk;
Gk = &k;

```

satırları ile *k* adında tam sayı türünde bir değişken tanımlanmış olup, bu *k* değişkeninin adresi *Gk* göstericisine atanmıştır. *k* normal bir değişken iken, *Gk* gösterici olarak tanımlanmıştır. Aşağıda Şekil 7.4’te gösterildiği üzere, *Gk* dolaylı olarak 25 değerini içermektedir.

Şekil 7.4

Hafızada Değişken ve Gösterici Kullanımı



k ve *Gk* ifadelerinin hafızadaki adresleri ve içerdikleri değerlerle ilgili uygulama örneği aşağıdaki Örnek 7.7’deki program kodunda yer almaktadır. Bu örnekte görüldüğü üzere, göstericilerde sıklıkla kullanılan iki farklı işlemci bulunmaktadır. Bunlar * ve & işlemcileridir. * işlemcisi o göstericiye atanan değişkenin değerini, & işlemcisi ise kullanıldığı değişkenin adres değerini verir.

Örnek 7.7: Değişken ve gösterici kullanımı

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
int main()
{ int k,*Gk;
  k=25;
  Gk=&k;
  printf("k degiskeninin adresi :%p", &k);
  printf("\nGk_gosterici nin icerdigi adres :%p", Gk);
  printf("\nGk_gosterici nin adresi :%d", Gk);
  printf("\nk degisken degeri:%d", k);
  printf("\nGk_gosterici nin degeri:%d", *Gk);
  getch(); /*Ekran ciktisini ekranda bekletebilmek için*/
  return 0;
}
```

Program Çıktısı (Örnek 7.7)

```
k degiskeninin adresi :0028FF44
Gk nin icerdigi adres :0028FF44
Gk nin adresi :2686788
k degisken degeri :25
Gk degeri :25
```

Yukarıdaki örnekte yer alan programdan da görülebileceği üzere, göstericilerin kullanımı değişkenlere dolaylı olarak erişimi sağlar. Program çıktısından da görüldüğü üzere, göstericinin gösterdiği değişken ile gösterici aynı değere sahip olacaktır.

Program kodlarında değişkenlerin ve göstericilerin belirttikleri adresler her bilgisayarda farklı adresleri gösterebilir.

**DİKKAT**

Değişkenlerin farklı şekilde içeriklerinin değiştirilebilmesi ile ilgili yukarıdaki Örnek 7.7'nin genişletilmiş şekli aşağıda Örnek 7.8'de yer almaktadır.

Örnek 7.8: Doğrudan veya gösterici aracılığıyla değişken içeriği değiştirme

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
int main()
{ /*1*/ int k,*Gk;
  /*2*/ k=25;
  /*3*/ Gk=&k;
  /*4*/ printf("\nk degisken degeri :%d", k);
  /*5*/ printf("\nGk nin degeri :%d\n", *Gk);
  /*6*/ printf("k degiskeninin adresi :%p", &k);
  /*7*/ printf("\nGk nin icerdigi adres :%p", Gk);
  /*8*/ printf("\nGk nin adresi :%d\n", Gk);
  /*9*/ printf("-----");
  /*10*/ printf("\nDogrudan deger degistirme\n");
  /*11*/ printf("-----");
  /*12*/ k=45;
  /*13*/ printf("\nk degisken degeri :%d", k);
  /*14*/ printf("\nGk nin degeri :%d\n", *Gk);
  /*15*/ printf("k degiskeninin adresi :%p", &k);
  /*16*/ printf("\nGk nin icerdigi adres :%p", Gk);
  /*17*/ printf("\nGk nin adresi :%d\n", Gk);
  /*18*/ printf("-----");
  /*19*/ printf("\nDolayli olarak gosterici yardimiyla deger degistirme\n");
  /*20*/ printf("-----");
  /*21*/ *Gk=85;
```

```

/*22*/printf("\nk degisken degeri      :%d", k);
/*23*/printf("\nGk nin degeri        :%d\n", *Gk);
/*24*/printf("k degiskeninin adresi :%p",&k);
/*25*/printf("\nGk nin icerdigi adres :%p",Gk);
/*26*/printf("\nGk nin adresi       :%d",Gk);
getch(); /*Ekran ciktisini ekranda bekletebilmek icin*/
return 0;}

```

Program Çıktısı (Örnek 7.8)

k degisken degeri :25
 Gk degeri :25
 k degiskeninin adresi :0028FF44
 Gk nin icerdigi adres :0028FF44
 Gk nin adresi :2686788

Dogrudan Deger Degistirme

k degisken degeri :45
 Gk degeri :45
 k degiskeninin adresi :0028FF44
 Gk nin icerdigi adres :0028FF44
 Gk nin adresi :2686788

Dolaylı olarak gosterici yardımıyla deger degistirme

k degisken degeri :85
 Gk degeri :85
 k degiskeninin adresi :0028FF44
 Gk nin icerdigi adres :0028FF44
 Gk nin adresi :2686788

Yukarıda yer alan Örnek 7.8'de 1, 2 ve 3. satırlar yardımıyla *k* değişkeni ve *Gk* göstericisi tanımlanmıştır. *k*=25; satırı ile *k* değişkenine doğrudan 25 değeri atanmıştır. 4.-8. satırlar arasında *k* ve *Gk* ya ait içerik ve adres değerleri listelenmiştir. 12. Satırda yer alan *k*=45; ifadesi ile *k* nin değeri değiştirilmiş olup, *Gk* göstericisine atama olarak fazladan bir işlem yapılmamasına rağmen, 14. satır yardımıyla ekrana yazdırılan *Gk*'nin değerinin de 45 olarak değiştiği gözlenmektedir. Devamında yer alan 21. satırda **Gk*=85; satırı kullanılarak *k* değişkeninin değeri bu sefer de gösterici yardımıyla dolaylı olarak değiştirilmektedir. Dikkat edilirse, 4.- 8. satırlar arası, 13.- 17. satırlar arası, ve 22.-26. satırlar arasındaki program blokları birbirinin aynısıdır. *k* değerinin doğrudan veya dolaylı olarak değiştirilmesi durumunda içerik ve adreslerin nasıl değişim gösterdiğini sergilemektedir. Adresler her üç durumda da aynı kalırken, *k*'nin içerdği değer değişmektedir.

Göstericilerin Dizilerle Birlikte Kullanılması

Göstericiler değişkenlerin başlangıç adresini belirtmeleri dolayısı ile dizilerle benzer özellikleri içermektedirler. Dizilerin ilk elemanlarını göstericilerle belirterek, diziler-göstericiler birbirlerinin yerine kullanılabilirler.

Aşağıda yer alan Örnek 7.9'da dizinin başlangıç adresini gösteren bir gösterici yardımıyla dizinin elemanlarına ulaşım gerçekleştirilmektedir. Böylece dizinin elemanlarına erişim hem doğrudan hem de gösterici yardımıyla yapılabilecektir.

Örnek 7.9: Gösterici ve dizilerinin birbirleri yerine kullanılması

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
int main(void)
{ /*1*/      int m[ 5]={ 1,2,3,4,5 };
  /*2*/      int i,*Gm;
  /*3*/      Gm=m;
  /*4*/      printf("Dizinin indis yardimiyla gosterimi\n");
  /*5*/      for(i=0;i<5;i++)
  /*6*/      printf("m[ %d] =%d  ",i,m[ i]);/*dizinin indis yardimiy-
la*/      /*gösterimi*/

  /*7*/      printf("\n\nDizinin          gosterici-oteleme          ile
kullanimi\n");
  /*8*/      for(i=0;i<5;i++)
  /*9*/      printf("*(m+%d)=%d  ",i,*(m+i));
          /* dizinin gösterici-öteleme olarak kullanimi*/
  /*10*/     printf("\n\nGostericinin dizi olarak kullanimi\n");
  /*11*/     for(i=0;i<5;i++)
  /*12*/     printf("Gm[ %d] =%d  ",i,Gm[ i]);
          /*göstericinin dizi olarak kullanimi*/
  /*13*/     printf("\n\nGostericinin oteleme ile kullanilmasi\n");
  /*14*/     for(i=0;i<5;i++)
  /*15*/     printf("*(Gm+%d)=%d  ", i, *(Gm+i));
          /*göstericinin öteleme ile kullanilmasi*/
  getch(); /*Program ciktisini ekranda bekletebilmek için */
  return 0; }
```

Program Çıktısı (Örnek 7.9) :

Dizinin indis yardımıyla gösterimi

m[0]=1 m[1]=2 m[2]=3 m[3]=4 m[4]=5

Dizinin gosterici-oteleme olarak kullanimi

*(m+0)=1 *(m+1)=2 *(m+2)=3 *(m+3)=4 *(m+4)=5

Gostericinin dizi olarak kullanimi

Gm[0]=1 Gm[1]=2 Gm[2]=3 Gm[3]=4 Gm[4]=5

Gostericinin oteleme ile kullanilmasi

*(Gm+0)=1 *(Gm+1)=2 *(Gm+2)=3 *(Gm+3)=4 *(Gm+4)=5

Yukarıdaki Örnek 7.9'daki programda 4.-6. satırlar arasında dizinin elemanları diziler konusunda gösterildiği şekilde ekrana yazdırılmıştır. 7.-9. satırlar arasında $*(m+i)$ ifadesinin kullanımı yardımıyla dizi gösterici olarak ifade edilmiştir. Çünkü bu satırlardan önce, 3. satır $Gm=m;$ kullanılarak Gm göstericisinin m dizisinin başlangıç adresini göstermesi sağlanmıştır. Böylece Gm ve m aynı bellek adresini göstermektedirler. Bu örnekte, $Gm=m;$ satırı $Gm=&m[0];$ anlamına gelmektedir.

Örnek 7.9 da, $Gm=m;$ yerine $Gm=&m[4];$ şeklinde tanımlama yapılır ise, göstericinin gösterdiği değerler hangileridir? Kodu çalıştırarak analiz ediniz.



SIRA SİZDE

Gösterici Aritmetiği ve sizeof() Fonksiyonu

Göstericiler bellekteki adres değerleri ile işlem yapabilme kabiliyetlerinden dolayı aritmetik işlemlerde de adresler arası işlemleri kullanırlar. Göstericilerin aritmetik işlemlerinde $+$, $-$, $++$, $--$ işlemcileri kullanılabilir. İşlemlerdeki arttırma ve eksiltme işlemleri sayısal olarak olmayıp, göstericinin bir arttırılması bir sonraki bellek adresine öteleme olarak sağla-

nır. Elbette ki bu öteleme esnasında veri türü de bellekte kapladığı büyüklüğünden dolayı önemlidir. Çünkü arttırma ve eksiltme işlemleri değişkenin hafızada kapladığı *int*, *float*, *char* vb. kadar arttırılıp eksiltilecektir.

Dizilerin veya göstericilerin hafızada kapsadığı alanların analizi için *sizeof()* fonksiyonu kullanılabilir. *sizeof()* fonksiyonu girdi olarak aldığı değişkenin hafızada byte olarak kapladığı alanı geri döndürecektir. Örnek 7.10'da tam sayı ve double tipinde 5 elemana sahip iki farklı dizi tanımlanarak göstericilere atanmışlardır. Program çıktısında gösterildiği üzere, tam sayı tipi hafızada 4 byte, 5 elemana sahip tam sayı tipinde dizi toplamda 20 byte uzunlunda hafızada yer kaplayacaktır. Ancak, tam sayı türündeki diziyi gösteren gösterici ise yalnızca 4 byte uzunluğundadır. Çünkü gösterici dizinin başlangıç adresini gösteren yalnızca bir değişkendir. double türündeki dizinin her elemanı 8 byte olmak üzere, dizi toplamda 40 byte uzunluğunda, gösterici ise yine aynı şekilde 4 byte uzunluğundadır.

Örnek 7.10: *sizeof()* fonksiyonu ile farklı veri tiplerindeki dizi ve göstericilerin boyutlarının hesaplanması

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
int main(void)
{ int dizi_t[ 5]={ 0 };
  int *G_t=dizi_t;
  printf(" Tamsayı dizi_t nin boyutu= %d byte\n", sizeof(dizi_t) );
  printf(" dizi_t nin herbir elemanı");
  printf(" %d byte uzunlugundadır\n",sizeof (dizi_t[ 0] ) );
  printf(" Gosterici G_t nin boyutu= %d byte\n\n", sizeof(G_t));

  double dizi_d[ 5]={ 0 };
  double *G_d=dizi_d;
  printf(" Double dizi_d nin boyutu= %d byte \n", sizeof(dizi_d));
  printf(" dizi_d nin herbir elemanı");
  printf(" %d byte uzunlugundadır\n",sizeof (dizi_d[ 0] ) );
  printf(" Gosterici G_d nin boyutu= %d byte\n", sizeof(G_d) );
  getch();
  return 0; }
```

Program Çıktısı: (Örnek 7.10)

Tam sayı dizi_t nin boyutu =20 byte
 dizi_t nin herbir elemanı 4 byte uzunluğundadır
 Gosterici G_t nin boyutu=4 byte

 Tam sayı dizi_d nin boyutu =40 byte
 dizi_d nin herbir elemanı 8 byte uzunluğundadır
 Gosterici G_d nin boyutu=4 byte

SIRA SİZDE



sizeof (dizi_t) / sizeof (dizi_t[0]) ifadesi program kodunda kullanıldığında hangi değeri verir?

Göstericilerle aritmetik işlem yapılabilmesi için hafızada ileri-geri hareket edilmesi gerekir. Örnek vermek gerekirse; her bir elemanı 4 byte uzunluğunda olan tam sayı türünde *dizi_t[5]*'in başlangıç adresi 1000 ise, bu dizinin başlangıç adresini gösteren gösterici *G_t = dizi_t*; tanımlandığında *G_t* 1000 adresini gösterirken, *G_t++* ; şeklinde aritmetik olarak arttırıldığında 1004 adresini gösterecektir.

G_t += 2; ifadesi kullanıldığında ise, $(1000 + (2*4\text{byte}))$ adresine atlanacaktır.

Aynı şekilde Gösterici 1016 adresini gösterdiği kabul edilirse,

G_t = G_t - 3 ; ifadesi *G_t* göstericisini 1004 adresine taşıyacaktır. Örneklerden görüldüğü üzere göstericilerle aritmetik işlem yapıldığında gösterdiği veri türünün hafızada byte olarak kapladığı alan önem kazanmaktadır.

DİZGİLER

C programlama dilinde, dizgiler için tanımlanmış bir veri tipi bulunmamaktadır. Dizgiler karakterlerden oluşan diziler veya göstericiler olarak tanımlanabilir. Bu nedenle, dizgi tanımlarında tek boyutlu diziler tanımlanmaktadır. Normal dizilerden farklı olarak son elemanı *NULL(0)* ('\\0') değeri ile sonlandırılmaktadır. Bu özelliklerinden dolayı karakter dizilerini, dizginin içereceği karakter sayısından bir karakter fazla kapasiteli olacak şekilde tanımlamak gerekmektedir.

```
char d[10];
```

şeklindeki bir dizgi tanımlamasında, dizginin *d[9]* indis nolu elemanı *NULL* değere sahiptir. Bu durumda *d* dizgisine 9 karakterli bir kelime yazılabilir. *d* dizgisi içerisindeki karakterler hafızada okunurken *NULL* a kadar okunur, *NULL* bir sınır değeri oluşturur.

```
char ad[] = "neslihan";
```

tanımlamasında 8 adet karakter içerilmektedir. Dizi boyutu verilmemiştir. Bu nedenle *NULL* karakteri de dahil olmak üzere toplamda 9 karakter kapasitesine sahip bir karakter dizisi yani dizgi tanımlanmıştır. Sırasıyla 'n', 'e', 's', 'l', 'i', 'h', 'a', 'n', '\\0' karakterlerini içeren *ad* dizgisinin ilk elemanı olan *ad[0]* 'n', 7 indis nolu *ad[7]* elemanı ise yine 'n' harfini içermektedir. *ad[8]* nolu elemanı ise kelime sonu anlamına gelen '\\0' *NULL* karakterini içerir. Yukarıdaki örnek başka bir gösterim ile,

```
char ad[] = {'n', 'e', 's', 'l', 'i', 'h', 'a', 'n', '\\0'};
```

şeklinde de tanımlanabilmektedir. Ancak bu tanımlama şeklinde en sonda yer alan '\\0' karakteri eklenmeye unutulmamalıdır. Bir önceki tanımlamada bu *NULL* karakter otomatik olarak sona eklenirken, süslü parantez gösteriminde, kodlamayı yapanın manuel olarak en sona eklemesi gerekmektedir. '\\0' karakterinden dolayı dizgilerin boyutu bir fazla olarak belirlenmelidir.

Dizgilerin içeriği kelime hâlinde yazılacak ise, "metin" şeklinde yazılırken, süslü parantez { } gösterimi ile ise, karakter karakter ifade edilir ve her karakter 'k' şeklinde ' ' işaretleri arasına alınmaktadır.



DİKKAT

Dizgileri tanımlarken belirlenen dizgi kapasitesi (karakter sayısı) daha sonra bu dizgiye atama yapılması esnasında aşılsa, diğer bir ifade ile daha uzun bir kelime atanması durumunda hafızada dizginin devamında yer alan bölüme taşmaya sebep olacağından o bölümdeki verilerde kayıplara sebep olacaktır. Örnek olarak,

```
char ad[4];
```

şeklinde tanımlama yapıldıktan sonra, hafızadaki yerleşim;

ad0	ad1	ad2	'\\0'	a	l	i	g	e	l	'\\0'
-----	-----	-----	-------	---	---	---	---	---	---	-------

şeklinde olacaktır. Tanımlanan *ad* dizgisinin devamında "aligel" şeklinde bir karakter verisi yer aldığı kabul edilir ise ve sonrasında *ad* dizgisine "neslihan" kelimesi atandığında, hafızadaki yerleşim;

n	e	s	l	i	h	a	n	e	l	'\\0'
---	---	---	---	---	---	---	---	---	---	-------

şeklinde olacaktır. Bu durumda "alig" bölümü silinmiş olup veri kaybı oluşacaktır. Bu nedenle dizgilerin kapasiteleri belirlenirken dikkatli davranmalı, veri kayıplarından kaçınılmalıdır.

Dizgilerle birlikte, kullanıcıdan veri girişi istenildiğinde sıklıkla `gets()` veya `scanf` fonksiyonları kullanılmaktadır. `gets` fonksiyonu giriş değeri olarak karakter dizisinin adını kullanır ve ENTER tuşuna basıp sonlandırılmadığı sürece klavyeden girilen karakterlerin ilgili diziye atamalarını yapar.

Aşağıda yer alan Örnek 7.11’de klavyeden giriş yapıp, karakter karakter ekrana yazdırılmaktadır. Klavyeden girişi `gets(ad)`; satırı sağlamaktadır.

Örnek 7.11: Klavyeden karakter dizisi girişi ekranda görüntüleme

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
int main(void)
{ char ad[10], i=0;
  printf("Bir isim giriniz:");
  gets(ad); /*klavyeden karakter girişi sağlar*/
  printf("Girdiğiniz isim: %s\n",ad);
  printf("Girdiğiniz ismin harfleri:\n");
  while(ad[i] != '\0') /*ad dizisinin içeriği '\0', NULL olmadığı sürece*/
  { printf("%c\n",ad[i]);
    i++;
  }
  getch(); /*Ekranda görüntüleme için*/
  return 0; }
```

Program Çıktısı (Örnek 7.11)

Bir isim giriniz: neslihan

Girdiğiniz isim: neslihan

Girdiğiniz ismin harfleri:

n
e
s
l
i
h
a
n

Yukarıda yer alan Örnek 7.11’de harfler karakter karakter ele alınıyorsa `printf` fonksiyonunda `%c` olarak kullanılmaktadır. `gets(ad)` yerine `scanf("%s", ad)`; şeklindeki kullanım ile de klavyeden karakter dizisi girişi yapılabilir. Nümerik değer içeren dizilerde olduğu gibi, `scanf` kullanımında `&ad` olarak kullanımına gerek kalmamaktadır. `scanf("%s", &ad)`; ifadesinde `&` kullanılması bir hataya sebep olmamakla birlikte gereksizdir. Ancak `scanf` fonksiyonu dizgilerin karakter karakter giriş yapılmasını sağlayacaktır.

DİKKAT



printf fonksiyonu kullanılarak karakter kümesi yazdırılacak ise %s, yalnızca karakter yazdırılacak ise %c belirteçleri kullanılır.

SIRA SİZDE



gets(ad) yerine scanf fonksiyonu kullanılarak klavyeden kullanıcı girişi yapılmasını sağlayan program kodunu yazınız.

Özet



Tek boyutlu dizi tanımlamalarını, dizilere ilk değer atamayı, kod içerisinde ihtiyaç duyulduğunda, dizi elemanlarının içeriklerinin değiştirilmesi işlemlerini açıklamak. Diziler, tek tek tanımlanan değişken türlerinin birden fazla sayıda bir araya gelerek aynı türden değişkenler kümesi oluşturulmasını sağlarlar. Değişkenlerin tek tek tanımlanmasının zor olduğu, fazla sayıdaki değişken verilerinde oldukça kullanışlı olan diziler için ilk indis değeri 0 ile başlar. İndis değeri toplam eleman sayısının bir eksik sayısına kadar devam eder. Dizilerin tanımlanması yapıldığında, bellekte yalnızca o diziye ait alan ayrılır. Dizinin elemanlarına herhangi bir değer atanmaz. Bu nedenle dizi elemanları ile işlemler yapılmadan önce, dizi tanımlamasının hemen ardından dizi elemanlarına ilk değer atanmalıdır. İlk değer atanmasından sonra, kod çalıştığı sürece değişik işlemler ile dizinin elemanlarının içerikleri başka değerler ile değiştirilebilir.



İki ve üç boyutlu dizi tanımlamalarını ve çok boyutlu dizilerin elemanlarına değer atanması işlemlerini anlatmak.

Tek boyutlu dizilerde yalnızca bir yönde değişkenler bulunurken, iki boyutlu dizilerde ise değişkenler hem satır hem sütunlar oluşturan hücrelerden oluşmaktadır. Tek boyutlu dizilerde elemanlara tek bir for yinelemesi ile ulaşılabilirken, iki boyutlu dizilerde iç içe iki for yinelemesi ile ulaşılır. Bu for yinelemeleri iç içe kullanılacak ise sayaç değerlerinin birbirinden farklı, tamsayı türünde iki değişken olarak kullanılması gerekmektedir.



Göstericilerin tanımlamalarını ve diziler ile birlikte kullanımlarını ifade etmek.

Değişkenlerin başlangıç adreslerini içeren değişkenler olarak tanımlanabilen göstericiler ile değişken tabloları olarak tanımlanabilen diziler arasında oldukça yakın bir ilişki bulunmasından dolayı, uygun dönüşümler sayesinde, birbirleri yerine kullanılabilirler. Göstericilerin tanımlamalarında değişkenlerden farklı olarak “*” işareti kullanılmaktadır. Bir göstericinin diziye gösterebilmesi için, dizinin ilk elemanına eşitlenmesi gerekmektedir. Göstericiler kullanılarak artırma veya eksiltme yapıldığında veri türünün uzunluğu kadar byte ileri yada geri hareket edilir. sizeof() fonksiyonu ile herhangi bir değişkenin veya göstericinin hafızada kapladığı alan değeri tespit edilebilir.



Karakterleri içeren dizi olarak kabul edilen dizgileri tanımlamak, karakter dizgileri ile işlem yapmak.

Dizgiler, sayılar yerine karakter ve metinler içeren dizilerdir. Diziler gerçekte char tipinde dizilerdir. Dizgilerin normal dizilerden önemli bir farkı, son elemanlarının ‘\0’-NULL karakter içermesidir. Çünkü karakter dizilerinin içeriği okunurken dizginin NULL karakteri tespit edilinceye kadar devam eder. Klavyeden karakter seti giriş işlemi için gets() fonksiyonu veya karakter karakter giriş sağlanabilen scanf fonksiyonu kullanılabilir.

Kendimizi Sınyalım

1. `int A [ ] = ( 1, 2, 3, 4, 5);`

Yukarıdaki program satırındaki hata nasıl düzeltilir?

- () parantezleri, {} ile değıştirilmelidir.
- () parantezleri köşeli parantezleri [] ile değıştirilmelidir.
- [] parantezler {} hale gelmelidir.
- Köşeli parantez içersine mutlaka bir sayı değeri yazılmalıdır.
- () parantezleri içersinde tamsayı olmamalıdır.

2. Aşağıdakilerden hangisi dizi tanımlaması için uygun bir yöntem **değildir**?

- `int n[ 5 ] = { 0 };`
- `int n[ ] = { 0, 7, 0, 3, 8, 2 };`
- `int n[ 5 ] = { 7 };`
- `int n[ 5 ] = { 6, 6, 6 };`
- `int n[ 5 ] = { 0, 7, 0, 3, 8, 2 };`

3. İki boyutlu b dizisinin elemanları `int b[ 2 ][ 2 ] = { { 1 }, { 3, 4 } };` şeklinde tanımlanmıştır. Bu duruma göre b dizisinin `b[1][0]` elemanının değeri aşağıdakilerden hangisidir?

- 3
- 4
- 1
- 0
- 10

4. Aşağıdaki tanımlamalardan hangisi 10 elemanlı tamsayı tipinde adı "puan" olan bir dizi tanımlar?

- `puan= int[ 10 ] ;`
- `Dizi puan = int[ 10 ] ;`
- `int puan[ 10 ] ;`
- `int Dizi puan[ 10 ] ;`
- `puan int[10] ;`

5.

```
#include "stdio.h"
#include <conio.h>
#define BOYUT 10
int main()
{
    int n[ BOYUT ] = { 5, 3, 0, 1, 2, 1,
4, 3, 3, 1 };
    int i, j, max=5;
    for ( i = 0; i < (max+1); i++ ) {
        printf( "%2d",max-i) ;
        for ( j = 0; j <BOYUT; j++ ) {
            if (n[ j] == (max-i) || n[ j] > (max-i))
                printf( "%c", '*' );
            else
                printf( " ");
            printf( " ");
        }
        printf( "\n" );
    }
    getch();
    return 0;
}
```

Yukarıdaki programın çıktısı nedir?

6. `dizi_t` bir dizi, `*Gst` ise `dizi_t` nin bir göstericisi olarak tanımlandığı durumda, `dizi_t` nin beşinci (5.) elemanının adresini veren ifade aşağıdakilerden hangisidir?

- `&dizi_t[ 4 ]`
- `*( Gst + 4 )`
- `Gst[ 4 ]`
- `*( dizi_t+ 4 )`
- `&dizi_t[ 5 ]`

7. A dizisinin bir tamsayı tipinde 3000 nolu adreste bulunduğu, `AGst` nin ise A dizisini gösteren bir gösterici olarak tanımlandıkları kabul edilir ise, tamsayı değışkenlerin hafızada 4 byte lık alanlar kapladıkları dikkate alınarak, `AGst+3` aritmetik işlemi sonucunda `Agst` gösterecek adres aşağıdakilerden hangisidir?

- 3000
- 3004
- 3003
- 3012
- 3005

8. `int s[2][3][2] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};` program satırı ile tanımlanan s dizisi için, `s[1][1][1]` elemanının değeri aşağıdakilerden hangisidir?

- 0
- 10
- 5
- 7
- 1

9. `for ( i = 6; i < 20; i++ )`
`{ printf( "k [%d] = %d\n", i, k [i-6] ); }`

Yukarıdaki for yinelemesi kullanılarak k dizisinin kaç adet elemanı ekranda yazdırılır?

- 20
- 14
- 19
- 16
- 6

10.

```
#include <stdio.h>
#include <conio.h>
int main(void)
{   int i;
    char karakter[]="Bu bir deneme cum-
lesidir";
    printf("kacinci karakter ?:");
    scanf("%d",&i);
    if (karakter[i-1]==' ')
        printf("bu karakter bosluktur");
    else
        printf("%c", karakter[i-1]);
    getch();
    return 0;}
```

Yukarıda yer alan program kodunda kullanıcı "kacinci karakter?" sorusuna klavyeden 7 sayısını giriş yapar ise, program çıktısı aşağıdakilerden hangisi olur?

- bu karakter bir bosluktur "
- 'e'
- 'd'
- 'r'
- 'b'

Kendimizi Sıneyalım Yanıt Anahtarı

- a Yanıtınız yanlış ise "Dizilere Değer Atama" konusunu yeniden gözden geçirin.
- e Yanıtınız yanlış ise "Dizilere Değer Atama" konusunu yeniden gözden geçirin.
- a Yanıtınız yanlış ise "Diziler" konusunu yeniden gözden geçirin.
- c Yanıtınız yanlış ise "Diziler" konusunu yeniden gözden geçirin.

5. Yanıtınız yanlış ise "Diziler" konusunu yeniden gözden geçirin.



- a Yanıtınız yanlış ise "Göstericilerin Dizilerle Birlikte Kullanılması" konusunu yeniden gözden geçirin.
- d Yanıtınız yanlış ise "Gösterici Aritmetiği ve sizeof() Fonksiyonu" konusunu yeniden gözden geçirin.
- b Yanıtınız yanlış ise "İki ve Üç Boyutlu Diziler" konusunu yeniden gözden geçirin.

Aşağıdaki program kodu ile test edilebilir.

```
#include <stdio.h>
#include <conio.h>
int main()
{   int i,j,k;
    int s[ 2][ 3][ 2] = { 1, 2, 3, 4, 5, 6, 7,
                        8, 9, 10};
    for(k=0;k<2;k++){
        printf("katman:%d\n",k);
        for(i=0;i<3;i++){
            for(j=0;j<2;j++){
                printf("%d ",s[k][i][j]);
            }
            printf("\n");
        }
    }
    getch();
    /*ekranda görüntüleme için*/
    return 0;
}
```

- b Yanıtınız yanlış ise "Diziler" konusunu yeniden gözden geçirin.
- a Yanıtınız yanlış ise "Dizgiler" konusunu yeniden gözden geçirin.

Sıra Sizde Yanıt Anahtarı

Sıra Sizde 1

k[0]= bir değer atanmaz, k[1]=3, k[2]=6, k[3]=9, k[4]=12, k[5]=15

Sıra Sizde 2

Örnek 7.4 de

İlk satır, int arasinav1 [BOYUT]={50, 80, 60, 90, 70,100} ; şeklinde değiştirilirse derleyici fazla değer atanma (**too many initializer**) derleme hatası verecektir

Sıra Sizde 3

Elemanların dizilimi aşağıdaki şekilde olacaktır.

D dizisi 0.satır: 40, 60, 60, 50

D dizisi 1.satır: 80, 85, 70, 50

D dizisi 2.satır: 85, 70, 50, 80

D dizisi 3.satır: 80, 0, 90, 70

D dizisi 4.satır: 70, 0, 80, 50

Sıra Sizde 4

Aşağıda kod ve program çıktısı yer almaktadır *m* adında 1,2,3,4,5,10,20,30,40,50 elemanlarına sahip bir dizi tanımlanmış olup, ardından bu *m* dizisinin 4 nolu indisindeki elemandan itibaren gösterecek *Gm* göstericisi tanımlanmıştır. Böylece göstericinin ilk elemanının sayısal değeri, $*(Gm+0)=5$ olarak atanmıştır. Programda ise *Gm* in sonraki elemanları listelenmiştir. Bu durumda *m* dizisinin indis olarak 5, 6, 7, 8, 9. elemanları *Gm* in 1,2,3,4,5. elemanlarına karşılık gelip sayısal olarak 10,20,30,40,50 değerlerine sahip olacaktır.

```
#include<stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
int main(void)
{ int m[ 10]={ 1,2,3,4,5,10,20,30,40,50} ;
  int i,*Gm;
  Gm=&m[ 4] ;
  printf("Dizinin indis yardimiyla gosterimi\n");
  for(i=5;i<10;i++)
    printf("m[ %d] =%d\n",i,m[ i] );
  /*dizinin elemanlarinin indis yardimiy-la gösterimi*/
  printf("Gostericinin oteleme ile kullanilmasi\n");
  for(i=1;i<=5;i++)
    printf("*( Gm+%d)=%d\n", i, *( Gm+i));
  /*gostericinin elemanlarinin listelen-mesi*/
  getch(); /*Ekran ciktisini ekranda bek-letebilmek için */
  return 0; }
```

Program Çıktısı :

Dizinin elemanlarinin indis yardimiyla gösterimi

m[5]=10 m[6]=20 m[7]=30 m[8]=40 m[9]=50

Gostericinin elemanlarinin listelenmesi

*(Gm+1)=10

*(Gm+2)=20

*(Gm+3)=30

*(Gm+4)=40

*(Gm+5)=50

Sıra Sizde 5

sizeof(dizi_t) / sizeof(dizi_t[0]) ifadesi dizi_t nin eleman sayısını verir.

Sıra Sizde 6

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
int main(void)
{ char ad[ 10] ,i=0;
  printf("Bir isim giriniz:\n");
  printf("(Her harf girisinden sonra BOSLUK(space) tusuna basiniz)\n");
  for(i=0;i<10;i++)
  { scanf("%c", &ad[ i] );

  printf("Girdiginiz isim: %s \n",ad);
  i=0;
  printf("Girdiginiz ismin harfleri:\n");
  while( ad[ i] !='\0' ) /*ad dizgisi-nin içeriği '\0', NULL olmadigi surece*/
  /*dongu devam eder*/
  { printf("%c\n",ad[ i] );
    i++;
  }
  getch(); /*ekranda go-runtuleme için*/
  return 0; }
```

Yararlanılan ve Başvurulabilecek Kaynaklar

Deitel, P.,Deitel, H. (2010). **C How to Program-6th Edition.**, Pearson-Prentice Hall.

Güney, İ. (2006).**C Programlama Dili**, Ankara:Nobel Yayın Dağıtım

Çölkesen, R. (2005). **C Programlama Dili**, İstanbul:Papatya Yayıncılık.

Zavrak, M., Aksoy, E., Karaca, H.N. (2004). **C ve C++**, (Çeviri:C How to Program-Third Edition), İstanbul: Sistem Yayıncılık.

Tunçer, B.(2002). **Uygulamalı C Programlama Dili**, İstanbul: Papatya Yayıncılık.

Ekinci, F. (1996). **C ve Siz**, İstanbul: Türkmen Kitabevi



Amaçlarımız

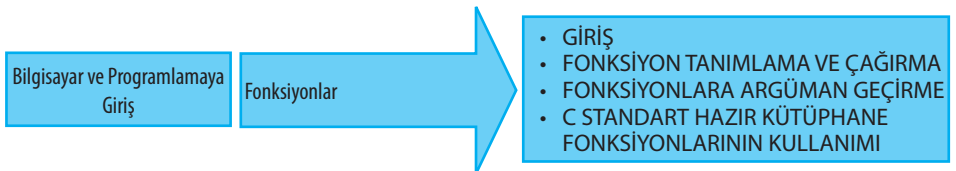
Bu üniteyi tamamladıktan sonra;

- Fonksiyonların kullanım amaçlarını açıklayabilecek,
 - Fonksiyonları tanımlayıp, fonksiyon bloğu oluşturarak fonksiyonu çağırabilecek,
 - Fonksiyonlara değer veya referans yöntemi ile argüman geçirebilecek,
 - Dizileri ve göstericileri fonksiyonlarda argüman olarak kullanabilecek,
 - C hazır kütüphane fonksiyonlarını kullanarak önceden tanımlı fonksiyonlar yardımıyla matematiksel, dizgi-karakterlerle ve bellek ayırma ile ilgili belli işlemleri gerçekleştirebilecek
- bilgi ve becerilere sahip olabileceksiniz.

Anahtar Kavramlar

- Fonksiyon
- Argüman
- Değer /Referans Yöntemi ile Fonksiyon Çağrılması
- C Hazır Kütüphane Fonksiyonları
- math.h
- string.h
- ctype.h
- Dinamik Bellek

İçindekiler



Fonksiyonlar

GİRİŞ

C Programlama dilinde, fonksiyonlar önemli bir yapıya sahip alt program kodlarıdır. Fonksiyonlar, yazılan program kodlarının daha modüler bir şekilde yapılandırılmasını sağlarlar. Fonksiyonların kullanımı sayesinde program kodları daha küçük alt bölümlere ayrılabilir.

Matematiksel ifadelerde kullanılan fonksiyonlara (örnek: $z=f(x, y)=2x+3y$, x, y :giriş değerleri, z :çıkış değeri) benzer şekilde, programlamadaki fonksiyonlar da bazı giriş değerlerini kullanarak belli bir işlemi gerçekleştirecek şekilde çıkış değeri oluşturacaklardır. İçerisine ana program kodlarının yazıldığı `main()` fonksiyonu da bir fonksiyondur. Ancak `main()` fonksiyonu mutlaka olması gerekli ana fonksiyondur. Kodlar çok uzun satırlar halinde, yalnızca `main` fonksiyonunun içerisine yazılabildikleri gibi modüler bir tasarımla fonksiyonlar halinde yazılıp `main()` fonksiyonunun içinden çağrılabilirler. `main()` fonksiyonunun içerisinde yalnızca fonksiyonun çağrılma işlemi gerçekleştirilecek, gerçekte o fonksiyonun yapması gerekli işlem, `main()` dışında yer alan ilgili fonksiyon bloğu içerisinde gerçekleştirilecektir. Böylece `main()`'in kontrolü dâhilinde diğer fonksiyonlara görev paylaşımı yaptırılmaktadır.

Programın alt bölümler halinde çalıştırılması, yapılması gerekli işlemlerin farklı fonksiyonlar halinde gerçekleştirilmesini sağlayarak kodlamada bir hata oluştuğunda hata tespitini kolaylaştıracaktır. Olası olumsuzluklar durumunda, hatayı tüm ana kod içerisinde aramak yerine, alt programcılar anlamına gelen, ilgili fonksiyonlar içerisinde, ilgili olabilecekler arasından kontrol edebilme imkânı sağlayacaktır.

FONKSİYON TANIMLAMA VE ÇAĞIRMA

Fonksiyonlar C kütüphanesinin içerisinde yer alan hazır fonksiyonlar(`printf`, `scanf`, `pow` vb.) şeklinde olabileceği gibi, programcı tarafından ihtiyaca göre sonradan oluşturulmuş fonksiyonlar da olabilirler. Fonksiyonun kullanılabilmesi için tanımlanması, fonksiyon bloğunun olması ve fonksiyonun çağrılması gereklidir.

Bir fonksiyonu kullanmadan önce, değişken tanımlamada olduğu gibi fonksiyonu tanımlamak gerekmektedir. Tanımlama aşamasında fonksiyonun giriş ve çıkış değerlerinin sayısı ve türü belirtilir. Yapılacak işlemler ise fonksiyon bloğunun içindeki kod satırları ile sağlanır. Fonksiyon tanımında genel bir kural olmamakla birlikte, fonksiyon adını amacına uygun bir şekilde belirlemek daha uygun olacaktır. Örneğin toplama işlemi yapan fonksiyonun `topla` şeklinde kullanılması, program kodlarının daha sonra, farklı kişiler ta-

rafından kontrolü veya güncellemesi esnasında fonksiyonu daha anlaşılır hâle getirecektir. Fonksiyon tanımlama satırı, *main()* öncesinde, “*#include*” ifadesini içeren önışlemci bölümünün hemen altında yapılabilir. Böylece, *main()* fonksiyonu içerisinde fonksiyonu kullanabilme yetkisi oluşturulmuş olur.

DİKKAT



Çağrılacak fonksiyon *main()*'den önce tanımlanmaz ise derleme hatası verecektir.

Fonksiyon Tanımı

fonksiyon_tipi fonksiyon_adı(tip1 arguman1, tip2 arguman2);
veya

fonksiyon_tipi fonksiyon_adı(tip1, tip2);

şeklinde yapılır. Tanımlama satırında argüman değişkenlerinin adını(*arguman1, arguman2*) yazma zorunluluğu yoktur. İkinci ifadede olduğu gibi, yalnızca tiplerini (*int, float, double, char vb.*) yazmak yeterlidir. Ancak parantez içerisindeki toplam argüman sayısı önemlidir. Fonksiyonun giriş argümanları olarak dışarıdan, bir diğer ifade ile *main()* ortamından kaç değişken alması gerektiği belirlenmelidir. Argüman sayısı fonksiyon tanımı, fonksiyonun çağırılması ve fonksiyon bloğu bölümlerinde birbirleri ile tutarlı olmalıdır. Argüman bölümüne *void* ifadesi yazıldığı durumda ise fonksiyonun bir giriş değeri almadığı kabul edilir. Herhangi bir giriş değeri(argüman) almayan fonksiyonlar, her yerden erişilebilen *genel(global)* değişkenleri kullanarak işlem yapabilecekleri gibi, açıklama içeren ekrana yazdırma işlemlerini gerçekleştirebilirler.

Çıkış değerini ise *fonksiyon_tipi* bölümü belirleyecektir. Fonksiyon tipi de *int, float, double, char vb.* değişken türlerinden herhangi bir tanesi olabilir. Fonksiyon yaptığı işlemin ardından, *main()* ortamına herhangi bir değer geri döndürmüyor ise *fonksiyon_tipi* bölümüne *void* ifadesi yazılır. Geri değer döndürmeyen fonksiyonlara, *main()*'den fonksiyona gönderilen değerleri *printf* fonksiyonu kullanarak ekrana yazdıran fonksiyonlar örnek olarak gösterilebilir.

Fonksiyonun görevi olan işlemi gerçekleştiren fonksiyon bloğu ise,

```
fonksiyon_tipi fonksiyon_adı(arguman1, arguman2){
yerel değişkenler ;
fonksiyon kod satirlari;
return (geri_donus_degeri) ;
}
```

şeklinde oluşturulur. Fonksiyon bloğunda, işlem sonucunda, hangi değişken, fonksiyonun çağrıldığı yere geri döndürülecek ise, o değişken fonksiyon bloğunun sonundaki *return* ifadesinin yanına yazılacaktır. *fonksiyon_tipi* ise dönüş değerinin tipini belirler. Fonksiyon tanımında herhangi bir tip türü yazılmaz ise geri dönüş değeri varsayılan değer olarak, *int (tam sayı)* kabul edilir.

Fonksiyonun çağırılması aşaması ise *main()* içerisinde,

```
fonksiyon_tipi fonksiyon_adı(arguman1, arguman2);
şeklinde gerçekleştirilir.
```

Aşağıda yer alan Örnek 8.1'de, *main()* içerisinden gönderilen iki değeri kullanarak toplama işlemi yapan *topla()*, çarpma işlemi yapan *carp()* fonksiyonları ile sonuçları ekrana yazdıran *yazdır()* fonksiyonları kullanılmıştır.

Örnek 8.1: Fonksiyon tanımlama ve çağırma

```

#include <stdio.h>
#include <conio.h>      /*getch fonksiyonunu kullanabilmek için
tanımlanmıştır.*/
int topla(int x, int y);      /*fonksiyon tanımlama*/
int carp(int x, int y);      /*fonksiyon tanımlama*/
void yazdir(int x,int y);    /*fonksiyon tanımlama*/

int main(void){
    int a,b,t,c;
    a=10;
    b=5;
    t=topla(a,b);      /*Fonksiyon cagirma*/
    c=carp(a,b);      /*Fonksiyon cagirma*/
    yazdir(t, c);      /*Fonksiyon cagirma*/
    getch();          /*Ekran ciktisini ekranda bekletebilmek için*/
    return 0;
}                      /*main ana fonksiyon sonu*/

int topla(int x,int y)  /*toplama islemini gerceklestirir*/
{ int sonuc;
  sonuc=x+y;
  return sonuc; }

int carp(int x,int y)  /*carpma gerceklestirir,carpim sonucunu
geri döndürür*/.
{ int sonuc;
  sonuc=x*y;
  return sonuc; }

void yazdir(int x, int y)      /*ekrana yazdirma islemini ger-
ceklestirir*/.
{
    printf("toplama sonucu: %d", x);
    printf("\n");
    printf("carpim sonucu: %d", y); }

```

Program Çıktısı: (Örnek 8.1)

toplama sonucu: 15

çarpım sonucu: 50

Örnek program kodlarındaki satırların bitiminden sonra /* ve */ işaretleri arasında yer alan metinler derleyici tarafından program satırı olarak dikkate alınmaz ve derlenmez. Bu işaretler arasına yazılan metinler ilgili satırı açıklamak amacıyla yazılmıştır.

**DİKKAT**

Yukarıda Örnek 8.1'de *main()* içerisinde yalnızca değişken tanımlamaları yapılmış olup, işlemler fonksiyonlar içerisinde gerçekleştirilmiştir. İşlem sonuçları *main()* ortamına geri gönderilerek, bu sonuçlar *yazdir()* fonksiyonuna argüman olarak verilip *yazdir()* fonksiyonu içerisinde ekrana yazdırılmıştır. Böylece giriş argümanı aldığı hâlde, çıkış değeri olarak değer geri döndürmeyen (void) bir fonksiyon kullanılarak ekrana yazdırma işlemi gerçekleştirilmiştir. Ayrıca fonksiyon tanımı ve fonksiyon bloğunun içerisinde *x*, *y* değişkenleri kullanılmasına rağmen, *main()* içerisinde fonksiyonun çağırılması aşamasında *a* ve *b* değişkenleri kullanılmıştır. Bu durumda fonksiyonda *a* değişkenini *x*, *b* değişkenini *y* karşılamaktadır. Fonksiyon içerisinde tanımlanan değişkenler yerel değişkenler olup fonksiyon çalıştığı sürece aktif olup fonksiyondan çıkıldığında iptal edilirler. Bu durumda asıl önemli olan nokta, fonksiyon tanımındaki argüman sayısı ve çeşididir.

DİKKAT



Fonksiyon tanımlamasından sonra “;” (noktalı virgül) işaretinin unutulması derleme hatası verecektir.

SIRA SİZDE



Fonksiyon tanımının, değişken adını belirtmeden “`int topla(int, int);`” şeklinde kullanımını Örnek 8.1’de ilgili satırları değiştirerek deneyiniz.

FONKSİYONLARA ARGÜMAN GEÇİRME

İçerisine argüman alan fonksiyonlarda, bu argümanların gönderilmesi için iki farklı yöntem bulunmaktadır. Bunlar değer yöntemi(*call by value*) ve referans yöntemi(*call by reference*) ile çağırma.

Fonksiyona Değer Yöntemi ile Argüman Geçirme

Değer yöntemi ile çağırma işlemi, daha önce Örnek 8.1’de kullanılmış olan *topla()*, *carp()* fonksiyonlarının kullanımına benzer bir yapıdadır. Fonksiyona iletilen değişkenlerin(argümanların) yalnızca sayısı kadar tip bildirimi iletilmekte olup, fonksiyon bloğu aktif olduğu anda, fonksiyonun kendi içerisinde, gönderilen argümanların birer kopyası oluşturulmaktadır. Bir diğer ifade ile, fonksiyon içerisindeki işlemlerde, *main()* fonksiyonunda ilk anda oluşturulan orijinal veri yerine kopyası ile işlem yapılmaktadır. Böylece fonksiyon içerisinde orijinal değerler değiştirilememektedir.

Aşağıda Örnek 8.2’de yer alan program kodunda değer yöntemi ile *kare* fonksiyonuna eleman geçirme işlemi gerçekleştirilmektedir. Bu program kodu kullanılarak, fonksiyona geçirilen *a* değişkeninin *main()* içerisinde, fonksiyon içerisinde ve fonksiyondan geri dönişte tekrar *main()* içerisindeki sayısal değerleri ekrana yazdırılmıştır.

Örnek 8.2: Fonksiyona değer yöntemi ile argüman geçirme

```
#include <stdio.h>
#include <conio.h>          /*getch fonksiyonunu kullanabilmek için
tanımlanmıştır.*/.
int kare(int);              /*fonksiyon tanımlama*/.

int main(void) {
    int a=4, k;
    printf("main fonksiyonundayım\n");
    printf("kare fonksiyonundan önce a =%d \n", a);
    k=kare(a);              /*Fonksiyon çağırma*/
    printf("main fonksiyonuna geri geldim \n");
    printf("kare fonksiyonundan donuste a=%d\n",a);
    printf("%d nin karesi %d dir ",a , k);
    getch();               /*Ekran ciiktisini ekranda bekletebilmek için*/.
    return 0;
}                          /* main fonksiyonu sonu*/

int kare(int a)             /*kare alma fonksiyon blogu*/
{ printf("kare fonksiyonunun icinden merhaba :)\n");
  printf("kare fonksiyonunda kopyasi olusturulan deger a=%d\n",a);
  a=a*a;
  printf("kare fonksiyonunda degeri degisen a=%d\n",a);
  printf("kare fonksiyonundan cikiyorum.....\n");
  return (a);
}
```

Program Çıktısı: (Örnek 8.2)

```

main fonksiyonundayım
kare fonksiyonundan önce a=4
kare fonksiyonunun içinden merhaba :)
kare fonksiyonunda kopyası oluşturulan deger a=4
kare fonksiyonunda degeri degisen a=16
kare fonksiyonundan cikiyorum.....
main fonksiyonuna geri geldim
kare fonksiyonundan donuste a=4
4 nin karesi 16 dir

```

Örnek 8.2'nin program çıktısından da görüldüğü üzere, fonksiyon içinde fonksiyona verilen argümanın kopyası oluşturularak işlem yapılmaktadır. İşlem sonucu yine aynı değışkene atanmasından dolayı, değeri değışiyor gibi görünse de fonksiyon dönüşünde aynı *a* değışkeni *main()* içerisinde ekrana yazdırıldığında değerin değışmediğı gözlenmektedir. Böylece *main()* ve *kare* fonksiyonunda aynı isimle kullanılan *a* değışkenlerinin gerçekte farklı değışkenler olarak kullanıldığı ve değışimlerden etkilenmedikleri gözlenmektedir.

Örnek 8.2'de fonksiyon çağırma satırı “k=kare(a);” yerine “a=kare(a);” şeklinde değıştirilir ise ve “printf(“ %d nin karesi %d dir “a,k);” satırı iptal edilir ise program çıktısı nasıl değışecektir?



SIRA SİZDE

Fonksiyona Referans Yöntemi ile Argüman Geçirme

Örnek 8.2'de de gösterildiğı gibi değeri yöntemi ile fonksiyonlara argüman geçirildiğinde fonksiyondaki değışiklikler orijinal değeri değıştirmemesine rağmen, fonksiyona referans yöntemi ile argüman geçirildiğinde orijinal değeri değışebilmektedir. Çünkü referans yöntemi, orijinal değışkenin adres değeri kullanılmaktadır. Bellekteki adreslerle işlem yapılabilmesi için önceki ünite anlatılan gösterici (pointer) yapıları kullanılmaktadır. Fonksiyona geçirilen argüman bir gösterici-değışken olarak kullanılmaktadır.

Aşağıda yer alan Örnek 8.3'te bulunan program kodunda sayının karesini alma işlemi gösterici kullanılarak referans yöntemi ile yapılmaktadır.

Örnek 8.3: Fonksiyona referans yöntemi ile argüman geçirme

```

#include <stdio.h>
#include <conio.h>          /*getch fonksiyonunu kullanabilmek için
tanımlanmıştır.*/
void kare(int *aGst);      /*fonksiyon tanımlama*/

int main(void){
    int a=4, k;
    printf("main fonksiyonundayım\n");
    printf("kare fonksiyonundan önce a=%d\n",a);
    kare(&a);               /*Fonksiyon çağırma*/
    printf("main fonksiyonundayım\n");
    printf("kare fonksiyonundan donuste a=%d\n",a);
    getch();               /*Ekran çıktısını ekranda bekle-
tebilmek için*/
    return 0;
}

void kare(int *aGst)/*kare alma islemini gerceklestiren fonksi-
yon blogu*/
{ printf("kare fonksiyonunun içinden merhaba :) \n");
  printf("kare fonksiyonunda kopyası oluşturulan deger a =%d\n",
*aGst);
  *aGst=*aGst * *aGst;
  printf("kare fonksiyonunda degeri degisen a=%d\n",*aGst);
  printf("kare fonksiyonundan cikiyorum.....\n"); }

```

Program Çıktısı: (Örnek 8.3)

```

main fonksiyonundayım
kare fonksiyonundan önce a=4
kare fonksiyonunun icinden merhaba :)
kare fonksiyonunda kopyasi olusturulan deger a=4
kare fonksiyonunda degeri degisen a=16
kare fonksiyonundan cikiorum.....
main fonksiyonundayım
kare fonksiyonundan donuste a=16

```

Örnek 8.3'ün program çıktısı incelendiğinde, kare fonksiyonuna değişken değerinin kopyası yerine aGst göstericisi yardımıyla adresi geçirildiği için *main()* fonksiyonu içerisindeki orijinal değişkene fonksiyon tarafından ulaşılabilmektedir. *main()* fonksiyonuna geri dönüşte, *main()* ortamında a değişkenine hiçbir değişiklik yapılmamasına rağmen fonksiyona gönderilen *a=4* sayısının karesini alma işlemi sonucu yine a GST'ye atandığı için dolaylı olarak *a=16* olmuştur. Böylece *main()* e geri dönüşte *a=16* olarak değiştirilmiştir.

DİKKAT

Referans yöntemi ile argüman geçirildiğinde, çoğunlukla fonksiyondan bir geri dönüş değeri ihtiyaç kalmamaktadır. Bu nedenle fonksiyonun son bölümünde de return ifadesi yer almamaktadır.

Özet olarak, Örnek 8.2 ve Örnek 8.3'te yer alan örneklerden de anlaşıldığı üzere, fonksiyonlara argüman geçirirken gönderilen değişkenin *main()* deki mevcut değeri değiştirilmek istenmez ise değer yolu ile fonksiyon çağırma işlemi, orijinal değer de değişmesi isteniyorsa referans yolu ile fonksiyon çağırma işlemi gerçekleştirilir.

Dizileri Fonksiyona Argüman olarak Geçirme

Bir önceki ünite de detayları verilen diziler, göstericilerle karşılıklı olarak birbirleri yerine kullanılabilen değişken küme yapılarıdır. Bir dizinin elemanları fonksiyona argüman olarak geçirilmek istendiğinde her bir elemanın ayrı ayrı gönderilmesi gerekir. Ancak bu şekildeki kullanım çok pratik değildir. Bu nedenle dizinin tüm elemanlarını geçirebilmek için dizinin ilk elemanının adresi yani başlangıç adresini geçirmek daha uygun olacaktır. Dizinin başlangıç adresinin geçirilmesi aslında diziyi gösterici olarak kullanmak anlamına da gelir.

Aşağıda yer alan Örnek 8.4'te bir diziye ait elemanların her biri ayrı ayrı fonksiyona argüman olarak atanmaktadır. for yinelemesi içerisinde yer alan,

Örnek 8.4: Dizinin elemanlarını fonksiyona argüman olarak geçirme

```

#include<stdio.h>
#include <conio.h>          /*getch fonksiyonunu kullanabilmek için
tanımlanmıştır.*//.
#define SIZE 5              int kare(a);/*fonksiyon tanımlama*/
int kare(int);              /*fonksiyon tanımlama*/

int main(void){
    int dizi_1[ SIZE] = { 0 }; /*dizi_1' in elemanlari sifirlaniyor*/
    int i;
    for(i=0; i< SIZE; i++)
    { dizi_1[ i] =i;
      printf("fonksiyon oncesi dizi_1[ %d] =%d",i, dizi_1[ i] );
      dizi_1[ i] = kare (dizi_1[ i] );          /*dizi_1 in elemanlari ayri
ayri gonderiliyor*/
      printf("  fonksiyon sonrasi dizi_1[ %d] =%d\n",i, dizi_1[ i] );
    }
}

```

```

    getch();          /*Ekran ciktisini ekranda bekletebilmek icin*/
    return 0;
}

int kare(int a)        /*kare alma islemi fonksiyon blogu*/
{
    return (a*a);
}

```

Program Çıktısı: (Örnek 8.4)

fonksiyon oncesi dizi_1[0]=0 fonksiyon sonrasi dizi_1[0]=0
 fonksiyon oncesi dizi_1[1]=1 fonksiyon sonrasi dizi_1[1]=1
 fonksiyon oncesi dizi_1[2]=2 fonksiyon sonrasi dizi_1[2]=4
 fonksiyon oncesi dizi_1[3]=3 fonksiyon sonrasi dizi_1[3]=9
 fonksiyon oncesi dizi_1[4]=4 fonksiyon sonrasi dizi_1[4]=16

Örnek 8.4'te, $dizi_1[i] = kare(dizi_1[i])$; satırı ile döngünün her tekrarında dizinin bir elemanı fonksiyona gönderilmektedir. Böylece fonksiyon 5(beş) kez çağırılmış olmaktadır. $dizi_1[i] = kare(dizi_1[i])$; satırı ile de fonksiyonun geri dönüş değeri tekrar $dizi_1$ in aynı elemanına eşitlendiği için $dizi_1$ in elemanları da fonksiyon dönüşünde sayısal olarak karesi alınan yeni değer ile güncellenmiş olmaktadır.

Örnek 8.4'te yer alan program kodunda $dizi_1[i] = kare(dizi_1[i])$; satırı yerine, $dizi_2$ adında farklı bir dizi tanımlanarak, her bir geri dönüş değeri $dizi_2[i] = kare(dizi_1[i])$; şeklinde atanırsa $dizi_1$ 'in elemanlarının değeri ne olur?



SIRA SİZDE

Aşağıda yer alan Örnek 8.5'te bulunan program kodunda ise, dizi fonksiyona gösterici-referans yöntemi ile argüman olarak geçirilmektedir. Dizi elemanlarını, yineleme (for döngüsü) kullanarak ayrı ayrı geçirmeye gerek kalmamaktadır.

Örnek 8.5: Dizinin elemanlarını referans yöntemi ile geçirme

```

#include<stdio.h>
#include <conio.h>          /*getch fonksiyonunu kullanabilmek icin
tanımlanmıştır.*/
#define SIZE 5
void kare(int *aGst);      /*fonksiyon tanımlama*/

int main(void) {
    int dizi_1[ SIZE]={ 0 }; /*dizi_1' in elemanlari sifirlaniyor*/
    int i;
    for(i=0;i< SIZE; i++)    /*fonksiyona gonderim oncesi dizi ele-
manlari listeleniyor*/
    {
        dizi_1[ i]=i;
        printf("fonksiyon oncesi dizi_1[ %d]=%d\n",i, dizi_1[ i] );
    }
    printf("\n");
    kare (dizi_1);           /*dizi_1' in baslangic adresi gonderiliyor*/

    for(i=0;i< SIZE; i++)
    {
        printf("fonksiyon sonrasi dizi_1[ %d]=%d\n",i, dizi_1[ i] );
    }
    getch();                /*Ekran ciktisini ekranda bekletebilmek icin*/
    return 0;
}                            /*main fonksion sonu*/

void kare(int *aGst) /*kare alma islemifonksiyon blogu */
{
    int i;
    for(i=0;i<SIZE;i++)
    {
        aGst[ i]=(aGst[ i] *aGst[ i] );
    }
}

```

Program Çıktısı: (Örnek 8.5)

```

fonksiyon oncesi dizi_1[0]=0
fonksiyon oncesi dizi_1[1]=1
fonksiyon oncesi dizi_1[2]=2
fonksiyon oncesi dizi_1[3]=3
fonksiyon oncesi dizi_1[4]=4

fonksiyon sonrasi dizi_1[0]=0
fonksiyon sonrasi dizi_1[1]=1
fonksiyon sonrasi dizi_1[2]=4
fonksiyon sonrasi dizi_1[3]=9
fonksiyon sonrasi dizi_1[4]=16

```

kare (dizi_1); satırı ile *dizi_1* in başlangıç adresi *kare* fonksiyonuna argüman olarak geçirilmektedir. Bu satır bir diğer şekli ile *kare (&dizi_1[0]);* satırı ile eş değerdir. *kare* fonksiyonu bloğunda ise argüman olarak *aGst* göstericisi kullanılmaktadır. Böylece *dizi_1*, *aGst* birbirlerinin yerine kullanılabilir. Program çıktısında da gösterildiği üzere, *kare* fonksiyonu **return** ile bir değer geri döndürmediği hâlde, *dizi_1*'in elemanları sayısal olarak değişmiş durumdadır ve *main()*' de, fonksiyon sonrasında for yinelemesi ile ekrana yazdırılmaktadır.

Aşağıda yer alan Örnek 8.6'da, referans yöntemi ile fonksiyona karakterlerden oluşan dizi (dizgi) nin argüman olarak geçirilmesi ile ilgili bir program kodu bulunmaktadır. *cumle* fonksiyonu iki adet dizgiyi argüman olarak almakta olup ilki dizgi hâlinde iken ikincisi karakterlerden oluşan kelime hâlinindedir. Bu durumda *cumle* fonksiyonu iki kelimeyi arka arkaya ekrana yazdırmaktadır.

Örnek 8.6: Dizgilerin ve karakterlerin referans yöntemi ile fonksiyona argüman olarak geçirilmesi

```

#include<stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için ta-
nimlanmıştır.*/
void cumle(char *g1, char *g2); /*fonksiyon tanımlama*/

int main(void) {
    char k[]="Programlama "; /*k nin elemanlari ilkleniyor*/
    cumle(k, "ogreniyorum"); /*Fonksiyon cagrilmasi*/
    getch(); /*Program ciktisini ekranda bekletebil-
mek icin*/
    return 0;
} /*main fonksiyonu sonu*/

void cumle(char *g1,char *g2)
{
    int i=0;
    printf(g1);
    while(g2[ i ] != '\0' ) /*kelime sonuna gelindimi ? */
    { printf("%c", g2[ i ] );
      i++;
    }
}

```

Program Çıktısı: (Örnek 8.6)

Programlama ogreniyorum

Örnek 8.6'da iki kelime yan yana ekrana yazdırılmaktadır. Dizgilerin içeriği cumle fonksiyonuna iki adet argüman ile aktarılmaktadır. Aktarılan argümanlar, fonksiyon içerisinde g1 ve g2 göstericileri ile eşleşerek, *cumle* fonksiyonu içerisinde ekrana yazdırma işlemi gerçekleştirilmektedir.

Örnek 8.6'da iki kelime arasında bir adet boşluk bırakılması için "Programlama " kelimesinin sonunda bir adet boşluk bulunmaktadır. Bu boşluk olmaması durumunda ekranda "Programlamaogreniyorum" şeklinde olacaktır.



DİKKAT

C STANDART HAZIR KÜTÜPHANE FONKSİYONLARININ KULLANIMI

C programlama dilinde, ihtiyaca göre programcı tarafından yazılan fonksiyonların yanı sıra hazır olarak bulunan kütüphane fonksiyonları da bulunmaktadır. Şu ana kadar bazı program kodlarında ekrana yazdırma veya klavyeden girişler için kullanılan *printf* ve *scanf* kelimeleri de gerçekte hazır kütüphane fonksiyonlarıdır. Belli kurallara göre argüman almakta ve yapacağı işlemi gerçekleştirmektedir. C programlama dilinde birçok hazır kütüphane bulunmaktadır. Bu kütüphane fonksiyonlarını kullanıp, gereksiz yere zaman harcayarak, yeniden fonksiyon oluşturmaya gerek kalmamaktadır. Yalnızca önışlemci bölümüne ilgili fonksiyonun içerildiği *"*.h"* uzantılı kütüphane dosyalarını dâhil etmek yeterlidir.

Bu kütüphane fonksiyonlarından en çok kullanılanları *math.h*, *string.h*, *ctype.h*, *stdlib.h* fonksiyonları ile kullanım örnekleri aşağıda yer almaktadır.

Matematiksel Kütüphane Fonksiyonları (math.h)

Hazır matematiksel fonksiyonları kullanabilmek için önışlemci tanımları bölümüne *#include <math.h>*

adlı başlık dosyası eklenmelidir. Matematiksel fonksiyonların kullandığı sayılar double tipindedir.

Başlıca matematiksel fonksiyonlar *acos()*, *asin()*, *atan()*, *ceil()*, *cos()*, *cosh()*, *exp()*, *floor()*, *log()*, *log10()*, *pow()*, *sin()*, *sqrt()*, *tan()*, *tanh()*, *ceil()* olarak sıralanabilir. Aşağıda yer alan Örnek 8.7' de bu fonksiyonlardan bazılarının kullanımı ile ilgili program satırları yer almaktadır.

Örnek 8.7: Matematiksel fonksiyonların kullanımı

```
#include<stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek icin tanimlanmistir.* /
#include <math.h>
#define PI 3.141593

int main(void)
{ double aci_D=90;
  double aci_R=aci_D*(PI/180); /*aciyi dereceden radyana donusturme*/
  double a=16;
  int p=3;
  double k=0.2;
  printf("cos(%.01f)= %.21f\n",aci_D, cos(aci_R));
  printf("sin(%.01f)= %.21f\n",aci_D, sin(aci_R));
  printf("pow(%d,3)=%.21f\n",p, pow(p,3)); /* 1. argumani 2.arguman kadar carpar*/
  printf("sqrt(%.01f)=%.21f\n",a, sqrt(a)); /*Argumanin karekokunu alir*/
  printf("floor(%.21f)=%.21f\n",k, floor(k)); /*En yakin kucuk tamsayiye yuvarlar*/
  printf("ceil(%.21f)=%.21f\n",k, ceil(k)); /*En yakin buyuk tamsayiye yuvarlar*/
  getch(); /*Program ciktisini ekranda bekletebilmek icin*/
  return 0;
} /*main() fonksiyonu sonu*/
```

Program Çıktısı: (Örnek 8.7)

```

cos(90)= -0.00
sin(90)= 1.00
pow(3,3)= 27.00
sqrt(16)= 4.00
floor(0.20)= 0.00
ceil(0.20)= 1.00

```

Örnek 8.7’de `math.h` başlık dosyası kapsamındaki matematiksel fonksiyonların kullanımı gösterilmektedir. `cos` ve `sin` gibi trigonometrik fonksiyonlar giriş argümanı olarak radyan değeri alırlar. Bu nedenle, $aci_R = aci_D * (PI/180)$ şeklinde dereceden radyana dönüştürme formülü uygulanmıştır. Bütün fonksiyonlar için giriş ve çıkış argümanı `double` veri tipinde kullanılmıştır.

Dizgi ve Karakter Fonksiyonları (string.h, ctype.h)

Karakterleri veya dizgileri kullanarak işlem yapan hazır kütüphane fonksiyonları `string.h` ve `ctype.h` başlık dosyaları tarafından içerilirler. Bu bölümde, karakterlerle ve karakter kümeleriyle kullanılan temel hazır fonksiyonları içeren örnek program kodları yer almaktadır.

Aşağıdaki Örnek 8.8’de klavyeden girilen ismin karakter uzunluğu `strlen()` (string-length) fonksiyonu ile hesaplanıp ekrana yazdırılmaktadır.

Örnek 8.8: `strlen()` fonksiyon kullanımı

```

#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
#include <string.h>
int main(void)
{
    char ad[10], n;
    printf("Bir isim giriniz:");
    scanf("%s", &ad);
    n=strlen(ad);
    printf("Girdiginiz isim: %s, %d karakter uzunlugundadir\n", ad, n);
    getch(); /*Program ciktisini ekranda goruntuleme icin*/
    return 0;
}

```

Program Çıktısı: (Örnek 8.8)

Bir isim giriniz:neslihan

Girdiginiz isim: neslihan, 8 karakter uzunluğundadır.

Yukarıdaki Örnek 8.8’de yer alan program kodunda kullanılan, “`#include <string.h>`” satırı `strlen` gibi özel dizgi fonksiyonlarının kullanılabilmesi için hazır başlık dosyası olarak eklenmesi gerekli bir dosyadır. `strlen` fonksiyonu giriş argümanı olarak, klavyeden girilen `ad` değişkenini almakta, fonksiyon geri dönüş değeri ise giriş argümanının karakter sayısıdır.

Zaman zaman farklı karakter dizgilerini bir arada kullanarak işlem yapmak gerekebilir. Bu durumdaki fonksiyonlara örnek olarak `strcpy` verilebilir. `strcpy(birinci, ikinci)` fonksiyonu, parantezler arasında iki parametre(veri-değişken) alır. Bunlardan ikincisini birinciye kopyalar. Aşağıda yer alan Örnek 8.9’da `k1` ve `k2` dizgilerine farklı kelimeler atanmasına rağmen `strcpy` kullanılarak aktarma yapılmasının ardından, her iki dizgide de “ogreniyorum” kelimesi yer almaktadır.

Örnek 8.9: strcpy() fonksiyon kullanımı

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
#include <string.h> /*strcpy fonk kullanabilmek için*/
int main(void)
{
    char k1[ 30], k2[ 3] ;
    strcpy(k1, "programlama");
    strcpy(k2, "ogreniyorum");
    printf("k1 Dizgisi:%s\n", k1);
    printf("k2 Dizgisi:%s\n", k2);
    strcpy(k1, k2);
    printf("k2 dizgisi k1 dizgisine kopyalandi\n");
    printf("k1 Dizgisi:%s\n", k1);
    printf("k2 Dizgisi:%s", k2);
    getch(); /*ekranda görüntüleme için*/
    return 0;
}
```

Program Çıktısı: (Örnek 8.9)

```
k1 Dizgisi: programlama
k2 Dizgisi: ogreniyorum
k2 dizgisi k1 dizgisine kopyalandi
k1 Dizgisi: ogreniyorum
k2 Dizgisi: ogreniyorum
```

Bir diğer dizgi fonksiyonu, *strcmp()* fonksiyonudur. *strcmp()* ile karakter dizilerinin sayısal olarak karşılaştırılması sağlanmaktadır. Karakterlerin her birinin ASCII olarak sayısal bir karşılığı bulunmaktadır. *strcmp()* ile bu harflerin sayısal karşılıkları kullanılarak dizilim sırası karşılaştırılmaktadır. Aşağıda yer alan Örnek 8.10'da *strcmp()* fonksiyonu kullanımı ile ilgili örnek bir kod bulunmaktadır.

Örnek 8.10: strcmp() fonksiyonunun kullanımı

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
#include <string.h> /*strcpy fonk kullanabilmek için*/

int main(void)
{
    int d1,d2,d3,d4,d5,d6,d7,d8;
    d1=strcmp("neslihan","neslihan");
    d2=strcmp("NESLIHAN","neslihan");
    d3=strcmp("neslihan","NESLIHAN");
    d4=strcmp("ABC","XYZ");
    d5=strcmp("XYZ","ABC");
    d6=strcmp("abc","xyz");
    d7=strcmp("xyz","abc");
    d8=strcmp("abc","XYZ");
    printf("d1=%d d2=%d d3=%d d4=%d d5=%d d6=%d d7=%d d8=%d\n",
d1, d2, d3, d4, d5, d6, d7, d8);
    getch(); /*ekranda görüntüleme için*/
    return 0;
}
```

Program Çıktısı: (Örnek 8.10)

```
d1= 0 d2= -1 d3= 1 d4= -1 d5= 1 d6= -1 d7= 1 d8= 1
```

Örnek 8.10'da program çıktıları incelendiğinde, her iki karakter seti aynı ise *strcmp* fonksiyonu 0, aynı kelimenin büyük ve küçük harfle yazılmış hâli karşılaştırıldığında küçük harflerin daha büyük sayısal değere sahip olduğu, alfabenin sonlarındaki harflerin

Küçük harflerin ASCII kod karşılıkları büyük harflere göre daha büyüktür.

ilk harflerden daha büyük sayısal değere sahip olduğu gözlenmektedir. d8 değeri incelendiğinde alfabede “abc”nin “XYZ” den sayısal olarak büyük olması nedeniyle strcmp() fonksiyonu 1 değeri geri döndürmektedir.

Sıklıkla kullanılan bir diğer dizgi fonksiyonu ise, *strcat*(birinci, ikinci) fonksiyonu, aldığı iki adet giriş dizgilerinden ikincisini birincisinin sonuna boşluk bırakmadan ekler. *strcat*() in kullanımı ile ilgili açıklayıcı bir örnek kod Örnek 8.11’de yer almaktadır.

Örnek 8.11: strcat() fonksiyonu kullanımı

```
#include<stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
#include <string.h> /*strcpy fonk kullanabilmek için*/

int main(void)
{ char k1[ 30] , k2[ 30] ;
  strcpy(k1, "programlama");
  strcpy(k2, "ogreniyorum");
  printf("k1 Dizgisi:%s\n", k1);
  printf("k2 Dizgisi:%s\n", k2);
  strcat(k1, k2);
  printf("strcat ile k2 dizgisi k1 dizgisine eklendi\n");
  printf("k1 Dizgisi:%s\n", k1);
  printf("k2 Dizgisi:%s", k2);
  getch(); /*ekranda görüntüleme için*/
  return 0;
} /*main fonksiyonu sonu*/
```

Program Çıktısı: (Örnek 8.11)

```
k1 Dizgisi: programlama
k2 Dizgisi: ogreniyorum
strcat ile k2 dizgisi k1 dizgisine eklendi
k1 Dizgisi: programlamaogreniyorum
k2 Dizgisi: ogreniyorum
```

Örnek 8.6’da hazır fonksiyon kullanılmadan kod yardımıyla yapılan işlem, Örnek 8.11’de **strcat** hazır kütüphane fonksiyonu kullanılarak gerçekleştirilmiştir.

Karakterlerden oluşan dizgilerle ilgili fonksiyonları *string.h* başlık dosyası içerirken, yalnızca karakterlerle ilgili işlemleri içeren fonksiyonları ise *ctype.h* başlık dosyası tarafından içerilmektedir. Aşağıda Şekil 8.1’de bazı karakter fonksiyonları listelenmiştir.

Tablo 8.1
Bazı karakter
fonksiyonları

Fonksiyon Kullanımı	Açıklama
char toascii (int c);	c sayısı ile verilen ASCII koda sahip karakteri verir.
int isdigit (int c);	c bir rakam ise 0’dan farklı, diğer durumda 0 gönderir.
int isalpha (int c);	c bir harf ise 0’dan farklı, diğer durumda 0 gönderir.
int islower (int c);	c küçük harf ise 0’dan farklı, diğer durumda 0 gönderir.
int isupper (int c);	c büyük harf ise 0’dan farklı, diğer durumda 0 gönderir.
int tolower (int c);	Giriş argümanını küçük harfe çevirir.
int toupper (int c);	Giriş argümanını büyük harfe çevirir.

SıraSizde_4_Program kodu:

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek icin*/
#include <ctype.h> /*strcpy fonk kullanabilmek icin*/
int main(void)
{
    int i;
    char a;
    printf("ASCII \t Karakter\n");
    for(i=65; i<127; i++)
    {
        a = toascii(i);
        printf("%d\t%c\n", i,a);
    }
    getch();
    return 0;
}
```

Aşağıda yer alan Örnek 8.12’de Şekil 8.1’deki tabloda verilen karakter fonksiyonları ile ilgili örnek program satırları bulunmaktadır.

Örnek 8.12: Karakter fonksiyonlarının kullanımı

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek icin*/
#include <ctype.h> /*karakter fonk kullanabilmek icin*/

int main(void)
{
    char cB=' A' ;
    char ck=' z' ;
    char m=' 6' ; /*6 rakami karakter olarak ' ' isaretleri arasina yaziliyor*/.
    if(isdigit(m)!=0) printf("%c bir rakamdir\n",m);
    else printf("%c bir rakam degildir\n",m);
    if(isalpha(m)!=0) printf("%c bir harfdir\n",m);
    else printf("%c bir harf degildir\n",m);
    if(isdigit(cB)!=0) printf("%c bir rakamdir\n",cB);
    else printf("%c bir rakam degildir\n",cB);
    if(isalpha(cB)!=0) printf("%c bir harfdir\n",cB);
    else printf("%c bir harf degildir\n",cB);
    if(isdigit(ck)!=0) printf("%c bir rakamdir\n",ck);
    else printf("%c bir rakam degildir\n",ck);
    if(isalpha(ck)!=0) printf("%c bir harfdir\n",ck);
    else printf("%c bir harf degildir\n",ck);
    if(islower(cB)!=0) printf("%c kucuk bir harfdir\n",cB);
    else printf("%c kucuk bir harf degildir\n",cB);
    if(isupper(cB)!=0) printf("%c buyuk bir harfdir\n",cB);
    else printf("%c buyuk bir harf degildir\n",cB);
    if(islower(ck)!=0) printf("%c kucuk bir harfdir\n",ck);
    else printf("%c kucuk bir harf degildir\n",ck);
    if(isupper(ck)!=0) printf("%c buyuk bir harfdir\n",ck);
    else printf("%c buyuk bir harf degildir\n",ck);
    printf("tolower('%c')=%c\n",cB,tolower(cB));
    printf("toupper('%c')=%c\n",ck,toupper(ck));
    getch();
    return 0;
} /* main fonksiyon sonu*/
```

Program Çıktısı: Örnek 8.12

6 bir rakamdir.

6 bir harf değildir.

A bir rakam değildir.

```

A bir harfdir.
z bir rakam değildir.
z bir harfdir.
A küçük bir harf değildir.
A büyük bir harfdir.
z küçük bir harfdir.
z büyük bir harf değildir.
tolower('A')=a
toupper('z')= Z

```

Örnek 8.12'nin program çıktısından da görüldüğü üzere, verilen karakterin rakam, harf, büyük harf, küçük harf olup olmadığı, büyük harf ise küçük harfe, küçük harf ise büyük harfe dönüştürme işlemleri bu fonksiyonlar yardımıyla gerçekleştirilebilmektedir.



Diğer karakter yada dizgi hazır kütüphane fonksiyonları ile ilgili ayrıntılı bilgiyi C ve C++, (Çeviri:C How to Program-Third Edition) (Zavrak, M., Aksoy, E. , Karaca, H.N. 2004, İstanbul: Sistem Yayıncılık.) ve bu kitabın orijinalini C How to Program-6th Edition (Deitel, P.,Deitel, H. 2010. Pearson-Prentice Hall.) kitaplarından elde edebilirsiniz.

Dinamik Bellek Ayırma Fonksiyonları

C programlama dilinde, değişkenlerle veya herhangi bir dosyadan okunan verileri kullanarak işlemler gerçekleştirirken, bellekte verilerin büyüklüğüne göre sürekli yeni yerler ayrılır. Ancak bu verilerin büyüklüğü veya aynı anda birden çok programın çalıştırılması gereken durumlarda, programın çalıştırıldığı bilgisayarın belleğinin kapasitesine göre yetersizlik gösterebilir. Bu nedenle verilere sürekli yeni bellek alanları tahsis eden statik değişken atama yöntemi yerine, gerektiğinde bellekte yer açıp işlem bittiğinde o bellek alanını boşa çıkaran *dinamik bellek atama* işlemi gerekebilir. Böylece bellek daha verimli bir şekilde kullanılabilir. Bahsedilen dinamik bellek ayırma işlemleri için C'de bulunan hazır kütüphane fonksiyonlarından *malloc*, *calloc*, *realloc* ve *free* fonksiyonları kullanılabilir. Bu fonksiyonlar *stdlib.h* başlık dosyasını kullanırlar.

Bellek ayırma için;

```

void *malloc(n);
void *calloc(n, tip_uzunluğu);

```

fonksiyonları kullanılabilir. *malloc* ve *calloc* fonksiyonlarının kullanımları farklı olsa da yaptıkları işlem aynıdır. *malloc* argüman olarak doğrudan verinin byte türünden büyüklüğünü alırken, *calloc* fonksiyonu iki argüman alır. Bunlardan ilki kaç adet veri olduğu, ikincisi ise bu verinin veri tipinin *byte* olarak değerini gösterir,

malloc veya *calloc* ile tahsis edilen bellek alanının kullanımı bittiğinde, *free* fonksiyonu ile ilgili bellek alanı serbest bırakılır.

Bellek ayırma işlemlerinde bir diğer fonksiyon ise *realloc* fonksiyonudur. Bu fonksiyon ise *malloc* veya *calloc* ile ayrılan bellek alanı programın ilerleyen satırlarında, yetersiz kalma veya azaltılması gereken durumlarda mevcut ayrılan alana ilave bellek alanı ekleme veya çıkarma amacıyla kullanılır. *realloc* fonksiyonunun kullanılabilmesi için daha önceden *malloc* veya *calloc* fonksiyonu ile bellek tahsis edilmiş olmalıdır.

malloc, *calloc* yada *realloc* fonksiyonları ile tahsis edilen bellek alanının kullanımı bittikten sonra *free* fonksiyonu ile bu tahsis edilen bellek alanlarının serbest bırakılması gerekir.

Aşağıdaki Örnek 8.13'de karakter sayısı daha sonra artan dizgi göstericisine, ilk olarak bellekte *malloc* fonksiyonu ile 13 byte'lık bellek tahsis edilirken, ilerleyen satırlarda

dizginin boyutunun büyümesi gereken yerlerde *realloc* ile tahsis edilen bellek alanı 25 byte'a arttırılmıştır. Dizgi ile ilgili işlem bittikten sonra ise *free* (*dizgi*) şeklinde bellek alanı serbest bırakılmıştır.

Örnek 8.13: Dinamik bellek ayırma

```
#include<stdio.h>
#include <conio.h>      /*getch fonksiyonunu kullanabilmek için*/
#include<stdlib.h>
#include<string.h>

int main(void)
{
    char *dizgi;
    int i=0,u;
    dizgi=(char*)malloc(sizeof(char)*13);
    if(!dizgi){ printf("Bellek ayrim hatasi!"); /* Bellek ayrilama kontrol*/
        exit(1); }
    strcpy(dizgi, "programlama");
    printf("dizgi= %s, Address = %u\n", dizgi, dizgi);
    dizgi = (char *) realloc(dizgi, sizeof(char)*25);
    strcat(dizgi, "ogreniyorum");
    printf("dizgi= %s, Address = %u\n", dizgi, dizgi);
    free(dizgi); /* Bellek serbest birakiliyor*/
    getch(); /* ekranda görüntüleme için*/
    return 0;
} /* main fonksiyonu sonu*/
```

Program Çıktısı: Örnek 8.13

Dizgi=programlama, Adres=3805152

Dizgi=programlamaogreniyorum, Adres=3805152

Kod içerisinde *free* (*dizgi*) satırından sonra dizgi göstericisinin içeriği kaybedilir, kullanılamaz. Bu nedenle, tamamıyla değişken ile ilgili işlem bittikten sonra **free** fonksiyonunu kullanmak gerekir.

Örnek 8.13, malloc yerine calloc fonksiyonu ile gerçekleşirse nasıl bir değişiklik yapmak gereklidir?



SIRA SİZDE

Özet



Fonksiyonların kullanım amaçlarını açıklamak.

Programlama dillerinde fonksiyonlar, program kodlarını modüler hâle getirebilme özellikleri ile oldukça yararlı program alt bloklarıdır. Fonksiyonlar main() ana fonksiyonunun kontrolünde görevli oldukları işlemleri gerçekleştirirler. Böylece main() fonksiyonu, içerisinde gereksiz yere birçok değişken tanımlı ve işlem karmaşasından arındırılmış olur. Bu hâliyle main() fonksiyonunu bir projenin yöneticisi, fonksiyonları da bu projede görevli çalışanlar olarak ele alabiliriz. Böylece main istediği anda gerekli verileri fonksiyona iletecek, istediği işlem sonucunu fonksiyonun geri dönüş değeri ile elde edecektir. Böylece yapılan herhangi bir işlemle ilgili bir problem oluştuğunda, hata yalnızca ilgili fonksiyonda aranacaktır. Fonksiyonun gerçekleştirdiği işlemde güncelleme gerektiği durumlarda ise yine yalnızca ilgili fonksiyon kod bloğunda değişiklik yapmak yeterli olacaktır. Fonksiyonların kullanım kolaylığı ile ilgili bir diğer durum ise, yeniden yazılmalarına gerek kalmadan, gerekli kütüphane başlık dosyalarını önışlemciler bölümüne ekleyerek hazır kütüphane fonksiyonlarının kullanılabilmeleridir.



Fonksiyonları tanımlayıp, fonksiyon bloğu oluşturarak, fonksiyonu çağırarak.

Amaca uygun olarak oluşturulması planlanan fonksiyon öncelikle hangi giriş argümanlarını alacağı, işlem sonunda main() ortamına hangi değeri geri döndüreceği planlanarak main() fonksiyonu öncesinde, önışlemci tanımlamaları sonrası fonksiyon tanım satırı ilave edilmelidir. Daha sonra bu giriş ve çıkış argümanlarına göre yapılması gereken işlem kod satırlarına aktarılmaktadır. Fonksiyon tanımı ile fonksiyon bloğundaki argüman tipi, sayısı birbirleri ile tutarlı olmalıdır. Fonksiyon blokları main() ana fonksiyonunun bitiminden sonraki bölüme yazılabilir. Yazılan fonksiyon bloğunun ardından, artık main() fonksiyonu içerisinde gerekli olan yerde fonksiyon çağırılması gerçekleştirilerek fonksiyon sonucu elde edilecektir. main() fonksiyonundan çağırılmadığı sürece, fonksiyon herhangi bir çıkış vermeyecektir.



Fonksiyonlara değer veya referans yöntemi ile argüman geçirmek.

Fonksiyonlar main() ana fonksiyonundan çağırılırken, iki türlü argüman değerleri aktarılabilir. Bunlar değer

veya referans yöntemi ile argüman aktarmadır. Değer yöntemi ile argüman fonksiyona geçirilirken fonksiyon bloğunda yapılan işlemlerde, main() deki orijinal değerler bir değişikliğe uğramaz. Oysaki referans yöntemi ile fonksiyona argüman geçirildiğinde, geçirilen değerlerin adresleri fonksiyona bildirildiğinden ve göstericilerle birlikte kullanıldıklarından dolayı main() ortamındaki orijinal veriler de değişiklikten etkileneceklerdir.



Dizileri ve göstericileri fonksiyonlarda argüman olarak kullanmak.

Fonksiyonlara argüman geçirirken, giriş argümanları bölümünde parantez içerisinde değişkenler verilebilir. Ancak buradaki değişken sayısının çok fazla olması fonksiyonda kullanım zorluğu oluşturacaktır. Bu nedenle main() ortamında diziler veya göstericiler yardımıyla oluşturulan verilerin her biri fonksiyona geçirilmek istendiğinde, her birinin ayrı ayrı geçirilmesi fonksiyonu defalarca çağırmaya sebep olacaktır. Bu kullanım da çok pratik değildir. Bu nedenle diziler ve göstericiler fonksiyona geçirilirken referans yöntemi ile başlangıç adresleri fonksiyona bildirilir. Böylece verinin devamındaki değerlere de fonksiyon tarafından ulaşılabilir.



C hazır kütüphane fonksiyonlarını kullanarak önceden tanımlı fonksiyonlar yardımıyla matematiksel, dizgi ve karakterlerle ve bellek ayırma ile ilgili belli işlemleri gerçekleştirmek.

Fonksiyon blokları kullanıcı tarafından oluşturulacağı gibi, C ortamının hazır kütüphane fonksiyonları da kullanılabilir. math.h, string.h, ctype.h, stdlib.h vb. gibi başlık dosyaları önışlemciler bölümüne eklenerek bu fonksiyonlar kullanılabilir. Böylece programcının yeniden, bu işlemleri yapacak fonksiyonları yazmasına gerek kalmayacaktır. Bellek ayırma fonksiyonları kullanılarak ise, değişkenlerin gereksiz yere bellekte yer işgal etmesinin önüne geçilmektedir. Gerektiğinde bellekte yer tahsis edilerek belleğin verimli bir şekilde kullanılması sağlanmaktadır. Bellekte tahsis işlemi gerçekleştirirken değişkenin byte türünden kapladığı alana bağlı olarak yer ayrılmaktadır. Değişken ile ilgili yapılan işlem bittiğinde ise free() ile belleğin serbest bırakılması gerekir. Böylece aynı bellek alanını başka değişkenler kullanabilmektedir.

Kendimizi Sıneyalım

1. `#include<stdio.h>`
`int topla(int x, int y);`
`int main(void)`
`{int a,b,t,c;`
`a=10;`
`b=5;`
`t=topla(a,b);`
`return 0; }`
`int topla(int x, int y)`
`{ int sonuc;`
`sonuc=x+y;`
`_____?_____}`

Yukarıda yer alan programda topla fonksiyonunda, ? ile gösterilen yere yazılabilecek ifade aşağıdakilerden hangisidir?

- `return sonuc ;`
- `sonuc ;`
- `topla ;`
- `x+y ;`
- `return int;`

2. `void yazdir(int x, int y) {`
`printf("toplam sonucu: %d", x);`
`printf("\n");`
`printf("carpim sonucu: %d", y); }`

Fonksiyon bloğu yukarıda verilen *yazdir* fonksiyonu *main()* fonksiyonu içerisinde doğru çağırılması durumu aşağıdakilerden hangisidir?

- `yazdir ( 3, 5);`
- `k=yazdir(3, 5);`
- `k=yazdir();`
- `k=yazdir(3, );`
- `yazdir(void);`

3. 1: `#include <stdio.h>`
2: `int carp(int x, int y)`
3: `int main(void)`
4: `{int a, b, t ;`
5: `a=10;`
6: `b=5;`
7: `t= carp(a,b);`
8: `return 0;`
9: `}`
10: `int carp(int x, int y)`
11: `{ int sonuc;`
12: `return(x+y) ; }`

Yukarıdaki satırları numaralandırılmış, program kodunda bulunan hatalı satır aşağıdakilerden hangisidir?

- 12: `return(x+y) ;`
- 7: `t= carp(a,b);`
- 2: `int carp(int x, int y)`
- 10: `int carp(int x, int y)`
- 8: `return 0;`

4. `#include <stdio.h>`
`#include <conio.h>`
`int kup(int);`
`int main(void)`
`{ int a=3, k;`
`printf(" 1: a=%d \n", a);`
`k=kup(a);`
`printf(" 4: a=%d\n", a);`
`printf(" 5: k=%d", k);`
`getch();`
`return 0; }`

`int kup(int a)`
`{ printf(" 2: a=%d\n", a);`
`a=a*a*a;`
`printf(" 3: a=%d\n", a);`
`return (a);`
`}`

Yukarıdaki programın çıktısı aşağıdakilerden hangisidir?

- 1: a=3 2:a= 3 3:a=27 4:a=3 5: k=27
- 1: a=3 4:a= 3 5:a=27 2:a=3 3: k=27
- 1: a=3 2:a= 3 3:a=27 4:a=3 5: k=27
- 1: a=3 2:a= 3 3:a=3 4:a=3 5: k=3
- 1: a=3 2:a= 3 3:a=3 4:a=3 5: k=27

5. `#include <stdio.h>`
`#include <conio.h>`
`void fonksiyon(int *aGst);`
`int main(void)`
`{ int a=25;`
`printf("1:a =%d\ t", a);`
`fonksiyon(&a);`
`printf("4:a =%d\ t", a);`
`getch();`
`return 0; }`
`void fonksiyon(int *aGst)`
`{ printf("2:a=%d\ t", *aGst);`
`*aGst=*aGst*2;`
`printf("3:a=%d\ t", *aGst); }`

Yukarıdaki programın çıktısı aşağıdakilerden hangisidir?

- 1:a=25 2:a=50 3:a=50 4:a=50
- 1:a=25 4:a=25 2:a=50 3:a=50
- 1:a=25 4:a=25 2:a=50 3:a=25
- 1:a=25 2:a=25 3:a=50 4:a=25
- 1:a=25 2:a=25 3:a=50 4:a=50

6.

```
#include<stdio.h>
#include <conio.h>
#define SIZE 3
int kare(int);
int main(void)
{ int m1[ SIZE]={ 2, 3, 4 };
  int m2[ SIZE]={ 0 };
  int i;
  for(i=0; i< SIZE; i++)
  { m2[ i] = kare (m1[ i] ); }
  for(i=0; i< SIZE; i++)
  { printf(" m1[ %d] =%d , m2[ %d] =%d\t", i,
m1[ i], i, m2[ i] ); }
  getch();
  return 0; }
int kare(int a)
{ return (a*a); }
```

Yukarıdaki programın çıktısı aşağıdakilerden hangisidir?

- m1[0]=2, m2[0]=4 m1[1]=3, m2[1]=9 m1[2]=4, m2[2]=16
- m1[0]=0, m2[0]=0 m1[1]=0, m2[1]=0 m1[2]=0, m2[2]=0
- m1[0]=2, m2[0]=0 m1[1]=3, m2[1]=0 m1[2]=4, m2[2]=0
- m1[0]=2, m2[0]=2 m1[1]=3, m2[1]=3 m1[2]=4, m2[2]=4
- m1[0]=4, m2[0]=4 m1[1]=9, m2[1]=9 m1[2]=16, m2[2]=16

7.

```
#include <stdio.h>
#include <conio.h>
#define SIZE 3
void kare(int *aGst);
int main(void)
{ int m1[ SIZE]={ 3,5,7 };
  int i;
  printf("\n");
  kare (m1);
  for(i=0; i< SIZE; i++)
  { printf("m1[ %d] =%d ", i, m1[ i] ); }
  getch();
  return 0; }
void kare(int *aGst)
{ int i;
  for (i=0; i<SIZE; i++)
  { aGst[ i] =(aGst[ i] *aGst[ i] ); }
}
```

Yukarıdaki programın çıktısı aşağıdakilerden hangisidir?

- m1[0]=7 m2[1]=5 m3[2]=3
- m1[0]=3 m2[1]=5 m3[2]=7
- m1[0]=49 m2[1]=25 m3[2]=9
- m1[0]=9 m2[1]=25 m3[2]=49
- m1[0]=0 m2[1]=1 m3[2]=2

```
8. int kup(int x){
    return(x *x* x) }
```

Yukarıdaki kup fonksiyonu ile eşdeğer fonksiyon aşağıdakilerden hangisidir?

- pow(3, x)
- pow(x, 3)
- sqrt(x, 3)
- 3*pow(x)
- x* pow(3)

9. #include<stdio.h>

```
int main()
{ char k1[30], k2[30];
  strcpy(k1,"programlama");
  strcpy(k2,"ogreniyorum");
  strcat(k1,k2);
  a=isdigit(k1[0]);
  getch();
  return 0; }
```

Yukarıdaki programda eklenmesi gerekli başlık dosyası aşağıdakilerden hangisidir?

- string.h
- math.h
- string.h ve ctype.h
- ctype.h
- conio.h

10.

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
int main()
{
  int *ptr;
  ptr= (int *)malloc(sizeof(int));
  if (ptr== 0)
  { printf("ERROR: Out of memory\n");
    return 1;}
  *ptr= 25;
  free(ptr);
  printf("ptr=%d\n", *ptr);
  getch();
  return 0;}
```

Yukarıdaki programın çıktısı aşağıdakilerden hangisi **olamaz**?

- 25
- 4001776
- 3265478
- 1000347
- 1004000

Kendimizi Sınavalım Yanıt Anahtarı

1. a Yanıtınız yanlış ise “Fonksiyon Tanımı” konusunu yeniden gözden geçiriniz.
2. a Yanıtınız yanlış ise “Fonksiyonlara Argüman Geçirme” konusunu yeniden gözden geçiriniz.
3. c Yanıtınız yanlış ise “Fonksiyon Tanımlama ve Çağırma” konusunu yeniden gözden geçiriniz.
4. a Yanıtınız yanlış ise “Fonksiyona Değer Yöntemi ile Argüman Geçirme” konusunu yeniden gözden geçiriniz.
5. e Yanıtınız yanlış ise “Fonksiyona Referans Yöntemi ile Argüman Geçirme” konusunu yeniden gözden geçiriniz.
6. a Yanıtınız yanlış ise “Dizileri Fonksiyona Argüman olarak Geçirme” konusunu yeniden gözden geçiriniz.
7. d Yanıtınız yanlış ise “Dizileri Fonksiyona Argüman olarak Geçirme” konusunu yeniden gözden geçiriniz.
8. b Yanıtınız yanlış ise “Matematiksel Kütüphane Fonksiyonları (math.h)” konusunu yeniden gözden geçiriniz.
9. c Yanıtınız yanlış ise “Dizgi ve Karakter Fonksiyonları (string.h, ctype.h)” konusunu yeniden gözden geçiriniz.
10. a Yanıtınız yanlış ise “Dinamik Bellek Ayırma Fonksiyonları” konusunu yeniden gözden geçiriniz.

Sıra Sizde Yanıt Anahtarı

Sıra Sizde 1

Örnek 8.1 in program çıktısı ile aynı çıktı elde edilir. Çünkü fonksiyon tanımında önemli olan, argümanın türü ve sayısıdır.

Sıra Sizde 2

```
main fonksiyonundayım
kare fonksiyonundan önce a=4
kare fonksiyonunun icinden merhaba :)
kare fonksiyonunda kopyası olusturulan deger a=4
kare fonksiyonunda degeri degisen a=16
kare fonksiyonundan cikiyorum.....
main fonksiyonuna geri geldim
kare fonksiyonundan donuste a=16
```

Sıra Sizde 3

```
kare fonksiyonuna argüman olarak dizi_1 geçirilmesine rağmen
geri dönüş degeri dizi_2 ye atandığı için, dizi_1 aktarıldığı
andaki degerlerle aynı kalacaktır.
fonksiyon öncesi dizi_1[0]=0 fonksiyon sonrası dizi_1[0]=0
fonksiyon öncesi dizi_1[1]=1 fonksiyon sonrası dizi_1[1]=1
fonksiyon öncesi dizi_1[2]=2 fonksiyon sonrası dizi_1[2]=2
fonksiyon öncesi dizi_1[3]=3 fonksiyon sonrası dizi_1[3]=3
fonksiyon öncesi dizi_1[4]=4 fonksiyon sonrası dizi_1[4]=4
```

Sıra Sizde 4

65-90 arası A-Z arası büyük harfler, 91-96 arası özel şekiller, 97-122 arası a-z küçük harfler, 123-126 arası özel şekiller şeklinde program çıktısı verecektir.

Sıra Sizde 5

dizgi=(char*)malloc(13); satırını

dizgi=(char*)calloc(13,sizeof(char)); ile değiştirmek yeterlidir.

```
#include <stdio.h>
#include <conio.h> /*getch fonksiyonunu kullanabilmek için*/
#include <stdlib.h>
#include <string.h>

int main(void)
{ char *dizgi;
  int i=0,u;

  dizgi=(char*) calloc (13,sizeof(char));
  if(!dizgi){ printf("Bellek ayrim hatasi!"); /* Bellek ayrilamama kontrol*/
    exit(1); }
  strcpy(dizgi, "programlama");
  printf("dizgi= %s, Address = %u\n", dizgi, dizgi);
  dizgi = (char *) realloc(dizgi, sizeof(char)*25);
  strcat(dizgi, "ogreniyorum");
  printf("dizgi= %s, Address = %u\n", dizgi, dizgi);
  free(dizgi); /* Bellek serbest birakiliyor*/
  getch(); /* ekranda goruntuleme için*/
  return 0;
} /* main fonksiyonu sonu*/
```

Yararlanılan ve Başvurulabilecek Kaynaklar

- Deitel, P.,Deitel, H. (2010). **C How to Program-6th Edition.**, Pearson-Prentice Hall.
- Güney, İ. (2006).**C Programlama Dili**, Ankara:Nobel Yayın Dağıtım
- Çölkesen, R. (2005). **C Programlama Dili**, İstanbul:Papatya Yayıncılık.
- Zavrak, M., Aksoy, E. , Karaca, H.N. (2004). **C ve C++**, (Çeviri:C How to Program-Third Edition), İstanbul: Sistem Yayıncılık.
- Tunçer, B.(2002). **Uygulamalı C Programlama Dili**, İstanbul: Papatya Yayıncılık
- Ekinci, F. (1996). **C ve Siz**, İstanbul: Türkmen Kitabevi